

ProgIng - Programación en Ingeniería

Generated by Doxygen 1.16.1

Chapter 1

ProgIng - Programación en Ingeniería

Programación en Ingeniería (IILI06104)

Licenciatura en Ingeniería en Comunicaciones y Electrónica (IS75LI0303)

Licenciatura en Ingeniería en Mecatrónica (IS75LI0403)

Licenciatura en Ingeniería Sistemas Computacionales (IS75LI0502)

Licenciatura en Ingeniería de Datos e Inteligencia Artificial (IS75LI0801)

[Explorar la documentación »](#)

[Ver Demo](#) · [Reportar Bug](#) · [Requiere Modificaciones](#)

Tabla de contenidos

- Introducción
- Datos generales
- Competencia de la UDA
- Contenido
- Entregas y evaluación
 - Tareas
 - Prácticas
 - Proyecto final
 - Elementos mínimos de los reportes
- Repositorio y plataformas
- Documentación del código (Doxygen)
- Estructura sugerida del repositorio
- Código de ética y conducta profesional
- Bibliografía
- [Contacto](#)
- Licencia

1.0.1 Introducción

Esta Unidad de Aprendizaje (UDA) desarrolla bases sólidas de programación **en C** para resolver problemas de ingeniería con énfasis en:

- Estructuras de datos y algoritmos,
- Memoria dinámica y punteros,
- Manejo eficiente de archivos (texto y binarios),
- Buenas prácticas de programación estructurada,
- Trabajo profesional y ético.

Nota: Este repositorio concentra materiales, código, guías y entregables de la UDA.

1.0.2 Datos generales

- **UDA:** Programación en Ingeniería
- **Clave:** IILI06104
- **Periodo:** Enero 2026
- **Días:** Lunes y Jueves
- **Horario:** 12:00–14:00
- **Aula:** B101
- **Créditos:** 6 (150 horas totales: 72 con profesor/a + 78 autónomas)
- **Prerrequisito normativo:** Ninguno
- **Prerrequisito recomendable:** Fundamentos de los Sistemas de Información (IILI06061)

1.0.3 Competencia de la UDA

Diseña e implementa programas computacionales en C para abordar problemas complejos en ingeniería de datos e inteligencia artificial, empleando estructuras de datos avanzadas, manipulación de memoria dinámica y manejo eficiente de archivos, con apego a buenas prácticas, ética profesional y metodologías de programación estructurada.

1.0.4 Contenido

El contenido se organiza en los siguientes ejes:

1. **Estructura de un programa en C**
2. **Variables**
 - Reglas para identificadores
 - Tipos de datos
 - Alcance
3. **Operadores básicos**
 - Aritméticos, comparación, lógicos, binarios
 - Ejemplos
4. **Sentencias de control**
 - Decisiones (if/switch), ciclos (for/while/do-while), anidamientos
5. **Arreglos**
 - 1D (inicialización, longitud, búsqueda, ordenamiento: fuerza bruta, inserción, conteo)
 - Cadenas (longitud, carácter nulo, copiado, concatenado, búsqueda, conversión)
 - 2D y multidimensionales
6. **Manipulación de bits**
7. **Funciones**
 - Prototipos, paso por valor/referencia, `main(argc, argv)`, recursividad
8. **Estructuras y uniones**
9. **Punteros**
 - Aritmética, punteros a arreglos/cadenas/estructuras/funciones
 - Memoria dinámica
 - Punteros a punteros
 - Listas enlazadas (simples y dobles), árboles binarios
10. **Manejo de archivos**
 - Conceptos, flujo, posicionamiento
 - Texto y binarios (lectura/escritura/búsqueda/actualización/temporales)
11. **Directivas de preprocesamiento**
 - Macros, depuración
12. **Tópicos avanzados**
 - Unicode, números complejos, hilos (threads)

1.0.5 Entregas y evaluación

1.0.5.1 Ponderaciones

Elemento	Porcentaje
Tareas	8%

Elemento	Porcentaje
Exámenes rápidos	8%
Primera evaluación (1–4)	8%
Primera práctica	10%
Segunda evaluación (5–9)	8%
Segunda práctica	10%
Tercera evaluación (10–13)	8%
Tercera práctica	10%
Código del proyecto final	10%
Presentación del proyecto final	10%
Reporte del proyecto final	10%
Total	100%

1.0.5.2 Tareas

- **Plataformas:** Microsoft Teams y CSAcademy
- **Entrega por correo:** mibarram@gmail.com
- **Asunto:** PI 2026 1 T## NUA Apellidos
- **Formato:** reporte **PDF** con diagramas de flujo y simulaciones (cuando aplique)
- **Restricción:** **no** entregar ejecutables; **solo** código fuente y reporte
- **Fecha de entrega:** 1 semana (a partir de la asignación)

1.0.5.3 Prácticas

- **Plataforma:** Microsoft Teams
- **Entrega por correo:** mibarram@gmail.com
- **Asunto:** PI 2026 1 P# NUA Apellidos
- **Modalidad:** equipo (máximo 3 integrantes)
- **Formato:** reporte **PDF** con diagramas de flujo, simulaciones, **tablas de resultados**
- **Restricción:** **no** ejecutables; **solo** código fuente y reporte
- **Fecha de entrega:** acordada en clase

1.0.5.4 Proyecto final

- **Plataforma:** Microsoft Teams
- **Entrega por correo:** mibarram@gmail.com
- **Asunto:** PI 2026 1 Py ## NUA Apellidos
- **Modalidad:** equipo (máximo 3 integrantes)
- **Entregables:**
 - reporte **PDF** (diagramas de flujo, simulaciones, tablas y análisis),
 - **código fuente**,
 - **presentación**,
 - sesión de **preguntas y respuestas**.

1.0.5.5 Elementos mínimos de los reportes

1. **Introducción**
2. **Objetivos**
3. **Procedimiento (Algoritmo + Estructura de datos)**
4. **Diagrama de flujo**
5. **Resultados y análisis**
6. **Tablas comparativas** (si aplica)
7. **Conclusiones**
8. **Bibliografía**

1.0.6 Repositorio y plataformas

- **Repositorio oficial (GitHub):** <https://github.com/ibarram/ProgIng/>
- **Microsoft Teams (código de equipo):** co3pdww

- **Repositorio/Materiales (Dropbox):** disponible en el grupo (liga compartida)
- **Comunidad (Facebook):** grupo de apoyo (liga compartida en Teams)

Plataformas recomendadas para práctica y competencia:

- Repl.it, HackerRank, CSAcademy, OmegaUp, Project Euler, Codeforces, Codingame

Software sugerido:

- Dev-C++, MinGW, Code::Blocks, Visual Studio Community, Visual Studio Code, Eclipse, GNU Emacs, NetBeans

1.0.7 Instalación del compilador (Windows, macOS y Linux)

En la UDA trabajaremos principalmente con **C**. Necesitas un compilador (recomendado: **GCC** o **Clang**) y una terminal.

1.0.7.1 Verificación rápida (en cualquier sistema)

Después de instalar, verifica:

```
gcc --version
g++ --version
clang --version
```

Prueba de compilación:

```
# Compilar
gcc hello.c -o hello

# Ejecutar (Linux/macOS)
./hello

# Ejecutar (Windows)
hello.exe
```

Si usas bibliotecas o `math.h`, suele requerirse `-lm`: `gcc main.c -o main -lm`

1.0.7.2 Windows (recomendado)

Opción A: MSYS2 (recomendada por estabilidad y paquetes)

1. Instala MSYS2 (método estándar para tener terminal + paquetes).
2. Abre la terminal **MSYS2 UCRT64** o **MSYS2 MINGW64**.
3. Actualiza e instala el toolchain:

```
pacman -Syu
pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain
```

1. Agrega al **PATH** de Windows la carpeta de binarios (por ejemplo, `... \msys64\ucrt64\bin`).
2. Verifica con `gcc --version`.

Opción B: MinGW-w64 (simple)

- Instala MinGW-w64, agrega `bin` al **PATH** y verifica con `gcc --version`.

Opción C: WSL (Windows Subsystem for Linux)

- Si ya usas WSL (Ubuntu/Debian), sigue las instrucciones de Linux (Debian/Ubuntu).

1.0.7.3 macOS

Opción A: Xcode Command Line Tools (recomendada)

1. Instala las herramientas de línea de comandos:

```
xcode-select --install
```

1. Verifica con:

```
clang --version
gcc --version
```

En macOS, `gcc` normalmente apunta a **Clang** (es normal). Puedes compilar con `clang` o `gcc`.

Opción B: Homebrew + GCC (opcional)

1. Instala Homebrew (si lo usas en tu equipo).
2. Instala GCC:

```
brew install gcc
```

1. Compila usando `gcc-<versión>` (por ejemplo `gcc-14`):

```
gcc-14 hello.c -o hello
```

1.0.7.4 Linux

En la mayoría de distribuciones, GCC/Clang se instalan desde el gestor de paquetes.

Debian/Ubuntu

```
sudo apt update
sudo apt install -y build-essential gdb
```

Fedora

```
sudo dnf install -y gcc gcc-c++ make gdb
```

openSUSE (Leap/Tumbleweed)

```
sudo zypper refresh
sudo zypper install -y gcc gcc-c++ make gdb
```

Arch/Manjaro

```
sudo pacman -Syu --needed base-devel gcc gdb
```

1.0.7.5 Recomendación de editor/IDE (opcional)

- **Visual Studio Code** + extensiones de C/C++ (Microsoft) + CMake Tools (si aplica)
- **Code::Blocks** o **CLion** (si cuentas con licencia)
- En Linux/macOS: `terminal` + `make` es suficiente para iniciar

Software sugerido:

- Dev-C++, MinGW, Code::Blocks, Visual Studio Community, Visual Studio Code, Eclipse, GNU Emacs, NetBeans

1.0.8 Documentación del código (Doxygen)

Este repositorio incluye ejemplos en C documentados ([HTML](#) y [PDF](#)) con comentarios estilo **Doxygen**. La documentación se genera en:

- `doc/doxygen/html/index.html`
- `doc/doxygen/latex/refman.pdf`

1.0.8.1 Requisitos

- **Doxygen**
- **Graphviz** (opcional, pero recomendado para CALL_GRAPH / CALLER_GRAPH)

Instalación típica:

- **openSUSE:** `sudo zypper install doxygen graphviz texlive-scheme-medium latexmk`
- **Ubuntu/Debian:** `sudo apt-get update && sudo apt-get install doxygen graphviz texlive-scheme-medium latexmk`
- **macOS (Homebrew):** `brew install doxygen graphviz MacTeX`

1.0.8.2 Generar documentación

```
make docs
make docs-pdf
```

1.0.9 Estructura sugerida del repositorio

```
.
├── doc/
│   ├── pdf/           # Temario, consideraciones, guías
│   ├── slide/         # Presentaciones (PDF)
│   ├── img/           # Imágenes (escudo, figuras)
│   └── markdown/      # Notas y guías (Markdown)
├── src/
│   ├── 20251/         # Ejemplos desarrollados en la UDA para el periodo E-J 2025
│   ├── 20251/         # Ejemplos desarrollados en la UDA para el periodo A-D 2025
│   ├── 20251/         # Ejemplos desarrollados en la UDA para el periodo E-J 2026
│   ├── tools/         # Scripts de apoyo (opcional)
│   └── templates/     # Plantillas de reporte y estilos
├── data/              # Archivos de entrada/salida
└── LICENSE
```

1.0.10 Código de ética y conducta profesional

- El código debe ser **original** y debidamente **referenciado**.
- Se penaliza el **plagio** (copiar-pegar sin atribución) y la **suplantación**.
- Se fomenta la colaboración en equipo **cuando esté permitida**, respetando las reglas de autoría.
- El uso de herramientas de apoyo (incluida IA) debe reflejarse con transparencia en el **reporte** (qué se usó y cómo).

1.0.11 Bibliografía

- Gazi, O. (2024). *Modern C Programming* (1st Ed.). Springer Cham.
- Horton, I. (2013). *Beginning C* (5th Ed.). Apress.
- Chavan, S. (2017). *C Recipes* (1st Ed.). Apress.
- Toppo, N. & Dewan, H. (2013). *Pointers in C* (1st Ed.). Apress.
- Goyal, A. (2013). *Moving from C to C++* (1st Ed.). Apress.
- Kalicharan, N. (2017). *Advanced Topics in C* (1st Ed.). Apress.
- Erickson, J. (2019). *Algorithms* (1st Ed.). University of Illinois.

1.0.12 Contacto

Dr. M.-A. Ibarra-Manzano - DICIS-UG - ORCID: 0000-0003-4317-0248 - SCOPUS:↔
15837259000

Unidad de Aprendizaje Link: [Programación en Ingeniería](#)

1.0.13 Licencia

Este repositorio se distribuye bajo **GPL-3.0**. Consulta el archivo `LICENSE`.

Chapter 2

Directory Hierarchy

2.1 Directories

20251	??
001_cod_prioridad.c	??
002_Pokemon.c	??
003_cuadratica.c	??
004_rangTemp.c	??
005_rangTemp2.c	??
006_valorPI.c	??
007_exp_1.c	??
008_exp_2.c	??
009_exp_3.c	??
010_Inx.c	??
011_seno.c	??
012_aleatorio.c	??
014_OrdBurbuja.c	??
015_ParesNones.c	??
016_Seleccion.c	??
017_Insercion.c	??
018_Casilleros.c	??
019_BurbujaBD.c	??
020_BurbujaBD.c	??
021_Conteno.c	??
022_multiplicacion.c	??
023_EliGauss.c	??
024_EliGauss.c	??
025_CyN.c	??
026_Binarios.c	??
027_Paridad.c	??
028_Hamming.c	??
029_Enc.c	??
030_ParEntrada.c	??
031_ParEntrada.c	??
032_ParEntrada.c	??
033_Arreglo1D.c	??
Ejemplo_035.c	??
Ejemplo_036.c	??
Ejemplo_037.c	??
20252	??
Ejemplo001.c	??
Ejemplo002.c	??
Ejemplo003.c	??
Ejemplo004.c	??

Ejemplo005.c	??
Ejemplo006.c	??
Ejemplo007.c	??
Ejemplo008.c	??
Ejemplo009.c	??
Ejemplo010.c	??
Ejemplo011.c	??
Ejemplo012.c	??
Ejemplo013.c	??
Ejemplo014.c	??
Ejemplo015.c	??
Ejemplo016.c	??
Ejemplo017.c	??
Ejemplo018.c	??
Ejemplo019.c	??
Ejemplo020.c	??
Ejemplo021.c	??
Ejemplo022.c	??
Ejemplo023.c	??
Ejemplo024.c	??
Ejemplo025.c	??
Ejemplo026.c	??
Ejemplo027.c	??
Ejemplo028.c	??
Ejemplo029.c	??
Ejemplo030.c	??
Ejemplo031.c	??
Ejemplo032.c	??
Ejemplo033.c	??
Ejemplo034.c	??
Ejemplo035.c	??
Ejemplo036.c	??
Ejemplo037.c	??
Ejemplo038.c	??
Ejemplo039.c	??
Ejemplo040.c	??
Ejemplo041.c	??
Ejemplo042.c	??
Ejemplo043.c	??
Ejemplo044.c	??
Ejemplo045.c	??
20261	??
src	??
Ejemplo001.c	??
Ejemplo002.c	??
Ejemplo003.c	??
Ejemplo004.c	??
Ejemplo005.c	??
Ejemplo006.c	??
Ejemplo007.c	??
Ejemplo008.c	??
Ejemplo009.c	??
Ejemplo010.c	??
Ejemplo011.c	??
Ejemplo012.c	??
Ejemplo013.c	??
Ejemplo014.c	??
Ejemplo015.c	??

Ejemplo016.c	??
Ejemplo017.c	??
Ejemplo018.c	??
Ejemplo019.c	??
Ejemplo020.c	??
Ejemplo021.c	??
Ejemplo022.c	??
Ejemplo023.c	??
src	??
20251	??
001_cod_prioridad.c	??
002_Pokemon.c	??
003_cuadratica.c	??
004_rangTemp.c	??
005_rangTemp2.c	??
006_valorPl.c	??
007_exp_1.c	??
008_exp_2.c	??
009_exp_3.c	??
010_Inx.c	??
011_seno.c	??
012_aleatorio.c	??
014_OrdBurbuja.c	??
015_ParesNones.c	??
016_Seleccion.c	??
017_Insercion.c	??
018_Casilleros.c	??
019_BurbujaBD.c	??
020_BurbujaBD.c	??
021_Conteno.c	??
022_multiplicacion.c	??
023_EliGauss.c	??
024_EliGauss.c	??
025_CyN.c	??
026_Binarios.c	??
027_Paridad.c	??
028_Hamming.c	??
029_Enc.c	??
030_ParEntrada.c	??
031_ParEntrada.c	??
032_ParEntrada.c	??
033_Arreglo1D.c	??
Ejemplo_035.c	??
Ejemplo_036.c	??
Ejemplo_037.c	??
20252	??
Ejemplo001.c	??
Ejemplo002.c	??
Ejemplo003.c	??
Ejemplo004.c	??
Ejemplo005.c	??
Ejemplo006.c	??
Ejemplo007.c	??
Ejemplo008.c	??
Ejemplo009.c	??
Ejemplo010.c	??
Ejemplo011.c	??
Ejemplo012.c	??

Ejemplo013.c	??
Ejemplo014.c	??
Ejemplo015.c	??
Ejemplo016.c	??
Ejemplo017.c	??
Ejemplo018.c	??
Ejemplo019.c	??
Ejemplo020.c	??
Ejemplo021.c	??
Ejemplo022.c	??
Ejemplo023.c	??
Ejemplo024.c	??
Ejemplo025.c	??
Ejemplo026.c	??
Ejemplo027.c	??
Ejemplo028.c	??
Ejemplo029.c	??
Ejemplo030.c	??
Ejemplo031.c	??
Ejemplo032.c	??
Ejemplo033.c	??
Ejemplo034.c	??
Ejemplo035.c	??
Ejemplo036.c	??
Ejemplo037.c	??
Ejemplo038.c	??
Ejemplo039.c	??
Ejemplo040.c	??
Ejemplo041.c	??
Ejemplo042.c	??
Ejemplo043.c	??
Ejemplo044.c	??
Ejemplo045.c	??
20261	??
src	??
Ejemplo001.c	??
Ejemplo002.c	??
Ejemplo003.c	??
Ejemplo004.c	??
Ejemplo005.c	??
Ejemplo006.c	??
Ejemplo007.c	??
Ejemplo008.c	??
Ejemplo009.c	??
Ejemplo010.c	??
Ejemplo011.c	??
Ejemplo012.c	??
Ejemplo013.c	??
Ejemplo014.c	??
Ejemplo015.c	??
Ejemplo016.c	??
Ejemplo017.c	??
Ejemplo018.c	??
Ejemplo019.c	??
Ejemplo020.c	??
Ejemplo021.c	??
Ejemplo022.c	??
Ejemplo023.c	??

template	??
Template.c	??
src	??
Ejemplo001.c	??
Ejemplo002.c	??
Ejemplo003.c	??
Ejemplo004.c	??
Ejemplo005.c	??
Ejemplo006.c	??
Ejemplo007.c	??
Ejemplo008.c	??
Ejemplo009.c	??
Ejemplo010.c	??
Ejemplo011.c	??
Ejemplo012.c	??
Ejemplo013.c	??
Ejemplo014.c	??
Ejemplo015.c	??
Ejemplo016.c	??
Ejemplo017.c	??
Ejemplo018.c	??
Ejemplo019.c	??
Ejemplo020.c	??
Ejemplo021.c	??
Ejemplo022.c	??
Ejemplo023.c	??
template	??
Template.c	??

Chapter 3

Data Structure Index

3.1 Data Structures

Here are the data structures with brief descriptions:

contacto		??
dato_s		??
dato_u		??

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

src/20251/001_cod_prioridad.c	??
src/20251/002_Pokemon.c	??
src/20251/003_cuadratica.c	??
src/20251/004_rangTemp.c	??
src/20251/005_rangTemp2.c	??
src/20251/006_valorPl.c	??
src/20251/007_exp_1.c	??
src/20251/008_exp_2.c	??
src/20251/009_exp_3.c	??
src/20251/010_Inx.c	??
src/20251/011_seno.c	??
src/20251/012_aleatorio.c	??
src/20251/014_OrdBurbuja.c	??
src/20251/015_ParesNones.c	??
src/20251/016_Seleccion.c	??
src/20251/017_Insercion.c	??
src/20251/018_Casilleros.c	??
src/20251/019_BurbujaBD.c	??
src/20251/020_BurbujaBD.c	??
src/20251/021_Conteno.c	??
src/20251/022_multiplicacion.c	??
src/20251/023_EliGauss.c	??
src/20251/024_EliGauss.c	??
src/20251/025_CyN.c	??
src/20251/026_Binarios.c	??
src/20251/027_Paridad.c	??
src/20251/028_Hamming.c	??
src/20251/029_Enc.c	??
src/20251/030_ParEntrada.c	??
src/20251/031_ParEntrada.c	??
src/20251/032_ParEntrada.c	??
src/20251/033_Arreglo1D.c	??
src/20251/Ejemplo_035.c	??
src/20251/Ejemplo_036.c	??
src/20251/Ejemplo_037.c	??
src/20252/Ejemplo001.c	??
src/20252/Ejemplo002.c	??
src/20252/Ejemplo003.c	??
src/20252/Ejemplo004.c	??
src/20252/Ejemplo005.c	??

src/20252/Ejemplo006.c	??
src/20252/Ejemplo007.c	??
src/20252/Ejemplo008.c	??
src/20252/Ejemplo009.c	??
src/20252/Ejemplo010.c	??
src/20252/Ejemplo011.c	??
src/20252/Ejemplo012.c	??
src/20252/Ejemplo013.c	??
src/20252/Ejemplo014.c	??
src/20252/Ejemplo015.c	??
src/20252/Ejemplo016.c	??
src/20252/Ejemplo017.c	??
src/20252/Ejemplo018.c	??
src/20252/Ejemplo019.c	??
src/20252/Ejemplo020.c	??
src/20252/Ejemplo021.c	??
src/20252/Ejemplo022.c	??
src/20252/Ejemplo023.c	??
src/20252/Ejemplo024.c	??
src/20252/Ejemplo025.c	??
src/20252/Ejemplo026.c	??
src/20252/Ejemplo027.c	??
src/20252/Ejemplo028.c	??
src/20252/Ejemplo029.c	??
src/20252/Ejemplo030.c	??
src/20252/Ejemplo031.c	??
src/20252/Ejemplo032.c	??
src/20252/Ejemplo033.c	??
src/20252/Ejemplo034.c	??
src/20252/Ejemplo035.c	??
src/20252/Ejemplo036.c	??
src/20252/Ejemplo037.c	??
src/20252/Ejemplo038.c	??
src/20252/Ejemplo039.c	??
src/20252/Ejemplo040.c	??
src/20252/Ejemplo041.c	??
src/20252/Ejemplo042.c	??
src/20252/Ejemplo043.c	??
src/20252/Ejemplo044.c	??
src/20252/Ejemplo045.c	??
src/20261/src/Ejemplo001.c	
Hola mundo mínimo: imprime un mensaje en consola	??
src/20261/src/Ejemplo002.c	
Codificador de prioridad de interrupción para 4 dispositivos (D1..D4)	??
src/20261/src/Ejemplo003.c	
Máximo número de Pokémon que pueden evolucionarse con venta de no evolucionados	??
src/20261/src/Ejemplo004.c	
Mínimo y máximo posible de niños en equipo rojo tras exactamente K cambios	??
src/20261/src/Ejemplo005.c	
Solución de la ecuación cuadrática con raíces reales o complejas	??
src/20261/src/Ejemplo006.c	
Etiquetado de una temperatura T usando 4 umbrales ($T_1 < T_2 < T_3 < T_4$) y 5 categorías	??
src/20261/src/Ejemplo007.c	
Cálculo del entero i tal que $1+3+5+\dots+(2i-1) \geq n$ (relación con $\sqrt{n}$)	??
src/20261/src/Ejemplo008.c	
Aproximación de e mediante la serie de Leibniz	??
src/20261/src/Ejemplo009.c	
Aproximación de $\exp(x)$ con serie de Taylor (cálculo directo de potencias y factorial)	??

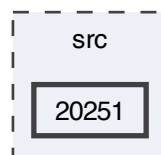
src/20261/src/ Ejemplo010.c	
Aproximación de $\exp(x)$ con Taylor usando producto incremental dentro del término	??
src/20261/src/ Ejemplo011.c	
Aproximación de $\exp(x)$ con Taylor usando recurrencia entre términos (forma eficiente)	??
src/20261/src/ Ejemplo012.c	
Aproximación de $\ln(x)$ con serie tipo atanh calculando potencias desde cero	??
src/20261/src/ Ejemplo013.c	
Aproximación de $\ln(x)$ con serie tipo atanh usando recurrencia entre términos	??
src/20261/src/ Ejemplo014.c	
Aproximación de $\sin(x)$ con serie de Taylor calculando potencia y factorial desde cero	??
src/20261/src/ Ejemplo015.c	
Aproximación de $\sin(x)$ con Taylor usando producto por término (sin factorial entero explícito)	??
src/20261/src/ Ejemplo016.c	
Aproximación de $\sin(x)$ con Taylor usando recurrencia entre términos ($O(n)$)	??
src/20261/src/ Ejemplo017.c	
Aproximación de $\arcsin(x)$ mediante serie de Taylor (Maclaurin) para $ x < 1$	??
src/20261/src/ Ejemplo018.c	
Generación de un arreglo de enteros aleatorios en el rango [min, max]	??
src/20261/src/ Ejemplo019.c	
Generación y ordenamiento de un arreglo de enteros aleatorios (orden ascendente)	??
src/20261/src/ Ejemplo020.c	
Genera un arreglo de enteros aleatorios y lo ordena con Odd–Even Sort (Brick Sort)	??
src/20261/src/ Ejemplo021.c	
Genera un arreglo de flotantes aleatorios y lo ordena con Selection Sort	??
src/20261/src/ Ejemplo022.c	
Genera un arreglo de flotantes aleatorios y lo ordena con Insertion Sort	??
src/20261/src/ Ejemplo023.c	
Clasificación por “casilleros” (bucket/distribution) de números reales en un rango [min, max] . .	??
src/template/ Template.c	??

Chapter 5

Directory Documentation

5.1 src/20251 Directory Reference

Directory dependency graph for 20251:



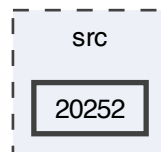
Files

- file [001_cod_prioridad.c](#)
- file [002_Pokemon.c](#)
- file [003_cuadratica.c](#)
- file [004_rangTemp.c](#)
- file [005_rangTemp2.c](#)
- file [006_valorPI.c](#)
- file [007_exp_1.c](#)
- file [008_exp_2.c](#)
- file [009_exp_3.c](#)
- file [010_Inx.c](#)
- file [011_seno.c](#)
- file [012_aleatorio.c](#)
- file [014_OrdBurbuja.c](#)
- file [015_ParesNones.c](#)
- file [016_Seleccion.c](#)
- file [017_Insercion.c](#)
- file [018_Casilleros.c](#)
- file [019_BurbujaBD.c](#)
- file [020_BurbujaBD.c](#)
- file [021_Conteno.c](#)
- file [022_multiplicacion.c](#)

- file [023_EliGauss.c](#)
- file [024_EliGauss.c](#)
- file [025_CyN.c](#)
- file [026_Binarios.c](#)
- file [027_Paridad.c](#)
- file [028_Hamming.c](#)
- file [029_Enc.c](#)
- file [030_ParEntrada.c](#)
- file [031_ParEntrada.c](#)
- file [032_ParEntrada.c](#)
- file [033_Arreglo1D.c](#)
- file [Ejemplo_035.c](#)
- file [Ejemplo_036.c](#)
- file [Ejemplo_037.c](#)

5.2 src/20252 Directory Reference

Directory dependency graph for 20252:



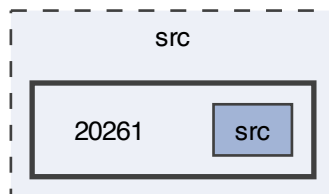
Files

- file [Ejemplo001.c](#)
- file [Ejemplo002.c](#)
- file [Ejemplo003.c](#)
- file [Ejemplo004.c](#)
- file [Ejemplo005.c](#)
- file [Ejemplo006.c](#)
- file [Ejemplo007.c](#)
- file [Ejemplo008.c](#)
- file [Ejemplo009.c](#)
- file [Ejemplo010.c](#)
- file [Ejemplo011.c](#)
- file [Ejemplo012.c](#)
- file [Ejemplo013.c](#)
- file [Ejemplo014.c](#)
- file [Ejemplo015.c](#)
- file [Ejemplo016.c](#)
- file [Ejemplo017.c](#)
- file [Ejemplo018.c](#)
- file [Ejemplo019.c](#)
- file [Ejemplo020.c](#)

- file [Ejemplo021.c](#)
- file [Ejemplo022.c](#)
- file [Ejemplo023.c](#)
- file [Ejemplo024.c](#)
- file [Ejemplo025.c](#)
- file [Ejemplo026.c](#)
- file [Ejemplo027.c](#)
- file [Ejemplo028.c](#)
- file [Ejemplo029.c](#)
- file [Ejemplo030.c](#)
- file [Ejemplo031.c](#)
- file [Ejemplo032.c](#)
- file [Ejemplo033.c](#)
- file [Ejemplo034.c](#)
- file [Ejemplo035.c](#)
- file [Ejemplo036.c](#)
- file [Ejemplo037.c](#)
- file [Ejemplo038.c](#)
- file [Ejemplo039.c](#)
- file [Ejemplo040.c](#)
- file [Ejemplo041.c](#)
- file [Ejemplo042.c](#)
- file [Ejemplo043.c](#)
- file [Ejemplo044.c](#)
- file [Ejemplo045.c](#)

5.3 src/20261 Directory Reference

Directory dependency graph for 20261:



Directories

- directory [src](#)

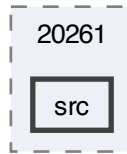
5.4 src Directory Reference

Directories

- directory [20251](#)
- directory [20252](#)
- directory [20261](#)
- directory [template](#)

5.5 src/20261/src Directory Reference

Directory dependency graph for src:



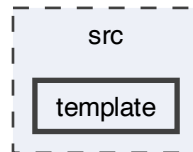
Files

- file [Ejemplo001.c](#)
Hola mundo mínimo: imprime un mensaje en consola.
- file [Ejemplo002.c](#)
Codificador de prioridad de interrupción para 4 dispositivos (D1..D4).
- file [Ejemplo003.c](#)
Máximo número de Pokémon que pueden evolucionarse con venta de no evolucionados.
- file [Ejemplo004.c](#)
Mínimo y máximo posible de niños en equipo rojo tras exactamente K cambios.
- file [Ejemplo005.c](#)
Solución de la ecuación cuadrática con raíces reales o complejas.
- file [Ejemplo006.c](#)
Etiquetado de una temperatura T usando 4 umbrales ($T1 < T2 < T3 < T4$) y 5 categorías.
- file [Ejemplo007.c](#)
Cálculo del entero i tal que $1+3+5+\dots+(2i-1) \geq n$ (relación con $\text{sqrt}(n)$).
- file [Ejemplo008.c](#)
Aproximación de π mediante la serie de Leibniz.
- file [Ejemplo009.c](#)
Aproximación de $\exp(x)$ con serie de Taylor (cálculo directo de potencias y factorial).
- file [Ejemplo010.c](#)
Aproximación de $\exp(x)$ con Taylor usando producto incremental dentro del término.
- file [Ejemplo011.c](#)
Aproximación de $\exp(x)$ con Taylor usando recurrencia entre términos (forma eficiente).
- file [Ejemplo012.c](#)
Aproximación de $\ln(x)$ con serie tipo atanh calculando potencias desde cero.
- file [Ejemplo013.c](#)
Aproximación de $\ln(x)$ con serie tipo atanh usando recurrencia entre términos.
- file [Ejemplo014.c](#)
Aproximación de $\sin(x)$ con serie de Taylor calculando potencia y factorial desde cero.
- file [Ejemplo015.c](#)
Aproximación de $\sin(x)$ con Taylor usando producto por término (sin factorial entero explícito).
- file [Ejemplo016.c](#)
Aproximación de $\sin(x)$ con Taylor usando recurrencia entre términos ($O(n)$).
- file [Ejemplo017.c](#)

- file [Ejemplo018.c](#)
Aproximación de $\arcsin(x)$ mediante serie de Taylor (Maclaurin) para $|x| < 1$.
- file [Ejemplo019.c](#)
Generación de un arreglo de enteros aleatorios en el rango $[min, max]$.
- file [Ejemplo020.c](#)
Generación y ordenamiento de un arreglo de enteros aleatorios (orden ascendente).
- file [Ejemplo021.c](#)
Genera un arreglo de enteros aleatorios y lo ordena con Odd–Even Sort (Brick Sort).
- file [Ejemplo022.c](#)
Genera un arreglo de flotantes aleatorios y lo ordena con Selection Sort.
- file [Ejemplo023.c](#)
Clasificación por “casilleros” (bucket/distribution) de números reales en un rango $[min, max]$.

5.6 src/template Directory Reference

Directory dependency graph for template:



Files

- file [Template.c](#)

Chapter 6

Data Structure Documentation

6.1 contacto Struct Reference

Data Fields

- char [nombre](#) [NM]
- long int [telefono](#)
- char [email](#) [NE]

6.1.1 Detailed Description

Definition at line 7 of file [Ejemplo034.c](#).

6.1.2 Field Documentation

6.1.2.1 email

```
char contacto::email[NE]
```

Definition at line 11 of file [Ejemplo034.c](#).

6.1.2.2 nombre

```
char contacto::nombre[NM]
```

Definition at line 9 of file [Ejemplo034.c](#).

6.1.2.3 telefono

```
long int contacto::telefono
```

Definition at line 10 of file [Ejemplo034.c](#).

The documentation for this struct was generated from the following file:

- [src/20252/Ejemplo034.c](#)

6.2 dato_s Struct Reference

Data Fields

- char [a](#)
- int [b](#)
- float [c](#)

6.2.1 Detailed Description

Definition at line 3 of file [Ejemplo035.c](#).

6.2.2 Field Documentation

6.2.2.1 a

```
char dato_s::a
```

Definition at line 4 of file [Ejemplo035.c](#).

6.2.2.2 b

```
int dato_s::b
```

Definition at line 5 of file [Ejemplo035.c](#).

6.2.2.3 c

```
float dato_s::c
```

Definition at line 6 of file [Ejemplo035.c](#).

The documentation for this struct was generated from the following file:

- [src/20252/Ejemplo035.c](#)

6.3 dato_u Union Reference

Data Fields

- char [a](#)
- int [b](#)
- float [c](#)

6.3.1 Detailed Description

Definition at line 9 of file [Ejemplo035.c](#).

6.3.2 Field Documentation

6.3.2.1 a

```
char dato_u::a
```

Definition at line 10 of file [Ejemplo035.c](#).

6.3.2.2 b

```
int dato_u::b
```

Definition at line 11 of file [Ejemplo035.c](#).

6.3.2.3 c

```
float dato_u::c
```

Definition at line 12 of file [Ejemplo035.c](#).

The documentation for this union was generated from the following file:

- [src/20252/Ejemplo035.c](#)

Chapter 7

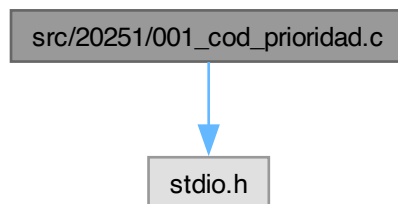
File Documentation

7.1 README.md File Reference

7.2 src/20251/001_cod_prioridad.c File Reference

```
#include <stdio.h>
```

Include dependency graph for 001_cod_prioridad.c:



Functions

- `int main (int argc, char *argv[])`

7.2.1 Function Documentation

7.2.1.1 main()

```
int main (  
    int argc,  
    char * argv[])  
Definition at line 3 of file 001_cod_prioridad.c.  
00004 {  
00005     int D4, D3, D2, D1, sal, b0, b1, b2;  
00006     printf("Ingrese el estado: ");  
00007     scanf("%d %d %d %d", &D4, &D3, &D2, &D1);  
00008     b0 = D1|(~D2&D3);  
00009     b1 = ~D1&(D2|D3);  
00010     b2 = ~D1&~D2&~D3&D4;  
00011     sal = 4*b2+2*b1+b0;  
00012     printf("Salida = %d (%d%d%d)\n", sal, b2, b1, b0);  
00013     return 0;  
00014 }
```

7.3 001_cod_prioridad.c

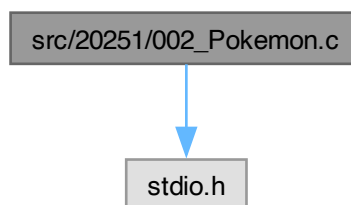
[Go to the documentation of this file.](#)

```
00001 #include<stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     int D4, D3, D2, D1, sal, b0, b1, b2;
00006     printf("Ingrese el estado: ");
00007     scanf("%d %d %d %d", &D4, &D3, &D2, &D1);
00008     b0 = D1|(~D2&D3);
00009     b1 = ~D1&(D2|D3);
00010     b2 = ~D1&~D2&~D3&D4;
00011     sal = 4*b2+2*b1+b0;
00012     printf("Salida = %d (%d%d%d)\n", sal, b2, b1, b0);
00013     return 0;
00014 }
```

7.4 src/20251/002_Pokemon.c File Reference

```
#include <stdio.h>
```

Include dependency graph for 002_Pokemon.c:



Functions

- int [main](#) (int argc, char *argv[])

7.4.1 Function Documentation

7.4.1.1 main()

```
int main (
    int argc,
    char * argv[ ])
```

Definition at line 3 of file [002_Pokemon.c](#).

```
00004 {
00005     int N, M, X, Y, S;
00006     printf("Ingrese los valores: ");
00007     scanf("%d %d %d %d", &N, &M, &X, &Y);
00008     S = (N*Y+M)/(X+Y);
00009     printf("Resultado = %d\n", N*X<M?N:S);
00010     return 0;
00011 }
```

7.5 002_Pokemon.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002
```

```

00003 int main(int argc, char *argv[])
00004 {
00005     int N, M, X, Y, S;
00006     printf("Ingrese los valores: ");
00007     scanf("%d %d %d %d", &N, &M, &X, &Y);
00008     S = (N*Y+M)/(X+Y);
00009     printf("Resultado = %d\n", N*X<M?N:S);
00010     return 0;
00011 }

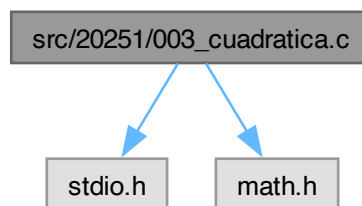
```

7.6 src/20251/003_cuadratica.c File Reference

```
#include <stdio.h>
```

```
#include <math.h>
```

Include dependency graph for 003_cuadratica.c:



Functions

- int `main` (int argc, char *argv[])

7.6.1 Function Documentation

7.6.1.1 main()

```

int main (
    int argc,
    char * argv[]
)

```

Definition at line 4 of file 003_cuadratica.c.

```

00005 {
00006     float a, b, c, x1r, x1i, x2r, x2i, r;
00007     printf("Ingrese el cuadratico: ");
00008     scanf("%f", &a);
00009     printf("Ingrese el lineal: ");
00010     scanf("%f", &b);
00011     printf("Ingrese el independiente: ");
00012     scanf("%f", &c);
00013     r = b*b-4*a*c;
00014     x1r = (-b+(r>=0?sqrt(r):0))/(2*a);
00015     x1i = r>=0?0:sqrt(-r)/(2*a);
00016     x2r = (-b+(r>=0?-sqrt(r):0))/(2*a);
00017     x2i = r>=0?0:-sqrt(-r)/(2*a);
00018     printf("X1 = %f%+fi\n", x1r, x1i);
00019     printf("X2 = %f%+fi\n", x2r, x2i);
00020     return 0;
00021 }

```

7.7 003_cuadratica.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
```

```

00002 #include <math.h>
00003
00004 int main(int argc, char *argv[])
00005 {
00006     float a, b, c, x1r, x1i, x2r, x2i, r;
00007     printf("Ingresa el cuadrático: ");
00008     scanf("%f", &a);
00009     printf("Ingresa el lineal: ");
00010     scanf("%f", &b);
00011     printf("Ingresa el independiente: ");
00012     scanf("%f", &c);
00013     r = b*b-4*a*c;
00014     x1r = (-b+(r>=0?sqrt(r):0))/(2*a);
00015     x1i = r>=0?0:sqrt(-r)/(2*a);
00016     x2r = (-b+(r>=0?-sqrt(r):0))/(2*a);
00017     x2i = r>=0?0:-sqrt(-r)/(2*a);
00018     printf("X1 = %f%+fi\n", x1r, x1i);
00019     printf("X2 = %f%+fi\n", x2r, x2i);
00020     return 0;
00021 }

```

7.8 src/20251/004_rangTemp.c File Reference

#include <stdio.h>

Include dependency graph for 004_rangTemp.c:

src/20251/004_rangTemp.c



stdio.h

Functions

- int `main` (int argc, char *argv[])

7.8.1 Function Documentation

7.8.1.1 main()

```

int main (
    int argc,
    char * argv[])

```

Definition at line 3 of file [004_rangTemp.c](#).

```

00004 {
00005     float T1, T2, T3, T4, T;
00006     int B;
00007     printf("Ingresa los rangos de temperatura: ");
00008     scanf("%f %f %f %f", &T1, &T2, &T3, &T4);
00009     printf("Ingresa la temperatura: ");
00010     scanf("%f", &T);
00011     B = ((T4>T)<<3)|((T3>T)<<2)|((T2>T)<<1)|(T1>T);
00012     printf("%d\n", B);
00013     switch(B)
00014     {
00015     case 15:
00016         printf("Muy Baja\n");
00017         break;
00018     case 14:
00019         printf("Baja\n");

```

```

00020         break;
00021     case 12:
00022         printf("Templada\n");
00023         break;
00024     case 0:
00025         printf("Muy Alta\n");
00026         break;
00027     default:
00028         printf("Alta");
00029     }
00030     return 0;
00031 }

```

7.9 004_rangTemp.c

[Go to the documentation of this file.](#)

```

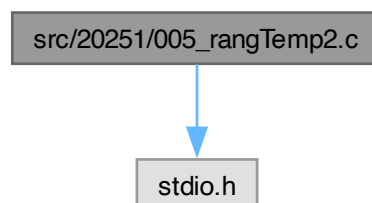
00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     float T1, T2, T3, T4, T;
00006     int B;
00007     printf("Ingrese los rangos de temperatura: ");
00008     scanf("%f %f %f %f", &T1, &T2, &T3, &T4);
00009     printf("Ingrese la temperatura: ");
00010     scanf("%f", &T);
00011     B = ((T4>T)<3) | ((T3>T)<2) | ((T2>T)<1) | (T1>T);
00012     printf("%d\n", B);
00013     switch(B)
00014     {
00015     case 15:
00016         printf("Muy Baja\n");
00017         break;
00018     case 14:
00019         printf("Baja\n");
00020         break;
00021     case 12:
00022         printf("Templada\n");
00023         break;
00024     case 0:
00025         printf("Muy Alta\n");
00026         break;
00027     default:
00028         printf("Alta");
00029     }
00030     return 0;
00031 }

```

7.10 src/20251/005_rangTemp2.c File Reference

```
#include <stdio.h>
```

Include dependency graph for 005_rangTemp2.c:



Functions

- int [main](#) (int argc, char *argv[])

7.10.1 Function Documentation

7.10.1.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 3 of file 005_rangTemp2.c.

```
00004 {
00005     float T1, T2, T3, T4, T;
00006     int B;
00007     printf("Ingrese los rangos de temperatura: ");
00008     scanf("%f %f %f %f", &T1, &T2, &T3, &T4);
00009     printf("Ingrese la temperatura: ");
00010     scanf("%f", &T);
00011     B = ((T4>T)<<3)|((T3>T)<<2)|((T2>T)<<1)|(T1>T);
00012     printf("%d\n", B);
00013     switch(B)
00014     {
00015     case 15:
00016         printf("Muy ");
00017     case 14:
00018         printf("Baja\n");
00019         break;
00020     case 12:
00021         printf("Templada\n");
00022         break;
00023     case 0:
00024         printf("Muy ");
00025     default:
00026         printf("Alta\n");
00027     }
00028     return 0;
00029 }
```

7.11 005_rangTemp2.c

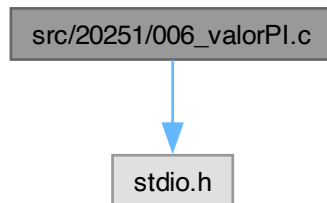
[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     float T1, T2, T3, T4, T;
00006     int B;
00007     printf("Ingrese los rangos de temperatura: ");
00008     scanf("%f %f %f %f", &T1, &T2, &T3, &T4);
00009     printf("Ingrese la temperatura: ");
00010     scanf("%f", &T);
00011     B = ((T4>T)<<3)|((T3>T)<<2)|((T2>T)<<1)|(T1>T);
00012     printf("%d\n", B);
00013     switch(B)
00014     {
00015     case 15:
00016         printf("Muy ");
00017     case 14:
00018         printf("Baja\n");
00019         break;
00020     case 12:
00021         printf("Templada\n");
00022         break;
00023     case 0:
00024         printf("Muy ");
00025     default:
00026         printf("Alta\n");
00027     }
00028     return 0;
00029 }
```

7.12 src/20251/006_valorPI.c File Reference

```
#include <stdio.h>
```

Include dependency graph for 006_valorPI.c:



Functions

- `int main (int argc, char *argv[])`

7.12.1 Function Documentation

7.12.1.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 3 of file [006_valorPI.c](#).

```
00004 {
00005     float pi;
00006     int i, n, den, sg;
00007     do{
00008         printf("Ingrese el numero de terminos: ");
00009         scanf("%d", &n);
00010     }while(n<0);
00011     for(i=0, pi=1; i<n; i++)
00012     {
00013         den = 2*i+3;
00014         sg = 2*(i%2)-1;
00015         pi+=(sg/(float)den);
00016     }
00017     pi*=4;
00018     printf("PI = %f\n", pi);
00019     return 0;
00020 }
```

7.13 006_valorPI.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     float pi;
00006     int i, n, den, sg;
00007     do{
00008         printf("Ingrese el numero de terminos: ");
00009         scanf("%d", &n);
00010     }while(n<0);
00011     for(i=0, pi=1; i<n; i++)
00012     {
00013         den = 2*i+3;
00014         sg = 2*(i%2)-1;
```

```

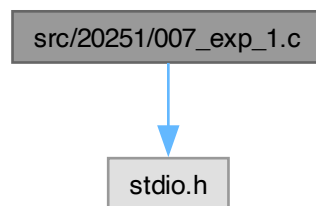
00015         pi+=(sg/(float)den);
00016     }
00017     pi*=4;
00018     printf("PI = %f\n", pi);
00019     return 0;
00020 }

```

7.14 src/20251/007_exp_1.c File Reference

#include <stdio.h>

Include dependency graph for 007_exp_1.c:



Functions

- int [main](#) (int argc, char *argv[])

7.14.1 Function Documentation

7.14.1.1 main()

```

int main (
    int argc,
    char * argv[])

```

Definition at line 3 of file 007_exp_1.c.

```

00004 {
00005     float ex, x, num;
00006     long int i, j, n, den;
00007     printf("Ingrese el valor de x: ");
00008     scanf("%f", &x);
00009     do{
00010         printf("Ingrese el numero de terminos: ");
00011         scanf("%ld", &n);
00012     }while(n<0);
00013     for(i=0, ex=0; i<n; i++)
00014     {
00015         for(j=0, num=1, den=1; j<i; j++)
00016         {
00017             num*=x;
00018             // for(j=0, den=1; j<i; j++)
00019                 den*=(j+1); //den=den*(j+1);
00020         }
00021         ex += num/den;
00022     }
00023     printf("exp(%f) = %f\n", x, ex);
00024     return 0;
00025 }

```

7.15 007_exp_1.c

[Go to the documentation of this file.](#)


```

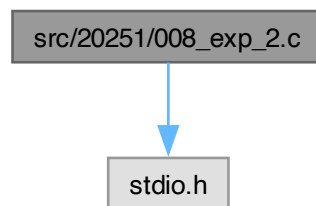
00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     float ex, x, num;
00006     long int i, j, n, den;
00007     printf("Ingresa el valor de x: ");
00008     scanf("%f", &x);
00009     do{
00010         printf("Ingresa el numero de terminos: ");
00011         scanf("%ld", &n);
00012     }while(n<0);
00013     for(i=0, ex=0; i<n; i++)
00014     {
00015         for(j=0, num=1, den=1; j<i; j++)
00016         {
00017             num*=x;
00018             // for(j=0, den=1; j<i; j++)
00019             den*=(j+1); //den=den*(j+1);
00020         }
00021         ex += num/den;
00022     }
00023     printf("exp(%f) = %f\n", x, ex);
00024     return 0;
00025 }

```

7.16 src/20251/008_exp_2.c File Reference

#include <stdio.h>

Include dependency graph for 008_exp_2.c:



Functions

- int [main](#) (int argc, char *argv[])

7.16.1 Function Documentation

7.16.1.1 main()

```

int main (
    int argc,
    char * argv[])

```

Definition at line 3 of file [008_exp_2.c](#).

```

00004 {
00005     float ex, x, fct;
00006     long int i, j, n;
00007     printf("Ingresa el valor de x: ");
00008     scanf("%f", &x);
00009     do{
00010         printf("Ingresa el numero de terminos: ");
00011         scanf("%ld", &n);
00012     }while(n<0);
00013     for(i=0, ex=0; i<n; i++)
00014     {

```

```

00015         for(j=0, fct=1; j<i; j++)
00016             fct *= x/(j+1);
00017         ex += fct;
00018     }
00019     printf("exp(%f) = %f\n", x, ex);
00020     return 0;
00021 }

```

Here is the call graph for this function:



7.17 008_exp_2.c

[Go to the documentation of this file.](#)

```

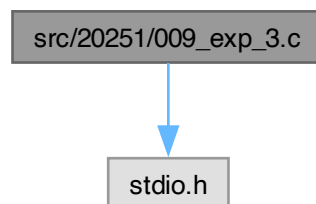
00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     float ex, x, fct;
00006     long int i, j, n;
00007     printf("Ingrese el valor de x: ");
00008     scanf("%f", &x);
00009     do{
00010         printf("Ingrese el numero de terminos: ");
00011         scanf("%ld", &n);
00012     }while(n<0);
00013     for(i=0, ex=0; i<n; i++)
00014     {
00015         for(j=0, fct=1; j<i; j++)
00016             fct *= x/(j+1);
00017         ex += fct;
00018     }
00019     printf("exp(%f) = %f\n", x, ex);
00020     return 0;
00021 }

```

7.18 src/20251/009_exp_3.c File Reference

```
#include <stdio.h>
```

Include dependency graph for 009_exp_3.c:



Functions

- `int main (int argc, char *argv[])`

7.18.1 Function Documentation

7.18.1.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 3 of file 009_exp_3.c.

```
00004 {
00005     float ex, x, fct;
00006     long int i, n;
00007     printf("Ingresa el valor de x: ");
00008     scanf("%f", &x);
00009     do{
00010         printf("Ingresa el numero de terminos: ");
00011         scanf("%ld", &n);
00012     }while(n<0);
00013     for(i=0, ex=0, fct=1; i<n; i++)
00014     {
00015         ex += fct;
00016         fct *= x/(i+1);
00017     }
00018     printf("exp(%f) = %f\n", x, ex);
00019     return 0;
00020 }
```

Here is the call graph for this function:



7.19 009_exp_3.c

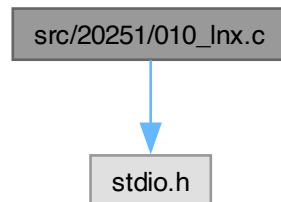
[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     float ex, x, fct;
00006     long int i, n;
00007     printf("Ingresa el valor de x: ");
00008     scanf("%f", &x);
00009     do{
00010         printf("Ingresa el numero de terminos: ");
00011         scanf("%ld", &n);
00012     }while(n<0);
00013     for(i=0, ex=0, fct=1; i<n; i++)
00014     {
00015         ex += fct;
00016         fct *= x/(i+1);
00017     }
00018     printf("exp(%f) = %f\n", x, ex);
00019     return 0;
00020 }
```

7.20 src/20251/010_Inx.c File Reference

#include <stdio.h>

Include dependency graph for 010_Inx.c:



Functions

- int [main](#) (int argc, char *argv[])

7.20.1 Function Documentation

7.20.1.1 main()

```

int main (
    int argc,
    char * argv[])
Definition at line 3 of file 010_Inx.c.
00004 {
00005     float lnx, x, num, fct, px;
00006     int i, den, n;
00007     do{
00008         printf("Intese el valor de x: ");
00009         scanf("%f", &x);
00010     }while(x<=0);
00011     do{
00012         printf("Ingrese el numero de terminos: ");
00013         scanf("%d", &n);
00014     }while(n<=0);
00015     for(i=0, px=(x-1)/(x+1), fct=px, lnx=0; i<n; i++)
00016     {
00017         lnx += fct;
00018         fct*= (2*i+1)*(px*px)/(2*i+3);
00019     }
00020     lnx*=2;
00021     printf("ln(%f) = %f\n", x, lnx);
00022     return 0;
00023 }
00024 }
  
```

Here is the call graph for this function:



7.21 010_Inx.c

[Go to the documentation of this file.](#)

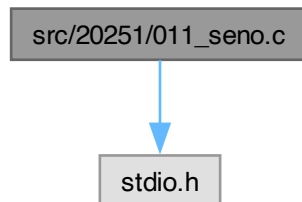
```

00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     float lnx, x, num, fct, px;
00006     int i, den, n;
00007     do{
00008         printf("Intese el valor de x: ");
00009         scanf("%f", &x);
00010     }while(x<=0);
00011     do{
00012         printf("Ingrese el numero de terminos: ");
00013         scanf("%d", &n);
00014     }while(n<=0);
00015     for(i=0, px=(x-1)/(x+1), fct=px, lnx=0; i<n; i++)
00016     {
00017         lnx += fct;
00018         fct*= (2*i+1)*(px*px)/(2*i+3);
00019     }
00020     lnx*=2;
00021     printf("ln(%f) = %f\n", x, lnx);
00022     return 0;
00023 }
00024 }
```

7.22 src/20251/011_seno.c File Reference

```
#include <stdio.h>
```

Include dependency graph for 011_seno.c:



Macros

- #define [PI2](#) 2*3.1415926535897932

Functions

- int [main](#) (int argc, char *argv[])

7.22.1 Macro Definition Documentation

7.22.1.1 PI2

```
#define PI2 2*3.1415926535897932
```

Definition at line 3 of file [011_seno.c](#).

7.22.2 Function Documentation

7.22.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 5 of file 011_seno.c.

```
00006 {
00007     float x, sx, fct;
00008     int n, i;
00009     printf("Ingrese el valor de x: ");
00010     scanf("%f", &x);
00011     x-=((int) (x/(PI2)))*PI2;
00012     do{
00013         printf("Ingrese el numero de terminos: ");
00014         scanf("%d", &n);
00015     }while(n<1);
00016     for(i=0, sx=0, fct=x; i<n; i++)
00017     {
00018         sx += (1-2*(i%2))*fct;
00019         fct *= (x/(2*i+2)) * (x/(2*i+3));
00020     }
00021     printf("sin(%f) = %f\n", x, sx);
00022     return 0;
00023 }
```

Here is the call graph for this function:



7.23 011_seno.c

[Go to the documentation of this file.](#)

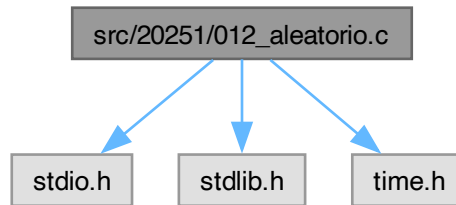
```
00001 #include <stdio.h>
00002
00003 #define PI2 2*3.1415926535897932
00004
00005 int main(int argc, char *argv[])
00006 {
00007     float x, sx, fct;
00008     int n, i;
00009     printf("Ingrese el valor de x: ");
00010     scanf("%f", &x);
00011     x-=((int) (x/(PI2)))*PI2;
00012     do{
00013         printf("Ingrese el numero de terminos: ");
00014         scanf("%d", &n);
00015     }while(n<1);
00016     for(i=0, sx=0, fct=x; i<n; i++)
00017     {
00018         sx += (1-2*(i%2))*fct;
00019         fct *= (x/(2*i+2)) * (x/(2*i+3));
00020     }
00021     printf("sin(%f) = %f\n", x, sx);
00022     return 0;
00023 }
```

7.24 src/20251/012_aleatorio.c File Reference

```
#include <stdio.h>
#include <stdlib.h>
```

```
#include <time.h>
```

Include dependency graph for 012_aleatorio.c:



Macros

- `#define N 100`
- `#define max 20`
- `#define min -10`

Functions

- `int main (int argc, char *argv[])`

7.24.1 Macro Definition Documentation

7.24.1.1 max

```
#define max 20
```

Definition at line 6 of file 012_aleatorio.c.

7.24.1.2 min

```
#define min -10
```

Definition at line 7 of file 012_aleatorio.c.

7.24.1.3 N

```
#define N 100
```

Definition at line 5 of file 012_aleatorio.c.

7.24.2 Function Documentation

7.24.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 9 of file 012_aleatorio.c.

```

00010 {
00011     int A[N], i;
00012     float X[N];
00013     srand(time(NULL));
00014     for(i=0; i<N; i++)
00015     {
00016         A[i] = rand()%(max-min)+min;
00017         X[i] = (max-min)*((float)rand())/RAND_MAX+min;
  
```

```

00018         printf("A[%d] = %d\n", i, A[i]);
00019         printf("X[%d] = %f\n", i, X[i]);
00020     }
00021     return 0;
00022 }

```

7.25 012_aleatorio.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <time.h>
00004
00005 #define N 100
00006 #define max 20
00007 #define min -10
00008
00009 int main(int argc, char *argv[])
00010 {
00011     int A[N], i;
00012     float X[N];
00013     srand(time(NULL));
00014     for(i=0; i<N; i++)
00015     {
00016         A[i] = rand()%(max-min)+min;
00017         X[i] = (max-min)*((float)rand())/RAND_MAX+min;
00018         printf("A[%d] = %d\n", i, A[i]);
00019         printf("X[%d] = %f\n", i, X[i]);
00020     }
00021     return 0;
00022 }

```

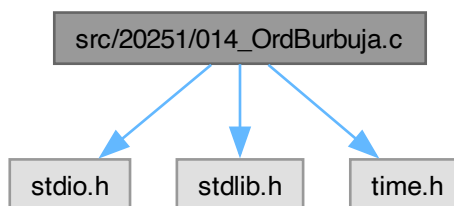
7.26 src/20251/014_OrdBurbuja.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for 014_OrdBurbuja.c:



Macros

- #define N 1000000

Functions

- int [main](#) (int argc, char *argv[])

7.26.1 Macro Definition Documentation

7.26.1.1 N

#define N 1000000

Definition at line 5 of file 014_OrdBurbuja.c.

7.26.2 Function Documentation

7.26.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 7 of file 014_OrdBurbuja.c.

```
00008 {
00009     long int n, i, j;
00010     float X[N], max, min;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%ld", &n);
00015     }while(n<2||n>N);
00016     printf("Ingrese le valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(max<min)
00021     {
00022         max*=min;
00023         min=max/min;
00024         max/=min;
00025     }
00026     printf("[%f, %f]\n", min, max);
00027     printf("Desordenados.\n");
00028     for(i=0; i<n; i++)
00029     {
00030         X[i] = ((max-min)*rand())/RAND_MAX+min;
00031         printf("X[%ld] = %f\n", i+1, X[i]);
00032     }
00033     for(i=0; i<n-1; i++)
00034         for(j=i+1; j<n; j++)
00035             if(X[i]>X[j])
00036             {
00037                 X[i]*=X[j];
00038                 X[j]=X[i]/X[j];
00039                 X[i]/=X[j];
00040             }
00041     printf("Ordenados.\n");
00042     for(i=0; i<n; i++)
00043         printf("X[%ld] = %f\n", i+1, X[i]);
00044     return 0;
00045 }
```

7.27 014_OrdBurbuja.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <time.h>
00004
00005 #define N 1000000
00006
00007 int main(int argc, char *argv[])
00008 {
00009     long int n, i, j;
00010     float X[N], max, min;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%ld", &n);
00015     }while(n<2||n>N);
00016     printf("Ingrese le valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(max<min)
00021     {
```

```

00022         max*=min;
00023         min=max/min;
00024         max/=min;
00025     }
00026     printf("[%f, %f]\n", min, max);
00027     printf("Desordenados.\n");
00028     for(i=0; i<n; i++)
00029     {
00030         X[i] = ((max-min)*rand())/RAND_MAX+min;
00031         printf("X[%ld] = %f\n", i+1, X[i]);
00032     }
00033     for(i=0; i<n-1; i++)
00034         for(j=i+1; j<n; j++)
00035             if(X[i]>X[j])
00036             {
00037                 X[i]*=X[j];
00038                 X[j]=X[i]/X[j];
00039                 X[i]/=X[j];
00040             }
00041     printf("Ordenados.\n");
00042     for(i=0; i<n; i++)
00043         printf("X[%ld] = %f\n", i+1, X[i]);
00044     return 0;
00045 }

```

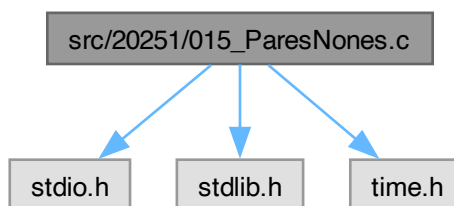
7.28 src/20251/015_ParesNones.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for 015_ParesNones.c:



Macros

- #define [N](#) 1000000

Functions

- int [main](#) (int argc, char *argv[])

7.28.1 Macro Definition Documentation

7.28.1.1 [N](#)

```
#define N 1000000
```

Definition at line 5 of file [015_ParesNones.c](#).

7.28.2 Function Documentation

7.28.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 7 of file 015_ParesNones.c.

```
00008 {
00009     long int n, i, j;
00010     float X[N], max, min;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%ld", &n);
00015     }while (n<2 || n>N);
00016     printf("Ingrese le valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(max<min)
00021     {
00022         max*=min;
00023         min=max/min;
00024         max/=min;
00025     }
00026     printf("[%f, %f]\n", min, max);
00027     printf("Desordenados.\n");
00028     for(i=0; i<n; i++)
00029     {
00030         X[i] = ((max-min)*rand())/RAND_MAX+min;
00031         printf("X[%ld] = %f\n", i+1, X[i]);
00032     }
00033     for(j=0; j<n; j++)
00034     {
00035         for(i=j%2; i<(n+j%2)/2; i++)
00036             if(X[2*i-j%2]>X[2*i+(j+1)%2])
00037             {
00038                 X[2*i-j%2]*=X[2*i+(j+1)%2];
00039                 X[2*i+(j+1)%2]=X[2*i-j%2]/X[2*i+(j+1)%2];
00040                 X[2*i-j%2]/=X[2*i+(j+1)%2];
00041             }
00042     /*     for(i=0; i<n/2; i++)
00043             if(X[2*i]>X[2*i+1])
00044             {
00045                 X[2*i]*=X[2*i+1];
00046                 X[2*i+1]=X[2*i+1]/X[2*i];
00047                 X[2*i]/=X[2*i+1];
00048             }
00049     for(i=1; i<(n+1)/2; i++)
00050             if(X[2*i-1]>X[2*i])
00051             {
00052                 X[2*i-1]*=X[2*i];
00053                 X[2*i]=X[2*i-1]/X[2*i];
00054                 X[2*i-1]/=X[2*i];
00055             }*/
00056     }
00057     printf("Ordenados.\n");
00058     for(i=0; i<n; i++)
00059         printf("X[%ld] = %f\n", i+1, X[i]);
00060     return 0;
00061 }
00062 }
```

7.29 015_ParesNones.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <time.h>
00004
00005 #define N 1000000
00006
00007 int main(int argc, char *argv[])
00008 {
00009     long int n, i, j;
00010     float X[N], max, min;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%ld", &n);
```

```

00015     }while (n<2||n>N);
00016     printf("Ingrese le valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(max<min)
00021     {
00022         max*=min;
00023         min=max/min;
00024         max/=min;
00025     }
00026     printf("[%f, %f]\n", min, max);
00027     printf("Desordenados.\n");
00028     for(i=0; i<n; i++)
00029     {
00030         X[i] = ((max-min)*rand())/RAND_MAX+min;
00031         printf("X[%ld] = %f\n", i+1, X[i]);
00032     }
00033     for(j=0; j<n; j++)
00034     {
00035         for(i=j%2; i<(n+j%2)/2; i++)
00036             if(X[2*i-j%2]>X[2*i+(j+1)%2])
00037             {
00038                 X[2*i-j%2]*=X[2*i+(j+1)%2];
00039                 X[2*i+(j+1)%2]=X[2*i-j%2]/X[2*i+(j+1)%2];
00040                 X[2*i-j%2]/=X[2*i+(j+1)%2];
00041             }
00042     /*     for(i=0; i<n/2; i++)
00043             if(X[2*i]>X[2*i+1])
00044             {
00045                 X[2*i]*=X[2*i+1];
00046                 X[2*i+1]=X[2*i+1]/X[2*i];
00047                 X[2*i]/=X[2*i+1];
00048             }
00049             for(i=1; i<(n+1)/2; i++)
00050                 if(X[2*i-1]>X[2*i])
00051                 {
00052                     X[2*i-1]*=X[2*i];
00053                     X[2*i]=X[2*i-1]/X[2*i];
00054                     X[2*i-1]/=X[2*i];
00055                 }*/
00056     }
00057     printf("Ordenados.\n");
00058     for(i=0; i<n; i++)
00059         printf("X[%ld] = %f\n", i+1, X[i]);
00060     return 0;
00061 }
00062 }

```

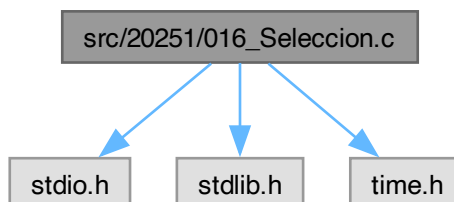
7.30 src/20251/016_Seleccion.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for 016_Seleccion.c:



Macros

- #define N 1000000

Functions

- `int main (int argc, char *argv[])`

7.30.1 Macro Definition Documentation

7.30.1.1 N

#define N 1000000

Definition at line 5 of file [016_Seleccion.c](#).

7.30.2 Function Documentation

7.30.2.1 main()

```
int main (
    int argc,
    char * argv[])
Definition at line 7 of file 016\_Seleccion.c.
00008 {
00009     long int n, i, j, aux_i;
00010     float X[N], max, min;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%ld", &n);
00015     }while (n<2 || n>N);
00016     printf("Ingrese le valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(max<min)
00021     {
00022         max*=min;
00023         min=max/min;
00024         max/=min;
00025     }
00026     printf("[%f, %f]\n", min, max);
00027     printf("Desordenados.\n");
00028     for(i=0; i<n; i++)
00029     {
00030         X[i] = ((max-min)*rand())/RAND_MAX+min;
00031         printf("X[%ld] = %f\n", i+1, X[i]);
00032     }
00033     for(i=0; i<n-1; i++)
00034     {
00035         for(j=i+1, aux_i=i; j<n; j++)
00036             if(X[j]<X[aux_i])
00037                 aux_i = j;
00038         if(i!=aux_i)
00039         {
00040             X[i] *= X[aux_i];
00041             X[aux_i] = X[i]/X[aux_i];
00042             X[i] /= X[aux_i];
00043         }
00044     }
00045     printf("Ordenados.\n");
00046     for(i=0; i<n; i++)
00047         printf("X[%ld] = %f\n", i+1, X[i]);
00048     return 0;
00049 }
00050 }
```

7.31 016_Seleccion.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <time.h>
00004
00005 #define N 1000000
00006
00007 int main(int argc, char *argv[])
00008 {
00009     long int n, i, j, aux_i;
```

```

00010     float X[N], max, min;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%ld", &n);
00015     }while(n<2||n>N);
00016     printf("Ingrese le valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(max<min)
00021     {
00022         max*=min;
00023         min=max/min;
00024         max/=min;
00025     }
00026     printf("[%f, %f]\n", min, max);
00027     printf("Desordenados.\n");
00028     for(i=0; i<n; i++)
00029     {
00030         X[i] = ((max-min)*rand())/RAND_MAX+min;
00031         printf("X[%ld] = %f\n", i+1, X[i]);
00032     }
00033     for(i=0; i<n-1; i++)
00034     {
00035         for(j=i+1, aux_i=i; j<n; j++)
00036             if(X[j]<X[aux_i])
00037                 aux_i = j;
00038         if(i!=aux_i)
00039         {
00040             X[i] *= X[aux_i];
00041             X[aux_i] = X[i]/X[aux_i];
00042             X[i] /= X[aux_i];
00043         }
00044     }
00045
00046     printf("Ordenados.\n");
00047     for(i=0; i<n; i++)
00048         printf("X[%ld] = %f\n", i+1, X[i]);
00049     return 0;
00050 }

```

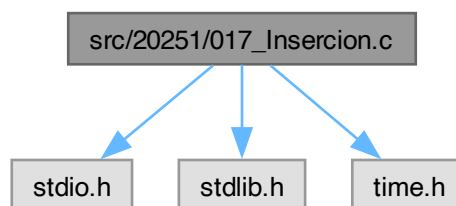
7.32 src/20251/017_Insercion.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for 017_Insercion.c:



Macros

- #define `N` 1000000

Functions

- int `main` (int argc, char *argv[])

7.32.1 Macro Definition Documentation

7.32.1.1 N

#define N 1000000

Definition at line 5 of file 017_Insercion.c.

7.32.2 Function Documentation

7.32.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 7 of file 017_Insercion.c.

```
00008 {
00009     long int n, i, j;
00010     float X[N], max, min, aux;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%ld", &n);
00015     }while(n<2||n>N);
00016     printf("Ingrese le valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(max<min)
00021     {
00022         max*=min;
00023         min=max/min;
00024         max/=min;
00025     }
00026     printf("[%f, %f]\n", min, max);
00027     printf("Desordenados.\n");
00028     for(i=0; i<n; i++)
00029     {
00030         X[i] = ((max-min)*rand())/RAND_MAX+min;
00031         printf("X[%ld] = %f\n", i+1, X[i]);
00032     }
00033     for(i=1; i<n; i++)
00034     {
00035         aux = X[i];
00036         j = i-1;
00037         while(X[j]>aux)
00038         {
00039             X[j+1] = X[j];
00040             j--;
00041             if(j<0)
00042                 break;
00043         }
00044         X[j+1]=aux;
00045     }
00046     printf("Ordenados.\n");
00047     for(i=0; i<n; i++)
00048         printf("X[%ld] = %f\n", i+1, X[i]);
00049     return 0;
00050 }
00051 }
```

7.33 017_Insercion.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <time.h>
00004
00005 #define N 1000000
00006
00007 int main(int argc, char *argv[])
00008 {
00009     long int n, i, j;
00010     float X[N], max, min, aux;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%ld", &n);
00015     }while(n<2||n>N);
```

```

00016     printf("Ingrese le valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(max<min)
00021     {
00022         max*=min;
00023         min=max/min;
00024         max/=min;
00025     }
00026     printf("[%f, %f]\n", min, max);
00027     printf("Desordenados.\n");
00028     for(i=0; i<n; i++)
00029     {
00030         X[i] = ((max-min)*rand())/RAND_MAX+min;
00031         printf("X[%ld] = %f\n", i+1, X[i]);
00032     }
00033     for(i=1; i<n; i++)
00034     {
00035         aux = X[i];
00036         j = i-1;
00037         while (X[j]>aux)
00038         {
00039             X[j+1] = X[j];
00040             j--;
00041             if(j<0)
00042                 break;
00043         }
00044         X[j+1]=aux;
00045     }
00046     printf("Ordenados.\n");
00047     for(i=0; i<n; i++)
00048         printf("X[%ld] = %f\n", i+1, X[i]);
00049     return 0;
00050 }
00051 }

```

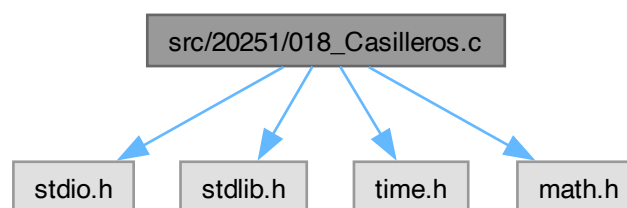
7.34 src/20251/018_Casilleros.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>

```

Include dependency graph for 018_Casilleros.c:



Macros

- #define **N** 1000000
- #define **M** 1000

Functions

- int **main** (int argc, char *argv[])

7.34.1 Macro Definition Documentation

7.34.1.1 M

```
#define M 1000
```

Definition at line 7 of file [018_Casilleros.c](#).

7.34.1.2 N

```
#define N 1000000
```

Definition at line 6 of file [018_Casilleros.c](#).

7.34.2 Function Documentation

7.34.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 9 of file [018_Casilleros.c](#).

```
00010 {
00011     long int n, m, i, j, k, car[M], ind[M], ind2;
00012     float X[N], max, min, aux, rg[M], rng_int, Y[N];
00013     srand(time(NULL));
00014     do{
00015         printf("Ingrese el numero de elementos: ");
00016         scanf("%ld", &n);
00017     }while(n<2||n>N);
00018     printf("Ingrese le valor maximo: ");
00019     scanf("%f", &max);
00020     printf("Ingrese el valor minimo: ");
00021     scanf("%f", &min);
00022     if(max<min)
00023     {
00024         max*=min;
00025         min=max/min;
00026         max/=min;
00027     }
00028     printf("[%f, %f]\n", min, max);
00029     printf("Desordenados.\n");
00030     for(i=0; i<n; i++)
00031     {
00032         X[i] = ((max-min)*rand())/RAND_MAX+min;
00033         printf("X[%ld] = %f\n", i+1, X[i]);
00034     }
00035     m = sqrt(n);
00036     printf("Casilleros: %ld\n", m);
00037     rng_int = ceil((max-min)/m);
00038     rg[0] = min;
00039     car[0] = 0;
00040     for(i=1; i<m+1; i++)
00041     {
00042         rg[i] = rg[i-1]+rng_int;
00043         car[i] = 0;
00044     }
00045     for(i=0; i<n; i++)
00046         car[(int)((X[i]-min)/rng_int)]++;
00047     ind[0]=0;
00048     for(i=1; i<m; i++)
00049         ind[i]=ind[i-1]+car[i-1];
00050     for(i=0; i<m; i++)
00051         printf("#[%f, %f] = %ld\t%ld\n", rg[i], rg[i+1], car[i], ind[i]);
00052     for(i=0; i<n; i++)
00053     {
00054         ind2 = (int)((X[i]-min)/rng_int);
00055         Y[ind[ind2]] = X[i];
00056         ind[ind2]++;
00057     }
00058     printf("Buket.\n");
00059     for(i=0; i<n; i++)
00060         printf("Y[%ld] = %f\n", i+1, Y[i]);
00061     for(k=0, ind2 = 0; k<m; k++)
00062     {
00063         for(i=ind2; i<(ind2+car[k]-1) ; i++)
00064             for(j=i+1; j<(ind2+car[k]); j++)
00065                 if(Y[i]>Y[j])
00066                 {
00067                     Y[i]*=Y[j];
00068                     Y[j]=Y[i]/Y[j];
```

```

00069             Y[i]/=Y[j];
00070         }
00071         ind2 += car[k];
00072     }
00073
00074
00075     printf("Ordenados.\n");
00076     for(i=0; i<n; i++)
00077         printf("Y[%ld] = %f\n", i+1, Y[i]);
00078     return 0;
00079 }

```

7.35 018_Casilleros.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <time.h>
00004 #include <math.h>
00005
00006 #define N 1000000
00007 #define M 1000
00008
00009 int main(int argc, char *argv[])
00010 {
00011     long int n, m, i, j, k, car[M], ind[M], ind2;
00012     float X[N], max, min, aux, rg[M], rng_int, Y[N];
00013     srand(time(NULL));
00014     do{
00015         printf("Ingrese el numero de elementos: ");
00016         scanf("%ld", &n);
00017     }while(n<2||n>N);
00018     printf("Ingrese le valor maximo: ");
00019     scanf("%f", &max);
00020     printf("Ingrese el valor minimo: ");
00021     scanf("%f", &min);
00022     if(max<min)
00023     {
00024         max*=min;
00025         min=max/min;
00026         max/=min;
00027     }
00028     printf("[%f, %f]\n", min, max);
00029     printf("Desordenados.\n");
00030     for(i=0; i<n; i++)
00031     {
00032         X[i] = ((max-min)*rand())/RAND_MAX+min;
00033         printf("X[%ld] = %f\n", i+1, X[i]);
00034     }
00035     m = sqrt(n);
00036     printf("Casilleros: %ld\n", m);
00037     rng_int = ceil((max-min)/m);
00038     rg[0] = min;
00039     car[0] = 0;
00040     for(i=1; i<m+1; i++)
00041     {
00042         rg[i] = rg[i-1]+rng_int;
00043         car[i] = 0;
00044     }
00045     for(i=0; i<n; i++)
00046         car[(int)((X[i]-min)/rng_int)]++;
00047     ind[0]=0;
00048     for(i=1; i<m; i++)
00049         ind[i]=ind[i-1]+car[i-1];
00050     for(i=0; i<m; i++)
00051         printf("#[%f, %f] = %ld\t%ld\n", rg[i], rg[i+1], car[i], ind[i]);
00052     for(i=0; i<n; i++)
00053     {
00054         ind2 = (int)((X[i]-min)/rng_int);
00055         Y[ind[ind2]] = X[i];
00056         ind[ind2]++;
00057     }
00058     printf("Buket.\n");
00059     for(i=0; i<n; i++)
00060         printf("Y[%ld] = %f\n", i+1, Y[i]);
00061     for(k=0, ind2 = 0; k<m; k++)
00062     {
00063         for(i=ind2; i<(ind2+car[k]-1) ; i++)
00064             for(j=i+1; j<(ind2+car[k]); j++)
00065                 if(Y[i]>Y[j])
00066                 {
00067                     Y[i]*=Y[j];
00068                     Y[j]=Y[i]/Y[j];
00069                     Y[i]/=Y[j];

```

```

00070         }
00071         ind2 += car[k];
00072     }
00073
00074
00075     printf("Ordenados.\n");
00076     for(i=0; i<n; i++)
00077         printf("Y[%ld] = %f\n", i+1, Y[i]);
00078     return 0;
00079 }

```

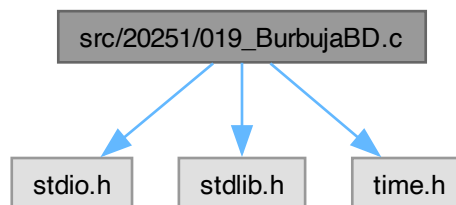
7.36 src/20251/019_BurbujaBD.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for 019_BurbujaBD.c:



Macros

- #define [N](#) 1000000

Functions

- int [main](#) (int argc, char *argv[])

7.36.1 Macro Definition Documentation

7.36.1.1 N

```
#define N 1000000
```

Definition at line 5 of file [019_BurbujaBD.c](#).

7.36.2 Function Documentation

7.36.2.1 main()

```

int main (
    int argc,
    char * argv[])

```

Definition at line 7 of file [019_BurbujaBD.c](#).

```

00008 {
00009     long int n, i, j, k;
00010     float X[N], max, min, aux;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%ld", &n);

```

```

00015     }while (n<2||n>N);
00016     printf("Ingrese le valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(max<min)
00021     {
00022         max*=min;
00023         min=max/min;
00024         max/=min;
00025     }
00026     printf("[%f, %f]\n", min, max);
00027     printf("Desordenados.\n");
00028     for(i=0; i<n; i++)
00029     {
00030         X[i] = ((max-min)*rand())/RAND_MAX+min;
00031         printf("X[%ld] = %f\n", i+1, X[i]);
00032     }
00033
00034     for(i=0, j=n-1; i<(n+1)/2; i++, j--)
00035     {
00036         for(k=i+1; k<n-i; k++)
00037         {
00038             if(X[i]>X[k])
00039             {
00040                 aux = X[i];
00041                 X[i] = X[k];
00042                 X[k] = aux;
00043             }
00044         }
00045         for(k=j-1; k>-1+i; k--)
00046         {
00047             if(X[j]<X[k])
00048             {
00049                 aux = X[j];
00050                 X[j] = X[k];
00051                 X[k] = aux;
00052             }
00053         }
00054     }
00055
00056     printf("Ordenados.\n");
00057     for(i=0; i<n; i++)
00058         printf("X[%ld] = %f\n", i+1, X[i]);
00059     return 0;
00060 }

```

7.37 019_BurbujaBD.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <time.h>
00004
00005 #define N 1000000
00006
00007 int main(int argc, char *argv[])
00008 {
00009     long int n, i, j, k;
00010     float X[N], max, min, aux;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%ld", &n);
00015     }while (n<2||n>N);
00016     printf("Ingrese le valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(max<min)
00021     {
00022         max*=min;
00023         min=max/min;
00024         max/=min;
00025     }
00026     printf("[%f, %f]\n", min, max);
00027     printf("Desordenados.\n");
00028     for(i=0; i<n; i++)
00029     {
00030         X[i] = ((max-min)*rand())/RAND_MAX+min;
00031         printf("X[%ld] = %f\n", i+1, X[i]);
00032     }
00033
00034     for(i=0, j=n-1; i<(n+1)/2; i++, j--)

```

```

00035     {
00036         for(k=i+1; k<n-i; k++)
00037         {
00038             if(X[i]>X[k])
00039             {
00040                 aux = X[i];
00041                 X[i] = X[k];
00042                 X[k] = aux;
00043             }
00044         }
00045         for(k=j-1; k>-1+i; k--)
00046         {
00047             if(X[j]<X[k])
00048             {
00049                 aux = X[j];
00050                 X[j] = X[k];
00051                 X[k] = aux;
00052             }
00053         }
00054     }
00055
00056     printf("Ordenados.\n");
00057     for(i=0; i<n; i++)
00058         printf("X[%ld] = %f\n", i+1, X[i]);
00059     return 0;
00060 }

```

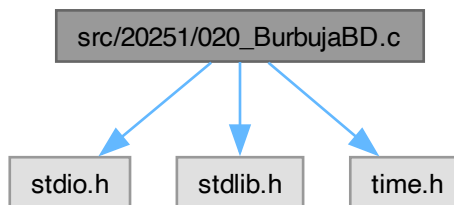
7.38 src/20251/020_BurbujaBD.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for 020_BurbujaBD.c:



Macros

- `#define N 1000000`

Functions

- `int main (int argc, char *argv[ ])`

7.38.1 Macro Definition Documentation

7.38.1.1 N

```
#define N 1000000
```

Definition at line 5 of file 020_BurbujaBD.c.

7.38.2 Function Documentation

7.38.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 7 of file [020_BurbujaBD.c](#).

```
00008 {
00009     long int n, i, j, k;
00010     float X[N], max, min;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%ld", &n);
00015     }while(n<2||n>N);
00016     printf("Ingrese le valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(max<min)
00021     {
00022         max*=min;
00023         min=max/min;
00024         max/=min;
00025     }
00026     printf("[%f, %f]\n", min, max);
00027     printf("Desordenados.\n");
00028     for(i=0; i<n; i++)
00029     {
00030         X[i] = ((max-min)*rand())/RAND_MAX+min;
00031         printf("X[%ld] = %f\n", i+1, X[i]);
00032     }
00033     for (i=0; i<n-1; i++)
00034         for(j=(i%2?n-1-(i+1)/2:i/2+1); (i%2?j>i/2:j<(n-i/2)); i%2?j--:j++)
00035         {
00036             k = i%2?n-(i+1)/2:i/2;
00037             if(i%2?X[k]<X[j]:X[k]>X[j])
00038             {
00039                 X[k]*=X[j];
00040                 X[j]=X[k]/X[j];
00041                 X[k]/=X[j];
00042             }
00043         }
00044     printf("Ordenados.\n");
00045     for(i=0; i<n; i++)
00046         printf("X[%ld] = %f\n", i+1, X[i]);
00047     return 0;
00048 }
00049 }
```

7.39 020_BurbujaBD.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <time.h>
00004
00005 #define N 1000000
00006
00007 int main(int argc, char *argv[])
00008 {
00009     long int n, i, j, k;
00010     float X[N], max, min;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%ld", &n);
00015     }while(n<2||n>N);
00016     printf("Ingrese le valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(max<min)
00021     {
00022         max*=min;
00023         min=max/min;
00024         max/=min;
00025     }
00026     printf("[%f, %f]\n", min, max);
00027     printf("Desordenados.\n");
```

```

00028     for(i=0; i<n; i++)
00029     {
00030         X[i] = ((max-min)*rand())/RAND_MAX+min;
00031         printf("X[%ld] = %f\n", i+1, X[i]);
00032     }
00033
00034     for (i=0; i<n-1; i++)
00035         for(j=(i%2?n-1-(i+1)/2:i/2+1); (i%2?j>i/2:j<(n-i/2)); i%2?j--:j++)
00036         {
00037             k = i%2?n-(i+1)/2:i/2;
00038             if(i%2?X[k]<X[j]:X[k]>X[j])
00039             {
00040                 X[k]*=X[j];
00041                 X[j]=X[k]/X[j];
00042                 X[k]/=X[j];
00043             }
00044         }
00045     printf("Ordenados.\n");
00046     for(i=0; i<n; i++)
00047         printf("X[%ld] = %f\n", i+1, X[i]);
00048     return 0;
00049 }

```

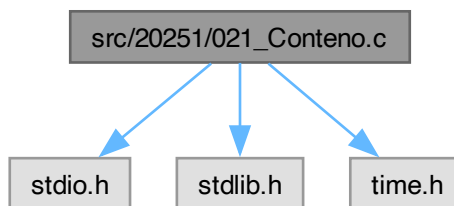
7.40 src/20251/021_Conteno.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for 021_Conteno.c:



Macros

- #define [N](#) 1000000

Functions

- int [main](#) (int argc, char *argv[])

7.40.1 Macro Definition Documentation

7.40.1.1 [N](#)

```
#define N 1000000
```

Definition at line 5 of file [021_Conteno.c](#).

7.40.2 Function Documentation

7.40.2.1 [main\(\)](#)

```
int main (
```

```

    int argc,
    char * argv[])

```

Definition at line 7 of file [021_Conteno.c](#).

```

00008 {
00009     int n, min, max, X[N], i, j, rg;
00010     do{
00011         printf("Ingrese el numero de elementos: ");
00012         scanf("%d", &n);
00013     }while(n<1||n>N);
00014     printf("Ingrese el valor maximo: ");
00015     scanf("%d", &max);
00016     printf("Ingrese el valor minimo: ");
00017     scanf("%d", &min);
00018     if(min>max)
00019     {
00020         max^=min;
00021         min^=max;
00022         max^=min;
00023     }
00024     rg = max-min;
00025     srand(time(NULL));
00026     for(i=0; i<n; i++)
00027         X[i] = (rand()%(max-min+1))+min;
00028     printf("Desordenado.\n");
00029     for(i=0; i<n; i++)
00030         printf("X[%d] = %d\n", i+1, X[i]);
00031     int h[rg+1];
00032     for(i=0; i<rg+1; i++)
00033         h[i] = 0;
00034     for(i=0; i<n; i++)
00035         h[X[i]-min]++;
00036     printf("Histograma\n");
00037     for(i=0; i<rg+1; i++)
00038         printf("h[%d] = %d\n", min+i, h[i]);
00039     for(i=0, j=0; i<n; )
00040     {
00041         while(!h[j])
00042             j++;
00043         while(h[j])
00044         {
00045             h[j]--;
00046             X[i++] = j+min;
00047         }
00048     }
00049     printf("Ordenado.\n");
00050     for(i=0; i<n; i++)
00051         printf("X[%d] = %d\n", i+1, X[i]);
00052     return 0;
00053 }
00054 }

```

7.41 021_Conteno.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <time.h>
00004
00005 #define N 1000000
00006
00007 int main(int argc, char *argv[])
00008 {
00009     int n, min, max, X[N], i, j, rg;
00010     do{
00011         printf("Ingrese el numero de elementos: ");
00012         scanf("%d", &n);
00013     }while(n<1||n>N);
00014     printf("Ingrese el valor maximo: ");
00015     scanf("%d", &max);
00016     printf("Ingrese el valor minimo: ");
00017     scanf("%d", &min);
00018     if(min>max)
00019     {
00020         max^=min;
00021         min^=max;
00022         max^=min;
00023     }
00024     rg = max-min;
00025     srand(time(NULL));
00026     for(i=0; i<n; i++)
00027         X[i] = (rand()%(max-min+1))+min;
00028     printf("Desordenado.\n");

```



```

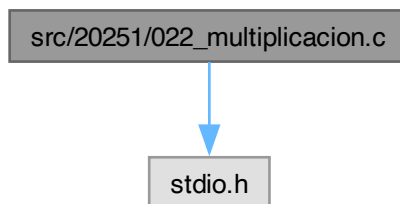
00029     for(i=0; i<n; i++)
00030         printf("X[%d] = %d\n", i+1, X[i]);
00031     int h[rg+1];
00032     for(i=0; i<rg+1; i++)
00033         h[i] = 0;
00034     for(i=0; i<n; i++)
00035         h[X[i]-min]++;
00036     printf("Histograma\n");
00037     for(i=0; i<rg+1; i++)
00038         printf("h[%d] = %d\n", min+i, h[i]);
00039     for(i=0, j=0; i<n; )
00040     {
00041         while(!h[j])
00042             j++;
00043         while(h[j])
00044         {
00045             h[j]--;
00046             X[i++] = j+min;
00047         }
00048     }
00049     printf("Ordenado.\n");
00050     for(i=0; i<n; i++)
00051         printf("X[%d] = %d\n", i+1, X[i]);
00052     return 0;
00053 }
00054 }

```

7.42 src/20251/022_multiplicacion.c File Reference

#include <stdio.h>

Include dependency graph for 022_multiplicacion.c:



Macros

- #define [Re](#) 100
- #define [Co](#) 100

Functions

- int [main](#) (int argc, char *argv[])

7.42.1 Macro Definition Documentation

7.42.1.1 Co

#define Co 100

Definition at line 4 of file [022_multiplicacion.c](#).

7.42.1.2 Re

#define Re 100

Definition at line 3 of file [022_multiplicacion.c](#).

7.42.2 Function Documentation

7.42.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 6 of file [022_multiplicacion.c](#).

```
00007 {
00008     int rA, cA, rB, cB, i, j, k;
00009     float A[Re][Co], B[Re][Co], C[Re][Co];
00010     do{
00011         printf("Ingrese el numero de renglones de A: ");
00012         scanf("%d", &rA);
00013     }while(rA<1||rA>Re);
00014     do{
00015         printf("Ingrese el numero de columnas de A: ");
00016         scanf("%d", &cA);
00017     }while(cA<1||cA>Co);
00018     for(i=0; i<cA; i++)
00019         for(j=0; j<rA; j++)
00020         {
00021             printf("A[%d][%d] = ", i+1, j+1);
00022             while(!scanf("%f", &A[i][j]));
00023         }
00024     do{
00025         printf("Ingrese el numero de renglones de B: ");
00026         scanf("%d", &rB);
00027     }while(rB<1||rB>Re);
00028     do{
00029         printf("Ingrese el numero de columnas de B: ");
00030         scanf("%d", &cB);
00031     }while(cB<1||cB>Co);
00032     for(i=0; i<cB; i++)
00033         for(j=0; j<rB; j++)
00034         {
00035             printf("B[%d][%d] = ", i+1, j+1);
00036             while(!scanf("%f", &B[i][j]));
00037         }
00038     if(cA!=rB)
00039     {
00040         printf("Error: La multiplicacion no puede realizarse\n");
00041         return 1;
00042     }
00043     for(i=0; i<rA; i++)
00044         for(j=0; j<cB; j++)
00045             for(k=0; C[i][j]=0; k<cA; k++)
00046                 C[i][j]+=A[i][k]*B[k][j];
00047     for(i=0; i<rA; i++)
00048         for(j=0; j<cB; j++)
00049             printf("C[%d][%d] = %f\n", i+1, j+1, C[i][j]);
00050     return 0;
00051 }
```

7.43 022_multiplicacion.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002
00003 #define Re 100
00004 #define Co 100
00005
00006 int main(int argc, char *argv[])
00007 {
00008     int rA, cA, rB, cB, i, j, k;
00009     float A[Re][Co], B[Re][Co], C[Re][Co];
00010     do{
00011         printf("Ingrese el numero de renglones de A: ");
00012         scanf("%d", &rA);
00013     }while(rA<1||rA>Re);
00014     do{
00015         printf("Ingrese el numero de columnas de A: ");
00016         scanf("%d", &cA);
00017     }while(cA<1||cA>Co);
00018     for(i=0; i<cA; i++)
00019         for(j=0; j<rA; j++)
00020         {
00021             printf("A[%d][%d] = ", i+1, j+1);
00022             while(!scanf("%f", &A[i][j]));
00023         }
00024     do{
```

```

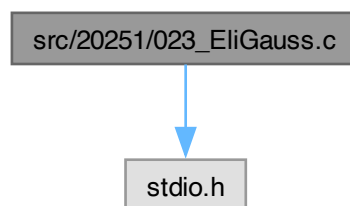
00025     printf("Ingrese el numero de renglones de B: ");
00026     scanf("%d", &rB);
00027 }while (rB<1||rB>Re);
00028 do{
00029     printf("Ingrese el numero de columnas de B: ");
00030     scanf("%d", &cB);
00031 }while (cB<1||cB>Co);
00032 for(i=0; i<cB; i++)
00033     for(j=0; j<rB; j++)
00034     {
00035         printf("B[%d][%d] = ", i+1, j+1);
00036         while(!scanf("%f", &B[i][j]));
00037     }
00038 if(cA!=rB)
00039 {
00040     printf("Error: La multiplicacion no puede realizarse\n");
00041     return 1;
00042 }
00043 for(i=0; i<rA; i++)
00044     for(j=0; j<cB; j++)
00045         for(k=0; C[i][j]=0; k<cA; k++)
00046             C[i][j]+=A[i][k]*B[k][j];
00047 for(i=0; i<rA; i++)
00048     for(j=0; j<cB; j++)
00049         printf("C[%d][%d] = %f\n", i+1, j+1, C[i][j]);
00050 return 0;
00051 }

```

7.44 src/20251/023_EliGauss.c File Reference

#include <stdio.h>

Include dependency graph for 023_EliGauss.c:



Macros

- #define [NV](#) 100

Functions

- int [main](#) (int argc, char *argv[])

7.44.1 Macro Definition Documentation

7.44.1.1 NV

#define NV 100

Definition at line 3 of file [023_EliGauss.c](#).

7.44.2 Function Documentation

7.44.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 5 of file [023_EliGauss.c](#).

```
00006 {
00007     int n, i, j, k;
00008     float A[NV][NV], b[NV], fct, x[NV];
00009     do{
00010         printf("Ingrese el numero de variables: ");
00011         scanf("%d", &n);
00012     }while(n<1 || n>NV);
00013     for(i=0; i<n; i++)
00014     {
00015         for(j=0; j<n; j++)
00016         {
00017             printf("A[%d][%d] = ", i+1, j+1);
00018             scanf("%f", &A[i][j]);
00019         }
00020         printf("b[%d] = ", i+1);
00021         scanf("%f", &b[i]);
00022     }
00023     printf("---\n");
00024     for(i=0; i<n; i++)
00025     {
00026         for(j=0; j<n; j++)
00027             printf("%.2f\t", A[i][j]);
00028         printf("%.2f\n", b[i]);
00029     }
00030     for(i=1; i<n; i++)
00031     {
00032         fct = A[i][0]/A[0][0];
00033         for(j=0; j<n; j++)
00034             A[i][j] -= (fct*A[0][j]);
00035         b[i] -= fct*b[0];
00036     }
00037     printf("---\n");
00038     for(i=0; i<n; i++)
00039     {
00040         for(j=0; j<n; j++)
00041             printf("%.2f\t", A[i][j]);
00042         printf("%.2f\n", b[i]);
00043     }
00044     for(i=2; i<n; i++)
00045     {
00046         fct = A[i][1]/A[1][1];
00047         for(j=0; j<n; j++)
00048             A[i][j] -= (fct*A[1][j]);
00049         b[i] -= fct*b[1];
00050     }
00051     printf("---\n");
00052     for(i=0; i<n; i++)
00053     {
00054         for(j=0; j<n; j++)
00055             printf("%.2f\t", A[i][j]);
00056         printf("%.2f\n", b[i]);
00057     }
00058     for(i=0; i<n; i++)
00059         x[i] = 0;
00060     for(i=n-1; i>=0; i--)
00061     {
00062         x[i] = b[i];
00063         for(j=0; j<n; j++)
00064             if(i!=j)
00065                 x[i] -= A[i][j]*x[j];
00066         x[i] /= A[i][i];
00067         printf("x[%d] = %f\n", i+1, x[i]);
00068     }
00069     return 0;
00070 }
```

Here is the call graph for this function:



7.45 023_EliGauss.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002
00003 #define NV 100
00004
00005 int main(int argc, char *argv[])
00006 {
00007     int n, i, j, k;
00008     float A[NV][NV], b[NV], fct, x[NV];
00009     do{
00010         printf("Ingrese el numero de variables: ");
00011         scanf("%d", &n);
00012     }while(n<1||n>NV);
00013     for(i=0; i<n; i++)
00014     {
00015         for(j=0; j<n; j++)
00016         {
00017             printf("A[%d][%d] = ", i+1, j+1);
00018             scanf("%f", &A[i][j]);
00019         }
00020         printf("b[%d] = ", i+1);
00021         scanf("%f", &b[i]);
00022     }
00023     printf("---\n");
00024     for(i=0; i<n; i++)
00025     {
00026         for(j=0; j<n; j++)
00027             printf("%.2f\t", A[i][j]);
00028         printf("%.2f\n", b[i]);
00029     }
00030     for(i=1; i<n; i++)
00031     {
00032         fct = A[i][0]/A[0][0];
00033         for(j=0; j<n; j++)
00034             A[i][j] -= (fct*A[0][j]);
00035         b[i] -= fct*b[0];
00036     }
00037     printf("---\n");
00038     for(i=0; i<n; i++)
00039     {
00040         for(j=0; j<n; j++)
00041             printf("%.2f\t", A[i][j]);
00042         printf("%.2f\n", b[i]);
00043     }
00044     for(i=2; i<n; i++)
00045     {
00046         fct = A[i][1]/A[1][1];
00047         for(j=0; j<n; j++)
00048             A[i][j] -= (fct*A[1][j]);
00049         b[i] -= fct*b[1];
00050     }
00051     printf("---\n");
00052     for(i=0; i<n; i++)
00053     {
00054         for(j=0; j<n; j++)
00055             printf("%.2f\t", A[i][j]);
00056         printf("%.2f\n", b[i]);
00057     }
00058     for(i=0; i<n; i++)
00059         x[i] = 0;
00060     for(i=n-1; i>-1; i--)
00061     {

```

```

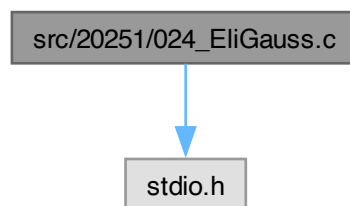
00062         x[i]=b[i];
00063         for(j=0; j<n; j++)
00064             if(i!=j)
00065                 x[i]-=A[i][j]*x[j];
00066         x[i]/=A[i][i];
00067         printf("x[%d] = %f\n", i+1, x[i]);
00068     }
00069     return 0;
00070 }

```

7.46 src/20251/024_EliGauss.c File Reference

#include <stdio.h>

Include dependency graph for 024_EliGauss.c:



Macros

- #define [NV](#) 100

Functions

- int [main](#) (int argc, char *argv[])

7.46.1 Macro Definition Documentation

7.46.1.1 NV

#define NV 100

Definition at line 3 of file [024_EliGauss.c](#).

7.46.2 Function Documentation

7.46.2.1 main()

```

int main (
    int argc,
    char * argv[])

```

Definition at line 5 of file [024_EliGauss.c](#).

```

00006 {
00007     int n, i, j, k;
00008     float A[NV][NV], b[NV], fct, x[NV];
00009     do{
00010         printf("Ingrese el numero de variables: ");
00011         scanf("%d", &n);
00012     }while(n<1||n>NV);
00013     for(i=0; i<n; i++)
00014     {
00015         for(j=0; j<n; j++)
00016         {

```

```

00017         printf("A[%d][%d] = ", i+1, j+1);
00018         scanf("%f", &A[i][j]);
00019     }
00020     printf("b[%d] = ", i+1);
00021     scanf("%f", &b[i]);
00022 }
00023 printf("---\n");
00024 for(i=0; i<n; i++)
00025 {
00026     for(j=0; j<n; j++)
00027         printf("%.2f\t", A[i][j]);
00028     printf("%.2f\n", b[i]);
00029 }
00030 for(i=0; i<n-1; i++)
00031     for(j=i+1; j<n; j++)
00032     {
00033         fct = A[j][i]/A[i][i];
00034         for(k=0; k<n; k++)
00035             A[j][k] -= (fct*A[i][k]);
00036         b[j] -= fct*b[i];
00037     }
00038
00039 for(i=0; i<n; i++)
00040     x[i] = 0;
00041 for(i=n-1; i>-1; i--)
00042 {
00043     x[i]=b[i];
00044     for(j=0; j<n; j++)
00045         if(i!=j)
00046             x[i]-=A[i][j]*x[j];
00047     x[i]/=A[i][i];
00048     printf("x[%d] = %f\n", i+1, x[i]);
00049 }
00050 return 0;
00051 }

```

Here is the call graph for this function:



7.47 024_EliGauss.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002
00003 #define NV 100
00004
00005 int main(int argc, char *argv[])
00006 {
00007     int n, i, j, k;
00008     float A[NV][NV], b[NV], fct, x[NV];
00009     do{
00010         printf("Ingrese el numero de variables: ");
00011         scanf("%d", &n);
00012     }while(n<1||n>NV);
00013     for(i=0; i<n; i++)
00014     {
00015         for(j=0; j<n; j++)
00016         {
00017             printf("A[%d][%d] = ", i+1, j+1);
00018             scanf("%f", &A[i][j]);
00019         }
00020         printf("b[%d] = ", i+1);
00021         scanf("%f", &b[i]);
00022     }
00023     printf("---\n");
00024     for(i=0; i<n; i++)
00025     {

```

```

00026         for(j=0; j<n; j++)
00027             printf("%.2f\t", A[i][j]);
00028         printf("%.2f\n", b[i]);
00029     }
00030     for(i=0; i<n-1; i++)
00031         for(j=i+1; j<n; j++)
00032         {
00033             fct = A[j][i]/A[i][i];
00034             for(k=0; k<n; k++)
00035                 A[j][k] -= (fct*A[i][k]);
00036             b[j] -= fct*b[i];
00037         }
00038
00039     for(i=0; i<n; i++)
00040         x[i] = 0;
00041     for(i=n-1; i>=0; i--)
00042     {
00043         x[i]=b[i];
00044         for(j=0; j<n; j++)
00045             if(i!=j)
00046                 x[i]-=A[i][j]*x[j];
00047         x[i]/=A[i][i];
00048         printf("x[%d] = %f\n", i+1, x[i]);
00049     }
00050     return 0;
00051 }

```

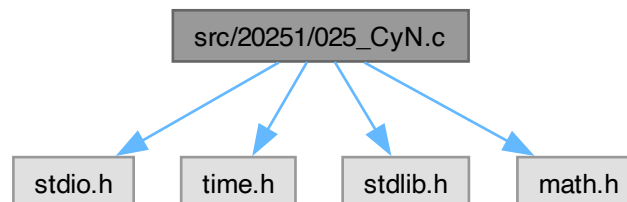
7.48 src/20251/025_CyN.c File Reference

```

#include <stdio.h>
#include <time.h>
#include <stdlib.h>
#include <math.h>

```

Include dependency graph for 025_CyN.c:



Macros

- `#define N 10000`

Functions

- `int main (int argc, char *argv[ ])`

7.48.1 Macro Definition Documentation

7.48.1.1 N

```
#define N 10000
```

Definition at line 6 of file 025_CyN.c.

7.48.2 Function Documentation

7.48.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 8 of file 025_CyN.c.

```
00009 {
00010     float X[N], min, max, xmin, xmax, xmed, xvar, xran;
00011     int n, i, flag, flag_e, flag_n, flag_c;
00012     do{
00013         printf("Ingrese el numero de muestra: ");
00014         scanf("%d", &n);
00015     }while(n<1);
00016     printf("Ingrese el valor minimo: ");
00017     scanf("%f", &min);
00018     printf("Ingrese le valor maximo: ");
00019     scanf("%f", &max);
00020     if(min>max)
00021     {
00022         max*=min;
00023         min=max/min;
00024         max/=min;
00025     }
00026     srand(time(NULL));
00027     for(i=0; i<n; i++)
00028     {
00029         X[i] = ((max-min)*rand())/RAND_MAX+min;
00030         printf("X[%d] = %f\n", i+1, X[i]);
00031     }
00032     printf("Normalización: ");
00033     scanf("%d", &flag_n);
00034     printf("Centrado: ");
00035     scanf("%d", &flag_c);
00036     printf("Estadístico: ");
00037     scanf("%d", &flag_e);
00038     flag = ((flag_c<1)|(flag_n)<1)|(flag_e<1);
00039     printf("flag = %d\n", flag);
00040     for(xmin=X[0], xmax=X[0], xmed=X[0], xvar=X[0]*X[0], i=1; i<n; i++)
00041     {
00042         if(X[i]<xmin)
00043             xmin = X[i];
00044         if(X[i]>xmax)
00045             xmax = X[i];
00046         xmed += X[i];
00047         xvar += X[i]*X[i];
00048     }
00049     xmed/=n;
00050     xvar/=n;
00051     xvar-=(xmed*xmed);
00052     xran=xmax-xmin;
00053     if((flag&1)|(flag&4))
00054         for(i=0; i<n; i++)
00055             X[i]-=(flag&4?xmed:xmin);
00056     if((flag&2)|(flag&8))
00057         for(i=0; i<n; i++)
00058             X[i]/=(flag&8?xvar:xran);
00059     printf("Maximo: %f\nMinimo: %f\nMedia: %f\nVarianza: %f\n",
00060           xmax, xmin, xmed, xvar);
00061     for(i=0; i<n; i++)
00062         printf("X[%d] = %f\n", i+1, X[i]);
00063     return 0;
00064 }
```

7.49 025_CyN.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002 #include <time.h>
00003 #include <stdlib.h>
00004 #include <math.h>
00005
00006 #define N 10000
00007
00008 int main(int argc, char *argv[])
00009 {
00010     float X[N], min, max, xmin, xmax, xmed, xvar, xran;
00011     int n, i, flag, flag_e, flag_n, flag_c;
00012     do{
00013         printf("Ingrese el numero de muestra: ");
```

```

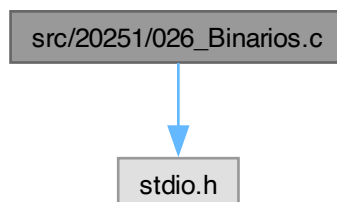
00014     scanf("%d", &n);
00015 }while (n<1);
00016 printf("Ingrese el valor minimo: ");
00017 scanf("%f", &min);
00018 printf("Ingrese le valor maximo: ");
00019 scanf("%f", &max);
00020 if(min>max)
00021 {
00022     max*=min;
00023     min=max/min;
00024     max/=min;
00025 }
00026 srand(time(NULL));
00027 for(i=0; i<n; i++)
00028 {
00029     X[i] = ((max-min)*rand())/RAND_MAX+min;
00030     printf("X[%d] = %f\n", i+1, X[i]);
00031 }
00032 printf("Normalización: ");
00033 scanf("%d", &flag_n);
00034 printf("Centrado: ");
00035 scanf("%d", &flag_c);
00036 printf("Estadístico: ");
00037 scanf("%d", &flag_e);
00038 flag = ((flag_c<1)|flag_n)<<(flag_e<1);
00039 printf("flag = %d\n", flag);
00040 for(xmin=X[0], xmax=X[0], xmed=X[0], xvar=X[0]*X[0], i=1; i<n; i++)
00041 {
00042     if(X[i]<xmin)
00043         xmin = X[i];
00044     if(X[i]>xmax)
00045         xmax = X[i];
00046     xmed += X[i];
00047     xvar += X[i]*X[i];
00048 }
00049 xmed/=n;
00050 xvar/=n;
00051 xvar-= (xmed*xmed);
00052 xran=xmax-xmin;
00053 if((flag&1)|(flag&4))
00054     for(i=0; i<n; i++)
00055         X[i]-=(flag&4?xmed:xmin);
00056 if((flag&2)|(flag&8))
00057     for(i=0; i<n; i++)
00058         X[i]/=(flag&8?xvar:xran);
00059 printf("Maximo: %f\nMinimo: %f\nMedia: %f\nVarianza: %f\n",
00060        xmax, xmin, xmed, xvar);
00061 for(i=0; i<n; i++)
00062     printf("X[%d] = %f\n", i+1, X[i]);
00063 return 0;
00064 }

```

7.50 src/20251/026_Binarios.c File Reference

#include <stdio.h>

Include dependency graph for 026_Binarios.c:



Macros

- #define [BIT\(n\)](#)

- `#define BIT_GET(x, n)`
- `#define BIT_SET(x, n)`
- `#define BIT_CLEAR(x, n)`
- `#define BIT_TOGGLE(x, n)`
- `#define BIT_WRITE(x, n, v)`

Functions

- `int main (int argc, char *argv[])`

7.50.1 Macro Definition Documentation

7.50.1.1 BIT

```
#define BIT(  
    n)
```

Value:

`(1<<(n))`

Definition at line 3 of file [026_Binarios.c](#).

7.50.1.2 BIT_CLEAR

```
#define BIT_CLEAR(  
    x,  
    n)
```

Value:

`((x) &= ~BIT(n))`

Definition at line 6 of file [026_Binarios.c](#).

7.50.1.3 BIT_GET

```
#define BIT_GET(  
    x,  
    n)
```

Value:

`((x) & BIT(n))`

Definition at line 4 of file [026_Binarios.c](#).

7.50.1.4 BIT_SET

```
#define BIT_SET(  
    x,  
    n)
```

Value:

`((x) |= BIT(n))`

Definition at line 5 of file [026_Binarios.c](#).

7.50.1.5 BIT_TOGGLE

```
#define BIT_TOGGLE(  
    x,  
    n)
```

Value:

`((x) ^= BIT(n))`

Definition at line 7 of file [026_Binarios.c](#).

7.50.1.6 BIT_WRITE

```
#define BIT_WRITE(  
    x,  
    n,  
    v)
```

Value:

```
((v)?BIT_SET(x,n):BIT_CLEAR(x,n))
```

Definition at line 8 of file [026_Binarios.c](#).

7.50.2 Function Documentation

7.50.2.1 main()

```
int main (  
    int argc,  
    char * argv[])
```

Definition at line 10 of file [026_Binarios.c](#).

```
00011 {  
00012     int i, n, nb;  
00013     int xi;  
00014     char x, xb;  
00015     printf("x = ");  
00016     scanf("%d", &xi);  
00017     x = (char)xi;  
00018     n = 8*sizeof(x);  
00019     for(i=n; i>0; i--)  
00020         printf("%d", BIT_GET(x,i-1)?1:0);  
00021     printf("\n");  
00022     do{  
00023         printf("Ingrese el numero de bit: ");  
00024         scanf("%d", &nb);  
00025     }while(nb<-1|nb>=n);  
00026     printf("%d\n", BIT_SET(x, nb));  
00027     do{  
00028         printf("Ingrese el numero de bit: ");  
00029         scanf("%d", &nb);  
00030     }while(nb<-1|nb>=n);  
00031     printf("%d\n", BIT_CLEAR(x, nb));  
00032     do{  
00033         printf("Ingrese el numero de bit: ");  
00034         scanf("%d", &nb);  
00035     }while(nb<-1|nb>=n);  
00036     printf("%d\n", BIT_TOGGLE(x, nb));  
00037     do{  
00038         printf("Ingrese el numero de bit: ");  
00039         scanf("%d", &nb);  
00040     }while(nb<-1|nb>=n);  
00041     printf("Ingrese el valor del bit: ");  
00042     scanf("%d", &xi);  
00043     xb = (char)xi;  
00044     printf("%d\n", BIT_WRITE(x, nb, xb));  
00045     return 0;  
00046 }
```

7.51 026_Binarios.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>  
00002  
00003 #define BIT(n)                (1<<(n))  
00004 #define BIT_GET(x,n)         ((x)&BIT(n))  
00005 #define BIT_SET(x,n)         ((x) |= BIT(n))  
00006 #define BIT_CLEAR(x,n)       ((x) &= ~BIT(n))  
00007 #define BIT_TOGGLE(x,n)      ((x) ^= BIT(n))  
00008 #define BIT_WRITE(x,n,v)     ((v)?BIT_SET(x,n):BIT_CLEAR(x,n))  
00009  
00010 int main(int argc, char *argv[])  
00011 {  
00012     int i, n, nb;  
00013     int xi;  
00014     char x, xb;  
00015     printf("x = ");  
00016     scanf("%d", &xi);  
00017     x = (char)xi;  
00018     n = 8*sizeof(x);
```

```

00019     for(i=n; i>0; i--)
00020         printf("%d", BIT_GET(x,i-1)?1:0);
00021     printf("\n");
00022     do{
00023         printf("Ingrese el numero de bit: ");
00024         scanf("%d", &nb);
00025     }while(nb<-1|nb>=n);
00026     printf("%d\n", BIT_SET(x, nb));
00027     do{
00028         printf("Ingrese el numero de bit: ");
00029         scanf("%d", &nb);
00030     }while(nb<-1|nb>=n);
00031     printf("%d\n", BIT_CLEAR(x, nb));
00032     do{
00033         printf("Ingrese el numero de bit: ");
00034         scanf("%d", &nb);
00035     }while(nb<-1|nb>=n);
00036     printf("%d\n", BIT_TOGGLE(x, nb));
00037     do{
00038         printf("Ingrese el numero de bit: ");
00039         scanf("%d", &nb);
00040     }while(nb<-1|nb>=n);
00041     printf("Ingrese el valor del bit: ");
00042     scanf("%d", &xi);
00043     xb = (char)xi;
00044     printf("%d\n", BIT_WRITE(x, nb, xb));
00045     return 0;
00046 }

```

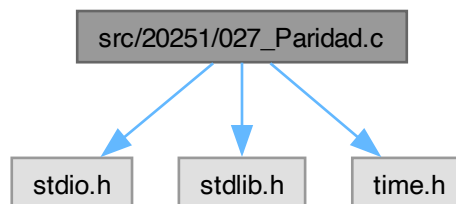
7.52 src/20251/027_Paridad.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for 027_Paridad.c:



Macros

- `#define BIT(n)`
- `#define BIT_TOGGLE(x, n)`
- `#define NC 100`

Functions

- `int main (int argc, char *argv[])`

7.52.1 Macro Definition Documentation

7.52.1.1 BIT

```

#define BIT(
    n)

```

Value:`(1<<(n))`Definition at line 5 of file [027_Paridad.c](#).**7.52.1.2 BIT_TOGGLE**

```
#define BIT_TOGGLE(
    x,
    n)
```

Value:`((x) ^= BIT(n))`Definition at line 6 of file [027_Paridad.c](#).**7.52.1.3 NC**

```
#define NC 100
```

Definition at line 7 of file [027_Paridad.c](#).**7.52.2 Function Documentation****7.52.2.1 main()**

```
int main (
    int argc,
    char * argv[])
```

Definition at line 9 of file [027_Paridad.c](#).

```
00010 {
00011     char msg[NC+1], p, v;
00012     int i, nc, _nc, _nb;
00013     srand(time(NULL));
00014     i=0;
00015     do{
00016         msg[i++] = getc(stdin);
00017     }while(msg[i-1]!=10&& i<NC);
00018     nc = i-1;
00019     msg[nc] = '\0';
00020     for(i=0, p=0; i<nc; i++)
00021         p^=msg[i];
00022     for(i=0, v=p; i<nc; i++)
00023         v^=msg[i];
00024     printf("MSG: %s(%d)-%c-%d\n", msg, nc, p, (int )v);
00025     _nc = rand()%nc;
00026     _nb = rand()% (8*sizeof(char));
00027     BIT_TOGGLE(msg[_nc], _nb);
00028     for(i=0, v=p; i<nc; i++)
00029         v^=msg[i];
00030     printf("MSG: %s(%d)-%c-%u\n", msg, nc, p, (unsigned int)v);
00031     return 0;
00032 }
```

7.53 027_Paridad.c[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <time.h>
00004
00005 #define BIT(n)          (1<<(n))
00006 #define BIT_TOGGLE(x,n) ((x) ^= BIT(n))
00007 #define NC              100
00008
00009 int main(int argc, char *argv[])
00010 {
00011     char msg[NC+1], p, v;
00012     int i, nc, _nc, _nb;
00013     srand(time(NULL));
00014     i=0;
00015     do{
00016         msg[i++] = getc(stdin);
00017     }while(msg[i-1]!=10&& i<NC);
00018     nc = i-1;
00019     msg[nc] = '\0';
```

```

00020     for(i=0, p=0; i<nc; i++)
00021         p^=msg[i];
00022     for(i=0, v=p; i<nc; i++)
00023         v^=msg[i];
00024     printf("MSG: %s(%d)-%c-%d\n", msg, nc, p, (int )v);
00025     _nc = rand()%nc;
00026     _nb = rand()%(8*sizeof(char));
00027     BIT_TOGGLE(msg[_nc], _nb);
00028     for(i=0, v=p; i<nc; i++)
00029         v^=msg[i];
00030     printf("MSG: %s(%d)-%c-%u\n", msg, nc, p, (unsigned int)v);
00031     return 0;
00032 }

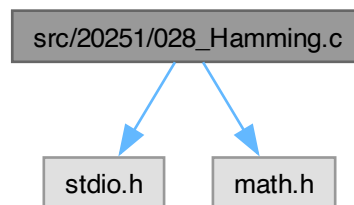
```

7.54 src/20251/028_Hamming.c File Reference

```
#include <stdio.h>
```

```
#include <math.h>
```

Include dependency graph for 028_Hamming.c:



Macros

- `#define MC 2048`
- `#define l2 log(2)`

Functions

- `int main (int argc, char *argv[ ])`

7.54.1 Macro Definition Documentation

7.54.1.1 l2

```
#define l2 log(2)
```

Definition at line 5 of file 028_Hamming.c.

7.54.1.2 MC

```
#define MC 2048
```

Definition at line 4 of file 028_Hamming.c.

7.54.2 Function Documentation

7.54.2.1 main()

```

int main (
    int argc,
    char * argv[ ])

```

Definition at line 7 of file [028_Hamming.c](#).

```

00008 {
00009     char str[MC], strP[MC];
00010     int i, j, k, nc, nP;
00011     i=0;
00012     do{
00013         str[i++] = getc(stdin);
00014     }while(str[i-1] != '\0');
00015     str[i-1] = '\0';
00016     nc = i-1;
00017     nP = log(nc+log(nc)/12+1)/12+1;
00018     printf("%s(%d, %d)\n", str, nc, nP);
00019     for(i=0, j=1, k=0; i<nc+nP; i++)
00020     {
00021         if((i+1) != j)
00022             strP[i] = str[k++];
00023         else
00024         {
00025             strP[i] = 0;
00026             j<<=1;
00027         }
00028     }
00029     for(i=0, j=1; i<nP; i++)
00030     {
00031         for(k=j; k<=(nc+nP); k++)
00032             if(((k-j)%(2*j))<j)
00033                 strP[j-1]^=strP[k-1];
00034         // printf("%d\t", ((k-j)%(2*j))<j);
00035         // printf("\n");
00036         j<<=1;
00037     }
00038     strP[nc+nP] = '\0';
00039     for(i=0; i<nc+nP; i++)
00040         printf("%d\n", strP[i]);
00041     printf("%s\n", strP);
00042     return 0;
00043 }

```

7.55 028_Hamming.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002 #include <math.h>
00003
00004 #define MC 2048
00005 #define L2 log(2)
00006
00007 int main(int argc, char *argv[])
00008 {
00009     char str[MC], strP[MC];
00010     int i, j, k, nc, nP;
00011     i=0;
00012     do{
00013         str[i++] = getc(stdin);
00014     }while(str[i-1] != '\0');
00015     str[i-1] = '\0';
00016     nc = i-1;
00017     nP = log(nc+log(nc)/12+1)/12+1;
00018     printf("%s(%d, %d)\n", str, nc, nP);
00019     for(i=0, j=1, k=0; i<nc+nP; i++)
00020     {
00021         if((i+1) != j)
00022             strP[i] = str[k++];
00023         else
00024         {
00025             strP[i] = 0;
00026             j<<=1;
00027         }
00028     }
00029     for(i=0, j=1; i<nP; i++)
00030     {
00031         for(k=j; k<=(nc+nP); k++)
00032             if(((k-j)%(2*j))<j)
00033                 strP[j-1]^=strP[k-1];
00034         // printf("%d\t", ((k-j)%(2*j))<j);
00035         // printf("\n");
00036         j<<=1;
00037     }
00038     strP[nc+nP] = '\0';
00039     for(i=0; i<nc+nP; i++)
00040         printf("%d\n", strP[i]);
00041     printf("%s\n", strP);
00042     return 0;

```

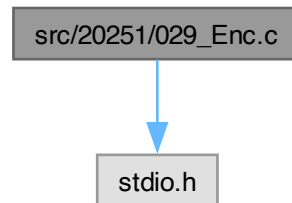


```
00043 }
```

7.56 src/20251/029_Enc.c File Reference

```
#include <stdio.h>
```

Include dependency graph for 029_Enc.c:



Macros

- `#define MC 2048`

Functions

- `int main (int argc, char *argv[])`

7.56.1 Macro Definition Documentation

7.56.1.1 MC

```
#define MC 2048
```

Definition at line 3 of file 029_Enc.c.

7.56.2 Function Documentation

7.56.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 5 of file 029_Enc.c.

```
00006 {
00007     char str[MC], strE[MC], strD[MC], aux[MC];
00008     int i, nc, a, ai, b, n;
00009     a = 17;
00010     // ai = 49;
00011     b = 21;
00012     n = 52;
00013     i=0;
00014     do{
00015         str[i++] = getc(stdin);
00016     }while(str[i-1] != 10);
00017     str[i-1] = '\0';
00018     nc = i-1;
00019     printf("MSG: %s\n", str);
00020     for(i=0; i<nc; i++)
00021         printf("%d ", str[i]);
00022     printf("\n");
00023     for(i=0; i<nc; i++)
00024     {
```

```

00025         aux[i] = str[i]-'A'-6*(str[i]>='a');
00026         printf("%d ", aux[i]);
00027     }
00028     printf("\n");
00029     for(i=0; i<nc; i++)
00030     {
00031         aux[i] = (a*((int)aux[i])+b)%n;
00032         printf("%d ", aux[i]);
00033     }
00034     printf("\n");
00035     for(i=0; i<nc; i++)
00036         strE[i] = aux[i]+'A'+6*(aux[i]>(n/2-1));
00037     strE[nc]='\0';
00038     printf("%s\n", strE);
00039     for(i=0; i<nc; i++)
00040     {
00041         aux[i] = strE[i]-'A'-6*(strE[i]>='a');
00042         printf("%d ", aux[i]);
00043     }
00044     printf("\n");
00045     ai=1;
00046     while(((ai*a)%n)!=1)
00047         ai++;
00048     printf("ai = %d\n", ai);
00049     for(i=0; i<nc; i++)
00050     {
00051         aux[i] = (ai*((aux[i]<b)*n+aux[i]-b))%n;
00052         printf("%d ", aux[i]);
00053     }
00054     printf("\n");
00055     for(i=0; i<nc; i++)
00056     {
00057         strD[i] = aux[i]+'A'+6*(aux[i]>25);
00058         printf("%d ", strD[i]);
00059     }
00060     strD[nc] = '\0';
00061     printf("\n");
00062     printf("%s\n", strD);
00063     return 0;
00064 }

```

7.57 029_Enc.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002
00003 #define MC 2048
00004
00005 int main(int argc, char *argv[])
00006 {
00007     char str[MC], strE[MC], strD[MC], aux[MC];
00008     int i, nc, a, ai, b, n;
00009     a = 17;
00010     // ai = 49;
00011     b = 21;
00012     n = 52;
00013     i=0;
00014     do{
00015         str[i++] = getc(stdin);
00016     }while(str[i-1]!=10);
00017     str[i-1] = '\0';
00018     nc = i-1;
00019     printf("MSG: %s\n", str);
00020     for(i=0; i<nc; i++)
00021         printf("%d ", str[i]);
00022     printf("\n");
00023     for(i=0; i<nc; i++)
00024     {
00025         aux[i] = str[i]-'A'-6*(str[i]>='a');
00026         printf("%d ", aux[i]);
00027     }
00028     printf("\n");
00029     for(i=0; i<nc; i++)
00030     {
00031         aux[i] = (a*((int)aux[i])+b)%n;
00032         printf("%d ", aux[i]);
00033     }
00034     printf("\n");
00035     for(i=0; i<nc; i++)
00036         strE[i] = aux[i]+'A'+6*(aux[i]>(n/2-1));
00037     strE[nc]='\0';
00038     printf("%s\n", strE);
00039     for(i=0; i<nc; i++)
00040     {

```

```

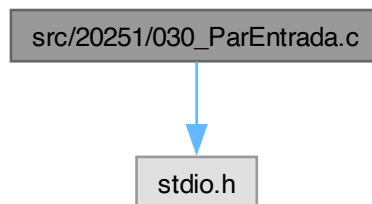
00041     aux[i] = strE[i]-'A'-6*(strE[i]>='a');
00042     printf("%d ", aux[i]);
00043 }
00044 printf("\n");
00045 ai=1;
00046 while((ai*a)%n!=1)
00047     ai++;
00048 printf("ai = %d\n", ai);
00049 for(i=0; i<nc; i++)
00050 {
00051     aux[i] = (ai*((aux[i]<b)*n+aux[i]-b))%n;
00052     printf("%d ", aux[i]);
00053 }
00054 printf("\n");
00055 for(i=0; i<nc; i++)
00056 {
00057     strD[i] = aux[i]+'A'+6*(aux[i]>25);
00058     printf("%d ", strD[i]);
00059 }
00060 strD[nc] = '\0';
00061 printf("\n");
00062 printf("%s\n", strD);
00063 return 0;
00064 }

```

7.58 src/20251/030_ParEntrada.c File Reference

#include <stdio.h>

Include dependency graph for 030_ParEntrada.c:



Functions

- int [main](#) (int argc, char *argv[])

7.58.1 Function Documentation

7.58.1.1 main()

```

int main (
    int argc,
    char * argv[])

```

Definition at line 3 of file [030_ParEntrada.c](#).

```

00004 {
00005     int i;
00006     printf("Numero de parametros: %d\n", argc);
00007     for(i=0; i<argc; i++)
00008         printf("%d. %s\n", i+1, argv[i]);
00009     return 0;
00010 }

```

7.59 030_ParEntrada.c

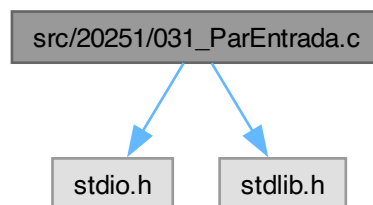
[Go to the documentation of this file.](#)

```
00001 #include<stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     int i;
00006     printf("Numero de parametros: %d\n", argc);
00007     for(i=0; i<argc; i++)
00008         printf("%d. %s\n", i+1, argv[i]);
00009     return 0;
00010 }
```

7.60 src/20251/031_ParEntrada.c File Reference

```
#include <stdio.h>
#include <stdlib.h>
```

Include dependency graph for 031_ParEntrada.c:



Functions

- int [main](#) (int argc, char *argv[])

7.60.1 Function Documentation

7.60.1.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 4 of file [031_ParEntrada.c](#).

```
00005 {
00006     int i, a, b, s;
00007     if(argc!=3)
00008     {
00009         printf("Uso: %s A B\n", argv[0]);
00010         return 1;
00011     }
00012     a = atoi(argv[1]);
00013     b = atoi(argv[2]);
00014     s = a + b;
00015     printf("%d+%d=%d\n", a, b, s);
00016     return 0;
00017 }
```

7.61 031_ParEntrada.c

[Go to the documentation of this file.](#)

```

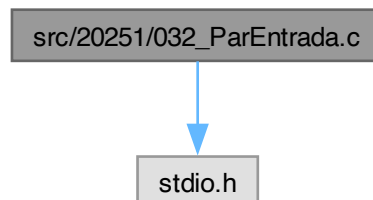
00001 #include<stdio.h>
00002 #include<stdlib.h>
00003
00004 int main(int argc, char *argv[])
00005 {
00006     int i, a, b, s;
00007     if(argc!=3)
00008     {
00009         printf("Uso: %s A B\n", argv[0]);
00010         return 1;
00011     }
00012     a = atoi(argv[1]);
00013     b = atoi(argv[2]);
00014     s = a + b;
00015     printf("%d+%d=%d\n", a, b, s);
00016     return 0;
00017 }

```

7.62 src/20251/032_ParEntrada.c File Reference

#include <stdio.h>

Include dependency graph for 032_ParEntrada.c:



Functions

- int [str2int](#) (char str[])
- float [str2float](#) (char str[])
- int [main](#) (int argc, char *argv[])

7.62.1 Function Documentation

7.62.1.1 main()

```

int main (
    int argc,
    char * argv[ ])

```

Definition at line 34 of file [032_ParEntrada.c](#).

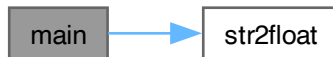
```

00035 {
00036     int i;
00037     float s, d;
00038     if(argc==1)
00039     {
00040         printf("Uso: %s A B\n", argv[0]);
00041         return 1;
00042     }
00043     for(i=1, s=0; i<argc; i++)
00044     {
00045         d = str2float(argv[i]);
00046         s += d;
00047         printf("%d-> %f\t%f\n", i, d, s);
00048     }
00049     printf("Suma = %f\n", s);
00050     return 0;

```

```
00051 }
```

Here is the call graph for this function:



7.62.1.2 str2float()

```
float str2float (
    char str[])
```

Definition at line 15 of file [032_ParEntrada.c](#).

```

00016 {
00017     float num = 0;
00018     int i = str[0]=='-'?1:0, p10=10;
00019     while(str[i]>='0' && str[i]<='9')
00020     {
00021         num *= 10;
00022         num += (str[i++] - '0');
00023     }
00024     if(str[i++] != '.')
00025         return str[0]=='-'? -num:num;
00026     while(str[i]>='0' && str[i]<='9')
00027     {
00028         num += (str[i++] - '0') / ((float)p10);
00029         p10 *= 10;
00030     }
00031     return str[0]=='-'? -num:num;
00032 }
```

Here is the caller graph for this function:



7.62.1.3 str2int()

```
int str2int (
    char str[])
```

Definition at line 3 of file [032_ParEntrada.c](#).

```

00004 {
00005     int num = 0;
00006     int i = str[0]=='-'?1:0;
00007     while(str[i]>='0' && str[i]<='9')
00008     {
00009         num *= 10;
00010         num += (str[i++] - '0');
00011     }
00012     return str[0]=='-'? -num:num;
00013 }
```

7.63 032_ParEntrada.c

[Go to the documentation of this file.](#)

```

00001 #include<stdio.h>
00002
00003 int str2int(char str[])
00004 {
00005     int num = 0;
00006     int i = str[0]=='-'?1:0;
00007     while(str[i]>='0' && str[i]<='9')
00008     {
00009         num *= 10;
00010         num += (str[i++]-'0');
00011     }
00012     return str[0]=='-'?-num:num;
00013 }
00014
00015 float str2float(char str[])
00016 {
00017     float num = 0;
00018     int i = str[0]=='-'?1:0, p10=10;
00019     while(str[i]>='0' && str[i]<='9')
00020     {
00021         num *= 10;
00022         num += (str[i++]-'0');
00023     }
00024     if(str[i++]!='.')
00025         return str[0]=='-'?-num:num;
00026     while(str[i]>='0' && str[i]<='9')
00027     {
00028         num += (str[i++]-'0')/((float)p10);
00029         p10 *= 10;
00030     }
00031     return str[0]=='-'?-num:num;
00032 }
00033
00034 int main(int argc, char *argv[])
00035 {
00036     int i;
00037     float s, d;
00038     if(argc==1)
00039     {
00040         printf("Uso: %s A B\n", argv[0]);
00041         return 1;
00042     }
00043     for(i=1, s=0; i<argc; i++)
00044     {
00045         d = str2float(argv[i]);
00046         s += d;
00047         printf("%d-> %f\t%f\n", i, d, s);
00048     }
00049     printf("Suma = %f\n", s);
00050     return 0;
00051 }

```

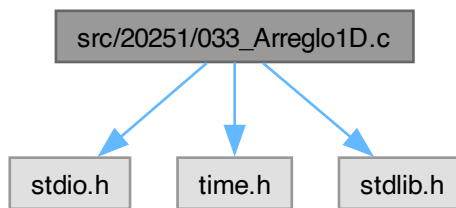
7.64 src/20251/033_Arreglo1D.c File Reference

```

#include <stdio.h>
#include <time.h>
#include <stdlib.h>

```

Include dependency graph for 033_Arreglo1D.c:



Macros

- `#define N 60`
- `#define min 1.5`
- `#define max 2.1`

Functions

- `int main (int argc, char *argv[])`

7.64.1 Macro Definition Documentation

7.64.1.1 max

```
#define max 2.1
```

Definition at line 7 of file [033_Arreglo1D.c](#).

7.64.1.2 min

```
#define min 1.5
```

Definition at line 6 of file [033_Arreglo1D.c](#).

7.64.1.3 N

```
#define N 60
```

Definition at line 5 of file [033_Arreglo1D.c](#).

7.64.2 Function Documentation

7.64.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 9 of file [033_Arreglo1D.c](#).

```
00010 {
00011     float X[N];
00012     float med, var;
00013     int i, n;
00014     srand(time(NULL));
00015     do{
00016         printf("Ingrese el numero de estudiantes: ");
00017         scanf("%d", &n);
00018     }while (n<1 || n>N);
00019     for(i=0, med=0, var=0; i<n; i++)
```



```

00020      {
00021          X[i] = (max-min)*rand()/RAND_MAX+min;
00022          med += X[i];
00023          var += (X[i]*X[i]);
00024          printf("X[%d] = %.3f\n", i+1, X[i]);
00025      }
00026      med /= n;
00027      var /= n;
00028      var -= (med*med);
00029      printf("Media = %.3f\n", med);
00030      printf("Varianza = %.3f\n", var);
00031      return 0;
00032 }

```

7.65 033_Arreglo1D.c

[Go to the documentation of this file.](#)

```

00001 #include<stdio.h>
00002 #include<time.h>
00003 #include<stdlib.h>
00004
00005 #define N 60
00006 #define min 1.5
00007 #define max 2.1
00008
00009 int main(int argc, char *argv[])
00010 {
00011     float X[N];
00012     float med, var;
00013     int i, n;
00014     srand(time(NULL));
00015     do{
00016         printf("Ingrese el numero de estudiantes: ");
00017         scanf("%d", &n);
00018     }while(n<1||n>N);
00019     for(i=0, med=0, var=0; i<n; i++)
00020     {
00021         X[i] = (max-min)*rand()/RAND_MAX+min;
00022         med += X[i];
00023         var += (X[i]*X[i]);
00024         printf("X[%d] = %.3f\n", i+1, X[i]);
00025     }
00026     med /= n;
00027     var /= n;
00028     var -= (med*med);
00029     printf("Media = %.3f\n", med);
00030     printf("Varianza = %.3f\n", var);
00031     return 0;
00032 }

```

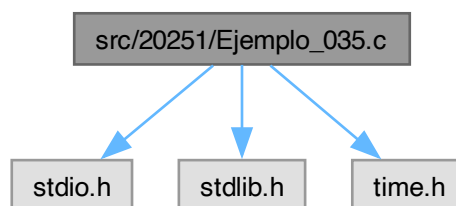
7.66 src/20251/Ejemplo_035.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo_035.c:



Functions

- float [sumar](#) (float *X, int n)
- float [media](#) (float *X, int n)
- int [main](#) (void)

7.66.1 Function Documentation

7.66.1.1 main()

```
int main (
    void )
```

Definition at line 15 of file [Ejemplo_035.c](#).

```
00016 {
00017     int n, m, i;
00018     float *A;
00019     srand(time(NULL));
00020     do{
00021         printf("Ingrese el numero de elementos: ");
00022         scanf("%d", &n);
00023     }while(n<1);
00024     do{
00025         printf("Ingrese el numero de muestras: ");
00026         scanf("%d", &m);
00027     }while(m<1||m>n);
00028     A = (float*)malloc(n*sizeof(float));
00029     if(A==NULL)
00030         return 1;
00031     for(i=0; i<n; i++)
00032     {
00033         A[i] = (1.0*rand())/RAND_MAX;
00034         printf("A[%d] = %f\t%p\n", i+1, A[i], A+i);
00035     }
00036     for(i=0; i<n-m; i++)
00037         printf("u(%d) = %f\n", i+1, media(A+i, m));
00038     free(A);
00039     return 0;
00040 }
```

Here is the call graph for this function:



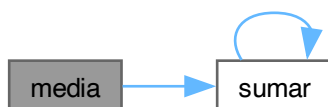
7.66.1.2 media()

```
float media (
    float * X,
    int n)
```

Definition at line 10 of file [Ejemplo_035.c](#).

```
00011 {
00012     return sumar(X, n)/n;
00013 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



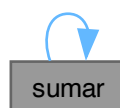
7.66.1.3 sumar()

```
float sumar (  
    float * X,  
    int n)
```

Definition at line 5 of file [Ejemplo_035.c](#).

```
00006 {  
00007     return n?X[0]+sumar(X+1,n-1):0;  
00008 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



7.67 Ejemplo_035.c

[Go to the documentation of this file.](#)

```

00001 #include<stdio.h>
00002 #include<stdlib.h>
00003 #include<time.h>
00004
00005 float sumar(float *X, int n)
00006 {
00007     return n?X[0]+sumar(X+1,n-1):0;
00008 }
00009
00010 float media(float *X, int n)
00011 {
00012     return sumar(X, n)/n;
00013 }
00014
00015 int main(void)
00016 {
00017     int n, m, i;
00018     float *A;
00019     srand(time(NULL));
00020     do{
00021         printf("Ingrese el numero de elementos: ");
00022         scanf("%d", &n);
00023     }while(n<1);
00024     do{
00025         printf("Ingrese el numero de muestras: ");
00026         scanf("%d", &m);
00027     }while(m<1||m>n);
00028     A = (float*)malloc(n*sizeof(float));
00029     if(A==NULL)
00030         return 1;
00031     for(i=0; i<n; i++)
00032     {
00033         A[i] = (1.0*rand())/RAND_MAX;
00034         printf("A[%d] = %f\t%p\n", i+1, A[i], A+i);
00035     }
00036     for(i=0; i<n-m; i++)
00037         printf("u(%d) = %f\n", i+1, media(A+i, m));
00038     free(A);
00039     return 0;
00040 }

```

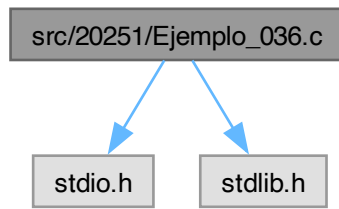
7.68 src/20251/Ejemplo_036.c File Reference

```

#include <stdio.h>
#include <stdlib.h>

```

Include dependency graph for Ejemplo_036.c:



Functions

- int [main](#) (void)

7.68.1 Function Documentation

7.68.1.1 main()

```
int main (
    void )
```

Definition at line 4 of file [Ejemplo_036.c](#).

```

00005 {
00006     float **A, **B, **C, *pA, *pC;
00007     int nA, mA, nB, mB, i, j, k;
00008     do{
00009         printf("Ingrese el numero de columnas de A: ");
00010         scanf("%d", &nA);
00011         printf("Ingrese el numro de renglones de A: ");
00012         scanf("%d", &mA);
00013     }while(nA<1||mA<1);
00014     do{
00015         printf("Ingrese el numero de columnas de B: ");
00016         scanf("%d", &nB);
00017         printf("Ingrese el numro de renglones de B: ");
00018         scanf("%d", &mB);
00019     }while(nB<1||mB<1||nB!=mA);
00020     pA = (float*)malloc(nA*mA*sizeof(float));
00021     if(pA==NULL)
00022         return 1;
00023     A = (float**)malloc(nA*sizeof(float*));
00024     if(A==NULL)
00025     {
00026         free(pA);
00027         return 2;
00028     }
00029     for(i=0; i<nA; i++)
00030         A[i] = pA+i*mA;
00031     B = (float**)malloc(nB*sizeof(float*));
00032     if(B==NULL)
00033     {
00034         free(pA);
00035         free(A);
00036         return 3;
00037     }
00038     for(i=0; i<nB; i++)
00039     {
00040         B[i] = (float*)malloc(mB*sizeof(float));
00041         if(B[i]== NULL)
00042         {
00043             free(pA);
00044             free(A);
00045             for(--i; i>-1; i--)
00046                 free(B[i]);
00047             free(B);
00048             return 4;
00049         }

```

```

00050     }
00051     free(B);
00052     free(pA);
00053     free(A);
00054     return 0;
00055 }

```

7.69 Ejemplo_036.c

[Go to the documentation of this file.](#)

```

00001 #include<stdio.h>
00002 #include<stdlib.h>
00003
00004 int main(void)
00005 {
00006     float **A, **B, **C, *pA, *pC;
00007     int nA, mA, nB, mB, i, j, k;
00008     do{
00009         printf("Ingrese el numero de columnas de A: ");
00010         scanf("%d", &nA);
00011         printf("Ingrese el numro de renglones de A: ");
00012         scanf("%d", &mA);
00013     }while(nA<1||mA<1);
00014     do{
00015         printf("Ingrese el numero de columnas de B: ");
00016         scanf("%d", &nB);
00017         printf("Ingrese el numro de renglones de B: ");
00018         scanf("%d", &mB);
00019     }while(nB<1||mB<1||nB!=mA);
00020     pA = (float*)malloc(nA*mA*sizeof(float));
00021     if(pA==NULL)
00022         return 1;
00023     A = (float**)malloc(nA*sizeof(float*));
00024     if(A==NULL)
00025     {
00026         free(pA);
00027         return 2;
00028     }
00029     for(i=0; i<nA; i++)
00030         A[i] = pA+i*mA;
00031     B = (float**)malloc(nB*sizeof(float*));
00032     if(B==NULL)
00033     {
00034         free(pA);
00035         free(A);
00036         return 3;
00037     }
00038     for(i=0; i<nB; i++)
00039     {
00040         B[i] = (float*)malloc(mB*sizeof(float));
00041         if(B[i]== NULL)
00042         {
00043             free(pA);
00044             free(A);
00045             for(--i; i>-1; i--)
00046                 free(B[i]);
00047             free(B);
00048             return 4;
00049         }
00050     }
00051     free(B);
00052     free(pA);
00053     free(A);
00054     return 0;
00055 }

```

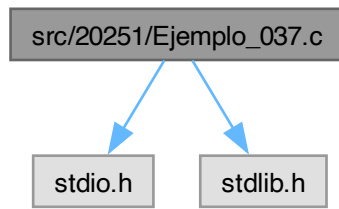
7.70 src/20251/Ejemplo_037.c File Reference

```

#include <stdio.h>
#include <stdlib.h>

```

Include dependency graph for Ejemplo_037.c:



Functions

- int [main](#) (void)

7.70.1 Function Documentation

7.70.1.1 main()

```
int main (
    void )
```

Definition at line 13 of file [Ejemplo_037.c](#).

```

00014 {
00015     float **A, **B, **C, *pA, *pC;
00016     int nA, mA, nB, mB, i, j, k;
00017     FILE *fp;
00018     do{
00019         printf("Ingrese el numero de columnas de A: ");
00020         scanf("%d", &nA);
00021         printf("Ingrese el numro de renglones de A: ");
00022         scanf("%d", &mA);
00023     }while(nA<1||mA<1);
00024     do{
00025         printf("Ingrese el numero de columnas de B: ");
00026         scanf("%d", &nB);
00027         printf("Ingrese el numro de renglones de B: ");
00028         scanf("%d", &mB);
00029     }while(nB<1||mB<1||nB!=mA);
00030     pA = (float*)malloc(nA*mA*sizeof(float));
00031     if(pA==NULL)
00032         return 1;
00033     A = (float**)malloc(nA*sizeof(float*));
00034     if(A==NULL)
00035     {
00036         free(pA);
00037         return 2;
00038     }
00039     for(i=0; i<nA; i++)
00040         A[i] = pA+i*mA;
00041     B = (float**)malloc(nB*sizeof(float*));
00042     if(B==NULL)
00043     {
00044         free(pA);
00045         free(A);
00046         return 3;
00047     }
00048     for(i=0; i<nB; i++)
00049     {
00050         B[i] = (float*)malloc(mB*sizeof(float));
00051         if(B[i]== NULL)
00052         {
00053             free(pA);
00054             free(A);
00055             for(--i; i>-1; i--)
00056                 free(B[i]);
00057             free(B);
00058             return 4;

```

```

00059     }
00060 }
00061 pC = (float*)malloc(nA*mB*sizeof(float));
00062 if(pC==NULL)
00063 {
00064     free(pA);
00065     free(A);
00066     for(i=0; i<nB; i++)
00067         free(B[i]);
00068     free(B);
00069     return 5;
00070 }
00071 C = (float**)malloc(nA*sizeof(float*));
00072 if(C==NULL)
00073 {
00074     free(pA);
00075     free(A);
00076     for(i=0; i<nB; i++)
00077         free(B[i]);
00078     free(B);
00079     free(pC);
00080     return 6;
00081 }
00082 for(i=0; i<nA; i++)
00083     C[i] = pC+i*mB;
00084 for(i=0; i<nA; i++)
00085     for(j=0; j<mA; j++)
00086     {
00087         printf("A[%d][%d] = ", i+1, j+1);
00088         scanf("%f", A[i]+j);
00089     }
00090 for(i=0; i<nB; i++)
00091     for(j=0; j<mB; j++)
00092     {
00093         printf("B[%d][%d] = ", i+1, j+1);
00094         scanf("%f", B[i]+j);
00095     }
00096 for(i=0; i<nA; i++)
00097     for(j=0; j<mB; j++)
00098         for(k=0; C[i][j]=0; k<nB; k++)
00099             C[i][j] += A[i][k]*B[k][j];
00100 for(i=0; i<nA; i++)
00101     for(j=0; j<mB; j++)
00102         printf("C[%d][%d] = %f\n", i+1, j+1, C[i][j]);
00103 fp = fopen("Matriz.txt", "w");
00104 if(fp==NULL)
00105 {
00106     free(pC);
00107     free(C);
00108     for(i=0; i<nB; i++)
00109         free(B[i]);
00110     free(B);
00111     free(pA);
00112     free(A);
00113     return 7;
00114 }
00115 for(i=0; i<nA; i++)
00116     for(j=0; j<mB; j++)
00117         fprintf(fp, "%f ", C[i][j]);
00118 fclose(fp);
00119 free(pC);
00120 free(C);
00121 for(i=0; i<nB; i++)
00122     free(B[i]);
00123 free(B);
00124 free(pA);
00125 free(A);
00126 return 0;
00127 }

```

7.71 Ejemplo_037.c

[Go to the documentation of this file.](#)

```

00001 #include<stdio.h>
00002 #include<stdlib.h>
00003
00004 /* Multiplicacion de matrices */
00005 /* A: mA x nA */
00006 /* B: mB x nB */
00007 /* C: mA x nB */
00008 /* A matriz ordenada */
00009 /* B matriz dispersa */
00010 /* C matriz ordenada */
00011 /* fp archivo de salida de C */

```



```

00012
00013 int main(void)
00014 {
00015     float **A, **B, **C, *pA, *pC;
00016     int nA, mA, nB, mB, i, j, k;
00017     FILE *fp;
00018     do{
00019         printf("Ingrese el numero de columnas de A: ");
00020         scanf("%d", &nA);
00021         printf("Ingrese el numro de renglones de A: ");
00022         scanf("%d", &mA);
00023     }while(nA<1||mA<1);
00024     do{
00025         printf("Ingrese el numero de columnas de B: ");
00026         scanf("%d", &nB);
00027         printf("Ingrese el numro de renglones de B: ");
00028         scanf("%d", &mB);
00029     }while(nB<1||mB<1||nB!=mA);
00030     pA = (float*)malloc(nA*mA*sizeof(float));
00031     if(pA==NULL)
00032         return 1;
00033     A = (float**)malloc(nA*sizeof(float*));
00034     if(A==NULL)
00035     {
00036         free(pA);
00037         return 2;
00038     }
00039     for(i=0; i<nA; i++)
00040         A[i] = pA+i*mA;
00041     B = (float**)malloc(nB*sizeof(float*));
00042     if(B==NULL)
00043     {
00044         free(pA);
00045         free(A);
00046         return 3;
00047     }
00048     for(i=0; i<nB; i++)
00049     {
00050         B[i] = (float*)malloc(mB*sizeof(float));
00051         if(B[i]== NULL)
00052         {
00053             free(pA);
00054             free(A);
00055             for(--i; i>-1; i--)
00056                 free(B[i]);
00057             free(B);
00058             return 4;
00059         }
00060     }
00061     pC = (float*)malloc(nA*mB*sizeof(float));
00062     if(pC==NULL)
00063     {
00064         free(pA);
00065         free(A);
00066         for(i=0; i<nB; i++)
00067             free(B[i]);
00068         free(B);
00069         return 5;
00070     }
00071     C = (float**)malloc(nA*sizeof(float*));
00072     if(C==NULL)
00073     {
00074         free(pA);
00075         free(A);
00076         for(i=0; i<nB; i++)
00077             free(B[i]);
00078         free(B);
00079         free(pC);
00080         return 6;
00081     }
00082     for(i=0; i<nA; i++)
00083         C[i] = pC+i*mB;
00084     for(i=0; i<nA; i++)
00085         for(j=0; j<mA; j++)
00086         {
00087             printf("A[%d][%d] = ", i+1, j+1);
00088             scanf("%f", A[i]+j);
00089         }
00090     for(i=0; i<nB; i++)
00091         for(j=0; j<mB; j++)
00092         {
00093             printf("B[%d][%d] = ", i+1, j+1);
00094             scanf("%f", B[i]+j);
00095         }
00096     for(i=0; i<nA; i++)
00097         for(j=0; j<mB; j++)
00098             for(k=0; C[i][j]=0; k<nB; k++)

```

```

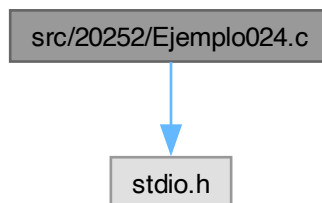
00099     C[i][j] += A[i][k]*B[k][j];
00100     for(i=0; i<nA; i++)
00101         for(j=0; j<mB; j++)
00102             printf("C[%d][%d] = %f\n", i+1, j+1, C[i][j]);
00103     fp = fopen("Matriz.txt", "w");
00104     if(fp==NULL)
00105     {
00106         free(pC);
00107         free(C);
00108         for(i=0; i<nB; i++)
00109             free(B[i]);
00110         free(B);
00111         free(pA);
00112         free(A);
00113         return 7;
00114     }
00115     for(i=0; i<nA; i++)
00116         for(j=0; j<mB; j++)
00117             fprintf(fp, "%f ", C[i][j]);
00118     fclose(fp);
00119     free(pC);
00120     free(C);
00121     for(i=0; i<nB; i++)
00122         free(B[i]);
00123     free(B);
00124     free(pA);
00125     free(A);
00126     return 0;
00127 }

```

7.72 src/20252/Ejemplo024.c File Reference

#include <stdio.h>

Include dependency graph for Ejemplo024.c:



Macros

- #define [BIT](#)(n)
- #define [BIT_GET](#)(x, n)
- #define [BIT_SET](#)(x, n)
- #define [BIT_CLEAR](#)(x, n)
- #define [BIT_TOGGLE](#)(x, n)
- #define [BIT_WRITE](#)(x, n, v)
- #define [ES_PAR](#)(x)

Functions

- int [main](#) (int argc, char *argv[])

7.72.1 Macro Definition Documentation

7.72.1.1 BIT

```
#define BIT(  
        n)
```

Value:

`(1<<(n))`

Definition at line 3 of file [Ejemplo024.c](#).

7.72.1.2 BIT_CLEAR

```
#define BIT_CLEAR(  
        x,  
        n)
```

Value:

`((x) &= ~BIT(n))`

Definition at line 6 of file [Ejemplo024.c](#).

7.72.1.3 BIT_GET

```
#define BIT_GET(  
        x,  
        n)
```

Value:

`((x) & BIT(n))`

Definition at line 4 of file [Ejemplo024.c](#).

7.72.1.4 BIT_SET

```
#define BIT_SET(  
        x,  
        n)
```

Value:

`((x) |= BIT(n))`

Definition at line 5 of file [Ejemplo024.c](#).

7.72.1.5 BIT_TOGGLE

```
#define BIT_TOGGLE(  
        x,  
        n)
```

Value:

`((x) ^= BIT(n))`

Definition at line 7 of file [Ejemplo024.c](#).

7.72.1.6 BIT_WRITE

```
#define BIT_WRITE(  
        x,  
        n,  
        v)
```

Value:

`((v) ? BIT_SET(x,n) : BIT_CLEAR(x,n))`

Definition at line 8 of file [Ejemplo024.c](#).

7.72.1.7 ES_PAR

```
#define ES_PAR(  
        x)
```

Value:

```
(!BIT_GET(x,0))
```

Definition at line 9 of file [Ejemplo024.c](#).

7.72.2 Function Documentation**7.72.2.1 main()**

```
int main (
    int argc,
    char * argv[])
```

Definition at line 11 of file [Ejemplo024.c](#).

```
00012 {
00013     int x, nb, i;
00014     printf("x = ");
00015     scanf("%d", &x);
00016     printf("b = ");
00017     scanf("%d", &nb);
00018     for(i=7; i>-1; i--)
00019         printf("%d", BIT_GET(x,i)?1:0);
00020     printf("\n");
00021     BIT_SET(x, nb);
00022     printf("%d\n", x);
00023     return 0;
00024 }
```

7.73 Ejemplo024.c

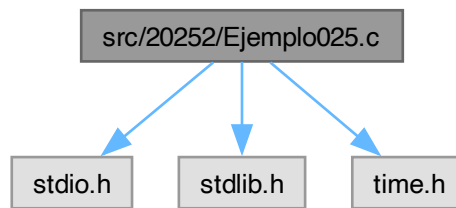
[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002
00003 #define BIT(n)                (1<<(n))
00004 #define BIT_GET(x,n)          ((x) & BIT(n))
00005 #define BIT_SET(x,n)          ((x) |= BIT(n))
00006 #define BIT_CLEAR(x,n)        ((x) &= ~BIT(n))
00007 #define BIT_TOGGLE(x,n)       ((x) ^= BIT(n))
00008 #define BIT_WRITE(x,n,v)      ((v)?BIT_SET(x,n):BIT_CLEAR(x,n))
00009 #define ES_PAR(x)              (!BIT_GET(x,0))
00010
00011 int main(int argc, char *argv[])
00012 {
00013     int x, nb, i;
00014     printf("x = ");
00015     scanf("%d", &x);
00016     printf("b = ");
00017     scanf("%d", &nb);
00018     for(i=7; i>-1; i--)
00019         printf("%d", BIT_GET(x,i)?1:0);
00020     printf("\n");
00021     BIT_SET(x, nb);
00022     printf("%d\n", x);
00023     return 0;
00024 }
```

7.74 src/20252/Ejemplo025.c File Reference

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

Include dependency graph for Ejemplo025.c:



Macros

- `#define BIT(n)`
- `#define BIT_GET(x, n)`
- `#define BIT_SET(x, n)`
- `#define BIT_CLEAR(x, n)`
- `#define BIT_TOGGLE(x, n)`
- `#define BIT_WRITE(x, n, v)`
- `#define ES_PAR(x)`
- `#define N 50`

Functions

- `int main (int argc, char *argv[])`

7.74.1 Macro Definition Documentation

7.74.1.1 BIT

```
#define BIT(
    n)
```

Value:

```
((1 << (n))
```

Definition at line 5 of file [Ejemplo025.c](#).

7.74.1.2 BIT_CLEAR

```
#define BIT_CLEAR(
    x,
    n)
```

Value:

```
((x) &= ~BIT(n))
```

Definition at line 8 of file [Ejemplo025.c](#).

7.74.1.3 BIT_GET

```
#define BIT_GET(
    x,
    n)
```

Value:

```
((x) & BIT(n))
```

Definition at line 6 of file [Ejemplo025.c](#).

7.74.1.4 BIT_SET

```
#define BIT_SET(
    x,
    n)
```

Value:

```
((x) |= BIT(n))
```

Definition at line 7 of file [Ejemplo025.c](#).

7.74.1.5 BIT_TOGGLE

```
#define BIT_TOGGLE(
    x,
    n)
```

Value:

```
((x) ^= BIT(n))
```

Definition at line 9 of file [Ejemplo025.c](#).

7.74.1.6 BIT_WRITE

```
#define BIT_WRITE(
    x,
    n,
    v)
```

Value:

```
((v) ? BIT_SET(x,n) : BIT_CLEAR(x,n))
```

Definition at line 10 of file [Ejemplo025.c](#).

7.74.1.7 ES_PAR

```
#define ES_PAR(
    x)
```

Value:

```
(!BIT_GET(x,0))
```

Definition at line 11 of file [Ejemplo025.c](#).

7.74.1.8 N

```
#define N 50
```

Definition at line 12 of file [Ejemplo025.c](#).

7.74.2 Function Documentation

7.74.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 14 of file [Ejemplo025.c](#).

```
00015 {
00016     char str[N], c, p, v;
00017     int i, nc, cc, cb;
00018     srand(time(NULL));
00019     i = 0;
00020     do{
00021         c = getchar();
00022         str[i] = c;
00023         i++;
00024     }while(c!=10&i<(N-1));
00025     nc = i-1;
00026     str[nc] = '\0';
00027     p = 0;
00028     i = 0;
00029     while(str[i]!='\0')
00030     {
```

```

00031         p^=str[i];
00032         i++;
00033     }
00034     printf("%s (%d)\n", str, p);
00035     cc = (rand()%nc);
00036     cb = (rand()%8);
00037     BIT_TOGGLE(str[cc],cb);
00038     v = p;
00039     i = 0;
00040     while(str[i]!='\0')
00041     {
00042         v^=str[i];
00043         i++;
00044     }
00045     printf("%s (%d, %d, %d)\n", str, v, cc, cb);
00046     return 0;
00047 }

```

7.75 Ejemplo025.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <time.h>
00004
00005 #define BIT(n)                (1<<(n))
00006 #define BIT_GET(x,n)          ((x) & BIT(n))
00007 #define BIT_SET(x,n)          ((x) |= BIT(n))
00008 #define BIT_CLEAR(x,n)        ((x) &= ~BIT(n))
00009 #define BIT_TOGGLE(x,n)       ((x) ^= BIT(n))
00010 #define BIT_WRITE(x,n,v)      ((v)?BIT_SET(x,n):BIT_CLEAR(x,n))
00011 #define ES_PAR(x)              (!BIT_GET(x,0))
00012 #define N 50
00013
00014 int main(int argc, char *argv[])
00015 {
00016     char str[N], c, p, v;
00017     int i, nc, cc, cb;
00018     srand(time(NULL));
00019     i = 0;
00020     do{
00021         c = getchar();
00022         str[i] = c;
00023         i++;
00024     }while(c!=10&&i<(N-1));
00025     nc = i-1;
00026     str[nc] = '\0';
00027     p = 0;
00028     i = 0;
00029     while(str[i]!='\0')
00030     {
00031         p^=str[i];
00032         i++;
00033     }
00034     printf("%s (%d)\n", str, p);
00035     cc = (rand()%nc);
00036     cb = (rand()%8);
00037     BIT_TOGGLE(str[cc],cb);
00038     v = p;
00039     i = 0;
00040     while(str[i]!='\0')
00041     {
00042         v^=str[i];
00043         i++;
00044     }
00045     printf("%s (%d, %d, %d)\n", str, v, cc, cb);
00046     return 0;
00047 }

```

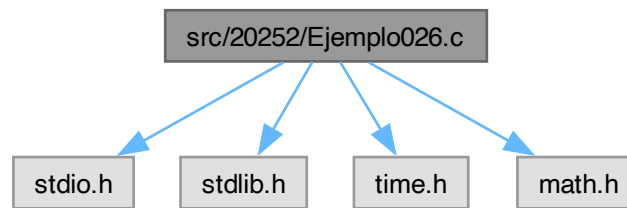
7.76 src/20252/Ejemplo026.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>

```

Include dependency graph for Ejemplo026.c:



Macros

- `#define BIT(n)`
- `#define BIT_GET(x, n)`
- `#define BIT_SET(x, n)`
- `#define BIT_CLEAR(x, n)`
- `#define BIT_TOGGLE(x, n)`
- `#define BIT_WRITE(x, n, v)`
- `#define ES_PAR(x)`
- `#define N 50`

Functions

- `int main (int argc, char *argv[])`

7.76.1 Macro Definition Documentation

7.76.1.1 BIT

```
#define BIT(
    n)
```

Value:

```
((1<<(n))
```

Definition at line 6 of file [Ejemplo026.c](#).

7.76.1.2 BIT_CLEAR

```
#define BIT_CLEAR(
    x,
    n)
```

Value:

```
((x) &= ~BIT(n))
```

Definition at line 9 of file [Ejemplo026.c](#).

7.76.1.3 BIT_GET

```
#define BIT_GET(
    x,
    n)
```

Value:

```
((x) & BIT(n))
```

Definition at line 7 of file [Ejemplo026.c](#).

7.76.1.4 BIT_SET

```
#define BIT_SET(
    x,
    n)
```

Value:

```
((x) |= BIT(n))
```

Definition at line 8 of file [Ejemplo026.c](#).

7.76.1.5 BIT_TOGGLE

```
#define BIT_TOGGLE(
    x,
    n)
```

Value:

```
((x) ^= BIT(n))
```

Definition at line 10 of file [Ejemplo026.c](#).

7.76.1.6 BIT_WRITE

```
#define BIT_WRITE(
    x,
    n,
    v)
```

Value:

```
((v) ? BIT_SET(x,n) : BIT_CLEAR(x,n))
```

Definition at line 11 of file [Ejemplo026.c](#).

7.76.1.7 ES_PAR

```
#define ES_PAR(
    x)
```

Value:

```
(!BIT_GET(x,0))
```

Definition at line 12 of file [Ejemplo026.c](#).

7.76.1.8 N

```
#define N 50
```

Definition at line 13 of file [Ejemplo026.c](#).

7.76.2 Function Documentation

7.76.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 15 of file [Ejemplo026.c](#).

```
00016 {
00017     char str[N], strP[N], strR[N], c, p, v;
00018     int i, j, k, l, nc, cc, cb, np, p2, p2f, cc2, cb2;
00019     srand(time(NULL));
00020     i = 0;
00021     do{
00022         c = getchar();
00023         str[i] = c;
00024         i++;
00025     }while(c!=10&& i<(N-1));
00026     nc = i-1;
00027     str[nc] = '\0';
00028     np = (int)(log2(nc+log2(nc)+1)+1);
00029     printf("NP = %d\n", np);
00030     printf("%s (%d, %d)\n", str, nc, np);
00031     for(i=0, j=0, p2=1; i<(nc+np); i++)
```

```

00032     {
00033         if((i+1)==p2)
00034         {
00035             strP[i] = 0;
00036             p2<<=1;
00037         }
00038         else
00039         {
00040             c = str[j++];
00041             strP[i] = c;
00042             for(k=0, p2f=1; k<np; k++)
00043             {
00044                 if(p2f&(i+1))
00045                     strP[p2f-1]^=c;
00046                 p2f<<=1;
00047             }
00048         }
00049     }
00050     strP[nc+np] = '\0';
00051     cc = rand()%(nc+np);
00052     cb = rand()%8;
00053     BIT_TOGGLE(strP[cc],cb);
00054     printf("%s\n", strP);
00055     for(i=0, j=0, l=0, p2=1; i<(nc+np); i++)
00056     {
00057         if((i+1)==p2)
00058         {
00059             strR[nc+1+l] = strP[i];
00060             l++;
00061             p2 <= 1;
00062         }
00063         else
00064         {
00065             c = strP[i];
00066             strR[j++] = c;
00067             for(k=0, p2f=1; k<np; k++)
00068             {
00069                 if(p2f&(i+1))
00070                     strR[nc+1+k]^=c;
00071                 p2f<<=1;
00072             }
00073         }
00074     }
00075     strR[nc] = '\0';
00076     strR[nc+np+1] = '\0';
00077     printf("%s (%d, %d)\n", strR, cc, cb);
00078     for(i=0, cc2=0; i<np; i++)
00079     {
00080         printf("%d\t", strR[nc+1+i]);
00081         BIT_WRITE(cc2,i,strR[nc+1+i]);
00082         if(strR[nc+1+i])
00083             cb2 = log2(strR[nc+1+i]);
00084     }
00085     printf("(%d, %d)\n", cc2-1, cb2);
00086     BIT_TOGGLE(strP[cc2-1],cb2);
00087     printf("%s\n", strP);
00088     return 0;
00089 }

```

7.77 Ejemplo026.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <time.h>
00004 #include <math.h>
00005
00006 #define BIT(n)                (1<<(n))
00007 #define BIT_GET(x,n)          ((x) & BIT(n))
00008 #define BIT_SET(x,n)          ((x) |= BIT(n))
00009 #define BIT_CLEAR(x,n)        ((x) &= ~BIT(n))
00010 #define BIT_TOGGLE(x,n)       ((x) ^= BIT(n))
00011 #define BIT_WRITE(x,n,v)      ((v)?BIT_SET(x,n):BIT_CLEAR(x,n))
00012 #define ES_PAR(x)              (!BIT_GET(x,0))
00013 #define N                      50
00014
00015 int main(int argc, char *argv[])
00016 {
00017     char str[N], strP[N], strR[N], c, p, v;
00018     int i, j, k, l, nc, cc, cb, np, p2, p2f, cc2, cb2;
00019     srand(time(NULL));
00020     i = 0;
00021     do{
00022         c = getchar();

```

```

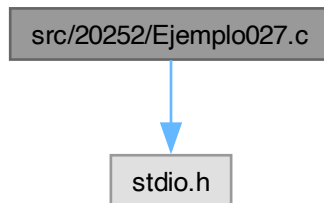
00023     str[i] = c;
00024     i++;
00025 }while(c!=10&& i<(N-1));
00026 nc = i-1;
00027 str[nc] = '\0';
00028 np = (int)(log2(nc+log2(nc)+1)+1);
00029 printf("NP = %d\n", np);
00030 printf("%s (%d, %d)\n", str, nc, np);
00031 for(i=0, j=0, p2=1; i<(nc+np); i++)
00032 {
00033     if((i+1)==p2)
00034     {
00035         strP[i] = 0;
00036         p2<<=1;
00037     }
00038     else
00039     {
00040         c = str[j++];
00041         strP[i] = c;
00042         for(k=0, p2f=1; k<np; k++)
00043         {
00044             if(p2f&(i+1))
00045                 strP[p2f-1]^=c;
00046             p2f<<=1;
00047         }
00048     }
00049 }
00050 strP[nc+np] = '\0';
00051 cc = rand()%(nc+np);
00052 cb = rand()%8;
00053 BIT_TOGGLE(strP[cc],cb);
00054 printf("%s\n", strP);
00055 for(i=0, j=0, l=0, p2=1; i<(nc+np); i++)
00056 {
00057     if((i+1)==p2)
00058     {
00059         strR[nc+1+l] = strP[i];
00060         l++;
00061         p2 <= 1;
00062     }
00063     else
00064     {
00065         c = strP[i];
00066         strR[j++] = c;
00067         for(k=0, p2f=1; k<np; k++)
00068         {
00069             if(p2f&(i+1))
00070                 strR[nc+1+k]^=c;
00071             p2f<<=1;
00072         }
00073     }
00074 }
00075 strR[nc] = '\0';
00076 strR[nc+np+1] = '\0';
00077 printf("%s (%d, %d)\n", strR, cc, cb);
00078 for(i=0, cc2=0; i<np; i++)
00079 {
00080     printf("%d\t", strR[nc+1+i]);
00081     BIT_WRITE(cc2,i,strR[nc+1+i]);
00082     if(strR[nc+1+i])
00083         cb2 = log2(strR[nc+1+i]);
00084 }
00085 printf("(%d, %d)\n", cc2-1, cb2);
00086 BIT_TOGGLE(strP[cc2-1],cb2);
00087 printf("%s\n", strP);
00088 return 0;
00089 }

```

7.78 src/20252/Ejemplo027.c File Reference

#include <stdio.h>

Include dependency graph for Ejemplo027.c:



Functions

- int [main](#) (int argc, char *argv[])

7.78.1 Function Documentation

7.78.1.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 3 of file [Ejemplo027.c](#).

```
00004 {
00005     int n, a, ai, b, m, c, mr, i;
00006     printf("n = ");
00007     scanf("%d", &n);
00008     printf("a = ");
00009     scanf("%d", &a);
00010     printf("b = ");
00011     scanf("%d", &b);
00012     for(i=0; i<n; i++)
00013         if((a*i)%n==1)
00014         {
00015             ai = i;
00016             printf("ai = %d\n", ai);
00017         }
00018     printf("m = ");
00019     scanf("%d", &m);
00020     c = (a*m+b)%n;
00021     printf("c = %d\n", c);
00022     if(c>=b)
00023         mr = c-b;
00024     else
00025         mr = n+c-b;
00026     mr = (ai*mr)%n;
00027     printf("m = %d\n", mr);
00028     return 0;
00029 }
```

7.79 Ejemplo027.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     int n, a, ai, b, m, c, mr, i;
```

```

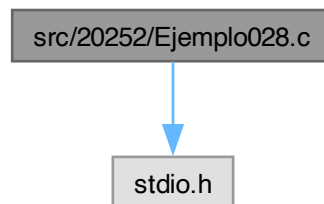
00006     printf("n = ");
00007     scanf("%d", &n);
00008     printf("a = ");
00009     scanf("%d", &a);
00010     printf("b = ");
00011     scanf("%d", &b);
00012     for(i=0; i<n; i++)
00013         if((a*i)%n==1)
00014         {
00015             ai = i;
00016             printf("ai = %d\n", ai);
00017         }
00018     printf("m = ");
00019     scanf("%d", &m);
00020     c = (a*m+b)%n;
00021     printf("c = %d\n", c);
00022     if(c>=b)
00023         mr = c-b;
00024     else
00025         mr = n+c-b;
00026     mr = (ai*mr)%n;
00027     printf("m = %d\n", mr);
00028     return 0;
00029 }

```

7.80 src/20252/Ejemplo028.c File Reference

```
#include <stdio.h>
```

Include dependency graph for Ejemplo028.c:



Macros

- `#define N 50`
- `#define NC 52`

Functions

- `int main (int argc, char *argv[])`

7.80.1 Macro Definition Documentation

7.80.1.1 N

```
#define N 50
```

Definition at line 3 of file [Ejemplo028.c](#).

7.80.1.2 NC

```
#define NC 52
```

Definition at line 4 of file [Ejemplo028.c](#).

7.80.2 Function Documentation

7.80.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 6 of file [Ejemplo028.c](#).

```
00007 {
00008     char str[N], strC[N], strD[N], c;
00009     int i, a, b, ai, nc;
00010     ai = -1;
00011     do{
00012         printf("a = ");
00013         scanf("%d", &a);
00014         for(i=0; i<NC; i++)
00015             if(((a*i)%NC)==1)
00016                 {
00017                     ai = i;
00018                     break;
00019                 }
00020     }while(ai<0);
00021     printf("b = ");
00022     scanf("%d", &b);
00023     while ((c = getchar()) != '\n' && c != EOF) {}
00024     i = 0;
00025     do{
00026         c = getchar();
00027         if((c>='A'&&c<='Z') || ((c>='a'&&c<='z') || c==' '))
00028             {
00029                 str[i] = c;
00030                 i++;
00031             }
00032     }while(c!=10&&i<(N-1));
00033     nc = i;
00034     str[nc] = '\0';
00035     printf("MSG: %s\n", str);
00036     for(i=0; i<nc; i++)
00037     {
00038         if(str[i]>='A'&&str[i]<='Z')
00039             strC[i] = str[i]-'A';
00040         else if(str[i]>='a'&&str[i]<='z')
00041             strC[i] = str[i]-'a'+26;
00042         else
00043             strC[i] = NC;
00044     }
00045     strC[nc] = '\0';
00046     for(i=0; i<nc; i++)
00047         if(strC[i]<NC)
00048             strC[i] = (a*strC[i]+b)%NC;
00049     for(i=0; i<nc; i++)
00050     {
00051         if(strC[i]<NC/2)
00052             strC[i] = strC[i]+'A';
00053         else if(strC[i]<NC)
00054             strC[i] = strC[i]+'a'-26;
00055         else
00056             strC[i] = ' ';
00057     }
00058     printf("MSG ENCRYPTADO: %s\n", strC);
00059     for(i=0; i<nc; i++)
00060     {
00061         if(strC[i]>='A'&&strC[i]<='Z')
00062             strD[i] = strC[i]-'A';
00063         else if(strC[i]>='a'&&strC[i]<='z')
00064             strD[i] = strC[i]-'a'+26;
00065         else
00066             strD[i] = NC;
00067     }
00068     strD[nc] = '\0';
00069     for(i=0; i<nc; i++)
00070         if(strD[i]<NC)
00071             {
00072                 if(strD[i]>=b)
00073                     strD[i] = (ai*(strD[i]-b))%NC;
00074                 else
00075                     strD[i] = (ai*(NC+strD[i]-b))%NC;
00076             }
00077     for(i=0; i<nc; i++)
00078     {
00079         if(strD[i]<NC/2)
00080             strD[i] = strD[i]+'A';
00081         else if(strD[i]<NC)
00082             strD[i] = strD[i]+'a'-26;
```

```

00083         else
00084             strD[i] = ' ';
00085     }
00086     printf("MSG DESENCRIPTADO: %s\n", strD);
00087     return 0;
00088 }

```

7.81 Ejemplo028.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002
00003 #define N 50
00004 #define NC 52
00005
00006 int main(int argc, char *argv[])
00007 {
00008     char str[N], strC[N], strD[N], c;
00009     int i, a, b, ai, nc;
00010     ai = -1;
00011     do{
00012         printf("a = ");
00013         scanf("%d", &a);
00014         for(i=0; i<NC; i++)
00015             if(((a*i)%NC)==1)
00016                 {
00017                     ai = i;
00018                     break;
00019                 }
00020     }while(ai<0);
00021     printf("b = ");
00022     scanf("%d", &b);
00023     while ((c = getchar()) != '\n' && c != EOF) {}
00024     i = 0;
00025     do{
00026         c = getchar();
00027         if((c>='A' && c<='Z') || ((c>='a' && c<='z') || c==' '))
00028             {
00029                 str[i] = c;
00030                 i++;
00031             }
00032     }while(c!=10&&i<(N-1));
00033     nc = i;
00034     str[nc] = '\0';
00035     printf("MSG: %s\n", str);
00036     for(i=0; i<nc; i++)
00037     {
00038         if(str[i]>='A' && str[i]<='Z')
00039             strC[i] = str[i]-'A';
00040         else if(str[i]>='a' && str[i]<='z')
00041             strC[i] = str[i]-'a'+26;
00042         else
00043             strC[i] = NC;
00044     }
00045     strC[nc] = '\0';
00046     for(i=0; i<nc; i++)
00047         if(strC[i]<NC)
00048             strC[i] = (a*strC[i]+b)%NC;
00049     for(i=0; i<nc; i++)
00050     {
00051         if(strC[i]<NC/2)
00052             strC[i] = strC[i]+'A';
00053         else if(strC[i]<NC)
00054             strC[i] = strC[i]+'a'-26;
00055         else
00056             strC[i] = ' ';
00057     }
00058     printf("MSG ENCRIPTADO: %s\n", strC);
00059     for(i=0; i<nc; i++)
00060     {
00061         if(strC[i]>='A' && strC[i]<='Z')
00062             strD[i] = strC[i]-'A';
00063         else if(strC[i]>='a' && strC[i]<='z')
00064             strD[i] = strC[i]-'a'+26;
00065         else
00066             strD[i] = NC;
00067     }
00068     strD[nc] = '\0';
00069     for(i=0; i<nc; i++)
00070         if(strD[i]<NC)
00071         {
00072             if(strD[i]>=b)
00073                 strD[i] = (ai*(strD[i]-b))%NC;
00074             else

```

```

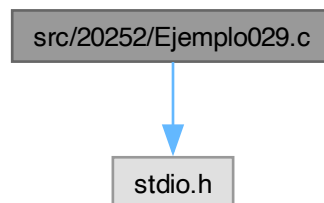
00075         strD[i] = (ai*(NC+strD[i]-b))%NC;
00076     }
00077     for(i=0; i<nc; i++)
00078     {
00079         if(strD[i]<NC/2)
00080             strD[i] = strD[i]+'A';
00081         else if(strD[i]<NC)
00082             strD[i] = strD[i]+'a'-26;
00083         else
00084             strD[i] = ' ';
00085     }
00086     printf("MSG DESENCRIPTADO: %s\n", strD);
00087     return 0;
00088 }

```

7.82 src/20252/Ejemplo029.c File Reference

#include <stdio.h>

Include dependency graph for Ejemplo029.c:



Functions

- double [exp1](#) (double x, int n)
- double [exp3](#) (double x, int n)
- double [exp4](#) (double x, int n)
- double [exp5](#) (double x, int n)
- double [potencia](#) (double x, int n)
- long int [factorial](#) (int n)
- double [potencia2](#) (double x, int n)
- long int [factorial2](#) (int n)
- double [fct](#) (double x, int n)
- int [main](#) (int argc, char *argv[])

7.82.1 Function Documentation

7.82.1.1 [exp1\(\)](#)

```

double exp1 (
    double x,
    int n)

```

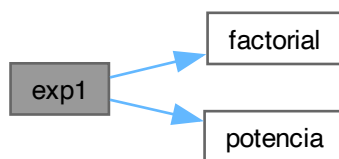
Definition at line 62 of file [Ejemplo029.c](#).

```

00063 {
00064     int i;
00065     double ex;
00066     for(i=0, ex=0; i<n; i++)
00067         ex += potencia(x, i)/factorial(i);
00068     return ex;
00069 }

```


Here is the call graph for this function:



Here is the caller graph for this function:



7.82.1.2 exp3()

```
double exp3 (  
    double x,  
    int n)
```

Definition at line 50 of file [Ejemplo029.c](#).

```
00051 {  
00052     int i;  
00053     double ex, fct;  
00054     for(i=0, ex=0, fct=1; i<n; i++)  
00055     {  
00056         ex += fct;  
00057         fct *= x/(i+1);  
00058     }  
00059     return ex;  
00060 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



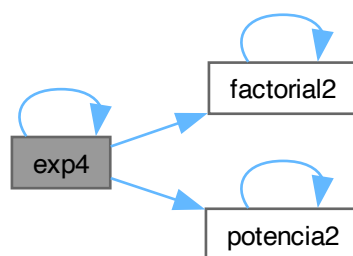
7.82.1.3 exp4()

```
double exp4 (
    double x,
    int n)
```

Definition at line 89 of file [Ejemplo029.c](#).

```
00090 {
00091     if (n>0)
00092         return exp4(x, n-1)+potencia2(x,n)/factorial2(n);
00093     else
00094         return 1;
00095 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



7.82.1.4 exp5()

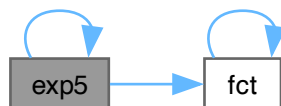
```
double exp5 (
```

```
double x,  
int n)
```

Definition at line 42 of file [Ejemplo029.c](#).

```
00043 {  
00044     if(n)  
00045         return exp5(x, n-1)+fct(x, n);  
00046     else  
00047         return 1;  
00048 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



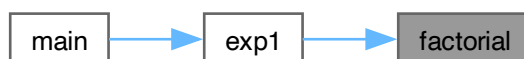
7.82.1.5 factorial()

```
long int factorial (  
    int n)
```

Definition at line 80 of file [Ejemplo029.c](#).

```
00081 {  
00082     int i;  
00083     long int fn;  
00084     for(i=0, fn=1; i<n; i++)  
00085         fn *= (i+1);  
00086     return fn;  
00087 }
```

Here is the caller graph for this function:



7.82.1.6 factorial2()

```
long int factorial2 (  
    int n)
```

Definition at line 105 of file [Ejemplo029.c](#).

```
00106 {  
00107     if (n>0)  
00108         return factorial2 (n-1) *n;  
00109     else  
00110         return 1;  
00111 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



7.82.1.7 fct()

```
double fct (  
    double x,  
    int n)
```

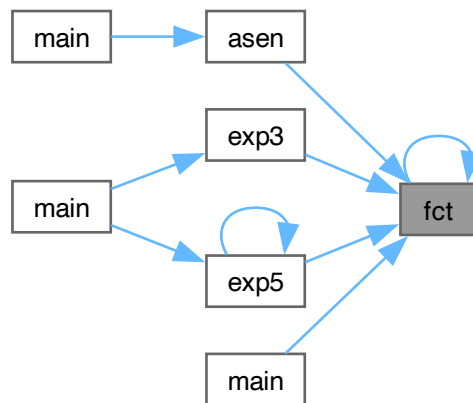
Definition at line 34 of file [Ejemplo029.c](#).

```
00035 {  
00036     if (n!=1)  
00037         return fct (x, n-1) * (x/n);  
00038     else  
00039         return x;  
00040 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



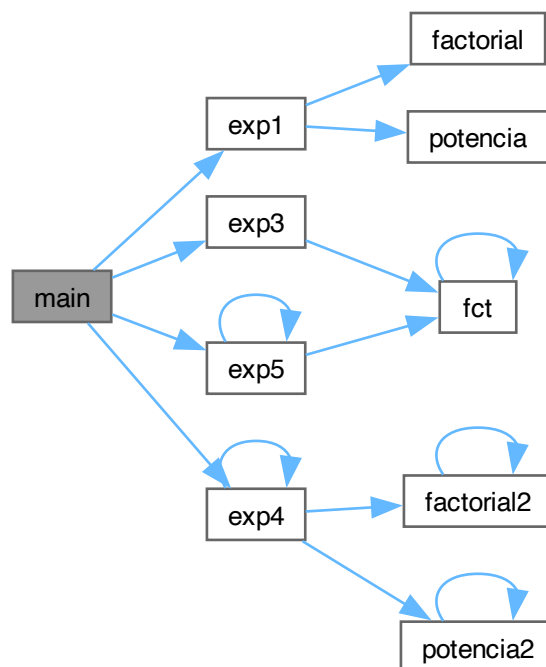
7.82.1.8 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 13 of file [Ejemplo029.c](#).

```
00014 {
00015     double x, ex, ex2, ex3, ex4;
00016     int n;
00017     printf("Ingrese el valor de x: ");
00018     scanf("%lf", &x);
00019     do{
00020         printf("Ingrese le numero de terminos: ");
00021         scanf("%d", &n);
00022     }while(n<1);
00023     ex = exp1(x, n);
00024     ex2 = exp3(x, n);
00025     ex3 = exp4(x, n);
00026     ex4 = exp5(x, n);
00027     printf("exp(%lf) = %lf\n", x, ex);
00028     printf("exp(%lf) = %lf\n", x, ex2);
00029     printf("exp(%lf) = %lf\n", x, ex3);
00030     printf("exp(%lf) = %lf\n", x, ex4);
00031     return 0;
00032 }
```

Here is the call graph for this function:



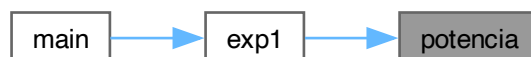
7.82.1.9 potencia()

```
double potencia (
    double x,
    int n)
```

Definition at line 71 of file [Ejemplo029.c](#).

```
00072 {
00073     int i;
00074     double xn;
00075     for(i=0, xn=1; i<n; i++)
00076         xn *= x;
00077     return xn;
00078 }
```

Here is the caller graph for this function:



7.82.1.10 potencia2()

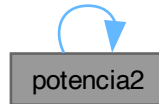
```
double potencia2 (
```

```
double x,
int n)
```

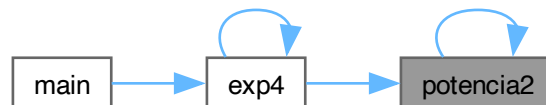
Definition at line 97 of file [Ejemplo029.c](#).

```
00098 {
00099     if (n>0)
00100         return potencia2(x, n-1)*x;
00101     else
00102         return 1;
00103 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



7.83 Ejemplo029.c

[Go to the documentation of this file.](#)

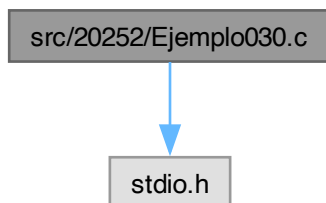
```
00001 #include <stdio.h>
00002
00003 double exp1(double x, int n);
00004 double exp3(double x, int n);
00005 double exp4(double x, int n);
00006 double exp5(double x, int n);
00007 double potencia(double x, int n);
00008 long int factorial(int n);
00009 double potencia2(double x, int n);
00010 long int factorial2(int n);
00011 double fct(double x, int n);
00012
00013 int main(int argc, char *argv[])
00014 {
00015     double x, ex, ex2, ex3, ex4;
00016     int n;
00017     printf("Ingrese el valor de x: ");
00018     scanf("%lf", &x);
00019     do{
00020         printf("Ingrese le numero de terminos: ");
00021         scanf("%d", &n);
00022     }while(n<1);
00023     ex = exp1(x, n);
00024     ex2 = exp3(x, n);
00025     ex3 = exp4(x, n);
00026     ex4 = exp5(x, n);
00027     printf("exp(%lf) = %lf\n", x, ex);
00028     printf("exp(%lf) = %lf\n", x, ex2);
00029     printf("exp(%lf) = %lf\n", x, ex3);
```

```
00030     printf("exp(%lf) = %lf\n", x, ex4);
00031     return 0;
00032 }
00033
00034 double fct(double x, int n)
00035 {
00036     if(n!=1)
00037         return fct(x, n-1)*(x/n);
00038     else
00039         return x;
00040 }
00041
00042 double exp5(double x, int n)
00043 {
00044     if(n)
00045         return exp5(x, n-1)+fct(x, n);
00046     else
00047         return 1;
00048 }
00049
00050 double exp3(double x, int n)
00051 {
00052     int i;
00053     double ex, fct;
00054     for(i=0, ex=0, fct=1; i<n; i++)
00055     {
00056         ex += fct;
00057         fct *= x/(i+1);
00058     }
00059     return ex;
00060 }
00061
00062 double exp1(double x, int n)
00063 {
00064     int i;
00065     double ex;
00066     for(i=0, ex=0; i<n; i++)
00067         ex += potencia(x, i)/factorial(i);
00068     return ex;
00069 }
00070
00071 double potencia(double x, int n)
00072 {
00073     int i;
00074     double xn;
00075     for(i=0, xn=1; i<n; i++)
00076         xn *= x;
00077     return xn;
00078 }
00079
00080 long int factorial(int n)
00081 {
00082     int i;
00083     long int fn;
00084     for(i=0, fn=1; i<n; i++)
00085         fn *= (i+1);
00086     return fn;
00087 }
00088
00089 double exp4(double x, int n)
00090 {
00091     if(n>0)
00092         return exp4(x, n-1)+potencia2(x,n)/factorial2(n);
00093     else
00094         return 1;
00095 }
00096
00097 double potencia2(double x, int n)
00098 {
00099     if(n>0)
00100         return potencia2(x, n-1)*x;
00101     else
00102         return 1;
00103 }
00104
00105 long int factorial2(int n)
00106 {
00107     if(n>0)
00108         return factorial2(n-1)*n;
00109     else
00110         return 1;
00111 }
```


7.84 src/20252/Ejemplo030.c File Reference

```
#include <stdio.h>
```

Include dependency graph for Ejemplo030.c:



Functions

- int [buscar](#) (int n)
- int [main](#) (int argc, char *argv[])

7.84.1 Function Documentation

7.84.1.1 [buscar\(\)](#)

```
int buscar (  
    int n)
```

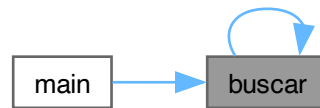
Definition at line 3 of file [Ejemplo030.c](#).

```
00004 {  
00005     return n%2?n:buscar (n/2);  
00006 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



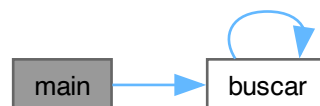
7.84.1.2 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 8 of file [Ejemplo030.c](#).

```
00009 {
00010     int n, a, b, i, j, x;
00011     scanf("%d", &n);
00012     for(i=0; i<n; i++)
00013     {
00014         scanf("%d %d", &a, &b);
00015         for(j=a, x=0; j<=b; j++)
00016         {
00017             printf("%d (%d)\t", j, buscar(j));
00018             x += buscar(j);
00019         }
00020         printf("%d\n", x);
00021     }
00022     return 0;
00023 }
```

Here is the call graph for this function:



7.85 Ejemplo030.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002
00003 int buscar(int n)
00004 {
00005     return n%2?n:buscar(n/2);
00006 }
00007
00008 int main(int argc, char *argv[])
00009 {
00010     int n, a, b, i, j, x;
00011     scanf("%d", &n);
00012     for(i=0; i<n; i++)
00013     {
00014         scanf("%d %d", &a, &b);
```

```

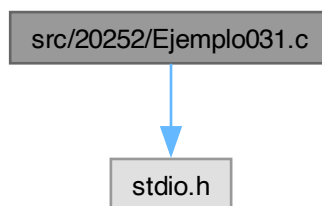
00015         for(j=a, x=0; j<=b; j++)
00016         {
00017             printf("%d (%d)\t", j, buscar(j));
00018             x += buscar(j);
00019         }
00020         printf("%d\n", x);
00021     }
00022     return 0;
00023 }

```

7.86 src/20252/Ejemplo031.c File Reference

```
#include <stdio.h>
```

Include dependency graph for Ejemplo031.c:



Functions

- int [main](#) (int argc, char *argv[])

7.86.1 Function Documentation

7.86.1.1 main()

```

int main (
    int argc,
    char * argv[])

```

Definition at line 3 of file [Ejemplo031.c](#).

```

00004 {
00005     int i;
00006     printf("Numero de parametros de enytrada: %d\n", argc);
00007     for(i=0; i<argc; i++)
00008         printf("%d. %s\n", i+1, argv[i]);
00009     return 0;
00010 }

```

7.87 Ejemplo031.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     int i;
00006     printf("Numero de parametros de enytrada: %d\n", argc);
00007     for(i=0; i<argc; i++)
00008         printf("%d. %s\n", i+1, argv[i]);
00009     return 0;
00010 }

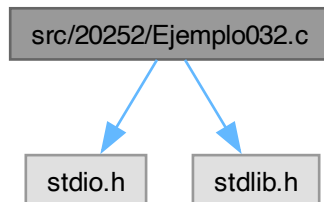
```

7.88 src/20252/Ejemplo032.c File Reference

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

Include dependency graph for Ejemplo032.c:



Functions

- `int main (int argc, char *argv[])`

7.88.1 Function Documentation

7.88.1.1 main()

```
int main (  
    int argc,  
    char * argv[])
```

Definition at line 4 of file [Ejemplo032.c](#).

```
00005 {  
00006     int i;  
00007     float pi;  
00008     if(argc>1)  
00009     {  
00010         for(i=1, pi=0; i<argc; i++)  
00011             pi+=atof(argv[i]);  
00012         printf("La suma es %f\n", pi);  
00013         return 0;  
00014     }  
00015     else  
00016         return 1;  
00017 }
```

7.89 Ejemplo032.c

[Go to the documentation of this file.](#)

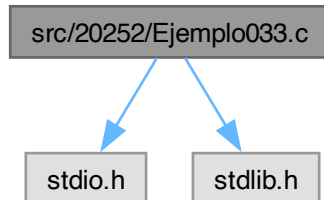
```
00001 #include <stdio.h>  
00002 #include <stdlib.h>  
00003  
00004 int main(int argc, char *argv[])  
00005 {  
00006     int i;  
00007     float pi;  
00008     if(argc>1)  
00009     {  
00010         for(i=1, pi=0; i<argc; i++)  
00011             pi+=atof(argv[i]);  
00012         printf("La suma es %f\n", pi);  
00013         return 0;  
00014     }  
00015     else  
00016         return 1;  
00017 }
```

7.90 src/20252/Ejemplo033.c File Reference

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

Include dependency graph for Ejemplo033.c:



Macros

- `#define N 1000`

Functions

- float `validar2` (float `a`, float `min`, float `max`, char `*str`)
- float `validar` (float `min`, float `max`, char `*str`)
- float `asen` (float `x`, int `n`)
- int `main` (int `argc`, char `*argv[]`)

7.90.1 Macro Definition Documentation

7.90.1.1 N

```
#define N 1000
```

Definition at line 4 of file [Ejemplo033.c](#).

7.90.2 Function Documentation

7.90.2.1 asen()

```
float asen (  
    float x,  
    int n)
```

Definition at line 29 of file [Ejemplo033.c](#).

```
00030 {  
00031     int i;  
00032     float asx, fct;  
00033     for(i=0, asx=0, fct=x; i<n; i++)  
00034     {  
00035         asx+=fct;  
00036         fct*= ((2*i+1)*x*x/(2*i+3));  
00037         fct*= ((2*i+1.0)/(2*(i+1)));  
00038     }  
00039     return asx;  
00040 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



7.90.2.2 main()

```

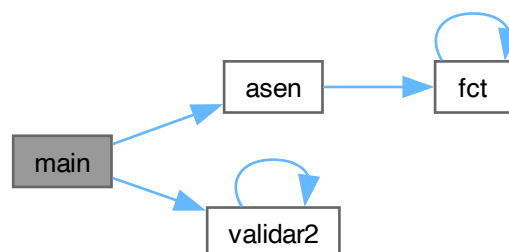
int main (
    int argc,
    char * argv[])
  
```

Definition at line 42 of file [Ejemplo033.c](#).

```

00043 {
00044     int n=0;
00045     float x=-1, asx;
00046     if(argc==3)
00047         x = atof(argv[2]);
00048     if(argc&2)
00049         n = atoi(argv[1]);
00050     n = (int)validar2(n, 1, N, "n");
00051     x = validar2(x, -1, 1, "x");
00052     asx = asen(x, n);
00053     printf("asen(%f) = %f\n", x, asx);
00054     return 0;
00055 }
  
```

Here is the call graph for this function:



7.90.2.3 validar()

```
float validar (  
    float min,  
    float max,  
    char * str)
```

Definition at line 18 of file [Ejemplo033.c](#).

```
00019 {  
00020     float a;  
00021     printf("Ingrese el valor de %s: ", str);  
00022     scanf("%f", &a);  
00023     if(a>min&&a<max)  
00024         return a;  
00025     else  
00026         return validar(min, max, str);  
00027 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



7.90.2.4 validar2()

```
float validar2 (  
    float a,  
    float min,  
    float max,  
    char * str)
```

Definition at line 6 of file [Ejemplo033.c](#).

```
00007 {  
00008     if(a>min&&a<max)  
00009         return a;  
00010     printf("Ingrese el valor de %s: ", str);  
00011     scanf("%f", &a);  
00012     if(a>min&&a<max)  
00013         return a;  
00014     else  
00015         return validar2(a, min, max, str);  
00016 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



7.91 Ejemplo033.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002 #include <stdlib.h>
00003
00004 #define N 1000
00005
00006 float validar2(float a, float min, float max, char *str)
00007 {
00008     if(a>min&&a<max)
00009         return a;
00010     printf("Ingresa el valor de %s: ", str);
00011     scanf("%f", &a);
00012     if(a>min&&a<max)
00013         return a;
00014     else
00015         return validar2(a, min, max, str);
00016 }
00017
00018 float validar(float min, float max, char *str)
00019 {
00020     float a;
00021     printf("Ingresa el valor de %s: ", str);
00022     scanf("%f", &a);
00023     if(a>min&&a<max)
00024         return a;
00025     else
00026         return validar(min, max, str);
00027 }
00028
00029 float asen(float x, int n)
00030 {
00031     int i;
00032     float asx, fct;
00033     for(i=0, asx=0, fct=x; i<n; i++)
00034     {
00035         asx+=fct;
00036         fct*=(2*i+1)*x*x/(2*i+3));
00037         fct*=(2*i+1.0)/(2*(i+1));
00038     }
00039     return asx;
00040 }
  
```



```

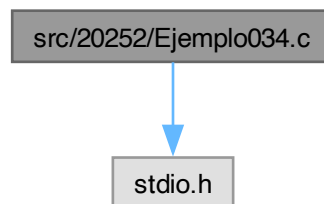
00041
00042 int main(int argc, char *argv[])
00043 {
00044     int n=0;
00045     float x=-1, asx;
00046     if(argc==3)
00047         x = atof(argv[2]);
00048     if(argc&2)
00049         n = atoi(argv[1]);
00050     n = (int)validar2(n, 1, N, "n");
00051     x = validar2(x, -1, 1, "x");
00052     asx = asen(x, n);
00053     printf("asen(%f) = %f\n", x, asx);
00054     return 0;
00055 }

```

7.92 src/20252/Ejemplo034.c File Reference

```
#include <stdio.h>
```

Include dependency graph for Ejemplo034.c:



Data Structures

- struct [contacto](#)

Macros

- #define [NM](#) 50
- #define [NE](#) 20
- #define [NC](#) 10

Functions

- int [main](#) (int argc, char *argv[])

7.92.1 Macro Definition Documentation

7.92.1.1 NC

```
#define NC 10
```

Definition at line 5 of file [Ejemplo034.c](#).

7.92.1.2 NE

```
#define NE 20
```

Definition at line 4 of file [Ejemplo034.c](#).

7.92.1.3 NM

#define NM 50

Definition at line 3 of file [Ejemplo034.c](#).

7.92.2 Function Documentation

7.92.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 14 of file [Ejemplo034.c](#).

```
00015 {
00016     contacto agenda[NC];
00017     int op, i, n;
00018     char c;
00019     n = 0;
00020     do{
00021         printf("Nombre: ");
00022         i = 0;
00023         do{
00024             c = getchar();
00025             agenda[n].nombre[i++] = c;
00026         }while(c!=10);
00027         agenda[n].nombre[i-1] = '\0';
00028         // scanf("%s", agenda[n].nombre);
00029         // &(agenda[i].nombre[]) -> agenda[i].nombre
00030         printf("Telefono: ");
00031         scanf("%ld", &(agenda[n].telefono));
00032         printf("Correo electronico: ");
00033         scanf("%s", agenda[n].email);
00034         n++;
00035         printf("Ingresar otro contacto: ");
00036         scanf("%d", &op);
00037         do{
00038             c = getchar();
00039         }while(c!=10);
00040     }while(op&&(n<NC));
00041     printf("Agenda.\n");
00042     for(i=0; i<n; i++)
00043         printf("%s, %ld, %s\n", agenda[i].nombre, agenda[i].telefono, agenda[i].email);
00044     return 0;
00045 }
```

7.93 Ejemplo034.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002
00003 #define NM 50
00004 #define NE 20
00005 #define NC 10
00006
00007 typedef struct
00008 {
00009     char nombre[NM];
00010     long int telefono;
00011     char email[NE];
00012 }contacto;
00013
00014 int main(int argc, char *argv[])
00015 {
00016     contacto agenda[NC];
00017     int op, i, n;
00018     char c;
00019     n = 0;
00020     do{
00021         printf("Nombre: ");
00022         i = 0;
00023         do{
00024             c = getchar();
00025             agenda[n].nombre[i++] = c;
00026         }while(c!=10);
00027         agenda[n].nombre[i-1] = '\0';
00028         // scanf("%s", agenda[n].nombre);
00029         // &(agenda[i].nombre[]) -> agenda[i].nombre
00030         printf("Telefono: ");
```

```

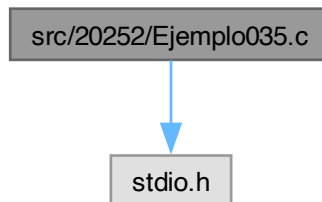
00031         scanf("%ld", &(agenda[n].telefono));
00032         printf("Correo electronico: ");
00033         scanf("%s", agenda[n].email);
00034         n++;
00035         printf("Ingresar otro contacto: ");
00036         scanf("%d", &op);
00037         do{
00038             c = getchar();
00039             }while(c!=10);
00040         }while(op&&(n<NC));
00041         printf("Agenda.\n");
00042         for(i=0; i<n; i++)
00043             printf("%s, %ld, %s\n", agenda[i].nombre, agenda[i].telefono, agenda[i].email);
00044         return 0;
00045     }

```

7.94 src/20252/Ejemplo035.c File Reference

#include <stdio.h>

Include dependency graph for Ejemplo035.c:



Data Structures

- struct [dato_s](#)
- union [dato_u](#)

Functions

- int [main](#) (int argc, char *argv[])

7.94.1 Function Documentation

7.94.1.1 main()

```

int main (
    int argc,
    char * argv[] )

```

Definition at line 15 of file [Ejemplo035.c](#).

```

00016 {
00017     dato_s x;
00018     dato_u y;
00019     printf("Sizeof(char) = %ld\n", sizeof(char));
00020     printf("Sizeof(int) = %ld\n", sizeof(int));
00021     printf("Sizeof(float) = %ld\n", sizeof(float));
00022     printf("Sizeof(dato_s) = %ld\n", sizeof(x));
00023     printf("Sizeof(dato_u) = %ld\n", sizeof(y));
00024     x.a = 1;
00025     y.a = 1;
00026     printf("x.a = %d\n", x.a);
00027     printf("y.a = %d\n", y.a);
00028     x.b = 2;

```

```

00029     y.b = 2;
00030     printf("x.a = %d\n", x.a);
00031     printf("y.a = %d\n", y.a);
00032     printf("x.b = %d\n", x.b);
00033     printf("y.b = %d\n", y.b);
00034     return 0;
00035 }

```

7.95 Ejemplo035.c

[Go to the documentation of this file.](#)

```

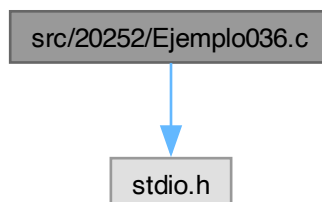
00001 #include <stdio.h>
00002
00003 typedef struct{
00004     char a;
00005     int b;
00006     float c;
00007 }dato_s;
00008
00009 typedef union{
00010     char a;
00011     int b;
00012     float c;
00013 }dato_u;
00014
00015 int main(int argc, char *argv[])
00016 {
00017     dato_s x;
00018     dato_u y;
00019     printf("Sizeof(char) = %ld\n", sizeof(char));
00020     printf("Sizeof(int) = %ld\n", sizeof(int));
00021     printf("Sizeof(float) = %ld\n", sizeof(float));
00022     printf("Sizeof(dato_s) = %ld\n", sizeof(x));
00023     printf("Sizeof(dato_u) = %ld\n", sizeof(y));
00024     x.a = 1;
00025     y.a = 1;
00026     printf("x.a = %d\n", x.a);
00027     printf("y.a = %d\n", y.a);
00028     x.b = 2;
00029     y.b = 2;
00030     printf("x.a = %d\n", x.a);
00031     printf("y.a = %d\n", y.a);
00032     printf("x.b = %d\n", x.b);
00033     printf("y.b = %d\n", y.b);
00034     return 0;
00035 }

```

7.96 src/20252/Ejemplo036.c File Reference

```
#include <stdio.h>
```

Include dependency graph for Ejemplo036.c:



Functions

- `int main (int argc, char *argv[ ])`

7.96.1 Function Documentation

7.96.1.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 3 of file [Ejemplo036.c](#).

```
00004 {
00005     int x, *px;
00006     char *py;
00007     double *pz;
00008     x = 20;
00009     px = &x;
00010     py = (char*)&x;
00011     pz = (double*)&x;
00012     printf("x = %d\n", x);
00013     printf("&x = %p\n", &x);
00014     printf("px = %p\n", px);
00015     printf("*px = %d\n", *px);
00016     printf("px = %p\n", px);
00017     printf("py = %p\n", py);
00018     printf("pz = %p\n", pz);
00019     printf("px = %p (%ld)\n", px+1, sizeof(int));
00020     printf("py = %p (%ld)\n", py+1, sizeof(char));
00021     printf("pz = %p (%ld)\n", pz+1, sizeof(double));
00022     (*px)++;
00023     printf("x = %d\n", x);
00024     printf("px = %p\n", px);
00025     (*py)++;
00026     printf("x = %d\n", x);
00027     printf("py = %p\n", py);
00028     py++;
00029     (*py)++;
00030     printf("x = %d\n", x);
00031     printf("py = %p\n", py);
00032     (*py)++;
00033     printf("x = %d\n", x);
00034     printf("py = %p\n", py);
00035     py++;
00036     (*py)++;
00037     printf("x = %d\n", x);
00038     printf("py = %p\n", py);
00039     return 0;
00040 }
```

7.97 Ejemplo036.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     int x, *px;
00006     char *py;
00007     double *pz;
00008     x = 20;
00009     px = &x;
00010     py = (char*)&x;
00011     pz = (double*)&x;
00012     printf("x = %d\n", x);
00013     printf("&x = %p\n", &x);
00014     printf("px = %p\n", px);
00015     printf("*px = %d\n", *px);
00016     printf("px = %p\n", px);
00017     printf("py = %p\n", py);
00018     printf("pz = %p\n", pz);
00019     printf("px = %p (%ld)\n", px+1, sizeof(int));
00020     printf("py = %p (%ld)\n", py+1, sizeof(char));
00021     printf("pz = %p (%ld)\n", pz+1, sizeof(double));
00022     (*px)++;
00023     printf("x = %d\n", x);
00024     printf("px = %p\n", px);
00025     (*py)++;
00026     printf("x = %d\n", x);
00027     printf("py = %p\n", py);
00028     py++;
00029     (*py)++;
00030     printf("x = %d\n", x);
00031     printf("py = %p\n", py);
00032     (*py)++;
```

```

00033     printf("x = %d\n", x);
00034     printf("py = %p\n", py);
00035     py++;
00036     (*py)++;
00037     printf("x = %d\n", x);
00038     printf("py = %p\n", py);
00039     return 0;
00040 }

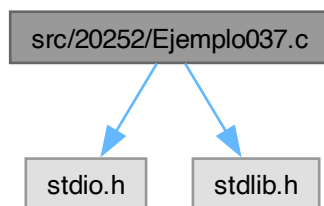
```

7.98 src/20252/Ejemplo037.c File Reference

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

Include dependency graph for Ejemplo037.c:



Functions

- int [main](#) (int argc, char *argv[])

7.98.1 Function Documentation

7.98.1.1 main()

```

int main (
    int argc,
    char * argv[ ])

```

Definition at line 4 of file [Ejemplo037.c](#).

```

00005 {
00006     double *x, m;
00007     int n, i;
00008     do{
00009         printf("Ingrese el numero de elementos: ");
00010         scanf("%d", &n);
00011     }while(n<1);
00012     x = (double*)malloc(n*sizeof(double));
00013     if(x==NULL)
00014         return 1;
00015     for(i=0, m=0; i<n; i++)
00016     {
00017         printf("x[%d] = ", i+1);
00018         scanf("%lf", x+i);
00019         m += x[i];
00020     }
00021     m/=n;
00022     printf("Media = %lf\n", m);
00023     free(x);
00024     return 0;
00025 }

```

7.99 Ejemplo037.c

[Go to the documentation of this file.](#)

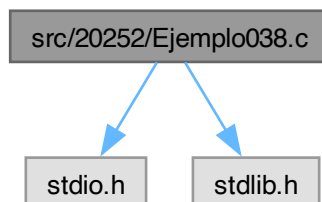
```

00001 #include <stdio.h>
00002 #include <stdlib.h>
00003
00004 int main(int argc, char *argv[])
00005 {
00006     double *x, m;
00007     int n, i;
00008     do{
00009         printf("Ingrese el numero de elementos: ");
00010         scanf("%d", &n);
00011     }while(n<1);
00012     x = (double*)malloc(n*sizeof(double));
00013     if(x==NULL)
00014         return 1;
00015     for(i=0, m=0; i<n; i++)
00016     {
00017         printf("x[%d] = ", i+1);
00018         scanf("%lf", x+i);
00019         m += x[i];
00020     }
00021     m/=n;
00022     printf("Media = %lf\n", m);
00023     free(x);
00024     return 0;
00025 }
```

7.100 src/20252/Ejemplo038.c File Reference

```
#include <stdio.h>
#include <stdlib.h>
```

Include dependency graph for Ejemplo038.c:



Functions

- int [main](#) (int argc, char *argv[])

7.100.1 Function Documentation

7.100.1.1 main()

```

int main (
    int argc,
    char * argv[])
```

Definition at line 4 of file [Ejemplo038.c](#).

```

00005 {
00006     int r, c, i, j;
00007     double *A;
00008     do{
00009         printf("Ingrese el numero de renglones: ");
```

```

00010         scanf("%d", &r);
00011     }while(r<1);
00012     do{
00013         printf("Ingrese el numero de columnas: ");
00014         scanf("%d", &c);
00015     }while(c<1);
00016     A = (double*)malloc(r*c*sizeof(double));
00017     if(A==NULL)
00018         return 1;
00019     for(i=0; i<r; i++)
00020         for(j=0; j<c; j++)
00021     {
00022         printf("A[%d][%d] = ", i+1, j+1);
00023         //         scanf("%lf", &A[c*i+j]);
00024         //         scanf("%lf", &*(A+c*i+j));
00025         scanf("%lf", A+c*i+j);
00026     }
00027     for(i=0; i<r; i++)
00028     {
00029         for(j=0; j<c; j++)
00030             printf("%.4lf\t", A[c*i+j]);
00031         printf("\n");
00032     }
00033     free(A);
00034     return 0;
00035 }

```

7.101 Ejemplo038.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002 #include <stdlib.h>
00003
00004 int main(int argc, char *argv[])
00005 {
00006     int r, c, i, j;
00007     double *A;
00008     do{
00009         printf("Ingrese el numero de renglones: ");
00010         scanf("%d", &r);
00011     }while(r<1);
00012     do{
00013         printf("Ingrese el numero de columnas: ");
00014         scanf("%d", &c);
00015     }while(c<1);
00016     A = (double*)malloc(r*c*sizeof(double));
00017     if(A==NULL)
00018         return 1;
00019     for(i=0; i<r; i++)
00020         for(j=0; j<c; j++)
00021     {
00022         printf("A[%d][%d] = ", i+1, j+1);
00023         //         scanf("%lf", &A[c*i+j]);
00024         //         scanf("%lf", &*(A+c*i+j));
00025         scanf("%lf", A+c*i+j);
00026     }
00027     for(i=0; i<r; i++)
00028     {
00029         for(j=0; j<c; j++)
00030             printf("%.4lf\t", A[c*i+j]);
00031         printf("\n");
00032     }
00033     free(A);
00034     return 0;
00035 }

```

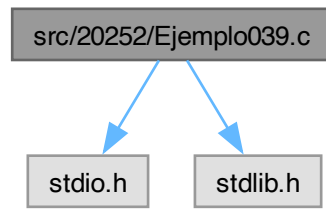
7.102 src/20252/Ejemplo039.c File Reference

```

#include <stdio.h>
#include <stdlib.h>

```


Include dependency graph for Ejemplo039.c:



Functions

- double * [crearMat](#) (int r, int c)
- int [capturar](#) (double *M, int r, int c, char *str)
- int [imprimir](#) (double *M, int r, int c)
- double * [multiplicar](#) (double *M1, int r1, int c1, double *M2, int r2, int c2)
- int [main](#) (int argc, char *argv[])

7.102.1 Function Documentation

7.102.1.1 [capturar\(\)](#)

```

int capturar (
    double * M,
    int r,
    int c,
    char * str)

```

Definition at line 13 of file [Ejemplo039.c](#).

```

00014 {
00015     int i, j;
00016     for(i=0; i<r; i++)
00017         for(j=0; j<c; j++)
00018         {
00019             printf("%s[%d][%d] = ", str, i+1, j+1);
00020             scanf("%lf", M+c*i+j);
00021         }
00022     return 0;
00023 }

```

Here is the caller graph for this function:



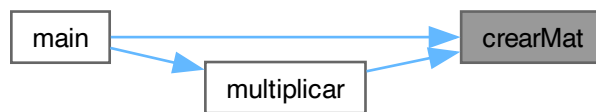
7.102.1.2 crearMat()

```
double * crearMat (
    int r,
    int c)
```

Definition at line 4 of file [Ejemplo039.c](#).

```
00005 {
00006     double *M;
00007     M = (double*)malloc(r*c*sizeof(double));
00008     if(M==NULL)
00009         return NULL;
00010     return M;
00011 }
```

Here is the caller graph for this function:



7.102.1.3 imprimir()

```
int imprimir (
    double * M,
    int r,
    int c)
```

Definition at line 25 of file [Ejemplo039.c](#).

```
00026 {
00027     int i, j;
00028     for(i=0; i<r; i++)
00029     {
00030         for(j=0; j<c; j++)
00031             printf("%.4lf\t", M[c*i+j]);
00032         printf("\n");
00033     }
00034     return 0;
00035 }
```

Here is the caller graph for this function:



7.102.1.4 main()

```
int main (
    int argc,
    char * argv[])
```

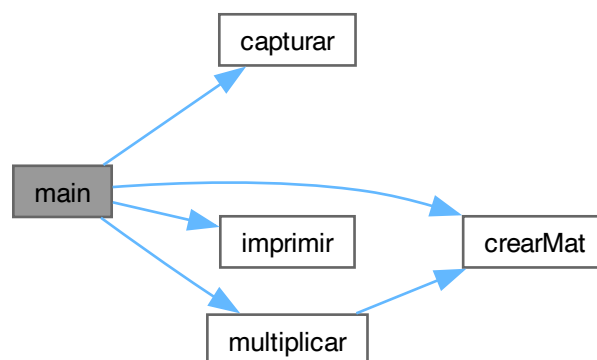
Definition at line 59 of file [Ejemplo039.c](#).

```

00060 {
00061     int rA, rB, cA, cB;
00062     double *A, *B, *C;
00063     do{
00064         printf("Ingrese el numero de renglones de A: ");
00065         scanf("%d", &rA);
00066     }while(rA<1);
00067     do{
00068         printf("Ingrese el numero de columnas de A: ");
00069         scanf("%d", &cA);
00070     }while(cA<1);
00071     A = crearMat(rA, cA);
00072     if(A==NULL)
00073         return 1;
00074     capturar(A, rA, cA, "A");
00075     do{
00076         printf("Ingrese el numero de renglones de B: ");
00077         scanf("%d", &rB);
00078     }while(rB<1);
00079     do{
00080         printf("Ingrese el numero de columnas de B: ");
00081         scanf("%d", &cB);
00082     }while(cB<1);
00083     B = crearMat(rB, cB);
00084     if(B==NULL)
00085     {
00086         free(A);
00087         return 2;
00088     }
00089     capturar(B, rB, cB, "B");
00090     C = multiplicar(A, rA, cA, B, rB, cB);
00091     if(C==NULL)
00092     {
00093         free(A);
00094         free(B);
00095         return 4;
00096     }
00097     printf("A\n");
00098     imprimir(A, rA, cA);
00099     printf("B\n");
00100     imprimir(B, rB, cB);
00101     printf("C\n");
00102     imprimir(C, rA, cB);
00103     free(A);
00104     free(B);
00105     free(C);
00106     return 0;
00107 }

```

Here is the call graph for this function:



7.102.1.5 multiplicar()

```

double * multiplicar (
    double * M1,

```

```

    int r1,
    int c1,
    double * M2,
    int r2,
    int c2)

```

Definition at line 37 of file [Ejemplo039.c](#).

```

00038 {
00039     int i, j, k;
00040     double *M3;
00041     if(c1!=r2)
00042     {
00043         printf("Error: En el tamaño de las matrices.\n");
00044         return NULL;
00045     }
00046     M3 = crearMat(r1, c2);
00047     if(M3==NULL)
00048         return NULL;
00049     for(i=0; i<r1; i++)
00050         for(j=0; j<c2; j++)
00051             for(k=0; M3[i*c1+j]=0; k<c1; k++)
00052                 M3[i*c2+j] += M1[i*c1+k]*M2[k*c2+j];
00053     //      M3[i][j] = M3[i][j]+M1[i][k]*M2[k][j]
00054     //      M3[i][j] += M1[i][k]*M2[k][j]
00055     //      M3[i*c2+j] += M1[i*c1+k]*M2[k*r2+j]
00056     return M3;
00057 }

```

Here is the call graph for this function:



Here is the caller graph for this function:



7.103 Ejemplo039.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002 #include <stdlib.h>
00003
00004 double *crearMat(int r, int c)
00005 {
00006     double *M;
00007     M = (double*) malloc(r*c*sizeof(double));
00008     if(M==NULL)
00009         return NULL;
00010     return M;
00011 }
00012
00013 int capturar(double *M, int r, int c, char *str)
00014 {
00015     int i, j;

```

```

00016     for(i=0; i<r; i++)
00017         for(j=0; j<c; j++)
00018         {
00019             printf("%s[%d][%d] = ", str, i+1, j+1);
00020             scanf("%lf", M+c*i+j);
00021         }
00022     return 0;
00023 }
00024
00025 int imprimir(double *M, int r, int c)
00026 {
00027     int i, j;
00028     for(i=0; i<r; i++)
00029     {
00030         for(j=0; j<c; j++)
00031             printf("%.4lf\t", M[c*i+j]);
00032         printf("\n");
00033     }
00034     return 0;
00035 }
00036
00037 double *multiplicar(double *M1, int r1, int c1, double *M2, int r2, int c2)
00038 {
00039     int i, j, k;
00040     double *M3;
00041     if(c1!=r2)
00042     {
00043         printf("Error: En el tamaño de las matrices.\n");
00044         return NULL;
00045     }
00046     M3 = crearMat(r1, c2);
00047     if(M3==NULL)
00048         return NULL;
00049     for(i=0; i<r1; i++)
00050         for(j=0; j<c2; j++)
00051             for(k=0, M3[i*c1+j]=0; k<c1; k++)
00052                 M3[i*c2+j] += M1[i*c1+k]*M2[k*c2+j];
00053     // M3[i][j] = M3[i][j]+M1[i][k]*M2[k][j]
00054     // M3[i][j] += M1[i][k]*M2[k][j]
00055     // M3[i*c2+j] += M1[i*c1+k]*M2[k*r2+j]
00056     return M3;
00057 }
00058
00059 int main(int argc, char *argv[])
00060 {
00061     int rA, rB, cA, cB;
00062     double *A, *B, *C;
00063     do{
00064         printf("Ingrese el numero de renglones de A: ");
00065         scanf("%d", &rA);
00066     }while(rA<1);
00067     do{
00068         printf("Ingrese el numero de columnas de A: ");
00069         scanf("%d", &cA);
00070     }while(cA<1);
00071     A = crearMat(rA, cA);
00072     if(A==NULL)
00073         return 1;
00074     capturar(A, rA, cA, "A");
00075     do{
00076         printf("Ingrese el numero de renglones de B: ");
00077         scanf("%d", &rB);
00078     }while(rB<1);
00079     do{
00080         printf("Ingrese el numero de columnas de B: ");
00081         scanf("%d", &cB);
00082     }while(cB<1);
00083     B = crearMat(rB, cB);
00084     if(B==NULL)
00085     {
00086         free(A);
00087         return 2;
00088     }
00089     capturar(B, rB, cB, "B");
00090     C = multiplicar(A, rA, cA, B, rB, cB);
00091     if(C==NULL)
00092     {
00093         free(A);
00094         free(B);
00095         return 4;
00096     }
00097     printf("A\n");
00098     imprimir(A, rA, cA);
00099     printf("B\n");
00100     imprimir(B, rB, cB);
00101     printf("C\n");
00102     imprimir(C, rA, cB);

```

```

00103     free(A);
00104     free(B);
00105     free(C);
00106     return 0;
00107 }

```

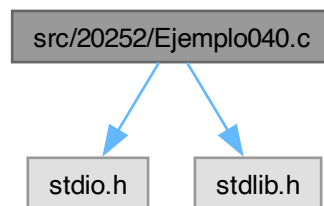
7.104 src/20252/Ejemplo040.c File Reference

```

#include <stdio.h>
#include <stdlib.h>

```

Include dependency graph for Ejemplo040.c:



Functions

- double * [crearMat](#) (int r, int c)
- double ** [crearMatC](#) (int r, int c)
- double ** [crearMatD](#) (int r, int c)
- int [capturar](#) (double *[M](#), int r, int c, char *str)
- int [imprimir](#) (double *[M](#), int r, int c)
- int [imprimirM](#) (double **[M](#), int r, int c)
- double * [multiplicar](#) (double *[M1](#), int r1, int c1, double *[M2](#), int r2, int c2)
- double ** [multiplicarM](#) (double **[M1](#), int r1, int c1, double **[M2](#), int r2, int c2)
- int [capturarM](#) (double **[M](#), int r, int c, char *str)
- int [liberar](#) (double **[M](#), int r)
- int [main](#) (int argc, char *argv[])

7.104.1 Function Documentation

7.104.1.1 [capturar\(\)](#)

```

int capturar (
    double * M,
    int r,
    int c,
    char * str)

```

Definition at line 52 of file [Ejemplo040.c](#).

```

00053 {
00054     int i, j;
00055     for(i=0; i<r; i++)
00056         for(j=0; j<c; j++)
00057         {
00058             printf("%s[%d][%d] = ", str, i+1, j+1);
00059             scanf("%lf", M+c*i+j);
00060         }
00061     return 0;
00062 }

```

7.104.1.2 capturarM()

```
int capturarM (  
    double ** M,  
    int r,  
    int c,  
    char * str)
```

Definition at line 129 of file [Ejemplo040.c](#).

```
00130 {  
00131     int i, j;  
00132     for(i=0; i<r; i++)  
00133         for(j=0; j<c; j++)  
00134     {  
00135         printf("%s[%d][%d] = ", str, i+1, j+1);  
00136         scanf("%lf", M[i]+j);  
00137     }  
00138     return 0;  
00139 }
```

Here is the caller graph for this function:



7.104.1.3 crearMat()

```
double * crearMat (  
    int r,  
    int c)
```

Definition at line 4 of file [Ejemplo040.c](#).

```
00005 {  
00006     double *M;  
00007     M = (double*)malloc(r*c*sizeof(double));  
00008     if(M==NULL)  
00009         return NULL;  
00010     return M;  
00011 }
```

Here is the caller graph for this function:



7.104.1.4 crearMatC()

```
double ** crearMatC (  
    int r,  
    int c)
```

Definition at line 13 of file [Ejemplo040.c](#).

```
00014 {
```

```

00015     double **M;
00016     int i;
00017     M = (double**)malloc(r*sizeof(double));
00018     if(M==NULL)
00019         return NULL;
00020     M[0] = (double*)malloc(r*c*sizeof(double));
00021     if(M[0]==NULL)
00022     {
00023         free(M);
00024         return NULL;
00025     }
00026     for(i=1; i<r; i++)
00027         M[i] = M[i-1]+c;
00028     return M;
00029 }

```

Here is the caller graph for this function:



7.104.1.5 crearMatD()

```

double ** crearMatD (
    int r,
    int c)

```

Definition at line 31 of file [Ejemplo040.c](#).

```

00032 {
00033     double **M;
00034     int i;
00035     M = (double**)malloc(r*sizeof(double));
00036     if(M==NULL)
00037         return NULL;
00038     for(i=0; i<r; i++)
00039     {
00040         M[i] = (double*)malloc(c*sizeof(double));
00041         if(M[i]==NULL)
00042         {
00043             for(--i; i>-1; i--)
00044                 free(M[i]);
00045             free(M);
00046             return NULL;
00047         }
00048     }
00049     return M;
00050 }

```

Here is the caller graph for this function:



7.104.1.6 imprimir()

```
int imprimir (  
    double * M,  
    int r,  
    int c)
```

Definition at line 64 of file [Ejemplo040.c](#).

```
00065 {  
00066     int i, j;  
00067     for(i=0; i<r; i++)  
00068     {  
00069         for(j=0; j<c; j++)  
00070             printf("%.4lf\t", M[c*i+j]);  
00071         printf("\n");  
00072     }  
00073     return 0;  
00074 }
```

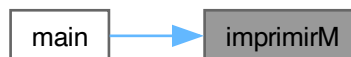
7.104.1.7 imprimirM()

```
int imprimirM (  
    double ** M,  
    int r,  
    int c)
```

Definition at line 76 of file [Ejemplo040.c](#).

```
00077 {  
00078     int i, j;  
00079     for(i=0; i<r; i++)  
00080     {  
00081         for(j=0; j<c; j++)  
00082             printf("%.4lf\t", M[i][j]);  
00083         printf("\n");  
00084     }  
00085     return 0;  
00086 }
```

Here is the caller graph for this function:



7.104.1.8 liberar()

```
int liberar (  
    double ** M,  
    int r)
```

Definition at line 141 of file [Ejemplo040.c](#).

```
00142 {  
00143     int i;  
00144     for(i=0; i<r; i++)  
00145         free(M[i]);  
00146     free(M);  
00147     return 0;  
00148 }
```

Here is the caller graph for this function:



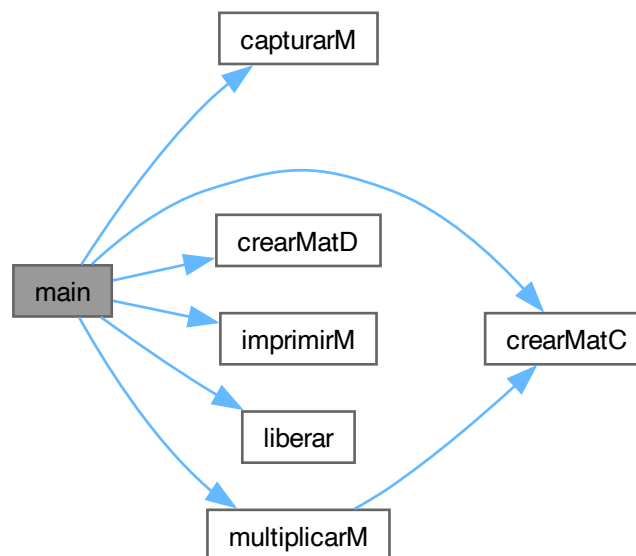
7.104.1.9 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 150 of file [Ejemplo040.c](#).

```
00151 {
00152     int rA, rB, cA, cB;
00153     double **A, **B, **C;
00154     do{
00155         printf("Ingrese el numero de renglones de A: ");
00156         scanf("%d", &rA);
00157     }while(rA<1);
00158     do{
00159         printf("Ingrese el numero de columnas de A: ");
00160         scanf("%d", &cA);
00161     }while(cA<1);
00162     A = crearMatC(rA, cA);
00163     if(A==NULL)
00164         return 1;
00165     capturarM(A, rA, cA, "A");
00166     do{
00167         printf("Ingrese el numero de renglones de B: ");
00168         scanf("%d", &rB);
00169     }while(rB<1);
00170     do{
00171         printf("Ingrese el numero de columnas de B: ");
00172         scanf("%d", &cB);
00173     }while(cB<1);
00174     B = crearMatD(rB, cB);
00175     if(B==NULL)
00176     {
00177         free(A[0]);
00178         free(A);
00179         return 2;
00180     }
00181     capturarM(B, rB, cB, "B");
00182     C = multiplicarM(A, rA, cA, B, rB, cB);
00183     if(C==NULL)
00184     {
00185         free(A[0]);
00186         free(A);
00187         liberar(B, rB);
00188         return 4;
00189     }
00190     printf("A\n");
00191     imprimirM(A, rA, cA);
00192     printf("B\n");
00193     imprimirM(B, rB, cB);
00194     printf("C\n");
00195     imprimirM(C, rA, cB);
00196     free(A[0]);
00197     free(A);
00198     liberar(B, rB);
00199     free(C);
00200     return 0;
00201 }
```

Here is the call graph for this function:



7.104.1.10 multiplicar()

```
double * multiplicar (
    double * M1,
    int r1,
    int c1,
    double * M2,
    int r2,
    int c2)
```

Definition at line 88 of file [Ejemplo040.c](#).

```
00089 {
00090     int i, j, k;
00091     double *M3;
00092     if(c1!=r2)
00093     {
00094         printf("Error: En el tamaño de las matrices.\n");
00095         return NULL;
00096     }
00097     M3 = crearMat(r1, c2);
00098     if(M3==NULL)
00099         return NULL;
00100     for(i=0; i<r1; i++)
00101         for(j=0; j<c2; j++)
00102             for(k=0; M3[i*c1+j]=0; k<c1; k++)
00103                 M3[i*c2+j] += M1[i*c1+k]*M2[k*c2+j];
00104     // M3[i][j] = M3[i][j]+M1[i][k]*M2[k][j]
00105     // M3[i][j] += M1[i][k]*M2[k][j]
00106     // M3[i*c2+j] += M1[i*c1+k]*M2[k*r2+j]
00107     return M3;
00108 }
```

Here is the call graph for this function:



7.104.1.11 multiplicarM()

```
double ** multiplicarM (
    double ** M1,
    int r1,
    int c1,
    double ** M2,
    int r2,
    int c2)
```

Definition at line 110 of file [Ejemplo040.c](#).

```
00111 {
00112     int i, j, k;
00113     double **M3;
00114     if(c1!=r2)
00115     {
00116         printf("Error: En el tamaño de las matrices.\n");
00117         return NULL;
00118     }
00119     M3 = crearMatC(r1, c2);
00120     if(M3==NULL)
00121         return NULL;
00122     for(i=0; i<r1; i++)
00123         for(j=0; j<c2; j++)
00124             for(k=0, M3[i][j]=0; k<c1; k++)
00125                 M3[i][j] += M1[i][k]*M2[k][j];
00126     return M3;
00127 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



7.105 Ejemplo040.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002 #include <stdlib.h>
00003
00004 double *creaMat(int r, int c)
00005 {
00006     double *M;
00007     M = (double*)malloc(r*c*sizeof(double));
00008     if(M==NULL)
00009         return NULL;
00010     return M;
00011 }
00012
00013 double **creaMatC(int r, int c)
00014 {
00015     double **M;
00016     int i;
00017     M = (double**)malloc(r*sizeof(double*));
00018     if(M==NULL)
00019         return NULL;
00020     M[0] = (double*)malloc(r*c*sizeof(double));
00021     if(M[0]==NULL)
00022     {
00023         free(M);
00024         return NULL;
00025     }
00026     for(i=1; i<r; i++)
00027         M[i] = M[i-1]+c;
00028     return M;
00029 }
00030
00031 double **creaMatD(int r, int c)
00032 {
00033     double **M;
00034     int i;
00035     M = (double**)malloc(r*sizeof(double*));
00036     if(M==NULL)
00037         return NULL;
00038     for(i=0; i<r; i++)
00039     {
00040         M[i] = (double*)malloc(c*sizeof(double));
00041         if(M[i]==NULL)
00042         {
00043             for(--i; i>-1; i--)
00044                 free(M[i]);
00045             free(M);
00046             return NULL;
00047         }
00048     }
00049     return M;
00050 }
00051
00052 int capturar(double *M, int r, int c, char *str)
00053 {
00054     int i, j;
00055     for(i=0; i<r; i++)
00056         for(j=0; j<c; j++)
00057         {
00058             printf("%s[%d][%d] = ", str, i+1, j+1);
00059             scanf("%lf", M+c*i+j);
00060         }
00061     return 0;
00062 }
00063
00064 int imprimir(double *M, int r, int c)
00065 {
00066     int i, j;
00067     for(i=0; i<r; i++)
00068     {
00069         for(j=0; j<c; j++)
00070             printf("%.4lf\t", M[c*i+j]);
00071         printf("\n");
00072     }
00073     return 0;
00074 }
00075
00076 int imprimirM(double **M, int r, int c)
00077 {
00078     int i, j;
00079     for(i=0; i<r; i++)
00080     {
00081         for(j=0; j<c; j++)
00082             printf("%.4lf\t", M[i][j]);
00083         printf("\n");

```

```

00084     }
00085     return 0;
00086 }
00087
00088 double *multiplicar(double *M1, int r1, int c1, double *M2, int r2, int c2)
00089 {
00090     int i, j, k;
00091     double *M3;
00092     if(c1!=r2)
00093     {
00094         printf("Error: En el tamaño de las matrices.\n");
00095         return NULL;
00096     }
00097     M3 = crearMat(r1, c2);
00098     if(M3==NULL)
00099         return NULL;
00100     for(i=0; i<r1; i++)
00101         for(j=0; j<c2; j++)
00102             for(k=0, M3[i*c1+j]=0; k<c1; k++)
00103                 M3[i*c2+j] += M1[i*c1+k]*M2[k*c2+j];
00104     // M3[i][j] = M3[i][j]+M1[i][k]*M2[k][j]
00105     // M3[i][j] += M1[i][k]*M2[k][j]
00106     // M3[i*c2+j] += M1[i*c1+k]*M2[k*r2+j]
00107     return M3;
00108 }
00109
00110 double **multiplicarM(double **M1, int r1, int c1, double **M2, int r2, int c2)
00111 {
00112     int i, j, k;
00113     double **M3;
00114     if(c1!=r2)
00115     {
00116         printf("Error: En el tamaño de las matrices.\n");
00117         return NULL;
00118     }
00119     M3 = crearMatC(r1, c2);
00120     if(M3==NULL)
00121         return NULL;
00122     for(i=0; i<r1; i++)
00123         for(j=0; j<c2; j++)
00124             for(k=0, M3[i][j]=0; k<c1; k++)
00125                 M3[i][j] += M1[i][k]*M2[k][j];
00126     return M3;
00127 }
00128
00129 int capturarM(double **M, int r, int c, char *str)
00130 {
00131     int i, j;
00132     for(i=0; i<r; i++)
00133         for(j=0; j<c; j++)
00134         {
00135             printf("%s[%d][%d] = ", str, i+1, j+1);
00136             scanf("%lf", &M[i+j]);
00137         }
00138     return 0;
00139 }
00140
00141 int liberar(double **M, int r)
00142 {
00143     int i;
00144     for(i=0; i<r; i++)
00145         free(M[i]);
00146     free(M);
00147     return 0;
00148 }
00149
00150 int main(int argc, char *argv[])
00151 {
00152     int rA, rB, cA, cB;
00153     double **A, **B, **C;
00154     do{
00155         printf("Ingrese el numero de renglones de A: ");
00156         scanf("%d", &rA);
00157     }while(rA<1);
00158     do{
00159         printf("Ingrese el numero de columnas de A: ");
00160         scanf("%d", &cA);
00161     }while(cA<1);
00162     A = crearMatC(rA, cA);
00163     if(A==NULL)
00164         return 1;
00165     capturarM(A, rA, cA, "A");
00166     do{
00167         printf("Ingrese el numero de renglones de B: ");
00168         scanf("%d", &rB);
00169     }while(rB<1);
00170     do{

```

```

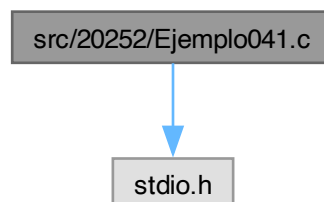
00171     printf("Ingrese el numero de columnas de B: ");
00172     scanf("%d", &cB);
00173     }while(cB<1);
00174     B = crearMatD(rB, cB);
00175     if(B==NULL)
00176     {
00177         free(A[0]);
00178         free(A);
00179         return 2;
00180     }
00181     capturarM(B, rB, cB, "B");
00182     C = multiplicarM(A, rA, cA, B, rB, cB);
00183     if(C==NULL)
00184     {
00185         free(A[0]);
00186         free(A);
00187         liberar(B, rB);
00188         return 4;
00189     }
00190     printf("A\n");
00191     imprimirM(A, rA, cA);
00192     printf("B\n");
00193     imprimirM(B, rB, cB);
00194     printf("C\n");
00195     imprimirM(C, rA, cB);
00196     free(A[0]);
00197     free(A);
00198     liberar(B, rB);
00199     free(C);
00200     return 0;
00201 }

```

7.106 src/20252/Ejemplo041.c File Reference

#include <stdio.h>

Include dependency graph for Ejemplo041.c:



Functions

- int [main](#) (int argc, char *argv[])

7.106.1 Function Documentation

7.106.1.1 main()

```

int main (
    int argc,
    char * argv[])

```

Definition at line 3 of file [Ejemplo041.c](#).

```

00004 {
00005     int i, n;
00006     FILE *fp;
00007     printf("Ingrese el valor de n: ");
00008     scanf("%d", &n);

```

```
00009     fp = fopen("Prueba.txt", "wt");
00010     if (fp==NULL)
00011         return 1;
00012     for (i=0; i<n; i++)
00013         fprintf(fp, "%d\n", i+1);
00014     fclose(fp);
00015     return 0;
00016 }
```

7.107 Ejemplo041.c

[Go to the documentation of this file.](#)

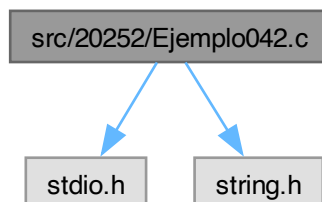
```
00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     int i, n;
00006     FILE *fp;
00007     printf("Ingrese el valor de n: ");
00008     scanf("%d", &n);
00009     fp = fopen("Prueba.txt", "wt");
00010     if (fp==NULL)
00011         return 1;
00012     for (i=0; i<n; i++)
00013         fprintf(fp, "%d\n", i+1);
00014     fclose(fp);
00015     return 0;
00016 }
```

7.108 src/20252/Ejemplo042.c File Reference

```
#include <stdio.h>
```

```
#include <string.h>
```

Include dependency graph for Ejemplo042.c:



Macros

- `#define N 50`

Functions

- `int main (int argc, char *argv[])`

7.108.1 Macro Definition Documentation

7.108.1.1 N

```
#define N 50
```

Definition at line 4 of file [Ejemplo042.c](#).

7.108.2 Function Documentation

7.108.2.1 main()

```
int main (  
    int argc,  
    char * argv[])
```

Definition at line 6 of file [Ejemplo042.c](#).

```
00007 {  
00008     int i, n;  
00009     FILE *fp;  
00010     char filename[N];  
00011     printf("Ingrese el valor de n: ");  
00012     scanf("%d", &n);  
00013     if(argc==2)  
00014         strcpy(filename, argv[1]);  
00015     else  
00016         strcpy(filename, "Prueba.txt");  
00017     fp = fopen(filename, "wt");  
00018     if(fp==NULL)  
00019         return 1;  
00020     for(i=0; i<n; i++)  
00021         fprintf(fp, "%d\n", i+1);  
00022     fclose(fp);  
00023     return 0;  
00024 }
```

7.109 Ejemplo042.c

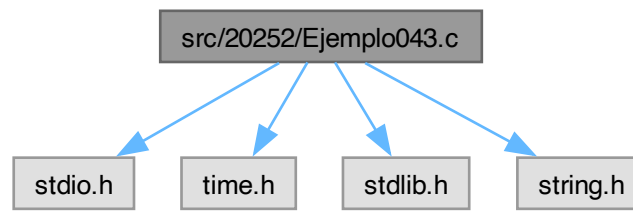
[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>  
00002 #include <string.h>  
00003  
00004 #define N 50  
00005  
00006 int main(int argc, char *argv[])  
00007 {  
00008     int i, n;  
00009     FILE *fp;  
00010     char filename[N];  
00011     printf("Ingrese el valor de n: ");  
00012     scanf("%d", &n);  
00013     if(argc==2)  
00014         strcpy(filename, argv[1]);  
00015     else  
00016         strcpy(filename, "Prueba.txt");  
00017     fp = fopen(filename, "wt");  
00018     if(fp==NULL)  
00019         return 1;  
00020     for(i=0; i<n; i++)  
00021         fprintf(fp, "%d\n", i+1);  
00022     fclose(fp);  
00023     return 0;  
00024 }
```

7.110 src/20252/Ejemplo043.c File Reference

```
#include <stdio.h>  
#include <time.h>  
#include <stdlib.h>  
#include <string.h>
```

Include dependency graph for Ejemplo043.c:



Macros

- `#define N 50`

Functions

- `int main (int argc, char *argv[ ])`

7.110.1 Macro Definition Documentation

7.110.1.1 N

`#define N 50`

Definition at line 6 of file [Ejemplo043.c](#).

7.110.2 Function Documentation

7.110.2.1 main()

```
int main (
    int argc,
    char * argv[ ])
```

Definition at line 8 of file [Ejemplo043.c](#).

```
00009 {
00010     int i, n;
00011     double x;
00012     FILE *fp;
00013     char filename[N];
00014     do{
00015         printf("Ingrese el valor de n: ");
00016         scanf("%d", &n);
00017     }while(n<1);
00018     if(argc==2)
00019         strcpy(filename, argv[1]);
00020     else
00021         strcpy(filename, "Prueba.txt");
00022     fp = fopen(filename, "wt");
00023     if(fp==NULL)
00024         return 1;
00025     srand(time(NULL));
00026     fprintf(fp, "%d\n", n);
00027     for(i=0; i<n; i++)
00028     {
00029         x = (1.0*rand())/RAND_MAX;
00030         fprintf(fp, "%d\t%lf\n", i+1, x);
00031         printf("x[%d] = %lf\n", i+1, x);
00032     }
00033     fclose(fp);
00034     return 0;
00035 }
```

7.111 Ejemplo043.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002 #include <time.h>
00003 #include <stdlib.h>
00004 #include <string.h>
00005
00006 #define N 50
00007
00008 int main(int argc, char *argv[])
00009 {
00010     int i, n;
00011     double x;
00012     FILE *fp;
00013     char filename[N];
00014     do{
00015         printf("Ingrese el valor de n: ");
00016         scanf("%d", &n);
00017     }while(n<1);
00018     if(argc==2)
00019         strcpy(filename, argv[1]);
00020     else
00021         strcpy(filename, "Prueba.txt");
00022     fp = fopen(filename, "wt");
00023     if(fp==NULL)
00024         return 1;
00025     srand(time(NULL));
00026     fprintf(fp, "%d\n", n);
00027     for(i=0; i<n; i++)
00028     {
00029         x = (1.0*rand())/RAND_MAX;
00030         fprintf(fp, "%d\t%lf\n", i+1, x);
00031         printf("x[%d] = %lf\n", i+1, x);
00032     }
00033     fclose(fp);
00034     return 0;
00035 }

```

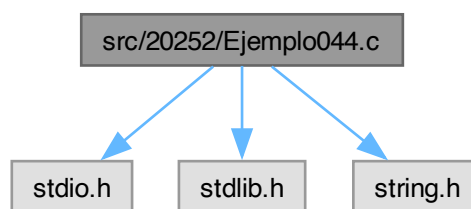
7.112 src/20252/Ejemplo044.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

Include dependency graph for Ejemplo044.c:



Macros

- #define N 50

Functions

- int main (int argc, char *argv[])

7.112.1 Macro Definition Documentation

7.112.1.1 N

#define N 50

Definition at line 5 of file [Ejemplo044.c](#).

7.112.2 Function Documentation

7.112.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 7 of file [Ejemplo044.c](#).

```
00008 {
00009     int i, n, c;
00010     double *x, m;
00011     FILE *fp;
00012     char filename[N];
00013     if(argc==2)
00014         strcpy(filename, argv[1]);
00015     else
00016         strcpy(filename, "Prueba.txt");
00017     fp = fopen(filename, "rt");
00018     if(fp==NULL)
00019         return 1;
00020     fscanf(fp, "%d\n", &n);
00021     printf("n = %d\n", n);
00022     x = (double*)malloc(n*sizeof(double));
00023     if(x==NULL)
00024     {
00025         fclose(fp);
00026         return 2;
00027     }
00028     for(i=0, m=0; i<n; i++)
00029     {
00030         //     fscanf(fp, "%d\t%lf\n", &c, &x[i]);
00031         fscanf(fp, "%d\t%lf\n", &c, &x[i]);
00032         printf("x[%d] = %lf\n", c, x[i]);
00033         m+=x[i];
00034     }
00035     m/=n;
00036     printf("Media = %lf\n", m);
00037     fclose(fp);
00038     return 0;
00039 }
```

7.113 Ejemplo044.c

[Go to the documentation of this file.](#)

```
00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <string.h>
00004
00005 #define N 50
00006
00007 int main(int argc, char *argv[])
00008 {
00009     int i, n, c;
00010     double *x, m;
00011     FILE *fp;
00012     char filename[N];
00013     if(argc==2)
00014         strcpy(filename, argv[1]);
00015     else
00016         strcpy(filename, "Prueba.txt");
00017     fp = fopen(filename, "rt");
00018     if(fp==NULL)
00019         return 1;
00020     fscanf(fp, "%d\n", &n);
00021     printf("n = %d\n", n);
00022     x = (double*)malloc(n*sizeof(double));
00023     if(x==NULL)
00024     {
00025         fclose(fp);
00026         return 2;
00027     }
```

```

00028     for(i=0, m=0; i<n; i++)
00029     {
00030 //         fscanf(fp, "%d\t%lf\n", &c, &x[i]);
00031         fscanf(fp, "%d\t%lf\n", &c, &x[i]);
00032         printf("x[%d] = %lf\n", c, x[i]);
00033         m+=x[i];
00034     }
00035     m/=n;
00036     printf("Media = %lf\n", m);
00037     fclose(fp);
00038     return 0;
00039 }

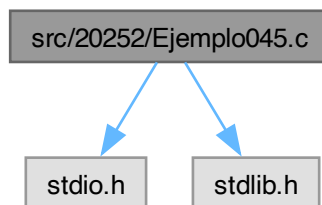
```

7.114 src/20252/Ejemplo045.c File Reference

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

Include dependency graph for Ejemplo045.c:



Macros

- `#define N 50`

Functions

- `int main (int argc, char *argv[])`

7.114.1 Macro Definition Documentation

7.114.1.1 N

```
#define N 50
```

Definition at line 4 of file [Ejemplo045.c](#).

7.114.2 Function Documentation

7.114.2.1 main()

```

int main (
    int argc,
    char * argv[])

```

Definition at line 6 of file [Ejemplo045.c](#).

```

00007 {
00008     char filename[N] = "full_data.csv";
00009     char *str;
00010     char c;
00011     int i, j, nc, nl, *ln, *Ceremony, *Year;
00012     FILE *fp;

```

```

00013     fp = fopen(filename, "rt");
00014     if (fp==NULL)
00015     {
00016         printf("Error al leer el archivo.\n");
00017         return 1;
00018     }
00019     //fscanf(fp, "%s", str);
00020     /*
00021     for(i=0; i<10; i++)
00022     {
00023         fgets(str, NC-1, fp);
00024         printf("%s", str);
00025     }
00026     */
00027     j=0;
00028     i=0;
00029     while((c=fgetc(fp))!=EOF)
00030     {
00031         j++;
00032         if(c==10)
00033             i++;
00034     }
00035     nl = i;
00036     nc = j;
00037     printf("Num L: %d\n", nl);
00038     printf("Num C: %d\n", nc);
00039     str = (char*)malloc((nc+1)*sizeof(char));
00040     if(str==NULL)
00041     {
00042         fclose(fp);
00043         return 1;
00044     }
00045     ln = (int*)calloc(nl, sizeof(int));
00046     if(ln==NULL)
00047     {
00048         fclose(fp);
00049         free(str);
00050         return 2;
00051     }
00052     fseek(fp, 0, SEEK_SET);
00053     for(i=0, j=1; i<nc; i++)
00054     {
00055         str[i] = getc(fp);
00056         if(str[i]==10)
00057             ln[j++] = i+1;
00058     }
00059     str[i] = '\0';
00060     fclose(fp);
00061     for(i=1; i<nl; i++)
00062         str[ln[i]-1] = '\0';
00063     Ceremony = (int*)malloc((nl-1)*sizeof(int));
00064     if(Ceremony==NULL)
00065     {
00066         free(ln);
00067         free(str);
00068         return 3;
00069     }
00070     Year = (int*)malloc((nl-1)*sizeof(int));
00071     if(Year==NULL)
00072     {
00073         free(Ceremony);
00074         free(ln);
00075         free(str);
00076         return 3;
00077     }
00078     for(i=1; i<nl; i++)
00079     {
00080         Ceremony[i-1] = atoi(str+ln[i]);
00081         j=0;
00082         while(str[ln[i]+j]!=9)
00083             j++;
00084         Year[i-1] = atoi(str+ln[i]+j+1);
00085         printf("%d. %d\t%d\n", i, Ceremony[i-1], Year[i-1]);
00086     }
00087     free(Year);
00088     free(Ceremony);
00089     free(ln);
00090     free(str);
00091     return 0;
00092 }

```

7.115 Ejemplo045.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002 #include <stdlib.h>
00003
00004 #define N 50
00005
00006 int main(int argc, char *argv[])
00007 {
00008     char filename[N] = "full_data.csv";
00009     char *str;
00010     char c;
00011     int i, j, nc, nl, *ln, *Ceremony, *Year;
00012     FILE *fp;
00013     fp = fopen(filename, "rt");
00014     if (fp==NULL)
00015     {
00016         printf("Error al leer el archivo.\n");
00017         return 1;
00018     }
00019     //fscanf(fp, "%s", str);
00020     /*
00021     for(i=0; i<10; i++)
00022     {
00023         fgets(str, NC-1, fp);
00024         printf("%s", str);
00025     }
00026     */
00027     j=0;
00028     i=0;
00029     while((c=fgetc(fp))!=EOF)
00030     {
00031         j++;
00032         if(c==10)
00033             i++;
00034     }
00035     nl = i;
00036     nc = j;
00037     printf("Num L: %d\n", nl);
00038     printf("Num C: %d\n", nc);
00039     str = (char*)malloc((nc+1)*sizeof(char));
00040     if(str==NULL)
00041     {
00042         fclose(fp);
00043         return 1;
00044     }
00045     ln = (int*)calloc(nl, sizeof(int));
00046     if(ln==NULL)
00047     {
00048         fclose(fp);
00049         free(str);
00050         return 2;
00051     }
00052     fseek(fp, 0, SEEK_SET);
00053     for(i=0, j=1; i<nc; i++)
00054     {
00055         str[i] = getc(fp);
00056         if(str[i]==10)
00057             ln[j++] = i+1;
00058     }
00059     str[i] = '\0';
00060     fclose(fp);
00061     for(i=1; i<nl; i++)
00062         str[ln[i]-1] = '\0';
00063     Ceremony = (int*)malloc((nl-1)*sizeof(int));
00064     if(Ceremony==NULL)
00065     {
00066         free(ln);
00067         free(str);
00068         return 3;
00069     }
00070     Year = (int*)malloc((nl-1)*sizeof(int));
00071     if(Year==NULL)
00072     {
00073         free(Ceremony);
00074         free(ln);
00075         free(str);
00076         return 3;
00077     }
00078     for(i=1; i<nl; i++)
00079     {
00080         Ceremony[i-1] = atoi(str+ln[i]);
00081         j=0;
00082         while(str[ln[i]+j]!=9)
00083             j++;
00084         Year[i-1] = atoi(str+ln[i]+j+1);
00085         printf("%d. %d\t%d\n", i, Ceremony[i-1], Year[i-1]);
00086     }
00087     free(Year);

```

```

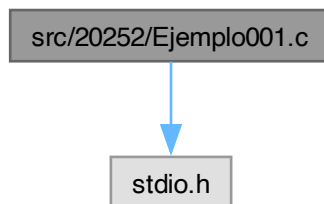
00088     free(Ceremony);
00089     free(ln);
00090     free(str);
00091     return 0;
00092 }

```

7.116 src/20252/Ejemplo001.c File Reference

```
#include <stdio.h>
```

Include dependency graph for Ejemplo001.c:



Functions

- int [main](#) (int argc, char *argv[])

7.116.1 Function Documentation

7.116.1.1 main()

```

int main (
    int argc,
    char * argv[])

```

Definition at line 3 of file [Ejemplo001.c](#).

```

00004 {
00005     int a = 10, b = -250, n = 4;
00006     printf("%d\t%d\n", a, b);
00007     printf("%d\t%d\n", a, b);
00008     printf("%05d\t%05d\n", a, b);
00009     printf("%0*d\t%0*d\n", 5, a, 5, b);
00010     printf("%0*d\t%0*d\n", n, a, n, b);
00011     return 0;
00012 }

```

7.117 Ejemplo001.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     int a = 10, b = -250, n = 4;
00006     printf("%d\t%d\n", a, b);
00007     printf("%d\t%d\n", a, b);
00008     printf("%05d\t%05d\n", a, b);
00009     printf("%0*d\t%0*d\n", 5, a, 5, b);
00010     printf("%0*d\t%0*d\n", n, a, n, b);
00011     return 0;
00012 }

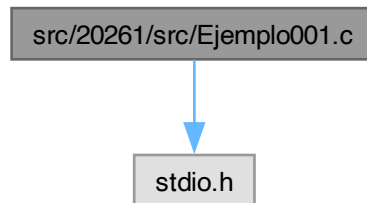
```


7.118 src/20261/src/Ejemplo001.c File Reference

Hola mundo mínimo: imprime un mensaje en consola.

```
#include <stdio.h>
```

Include dependency graph for Ejemplo001.c:



Functions

- `int main (int argc, char *argv[ ])`
Punto de entrada del programa.

7.118.1 Detailed Description

Hola mundo mínimo: imprime un mensaje en consola.

Programa introductorio en C para verificar compilación y ejecución.

Entrada

Ninguna.

Salida

Imprime la cadena "Universidad de Guanajuato" seguida de salto de línea.

Complejidad

Tiempo: $O(1)$. Memoria: $O(1)$.

Note

Compilación sugerida:

```
gcc Ejemplo001.c -o Ejemplo001
./Ejemplo001
```

Definition in file [Ejemplo001.c](#).

7.118.2 Function Documentation

7.118.2.1 main()

```
int main (  
    int argc,  
    char * argv[ ])
```

Punto de entrada del programa.

Parameters

<code>argc</code>	Cantidad de argumentos de línea de comandos.
-------------------	--

<i>argv</i>	Arreglo de cadenas con argumentos.
-------------	------------------------------------

Returns

0 si termina correctamente.

Definition at line 33 of file [Ejemplo001.c](#).

```
00034 {
00035     (void)argc;
00036     (void)argv;
00037
00038     printf("Universidad de Guanajuato\n");
00039     return 0;
00040 }
```

7.119 Ejemplo001.c

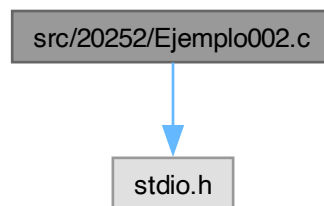
[Go to the documentation of this file.](#)

```
00001
00024
00025 #include <stdio.h>
00026
00033 int main(int argc, char *argv[])
00034 {
00035     (void)argc;
00036     (void)argv;
00037
00038     printf("Universidad de Guanajuato\n");
00039     return 0;
00040 }
```

7.120 src/20252/Ejemplo002.c File Reference

```
#include <stdio.h>
```

Include dependency graph for Ejemplo002.c:

**Functions**

- int [main](#) (int argc, char *argv[])

7.120.1 Function Documentation

7.120.1.1 main()

```
int main (
    int argc,
    char * argv[ ])
```

Definition at line 3 of file [Ejemplo002.c](#).

```
00004 {
00005     int N, M, X, Y, T;
00006     scanf("%d %d %d %d", &N, &M, &X, &Y);
00007     T = (Y*N+M) / (X+Y);
00008     T = (T>N?N:T);
00009     printf("%d\n", T);
00010     return 0;
00011 }
```

7.121 Ejemplo002.c

[Go to the documentation of this file.](#)

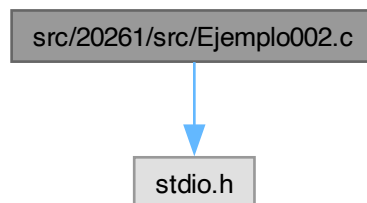
```
00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     int N, M, X, Y, T;
00006     scanf("%d %d %d %d", &N, &M, &X, &Y);
00007     T = (Y*N+M) / (X+Y);
00008     T = (T>N?N:T);
00009     printf("%d\n", T);
00010     return 0;
00011 }
```

7.122 src/20261/src/Ejemplo002.c File Reference

Codificador de prioridad de interrupción para 4 dispositivos (D1..D4).

#include <stdio.h>

Include dependency graph for Ejemplo002.c:



Functions

- int [main](#) (int argc, char *argv[])

Punto de entrada. Implementa el codificador de prioridad 4→3 bits.

7.122.1 Detailed Description

Codificador de prioridad de interrupción para 4 dispositivos (D1..D4).

Se reciben 4 solicitudes de interrupción (una por dispositivo). La prioridad se asigna de manera que el dispositivo con número más bajo tiene mayor prioridad: D1 > D2 > D3 > D4.

La salida es el número del dispositivo seleccionado codificado en binario (3 bits):

- Ninguno -> 0 (000)
- D1 -> 1 (001)
- D2 -> 2 (010)

- D3 -> 3 (011)
- D4 -> 4 (100)

Entrada

Cuatro enteros (0/1) en este orden:

D4 D3 D2 D1

Salida

Un renglón con:

- el valor decimal seleccionado (0..4)
- y los bits de salida (b2 b1 b0) solo como depuración.

Precondiciones

Las entradas deben ser booleanas (0 o 1) para que el comportamiento sea el esperado.

Complejidad

Tiempo: $O(1)$. Memoria: $O(1)$.

Definition in file [Ejemplo002.c](#).

7.122.2 Function Documentation

7.122.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Punto de entrada. Implementa el codificador de prioridad 4→3 bits.

Parameters

<i>argc</i>	Cantidad de argumentos de línea de comandos (no se usa).
<i>argv</i>	Argumentos de línea de comandos (no se usa).

Returns

0 si finaliza correctamente; 1 si falla la lectura.

Definition at line 43 of file [Ejemplo002.c](#).

```
00044 {
00045     (void)argc;
00046     (void)argv;
00047
00048     /* Entradas: solicitud de interrupción por dispositivo (0/1). */
00049     int D4, D3, D2, D1;
00050
00051     /* Bits de salida: b2 b1 b0 (representan 0..4). */
00052     int b2, b1, b0;
00053
00054     /* Salida decimal equivalente a (b2 b1 b0). */
00055     int salida;
00056
00057     /* Se leen las entradas en el orden: D4 D3 D2 D1. */
00058     if (scanf("%d %d %d %d", &D4, &D3, &D2, &D1) != 4) {
00059         printf("Error: se esperaban 4 valores (D4 D3 D2 D1).\n");
00060         return 1;
00061     }
00062
00063     /*
00064         Lógica de prioridad (D1 mayor prioridad):
```

```

00065
00066     Para "habilitar" un dispositivo de menor prioridad, se exige que NO haya
00067     dispositivos de mayor prioridad activos.
00068
00069     b2 debe ser 1 solo cuando gana D4 (100):
00070     b2 = D4 · ¬D3 · ¬D2 · ¬D1
00071 */
00072 b2 = D4 && (!D3) && (!D2) && (!D1);
00073
00074 /*
00075     b1 es 1 cuando la salida es 2 (010) o 3 (011):
00076     - Para 3: D3 · ¬D2 · ¬D1
00077     - Para 2: D2 · ¬D1
00078     => b1 = (D3 · ¬D2 · ¬D1) + (D2 · ¬D1)
00079 */
00080 b1 = (D3 && (!D2) && (!D1)) || (D2 && (!D1));
00081
00082 /*
00083     b0 es 1 cuando la salida es 1 (001) o 3 (011):
00084     - Para 1: D1
00085     - Para 3: D3 · ¬D2 · ¬D1
00086     => b0 = D1 + (D3 · ¬D2 · ¬D1)
00087 */
00088 b0 = (D1) || (D3 && (!D2) && (!D1));
00089
00090 /* Convertir (b2 b1 b0) a decimal: salida = b2*4 + b1*2 + b0. */
00091 salida = (b2 << 2) | (b1 << 1) | b0;
00092
00093 printf("Salida = %d (%ld %ld %ld)\n", salida, b2, b1, b0);
00094 return 0;
00095 }

```

7.123 Ejemplo002.c

[Go to the documentation of this file.](#)

```

00001
00034
00035 #include <stdio.h>
00036
00043 int main(int argc, char *argv[])
00044 {
00045     (void)argc;
00046     (void)argv;
00047
00048     /* Entradas: solicitud de interrupción por dispositivo (0/1). */
00049     int D4, D3, D2, D1;
00050
00051     /* Bits de salida: b2 b1 b0 (representan 0..4). */
00052     int b2, b1, b0;
00053
00054     /* Salida decimal equivalente a (b2 b1 b0). */
00055     int salida;
00056
00057     /* Se leen las entradas en el orden: D4 D3 D2 D1. */
00058     if (scanf("%d %d %d %d", &D4, &D3, &D2, &D1) != 4) {
00059         printf("Error: se esperaban 4 valores (D4 D3 D2 D1).\n");
00060         return 1;
00061     }
00062
00063     /*
00064     Lógica de prioridad (D1 mayor prioridad):
00065
00066     Para "habilitar" un dispositivo de menor prioridad, se exige que NO haya
00067     dispositivos de mayor prioridad activos.
00068
00069     b2 debe ser 1 solo cuando gana D4 (100):
00070     b2 = D4 · ¬D3 · ¬D2 · ¬D1
00071 */
00072 b2 = D4 && (!D3) && (!D2) && (!D1);
00073
00074 /*
00075     b1 es 1 cuando la salida es 2 (010) o 3 (011):
00076     - Para 3: D3 · ¬D2 · ¬D1
00077     - Para 2: D2 · ¬D1
00078     => b1 = (D3 · ¬D2 · ¬D1) + (D2 · ¬D1)
00079 */
00080 b1 = (D3 && (!D2) && (!D1)) || (D2 && (!D1));
00081
00082 /*
00083     b0 es 1 cuando la salida es 1 (001) o 3 (011):
00084     - Para 1: D1
00085     - Para 3: D3 · ¬D2 · ¬D1
00086     => b0 = D1 + (D3 · ¬D2 · ¬D1)
00087 */

```

```

00088     b0 = (D1) || (D3 && (!D2) && (!D1));
00089
00090     /* Convertir (b2 b1 b0) a decimal: salida = b2*4 + b1*2 + b0. */
00091     salida = (b2 << 2) | (b1 << 1) | b0;
00092
00093     printf("Salida = %d (%1d %1d %1d)\n", salida, b2, b1, b0);
00094     return 0;
00095 }

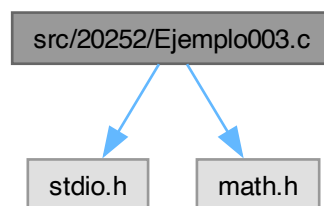
```

7.124 src/20252/Ejemplo003.c File Reference

```
#include <stdio.h>
```

```
#include <math.h>
```

Include dependency graph for Ejemplo003.c:



Functions

- `int main (int argc, char *argv[])`

7.124.1 Function Documentation

7.124.1.1 main()

```

int main (
    int argc,
    char * argv[] )

```

Definition at line 4 of file [Ejemplo003.c](#).

```

00005 {
00006     float a, b, c, x1r, x2r, x1i, x2i, r;
00007     printf("Ingrese el valor de a: ");
00008     scanf("%f", &a);
00009     printf("Ingrese el valor de b: ");
00010     scanf("%f", &b);
00011     printf("Ingrese el valor de c: ");
00012     scanf("%f", &c);
00013     r = b*b-4*a*c;
00014     x1r=(-b+(r>=0?sqrt(r):0))/(2*a);
00015     x2r=(-b-(r>=0?sqrt(r):0))/(2*a);
00016     x1i=(r>=0?0:sqrt(-r)/(2*a));
00017     x2i=-x1i;
00018     printf("x1=%.2f%+.2fi\n", x1r, x1i);
00019     printf("x2=%.2f%+.2fi\n", x2r, x2i);
00020     return 0;
00021 }

```

7.125 Ejemplo003.c

[Go to the documentation of this file.](#)

```

00001 #include<stdio.h>
00002 #include<math.h>

```

```

00003
00004 int main(int argc, char*argv[])
00005 {
00006     float a, b, c, x1r, x2r, x1i, x2i, r;
00007     printf("Ingrese el valor de a: ");
00008     scanf("%f", &a);
00009     printf("Ingrese el valor de b: ");
00010     scanf("%f", &b);
00011     printf("Ingrese el valor de c: ");
00012     scanf("%f", &c);
00013     r = b*b-4*a*c;
00014     x1r=(-b+(r>=0?sqrt(r):0))/(2*a);
00015     x2r=(-b-(r>=0?sqrt(r):0))/(2*a);
00016     x1i=(r>=0?0:sqrt(-r)/(2*a));
00017     x2i=-x1i;
00018     printf("x1=%.2f%.2fi\n", x1r, x1i);
00019     printf("x2=%.2f%.2fi\n", x2r, x2i);
00020     return 0;
00021 }

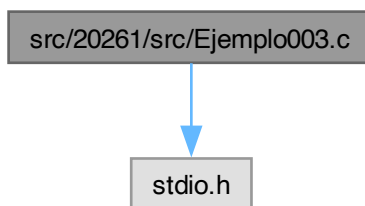
```

7.126 src/20261/src/Ejemplo003.c File Reference

Máximo número de Pokémon que pueden evolucionarse con venta de no evolucionados.

#include <stdio.h>

Include dependency graph for Ejemplo003.c:



Functions

- int `main` (int argc, char *argv[])

Punto de entrada. Calcula el máximo de evoluciones.

7.126.1 Detailed Description

Máximo número de Pokémon que pueden evolucionarse con venta de no evolucionados.

Se tienen N Pokémon y M barras de dulce. Evolucionar un Pokémon cuesta X dulces. Vender un Pokémon NO evolucionado produce Y dulces. No se puede vender un Pokémon evolucionado. Se pide el máximo número de evoluciones posibles.

Este código calcula una cota óptima usando la desigualdad: $M + (N - P) \cdot Y \geq P \cdot X$ que se transforma en: $P \leq (M + N \cdot Y) / (X + Y)$ y finalmente $P = \min(N, \text{floor}((M + N \cdot Y) / (X + Y)))$.

Entrada

El enunciado indica:

N M X Y

Warning

Este programa lee en el orden:

`M N X Y`

(intercambia N y M respecto al enunciado). Si vas a usar la entrada como está definida en el problema, ajusta el `scanf` o el orden de variables.

Salida

Un entero: máximo número de Pokémon que se pueden evolucionar.

Restricciones

1 N, M, X, Y 1e9.

Note

Con límites de 1e9, productos como $N*Y$ pueden exceder 32 bits. Conviene usar tipos de 64 bits (long long) si se requiere robustez numérica.

Complejidad

Tiempo: $O(1)$. Memoria: $O(1)$.

Definition in file [Ejemplo003.c](#).

7.126.2 Function Documentation**7.126.2.1 main()**

```
int main (
    int argc,
    char * argv[])
```

Punto de entrada. Calcula el máximo de evoluciones.

Parameters

<code>argc</code>	No usado.
<code>argv</code>	No usado.

Returns

0 si finaliza correctamente.

Definition at line 52 of file [Ejemplo003.c](#).

```
00053 {
00054     (void)argc;
00055     (void)argv;
00056
00057     /*
00058      * VARIABLES (según como ESTE código está leyendo la entrada):
00059      * M = cantidad de Pokémon capturados (enunciado lo llama N)
00060      * N = cantidad de barras de dulce (candies) (enunciado lo llama M)
00061      * X = costo en dulces para evolucionar 1 Pokémon
00062      * Y = dulces que se ganan al vender 1 Pokémon (NO evolucionado)
00063      * P = número máximo de Pokémon que se pueden evolucionar (respuesta)
00064      */
00065     int M, N, X, Y, P;
00066
00067     /* Lee: M N X Y (nota: el enunciado usualmente pide N M X Y). */
00068     scanf("%d %d %d %d", &M, &N, &X, &Y);
00069
00070     /*
00071      * Si evolucionamos P Pokémon y vendemos los (M - P) restantes:
00072      * dulces disponibles = N + (M - P)*Y
00073      * Requisito para evolucionar P:
```



```

00074      N + (M - P)*Y >= P*X
00075      => N + M*Y >= P*(X + Y)
00076      => P <= floor((N + M*Y)/(X + Y))
00077
00078      Además, P no puede ser mayor que M (no puedes evolucionar más Pokémon de los que tienes).
00079      */
00080      P = (N + Y * M) / (X + Y);
00081
00082      /* Imprime min(P, M). */
00083      printf("%d\n", P > M ? M : P);
00084      return 0;
00085 }

```

7.127 Ejemplo003.c

[Go to the documentation of this file.](#)

```

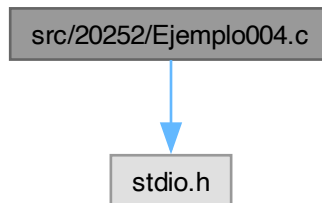
00001
00043
00044 #include <stdio.h>
00045
00052 int main(int argc, char *argv[])
00053 {
00054     (void)argc;
00055     (void)argv;
00056
00057     /*
00058      VARIABLES (según como ESTE código está leyendo la entrada):
00059      M = cantidad de Pokémon capturados (enunciado lo llama N)
00060      N = cantidad de barras de dulce (candies) (enunciado lo llama M)
00061      X = costo en dulces para evolucionar 1 Pokémon
00062      Y = dulces que se ganan al vender 1 Pokémon (NO evolucionado)
00063      P = número máximo de Pokémon que se pueden evolucionar (respuesta)
00064      */
00065     int M, N, X, Y, P;
00066
00067     /* Lee: M N X Y (nota: el enunciado usualmente pide N M X Y). */
00068     scanf("%d %d %d %d", &M, &N, &X, &Y);
00069
00070     /*
00071      Si evolucionamos P Pokémon y vendemos los (M - P) restantes:
00072      dulces disponibles = N + (M - P)*Y
00073      Requisito para evolucionar P:
00074      N + (M - P)*Y >= P*X
00075      => N + M*Y >= P*(X + Y)
00076      => P <= floor((N + M*Y)/(X + Y))
00077
00078      Además, P no puede ser mayor que M (no puedes evolucionar más Pokémon de los que tienes).
00079      */
00080     P = (N + Y * M) / (X + Y);
00081
00082     /* Imprime min(P, M). */
00083     printf("%d\n", P > M ? M : P);
00084     return 0;
00085 }

```

7.128 src/20252/Ejemplo004.c File Reference

#include <stdio.h>

Include dependency graph for Ejemplo004.c:



Functions

- int [main](#) (int argc, char *argv[])

7.128.1 Function Documentation

7.128.1.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 3 of file [Ejemplo004.c](#).

```
00004 {
00005     float T1, T2, T3, T4, T;
00006     int B;
00007     scanf("%f %f %f %f %f", &T1, &T2, &T3, &T4, &T);
00008     B = ((T4>T)<<3) | ((T3>T)<<2) | ((T2>T)<<1) | (T1>T);
00009     switch(B)
00010     {
00011     case 15:
00012         printf("Muy ");
00013     case 14:
00014         printf("Baja");
00015         break;
00016     case 12:
00017         printf("Templada");
00018         break;
00019     case 0:
00020         printf("Muy ");
00021     default:
00022         printf("Alta");
00023     }
00024     printf(".\n");
00025     return 0;
00026 }
```

7.129 Ejemplo004.c

[Go to the documentation of this file.](#)

```
00001 #include<stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     float T1, T2, T3, T4, T;
00006     int B;
00007     scanf("%f %f %f %f %f", &T1, &T2, &T3, &T4, &T);
00008     B = ((T4>T)<<3) | ((T3>T)<<2) | ((T2>T)<<1) | (T1>T);
```

```

00009     switch (B)
00010     {
00011     case 15:
00012         printf("Muy ");
00013     case 14:
00014         printf("Baja");
00015         break;
00016     case 12:
00017         printf("Templada");
00018         break;
00019     case 0:
00020         printf("Muy ");
00021     default:
00022         printf("Alta");
00023     }
00024     printf(".\n");
00025     return 0;
00026 }

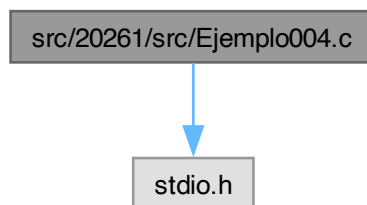
```

7.130 src/20261/src/Ejemplo004.c File Reference

Mínimo y máximo posible de niños en equipo rojo tras exactamente K cambios.

```
#include <stdio.h>
```

Include dependency graph for Ejemplo004.c:



Functions

- int `main` (int argc, char *argv[])

Punto de entrada. Calcula mínimo y máximo tamaño del equipo rojo.

7.130.1 Detailed Description

Mínimo y máximo posible de niños en equipo rojo tras exactamente K cambios.

N niños se dividen en dos equipos: R en rojo y N-R en azul. Exactamente K niños se cambian de equipo (cada niño como máximo una vez). Se pide:

- mínimo posible de niños en rojo al final
- máximo posible de niños en rojo al final

Modelado: Sea x = # que cambian de rojo->azul. Entonces $K-x$ cambian de azul->rojo. Rojo final: $R_{\text{final}} = R - x + (K - x) = R + K - 2x$ con restricciones: $0 \leq x \leq R$ $0 \leq K-x \leq N-R$

Entrada

N R K

Salida

Dos enteros: mínimo y máximo posibles en rojo.

Restricciones

1 N 1e5, 0 R, K N.

Complejidad

Tiempo: $O(1)$. Memoria: $O(1)$.

Definition in file [Ejemplo004.c](#).

7.130.2 Function Documentation**7.130.2.1 main()**

```
int main (
    int argc,
    char * argv[])
```

Punto de entrada. Calcula mínimo y máximo tamaño del equipo rojo.

Parameters

<i>argc</i>	No usado.
<i>argv</i>	No usado.

Returns

0 si finaliza correctamente.

Definition at line 42 of file [Ejemplo004.c](#).

```
00043 {
00044     (void)argc;
00045     (void)argv;
00046
00047     int N, R, K, Rmn, Rmx;
00048
00049     scanf("%d %d %d", &N, &R, &K);
00050
00051     /*
00052      * Mínimo:
00053      * Se logra maximizando x (tantos rojos como sea posible se van a azul).
00054      * x_max = min(K, R)
00055      * Rmn = R + K - 2*x_max
00056      * Esto equivale a |R - K|.
00057      * El código implementa |R-K| sin if usando el signo:
00058      * (2*(R>=K)-1) es +1 si R>=K, -1 si R<K.
00059      */
00060     Rmn = (2*(R >= K) - 1) * (R - K);
00061
00062     /*
00063      * Máximo:
00064      * Se logra minimizando x (que se vayan lo menos posible del rojo).
00065      * x_min = max(0, K - (N - R))
00066      * Si K <= (N - R): Rmx = R + K
00067      * Si K > (N - R): Rmx = 2N - (R + K)
00068      * El código evita if usando booleanos.
00069      */
00070     Rmx = ((N - R) < K) * 2 * N
00071           + (2 * ((N - R) >= K) - 1) * (R + K);
00072
00073     printf("%d\t%d\n", Rmn, Rmx);
00074     return 0;
00075 }
```

7.131 Ejemplo004.c

[Go to the documentation of this file.](#)

```
00001
00033
00034 #include <stdio.h>
00035
```

```

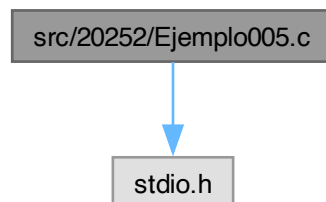
00042 int main(int argc, char *argv[])
00043 {
00044     (void)argc;
00045     (void)argv;
00046
00047     int N, R, K, Rmn, Rmx;
00048
00049     scanf("%d %d %d", &N, &R, &K);
00050
00051     /*
00052     Mínimo:
00053     Se logra maximizando x (tantos rojos como sea posible se van a azul).
00054     x_max = min(K, R)
00055     Rmn = R + K - 2*x_max
00056     Esto equivale a |R - K|.
00057     El código implementa |R-K| sin if usando el signo:
00058     (2*(R>=K)-1) es +1 si R>=K, -1 si R<K.
00059     */
00060     Rmn = (2*(R >= K) - 1) * (R - K);
00061
00062     /*
00063     Máximo:
00064     Se logra minimizando x (que se vayan lo menos posible del rojo).
00065     x_min = max(0, K - (N - R))
00066     Si K <= (N - R): Rmx = R + K
00067     Si K > (N - R): Rmx = 2N - (R + K)
00068     El código evita if usando booleanos.
00069     */
00070     Rmx = ((N - R) < K) * 2 * N
00071           + (2 * ((N - R) >= K) - 1) * (R + K);
00072
00073     printf("%d\t%d\n", Rmn, Rmx);
00074     return 0;
00075 }

```

7.132 src/20252/Ejemplo005.c File Reference

#include <stdio.h>

Include dependency graph for Ejemplo005.c:



Functions

- int [main](#) (int argc, char *argv[])

7.132.1 Function Documentation

7.132.1.1 main()

```

int main (
    int argc,
    char * argv[])

```

Definition at line 3 of file [Ejemplo005.c](#).

```

00004 {
00005     // s es el signo

```

```

00006 // d es el denominador
00007 // n es el numero de terminos
00008 // i es el contador
00009 // pi es el valor de pi
00010 int n, i, s, d;
00011 float pi;
00012 do{
00013     printf("Ingrese el numero de terminos: ");
00014     scanf("%d", &n);
00015 }while(n<=0);
00016 for(i=0, pi=0; i<n; i++)
00017 {
00018     s = 1-2*(i%2);
00019     d = 2*i+1;
00020     pi+=(s*1.0/d);
00021 }
00022 pi*=4;
00023 printf("PI = %f\n", pi);
00024 return 0;
00025 }

```

7.133 Ejemplo005.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     // s es el signo
00006     // d es el denominador
00007     // n es el numero de terminos
00008     // i es el contador
00009     // pi es el valor de pi
00010     int n, i, s, d;
00011     float pi;
00012     do{
00013         printf("Ingrese el numero de terminos: ");
00014         scanf("%d", &n);
00015     }while(n<=0);
00016     for(i=0, pi=0; i<n; i++)
00017     {
00018         s = 1-2*(i%2);
00019         d = 2*i+1;
00020         pi+=(s*1.0/d);
00021     }
00022     pi*=4;
00023     printf("PI = %f\n", pi);
00024     return 0;
00025 }

```

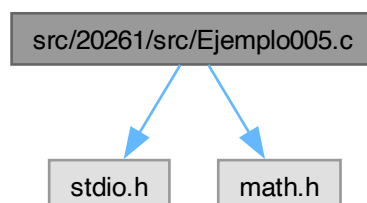
7.134 src/20261/src/Ejemplo005.c File Reference

Solución de la ecuación cuadrática con raíces reales o complejas.

```
#include <stdio.h>
```

```
#include <math.h>
```

Include dependency graph for Ejemplo005.c:



Functions

- `int main (int argc, char *argv[ ])`

Punto de entrada. Calcula y muestra raíces (reales o complejas).

7.134.1 Detailed Description

Solución de la ecuación cuadrática con raíces reales o complejas.

Resuelve: $ax^2 + bx + c = 0$ usando el discriminante: $\Delta = b^2 - 4ac$

Casos:

- $\Delta > 0$: dos raíces reales distintas
- $\Delta = 0$: una raíz real doble
- $\Delta < 0$: dos raíces complejas conjugadas

Para $\Delta < 0$: $\text{sqrt}() = i * \text{sqrt}(-)$ y las raíces se expresan como: $x = (-b \pm i * \text{sqrt}(-)) / (2a)$

Entrada

Tres valores float, leídos por consola (a, b, c).

Salida

Imprime las dos raíces en forma: $x = \text{real} + \text{imag} * i$

Warning

Si $a = 0$ la ecuación no es cuadrática; este programa divide entre $(2a)$.

Complejidad

Tiempo: $O(1)$. Memoria: $O(1)$.

Definition in file [Ejemplo005.c](#).

7.134.2 Function Documentation

7.134.2.1 main()

```
int main (
    int argc,
    char * argv[ ])
```

Punto de entrada. Calcula y muestra raíces (reales o complejas).

Parameters

<i>argc</i>	No usado.
<i>argv</i>	No usado.

Returns

0 si finaliza correctamente.

Definition at line 44 of file [Ejemplo005.c](#).

```
00045 {
00046     (void)argc;
00047     (void)argv;
00048
00049     float a, b, c;           /* coeficientes */
00050     float r;                 /* discriminante */
00051     float x1r, x1i, x2r, x2i; /* partes real/imag de cada raíz */
```

```

00052
00053     printf("Ingrese el termino cuadratico: ");
00054     scanf("%f", &a);
00055
00056     printf("Ingrese el termino lineal: ");
00057     scanf("%f", &b);
00058
00059     printf("Ingrese el termino independiente: ");
00060     scanf("%f", &c);
00061
00062     /* = b^2 - 4ac */
00063     r = b*b - 4*a*c;
00064
00065     /*
00066         Para >= 0:
00067         x1 = (-b + sqrt()) / (2a), parte imaginaria 0
00068         x2 = (-b - sqrt()) / (2a), parte imaginaria 0
00069
00070         Para < 0:
00071         parte real: (-b)/(2a)
00072         parte imag: ± sqrt(-)/(2a)
00073     */
00074     x1r = (-b + (r >= 0 ? sqrt(r) : 0)) / (2*a);
00075     x1i = (r >= 0 ? 0 : sqrt(-r) / (2*a));
00076
00077     x2r = (-b - (r >= 0 ? sqrt(r) : 0)) / (2*a);
00078     x2i = (r >= 0 ? 0 : -x1i); /* conjugada */
00079
00080     printf("x1 = %.4f%+.4fi\n", x1r, x1i);
00081     printf("x2 = %.4f%+.4fi\n", x2r, x2i);
00082
00083     return 0;
00084 }

```

7.135 Ejemplo005.c

[Go to the documentation of this file.](#)

```

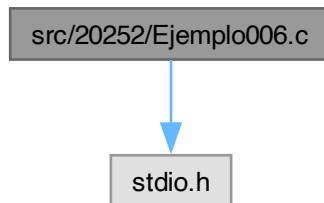
00001
00034
00035 #include <stdio.h>
00036 #include <math.h> /* sqrt() */
00037
00044 int main(int argc, char *argv[])
00045 {
00046     (void)argc;
00047     (void)argv;
00048
00049     float a, b, c; /* coeficientes */
00050     float r; /* discriminante */
00051     float x1r, x1i, x2r, x2i; /* partes real/imag de cada raíz */
00052
00053     printf("Ingrese el termino cuadratico: ");
00054     scanf("%f", &a);
00055
00056     printf("Ingrese el termino lineal: ");
00057     scanf("%f", &b);
00058
00059     printf("Ingrese el termino independiente: ");
00060     scanf("%f", &c);
00061
00062     /* = b^2 - 4ac */
00063     r = b*b - 4*a*c;
00064
00065     /*
00066         Para >= 0:
00067         x1 = (-b + sqrt()) / (2a), parte imaginaria 0
00068         x2 = (-b - sqrt()) / (2a), parte imaginaria 0
00069
00070         Para < 0:
00071         parte real: (-b)/(2a)
00072         parte imag: ± sqrt(-)/(2a)
00073     */
00074     x1r = (-b + (r >= 0 ? sqrt(r) : 0)) / (2*a);
00075     x1i = (r >= 0 ? 0 : sqrt(-r) / (2*a));
00076
00077     x2r = (-b - (r >= 0 ? sqrt(r) : 0)) / (2*a);
00078     x2i = (r >= 0 ? 0 : -x1i); /* conjugada */
00079
00080     printf("x1 = %.4f%+.4fi\n", x1r, x1i);
00081     printf("x2 = %.4f%+.4fi\n", x2r, x2i);
00082
00083     return 0;
00084 }

```


7.136 src/20252/Ejemplo006.c File Reference

```
#include <stdio.h>
```

Include dependency graph for Ejemplo006.c:



Functions

- `int main (int argc, char *argv[])`

7.136.1 Function Documentation

7.136.1.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 3 of file [Ejemplo006.c](#).

```

00004 {
00005     int n, i, den;
00006     float x, ex, num;
00007     printf("Ingrese el valor de x: ");
00008     scanf("%f", &x);
00009     do{
00010         printf("Ingrese el numero de terminos: ");
00011         scanf("%d", &n);
00012     }while(n<1);
00013     for(i=0, ex=0, num=1, den=1; i<n; i++)
00014     {
00015         ex+=(num/den);
00016         num*=x;
00017         den*=i+1;
00018     }
00019     printf("exp(%f) = %f\n", x, ex);
00020     return 0;
00021 }
```

7.137 Ejemplo006.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     int n, i, den;
00006     float x, ex, num;
00007     printf("Ingrese el valor de x: ");
00008     scanf("%f", &x);
00009     do{
00010         printf("Ingrese el numero de terminos: ");
00011         scanf("%d", &n);
00012     }while(n<1);
00013     for(i=0, ex=0, num=1, den=1; i<n; i++)
```

```

00014      {
00015          ex+=(num/den);
00016          num*=x;
00017          den*= (i+1);
00018      }
00019      printf("exp(%f) = %f\n", x, ex);
00020      return 0;
00021  }

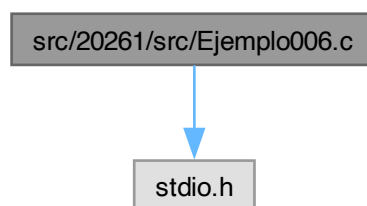
```

7.138 src/20261/src/Ejemplo006.c File Reference

Etiquetado de una temperatura T usando 4 umbrales ($T_1 < T_2 < T_3 < T_4$) y 5 categorías.

```
#include <stdio.h>
```

Include dependency graph for Ejemplo006.c:



Functions

- int [main](#) (int argc, char *argv[])

Punto de entrada. Clasifica la temperatura T en 5 regiones usando umbrales.

7.138.1 Detailed Description

Etiquetado de una temperatura T usando 4 umbrales ($T_1 < T_2 < T_3 < T_4$) y 5 categorías.

El programa lee cuatro umbrales de temperatura (T_1 , T_2 , T_3 , T_4) y luego una temperatura T que se desea clasificar. Se generan 5 regiones:

- 1) $T < T_1$ -> "Muy baja"
- 2) $T_1 \leq T < T_2$ -> "Baja"
- 3) $T_2 \leq T < T_3$ -> "Templada"
- 4) $T_3 \leq T < T_4$ -> "Alta"
- 5) $T \geq T_4$ -> "Muy alta"

La clasificación se implementa con 4 banderas booleanas ($B_1..B_4$) y se empaquetan en un código de 4 bits ($B_4 \leftarrow B_3B_2B_1$) para seleccionar el mensaje mediante switch().

Entrada

Dos lecturas desde stdin:

```

T1 T2 T3 T4
T

```

Salida

Imprime un código de depuración B (y sus bits) y una etiqueta textual: "muy baja", "baja", "templada", "alta" o "muy alta".

Precondiciones

- Se asume el orden: $T_1 < T_2 < T_3 < T_4$.

Complejidad

Tiempo: $O(1)$. Memoria: $O(1)$.

Note

Si los umbrales no están ordenados, pueden aparecer códigos no contemplados y el programa podría no imprimir etiqueta (cae en default).

Definition in file [Ejemplo006.c](#).

7.138.2 Function Documentation

7.138.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Punto de entrada. Clasifica la temperatura T en 5 regiones usando umbrales.

Parameters

<i>argc</i>	No usado.
<i>argv</i>	No usado.

Returns

0 si termina correctamente.

Definition at line 48 of file [Ejemplo006.c](#).

```
00049 {
00050     (void)argc;
00051     (void)argv;
00052
00053     /*
00054      T1, T2, T3, T4: Umbrales de temperatura que delimitan 5 regiones.
00055      - T1: umbral inferior (más frío)
00056      - T4: umbral superior (más caliente)
00057      - Se asume el orden: T1 < T2 < T3 < T4
00058
00059      T: temperatura a clasificar (etiquetar).
00060     */
00061     float T1, T2, T3, T4, T;
00062
00063     /*
00064      B1..B4 son banderas (booleanos como enteros 0/1) que indican si T es menor
00065      que cada umbral, usando comparaciones del tipo (Ti > T).
00066
00067      Ejemplo:
00068      B1 = (T1 > T)  -> 1 si T es menor que T1
00069                      0 si T es mayor o igual que T1
00070     */
00071     int B1, B2, B3, B4;
00072
00073     /*
00074      B será un "código" de 4 bits (B4 B3 B2 B1) que identifica en qué intervalo cae T.
00075      Luego se usa switch(B) para imprimir la etiqueta.
00076     */
00077     int B;
00078
00079     /* --- Lectura de umbrales --- */
00080     printf("Ingrese los rangos de temperaturas: ");
00081     scanf("%f %f %f %f", &T1, &T2, &T3, &T4);
00082
00083     /* --- Lectura de la temperatura a evaluar --- */
00084     printf("Ingrese la temperatura: ");
00085     scanf("%f", &T);
00086
00087     /*
00088      Se calcula cada bandera:
00089      - B1 = 1 si T < T1
00090      - B2 = 1 si T < T2
00091      - B3 = 1 si T < T3
```

```

00092     - B4 = 1 si T < T4
00093
00094     (Usa Ti > T, que es equivalente a T < Ti)
00095     */
00096     B1 = (T1 > T);
00097     B2 = (T2 > T);
00098     B3 = (T3 > T);
00099     B4 = (T4 > T);
00100
00101     /*
00102     Empaquetar (B4 B3 B2 B1) en un entero B usando desplazamientos:
00103         B4 « 3  coloca B4 en el bit 3 (valor 8)
00104         B3 « 2  coloca B3 en el bit 2 (valor 4)
00105         B2 « 1  coloca B2 en el bit 1 (valor 2)
00106         B1      queda en el bit 0 (valor 1)
00107
00108     Resultado:
00109         B = (B4*8) + (B3*4) + (B2*2) + (B1*1)
00110     */
00111     B = (B4 « 3) | (B3 « 2) | (B2 « 1) | (B1);
00112
00113     /* Depuración: imprime B y sus bits B4B3B2B1 */
00114     printf("%d (%d%d%d%d)\n", B, B4, B3, B2, B1);
00115
00116     printf("La temperatura es ");
00117
00118     /*
00119     Para T1<T2<T3<T4, los códigos típicos son:
00120         T < T1      -> 1111 (15) -> "muy baja"
00121         T1<=T<T2    -> 1110 (14) -> "baja"
00122         T2<=T<T3    -> 1100 (12) -> "templada"
00123         T3<=T<T4    -> 1000 ( 8) -> "alta"
00124         T>=T4       -> 0000 ( 0) -> "muy alta"
00125     */
00126     switch (B)
00127     {
00128     case 15:
00129         printf("muy ");
00130         /* fall-through intencional: completa con "baja" */
00131     case 14:
00132         printf("baja\n");
00133         break;
00134
00135     case 12:
00136         printf("templada\n");
00137         break;
00138
00139     case 0:
00140         printf("muy ");
00141         /* fall-through intencional: completa con "alta" */
00142     case 8:
00143         printf("alta\n");
00144         break;
00145
00146     default:
00147         /* Código no contemplado (p.ej. umbrales desordenados) */
00148         break;
00149     }
00150
00151     printf("\n");
00152     return 0;
00153 }

```

7.139 Ejemplo006.c

[Go to the documentation of this file.](#)

```

00001
00039
00040 #include <stdio.h>
00041
00048 int main(int argc, char *argv[])
00049 {
00050     (void)argc;
00051     (void)argv;
00052
00053     /*
00054     T1, T2, T3, T4: Umbrales de temperatura que delimitan 5 regiones.
00055         - T1: umbral inferior (más frío)
00056         - T4: umbral superior (más caliente)
00057         - Se asume el orden: T1 < T2 < T3 < T4
00058
00059     T: temperatura a clasificar (etiquetar).
00060     */
00061     float T1, T2, T3, T4, T;

```

```

00062
00063      /*
00064      B1..B4 son banderas (booleanos como enteros 0/1) que indican si T es menor
00065      que cada umbral, usando comparaciones del tipo (Ti > T).
00066
00067      Ejemplo:
00068      B1 = (T1 > T)  -> 1 si T es menor que T1
00069                  0 si T es mayor o igual que T1
00070
00071      */
00072      int B1, B2, B3, B4;
00073
00074      /*
00075      B será un "código" de 4 bits (B4 B3 B2 B1) que identifica en qué intervalo cae T.
00076      Luego se usa switch(B) para imprimir la etiqueta.
00077      */
00078      int B;
00079
00080      /* --- Lectura de umbrales --- */
00081      printf("Ingrese los rangos de temperaturas: ");
00082      scanf("%f %f %f %f", &T1, &T2, &T3, &T4);
00083
00084      /* --- Lectura de la temperatura a evaluar --- */
00085      printf("Ingrese la temperatura: ");
00086      scanf("%f", &T);
00087
00088      /*
00089      Se calcula cada bandera:
00090      - B1 = 1 si T < T1
00091      - B2 = 1 si T < T2
00092      - B3 = 1 si T < T3
00093      - B4 = 1 si T < T4
00094
00095      (Usa Ti > T, que es equivalente a T < Ti)
00096      */
00097      B1 = (T1 > T);
00098      B2 = (T2 > T);
00099      B3 = (T3 > T);
00100      B4 = (T4 > T);
00101
00102      /*
00103      Empaquetar (B4 B3 B2 B1) en un entero B usando desplazamientos:
00104      B4 << 3  coloca B4 en el bit 3 (valor 8)
00105      B3 << 2  coloca B3 en el bit 2 (valor 4)
00106      B2 << 1  coloca B2 en el bit 1 (valor 2)
00107      B1        queda en el bit 0 (valor 1)
00108
00109      Resultado:
00110      B = (B4*8) + (B3*4) + (B2*2) + (B1*1)
00111      */
00112      B = (B4 << 3) | (B3 << 2) | (B2 << 1) | (B1);
00113
00114      /* Depuración: imprime B y sus bits B4B3B2B1 */
00115      printf("%d (%d%d%d%d)\n", B, B4, B3, B2, B1);
00116
00117      printf("La temperatura es ");
00118
00119      /*
00120      Para T1<T2<T3<T4, los códigos típicos son:
00121      T < T1          -> 1111 (15) -> "muy baja"
00122      T1<=T<T2       -> 1110 (14) -> "baja"
00123      T2<=T<T3       -> 1100 (12) -> "templada"
00124      T3<=T<T4       -> 1000 ( 8) -> "alta"
00125      T>=T4          -> 0000 ( 0) -> "muy alta"
00126      */
00127      switch (B)
00128      {
00129          case 15:
00130              printf("muy ");
00131              /* fall-through intencional: completa con "baja" */
00132          case 14:
00133              printf("baja\n");
00134              break;
00135          case 12:
00136              printf("templada\n");
00137              break;
00138          case 0:
00139              printf("muy ");
00140              /* fall-through intencional: completa con "alta" */
00141          case 8:
00142              printf("alta\n");
00143              break;
00144          default:
00145              /* Código no contemplado (p.ej. umbrales desordenados) */
00146              break;
00147      }
00148

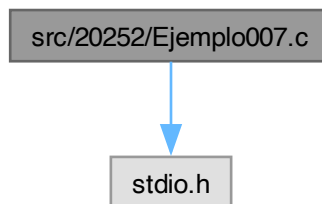
```

```
00149     }
00150
00151     printf("\n");
00152     return 0;
00153 }
```

7.140 src/20252/Ejemplo007.c File Reference

#include <stdio.h>

Include dependency graph for Ejemplo007.c:



Functions

- int `main` (int argc, char *argv[])

7.140.1 Function Documentation

7.140.1.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 3 of file [Ejemplo007.c](#).

```
00004 {
00005     int n, i;
00006     float x, ex, fct;
00007     printf("Ingrese el valor de x: ");
00008     scanf("%f", &x);
00009     do{
00010         printf("Ingrese el numero de terminos: ");
00011         scanf("%d", &n);
00012     }while(n<1);
00013     for(i=0, ex=0, fct=1; i<n; i++)
00014     {
00015         ex+=fct;
00016         fct *= x/(i+1);
00017     }
00018     printf("exp(%f) = %f\n", x, ex);
00019     return 0;
00020 }
```

Here is the call graph for this function:



7.141 Ejemplo007.c

[Go to the documentation of this file.](#)

```

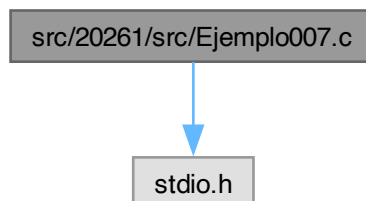
00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     int n, i;
00006     float x, ex, fct;
00007     printf("Ingrese el valor de x: ");
00008     scanf("%f", &x);
00009     do{
00010         printf("Ingrese el numero de terminos: ");
00011         scanf("%d", &n);
00012     }while(n<1);
00013     for(i=0, ex=0, fct=1; i<n; i++)
00014     {
00015         ex+=fct;
00016         fct *= x/(i+1);
00017     }
00018     printf("exp(%f) = %f\n", x, ex);
00019     return 0;
00020 }
  
```

7.142 src/20261/src/Ejemplo007.c File Reference

Cálculo del entero i tal que $1+3+5+\dots+(2i-1) \geq n$ (relación con $\sqrt{n}$).

#include <stdio.h>

Include dependency graph for Ejemplo007.c:



Functions

- int `main` (int argc, char *argv[])

Punto de entrada. Calcula i restando impares consecutivos.

7.142.1 Detailed Description

Cálculo del entero i tal que $1+3+5+\dots+(2i-1) \geq n$ (relación con $\text{sqrt}(n)$).

El programa usa la identidad: $1 + 3 + 5 + \dots + (2i-1) = i^2$

Restando números impares consecutivos a rn (inicialmente $rn=n$), el contador i se incrementa hasta que $rn \leq 0$.

Interpretación:

- Si n es un cuadrado perfecto ($n = m^2$), entonces al final $i = m$ y rn llega a 0.
- Si n NO es cuadrado perfecto, el algoritmo termina con $i = \text{ceil}(\text{sqrt}(n))$.

Entrada

Un entero $n > 0$.

Salida

Imprime una línea con el formato:

$i^2 = n$

Warning

El texto impreso sugiere igualdad exacta, pero si n no es cuadrado perfecto, $i^2 \neq n$. El valor i corresponde a $\text{ceil}(\text{sqrt}(n))$.

Complejidad

Tiempo: $O(\text{sqrt}(n))$. Memoria: $O(1)$.

Definition in file [Ejemplo007.c](#).

7.142.2 Function Documentation

7.142.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Punto de entrada. Calcula i restando impares consecutivos.

Parameters

<i>argc</i>	No usado.
<i>argv</i>	No usado.

Returns

0 si finaliza correctamente.

Definition at line 41 of file [Ejemplo007.c](#).

```
00042 {
00043     (void)argc;
00044     (void)argv;
00045
00046     int n;    /* número de entrada */
00047     int rn;   /* "residuo" que se va reduciendo restando impares */
00048     int i;    /* contador de impares restados */
00049
00050     /* Validar que n sea positivo */
00051     do {
00052         printf("Ingrese el valor de n: ");
00053         scanf("%d", &n);
00054     } while (n <= 0);
00055
00056     rn = n;
```



```

00057     i = 0;
00058
00059     /*
00060      En cada iteración se resta el siguiente impar:
00061      1, 3, 5, 7, ... que corresponde a (2*i+1) cuando i inicia en 0.
00062      Después de k iteraciones:
00063      rn = n - (1+3+...+(2k-1)) = n - k^2
00064     */
00065     while (rn > 0)
00066     {
00067         rn -= (2*i + 1); /* resta impar actual */
00068         i++;           /* incrementa contador */
00069     }
00070
00071     /* Nota: i será sqrt(n) si n es cuadrado perfecto; si no, i = ceil(sqrt(n)) */
00072     printf("%d^2 = %d\n", i, n);
00073
00074     return 0;
00075 }

```

7.143 Ejemplo007.c

[Go to the documentation of this file.](#)

```

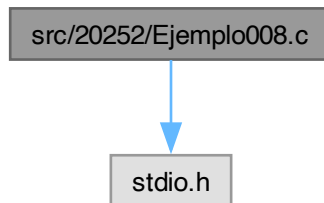
00001
00032
00033 #include <stdio.h>
00034
00041 int main(int argc, char *argv[])
00042 {
00043     (void)argc;
00044     (void)argv;
00045
00046     int n; /* número de entrada */
00047     int rn; /* "residuo" que se va reduciendo restando impares */
00048     int i; /* contador de impares restados */
00049
00050     /* Validar que n sea positivo */
00051     do {
00052         printf("Ingrese el valor de n: ");
00053         scanf("%d", &n);
00054     } while (n <= 0);
00055
00056     rn = n;
00057     i = 0;
00058
00059     /*
00060      En cada iteración se resta el siguiente impar:
00061      1, 3, 5, 7, ... que corresponde a (2*i+1) cuando i inicia en 0.
00062      Después de k iteraciones:
00063      rn = n - (1+3+...+(2k-1)) = n - k^2
00064     */
00065     while (rn > 0)
00066     {
00067         rn -= (2*i + 1); /* resta impar actual */
00068         i++;           /* incrementa contador */
00069     }
00070
00071     /* Nota: i será sqrt(n) si n es cuadrado perfecto; si no, i = ceil(sqrt(n)) */
00072     printf("%d^2 = %d\n", i, n);
00073
00074     return 0;
00075 }

```

7.144 src/20252/Ejemplo008.c File Reference

```
#include <stdio.h>
```

Include dependency graph for Ejemplo008.c:



Functions

- int [main](#) (int argc, char *argv[])

7.144.1 Function Documentation

7.144.1.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 3 of file [Ejemplo008.c](#).

```

00004 {
00005     int i, n;
00006     float x, lnx, fct, num;
00007     do{
00008         printf("Ingrese el numero de terminos: ");
00009         scanf("%d", &n);
00010     }while(n<1);
00011     do{
00012         printf("Ingrese l valor de x: ");
00013         scanf("%f", &x);
00014     }while(x<=0);
00015     for(i=0, fct=(x-1)/(x+1), num=fct, lnx=0; i<n; i++)
00016     {
00017         lnx+=(num/(2*i+1));
00018         num*=(fct*fct);
00019     }
00020     lnx*=2;
00021     printf("ln(%f) = %f\n", x, lnx);
00022     return 0;
00023 }
```

Here is the call graph for this function:



7.145 Ejemplo008.c

[Go to the documentation of this file.](#)

```

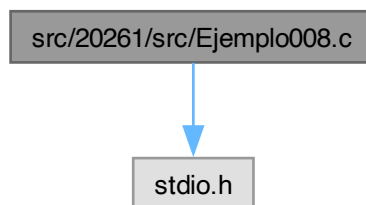
00001 #include<stdio.h>
00002
00003 int main(int argc, char*argv[])
00004 {
00005     int i, n;
00006     float x, lnx, fct, num;
00007     do{
00008         printf("Ingrese el numero de terminos: ");
00009         scanf("%d", &n);
00010     }while(n<1);
00011     do{
00012         printf("Ingrese l valor de x: ");
00013         scanf("%f", &x);
00014     }while(x<=0);
00015     for(i=0, fct=(x-1)/(x+1), num=fct, lnx=0; i<n; i++)
00016     {
00017         lnx+=(num/(2*i+1));
00018         num*=(fct*fct);
00019     }
00020     lnx*=2;
00021     printf("ln(%f) = %f\n", x, lnx);
00022     return 0;
00023 }
```

7.146 src/20261/src/Ejemplo008.c File Reference

Aproximación de mediante la serie de Leibniz.

```
#include <stdio.h>
```

Include dependency graph for Ejemplo008.c:



Functions

- int [main](#)(int argc, char *argv[])

Punto de entrada. Calcula una aproximación de con n términos.

7.146.1 Detailed Description

Aproximación de mediante la serie de Leibniz.

Usa la serie: $1/4 = 1 - 1/3 + 1/5 - 1/7 + \dots$

Con n términos: $4 * \sum_{i=0}^{n-1} (-1)^i / (2i+1)$

Entrada

Un entero $n \geq 1$ (número de términos).

Salida

Imprime el valor aproximado:

PI = <valor>

Complejidad

Tiempo: $O(n)$. Memoria: $O(1)$.

Note

La convergencia es lenta: para obtener muchos decimales se requieren muchos términos.

Definition in file [Ejemplo008.c](#).

7.146.2 Function Documentation**7.146.2.1 main()**

```
int main (
    int argc,
    char * argv[])
```

Punto de entrada. Calcula una aproximación de π con n términos.

Parameters

<i>argc</i>	No usado.
<i>argv</i>	No usado.

Returns

0 si finaliza correctamente.

Definition at line 36 of file [Ejemplo008.c](#).

```
00037 {
00038     (void)argc;
00039     (void)argv;
00040
00041     float pi;      /* acumulador para */
00042     int i, n;      /* i: índice; n: términos */
00043     int den;       /* denominador (2i+1) */
00044     int signo;     /* +1 o -1 alternando */
00045
00046     /* Validar n >= 1 */
00047     do {
00048         printf("Ingrese el numero de terminos: ");
00049         scanf("%d", &n);
00050     } while (n < 1);
00051
00052     /*
00053      Inicialización:
00054      i = 0
00055      pi = 0 (acumulador de /4 antes de multiplicar por 4)
00056     */
00057     for (i = 0, pi = 0; i < n; i++)
00058     {
00059         den = 2*i + 1;      /* 1, 3, 5, 7, ... */
00060         signo = 1 - 2*(i % 2); /* i par -> +1, i impar -> -1 */
00061         pi += (signo * 1.0f / den);
00062     }
00063
00064     pi *= 4; /* = 4*(/4) */
00065     printf("PI = %f\n", pi);
00066
00067     return 0;
00068 }
```

7.147 Ejemplo008.c

[Go to the documentation of this file.](#)

```

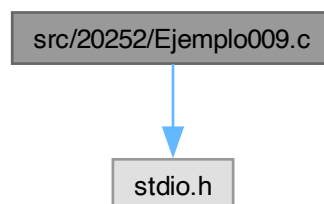
00001
00027
00028 #include <stdio.h>
00029
00036 int main(int argc, char *argv[])
00037 {
00038     (void)argc;
00039     (void)argv;
00040
00041     float pi;      /* acumulador para */
00042     int i, n;      /* i: índice; n: términos */
00043     int den;       /* denominador (2i+1) */
00044     int signo;     /* +1 o -1 alternando */
00045
00046     /* Validar n >= 1 */
00047     do {
00048         printf("Ingrese el numero de terminos: ");
00049         scanf("%d", &n);
00050     } while (n < 1);
00051
00052     /*
00053      Inicialización:
00054      i = 0
00055      pi = 0 (acumulador de /4 antes de multiplicar por 4)
00056     */
00057     for (i = 0, pi = 0; i < n; i++)
00058     {
00059         den = 2*i + 1;      /* 1, 3, 5, 7, ... */
00060         signo = 1 - 2*(i % 2); /* i par -> +1, i impar -> -1 */
00061         pi += (signo * 1.0f / den);
00062     }
00063
00064     pi *= 4; /* = 4*(/4) */
00065     printf("PI = %f\n", pi);
00066
00067     return 0;
00068 }

```

7.148 src/20252/Ejemplo009.c File Reference

#include <stdio.h>

Include dependency graph for Ejemplo009.c:



Functions

- int `main` (int argc, char *argv[])

7.148.1 Function Documentation

7.148.1.1 main()

```

int main (
    int argc,
    char * argv[])

```

Definition at line 3 of file [Ejemplo009.c](#).

```

00004 {
00005     int i, n, s;
00006     long den;
00007     float x, sx, num;
00008     do{
00009         printf("Ingrese el numero de terminos: ");
00010         scanf("%d", &n);
00011     }while(n<1);
00012     printf("Ingrese el valor de x: ");
00013     scanf("%f", &x);
00014     for(i=0, num=x, den=1, sx=0; i<n; i++)
00015     {
00016         s = 1-2*(i%2);
00017         sx += (s*num/den);
00018         //printf("%d. %d %f %ld %f\n", i, s, num, den, sx);
00019         num*=(x*x);
00020         den*=( (2*i+2)*(2*i+3) );
00021     }
00022     printf("sin(%f) = %f\n", x, sx);
00023     return 0;
00024 }
```

7.149 Ejemplo009.c

[Go to the documentation of this file.](#)

```

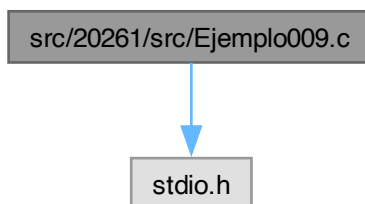
00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     int i, n, s;
00006     long den;
00007     float x, sx, num;
00008     do{
00009         printf("Ingrese el numero de terminos: ");
00010         scanf("%d", &n);
00011     }while(n<1);
00012     printf("Ingrese el valor de x: ");
00013     scanf("%f", &x);
00014     for(i=0, num=x, den=1, sx=0; i<n; i++)
00015     {
00016         s = 1-2*(i%2);
00017         sx += (s*num/den);
00018         //printf("%d. %d %f %ld %f\n", i, s, num, den, sx);
00019         num*=(x*x);
00020         den*=( (2*i+2)*(2*i+3) );
00021     }
00022     printf("sin(%f) = %f\n", x, sx);
00023     return 0;
00024 }
```

7.150 src/20261/src/Ejemplo009.c File Reference

Aproximación de $\exp(x)$ con serie de Taylor (cálculo directo de potencias y factorial).

```
#include <stdio.h>
```

Include dependency graph for Ejemplo009.c:



Functions

- `int main (int argc, char *argv[ ])`

Punto de entrada. Calcula $\exp(x)$ con Taylor y cálculo “desde cero” de cada término.

7.150.1 Detailed Description

Aproximación de $\exp(x)$ con serie de Taylor (cálculo directo de potencias y factorial).

Usa la expansión: $\exp(x) = \sum_{i=0}^{\infty} x^i / i!$

Este programa aproxima con n términos: $\exp(x) \approx \sum_{i=0}^{n-1} x^i / i!$

Para cada término i , calcula:

- $\text{num} = x^i$ (multiplicando x repetidamente)
- $\text{den} = i!$ (multiplicando $1*2*\dots*i$)

Entrada

- Un entero $n \geq 1$ (número de términos).
- Un real x (float).

Salida

Imprime:

`exp(x) = <aprox>`

Complejidad

Tiempo: $O(n^2)$ (por el doble ciclo). Memoria: $O(1)$.

Warning

El factorial crece muy rápido; 'den' (long int) puede desbordarse para i moderados. También puede perderse precisión por usar float.

Definition in file [Ejemplo009.c](#).

7.150.2 Function Documentation

7.150.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Punto de entrada. Calcula $\exp(x)$ con Taylor y cálculo “desde cero” de cada término.

Parameters

<i>argc</i>	No usado.
<i>argv</i>	No usado.

Returns

0 si finaliza correctamente.

Definition at line 42 of file [Ejemplo009.c](#).

```

00043 {
00044     (void)argc;
00045     (void)argv;
00046
00047     int n, i, j;
00048     long int den;      /* factorial i! (puede desbordar para i grandes) */
00049     float x, num, ex; /* num = x^i, ex = acumulador */
00050
00051     /* Leer n >= 1 */
00052     do {
00053         printf("Ingrese el numero de terminos: ");
00054         scanf("%d", &n);
00055     } while (n < 1);
00056
00057     /* Leer x */
00058     printf("Ingrese el valor de x: ");
00059     scanf("%f", &x);
00060
00061     /*
00062      * ex acumula la suma:
00063      * ex = sum_{i=0}^{n-1} x^i / i!
00064      */
00065     for (i = 0, ex = 0; i < n; i++)
00066     {
00067         /*
00068          * Para el término i:
00069          * num = x^i
00070          * den = i!
00071          *
00072          * Se reinician en 1 y se multiplican i veces.
00073          */
00074         for (j = 0, num = 1, den = 1; j < i; j++)
00075         {
00076             num *= x;      /* num = x^(j+1) al avanzar */
00077             den *= (j + 1); /* den = (j+1)! al avanzar */
00078         }
00079
00080         ex += (num / den);
00081     }
00082
00083     printf("exp(%f) = %f\n", x, ex);
00084     return 0;
00085 }

```

7.151 Ejemplo009.c

[Go to the documentation of this file.](#)

```

00001
00033
00034 #include <stdio.h>
00035
00042 int main(int argc, char *argv[])
00043 {
00044     (void)argc;
00045     (void)argv;
00046
00047     int n, i, j;
00048     long int den;      /* factorial i! (puede desbordar para i grandes) */
00049     float x, num, ex; /* num = x^i, ex = acumulador */
00050
00051     /* Leer n >= 1 */
00052     do {
00053         printf("Ingrese el numero de terminos: ");
00054         scanf("%d", &n);
00055     } while (n < 1);
00056
00057     /* Leer x */
00058     printf("Ingrese el valor de x: ");
00059     scanf("%f", &x);
00060
00061     /*
00062      * ex acumula la suma:
00063      * ex = sum_{i=0}^{n-1} x^i / i!
00064      */
00065     for (i = 0, ex = 0; i < n; i++)
00066     {
00067         /*
00068          * Para el término i:

```



```

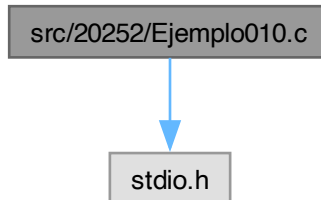
00069         num = x^i
00070         den = i!
00071
00072         Se reinician en 1 y se multiplican i veces.
00073     */
00074     for (j = 0, num = 1, den = 1; j < i; j++)
00075     {
00076         num *= x;          /* num = x^(j+1) al avanzar */
00077         den *= (j + 1);    /* den = (j+1)! al avanzar */
00078     }
00079
00080     ex += (num / den);
00081 }
00082
00083 printf("exp(%f) = %f\n", x, ex);
00084 return 0;
00085 }

```

7.152 src/20252/Ejemplo010.c File Reference

#include <stdio.h>

Include dependency graph for Ejemplo010.c:



Functions

- int [main](#) (int argc, char *argv[])

7.152.1 Function Documentation

7.152.1.1 main()

```

int main (
    int argc,
    char * argv[])

```

Definition at line 3 of file [Ejemplo010.c](#).

```

00004 {
00005     int i, n, s;
00006     float x, sx, fct;
00007     do{
00008         printf("Ingrese el numero de terminos: ");
00009         scanf("%d", &n);
00010     }while(n<1);
00011     printf("Ingrese el valor de x: ");
00012     scanf("%f", &x);
00013     for(i=0, fct=x, sx=0; i<n; i++)
00014     {
00015         s = 1-2*(i%2);
00016         sx += (s*fct);
00017         fct *= (x/(2*i+2)) * (x/(2*i+3));
00018     }
00019     printf("sin(%f) = %f\n", x, sx);
00020     return 0;
00021 }

```

Here is the call graph for this function:



7.153 Ejemplo010.c

[Go to the documentation of this file.](#)

```

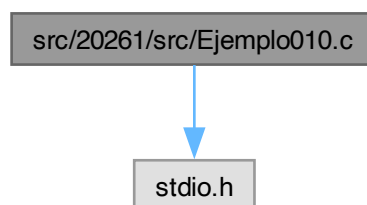
00001 #include <stdio.h>
00002
00003 int main(int argc, char *argv[])
00004 {
00005     int i, n, s;
00006     float x, sx, fct;
00007     do{
00008         printf("Ingrese el numero de terminos: ");
00009         scanf("%d", &n);
00010     }while(n<1);
00011     printf("Ingrese el valor de x: ");
00012     scanf("%f", &x);
00013     for(i=0, fct=x, sx=0; i<n; i++)
00014     {
00015         s = 1-2*(i%2);
00016         sx += (s*fct);
00017         fct *= (x/(2*i+2)) * (x/(2*i+3));
00018     }
00019     printf("sin(%f) = %f\n", x, sx);
00020     return 0;
00021 }
  
```

7.154 src/20261/src/Ejemplo010.c File Reference

Aproximación de $\exp(x)$ con Taylor usando producto incremental dentro del término.

```
#include <stdio.h>
```

Include dependency graph for Ejemplo010.c:



Functions

- int `main` (int argc, char *argv[])

Punto de entrada. Calcula $\exp(x)$ con Taylor y producto por término.

7.154.1 Detailed Description

Aproximación de $\exp(x)$ con Taylor usando producto incremental dentro del término.

Serie: $\exp(x) = \sum_{i=0}^{\infty} x^i / i!$

Este código calcula cada término i como: $fct = \sum_{j=0}^{i-1} \{ x / (j+1) \}$ lo cual equivale exactamente a: $fct = x^i / i!$

Entrada

- Un entero $n \geq 1$ (términos).
- Un real x (float).

Salida

Imprime:

$\exp(x) = \langle \text{aprox} \rangle$

Complejidad

Tiempo: $O(n^2)$ (por el doble ciclo). Memoria: $O(1)$.

Note

Esta forma evita manejar factorial explícito como entero, pero sigue siendo $O(n^2)$.

Definition in file [Ejemplo010.c](#).

7.154.2 Function Documentation

7.154.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Punto de entrada. Calcula $\exp(x)$ con Taylor y producto por término.

Parameters

<i>argc</i>	No usado.
<i>argv</i>	No usado.

Returns

0 si finaliza correctamente.

Definition at line 39 of file [Ejemplo010.c](#).

```
00040 {
00041     (void)argc;
00042     (void)argv;
00043
00044     int n, i, j;
00045     float x, fct, ex;
00046
00047     do {
00048         printf("Ingrese el numero de terminos: ");
00049         scanf("%d", &n);
00050     } while (n < 1);
00051
00052     printf("Ingrese el valor de x: ");
00053     scanf("%f", &x);
00054
00055     /*
00056     Para cada i:
00057         fct = x^i / i! calculado como producto:
00058         fct = (x/1)*(x/2)*...*(x/i)
00059     Luego:
00060         ex += fct
00061     */
```

```

00062     for (i = 0, ex = 0; i < n; i++)
00063     {
00064         for (j = 0, fct = 1; j < i; j++)
00065             fct *= (x / (j + 1)); /* multiplica por x/(j+1) */
00066
00067         ex += fct;
00068     }
00069
00070     printf("exp(%f) = %f\n", x, ex);
00071     return 0;
00072 }

```

Here is the call graph for this function:



7.155 Ejemplo010.c

[Go to the documentation of this file.](#)

```

00001
00030
00031 #include <stdio.h>
00032
00039 int main(int argc, char *argv[])
00040 {
00041     (void)argc;
00042     (void)argv;
00043
00044     int n, i, j;
00045     float x, fct, ex;
00046
00047     do {
00048         printf("Ingrese el numero de terminos: ");
00049         scanf("%d", &n);
00050     } while (n < 1);
00051
00052     printf("Ingrese el valor de x: ");
00053     scanf("%f", &x);
00054
00055     /*
00056     Para cada i:
00057         fct = x^i / i! calculado como producto:
00058         fct = (x/1)*(x/2)*...*(x/i)
00059     Luego:
00060         ex += fct
00061     */
00062     for (i = 0, ex = 0; i < n; i++)
00063     {
00064         for (j = 0, fct = 1; j < i; j++)
00065             fct *= (x / (j + 1)); /* multiplica por x/(j+1) */
00066
00067         ex += fct;
00068     }
00069
00070     printf("exp(%f) = %f\n", x, ex);
00071     return 0;
00072 }

```

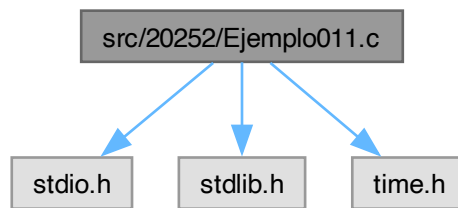
7.156 src/20252/Ejemplo011.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo011.c:



Functions

- int `main` (int argc, char *argv[])

7.156.1 Function Documentation

7.156.1.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 5 of file [Ejemplo011.c](#).

```
00006 {
00007     int i, n, max, min, x, aux;
00008     srand(time(NULL));
00009     do{
00010         printf("Ingrese el numero de elementos: ");
00011         scanf("%d", &n);
00012     }while(n<1);
00013     printf("Ingrese el valor maximo: ");
00014     scanf("%d", &max);
00015     printf("Ingrese el valor minimo: ");
00016     scanf("%d", &min);
00017     if(min>max)
00018     {
00019         aux = max;
00020         max = min;
00021         min = aux;
00022     }
00023     for(i=0; i<n; i++)
00024     {
00025         x = rand()%(max-min+1)+min;
00026         printf("%d. %d\n", i+1, x);
00027     }
00028     return 0;
00029 }
```

7.157 Ejemplo011.c

[Go to the documentation of this file.](#)

```
00001 #include<stdio.h>
00002 #include<stdlib.h>
00003 #include<time.h>
00004
00005 int main(int argc, char*argv[])
00006 {
00007     int i, n, max, min, x, aux;
00008     srand(time(NULL));
00009     do{
00010         printf("Ingrese el numero de elementos: ");
00011         scanf("%d", &n);
00012     }while(n<1);
```

```

00013     printf("Ingrese el valor maximo: ");
00014     scanf("%d", &max);
00015     printf("Ingrese el valor minimo: ");
00016     scanf("%d", &min);
00017     if(min>max)
00018     {
00019         aux = max;
00020         max = min;
00021         min = aux;
00022     }
00023     for(i=0; i<n; i++)
00024     {
00025         x = rand()%(max-min+1)+min;
00026         printf("%d. %d\n", i+1, x);
00027     }
00028     return 0;
00029 }

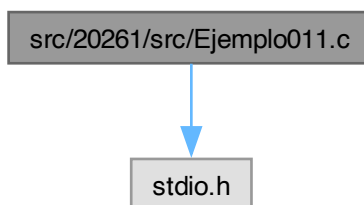
```

7.158 src/20261/src/Ejemplo011.c File Reference

Aproximación de $\exp(x)$ con Taylor usando recurrencia entre términos (forma eficiente).

#include <stdio.h>

Include dependency graph for Ejemplo011.c:



Functions

- int `main` (int argc, char *argv[])

Punto de entrada. Calcula $\exp(x)$ con Taylor usando recurrencia $O(n)$.

7.158.1 Detailed Description

Aproximación de $\exp(x)$ con Taylor usando recurrencia entre términos (forma eficiente).

Serie: $\exp(x) = \sum_{i=0}^{\infty} x^i / i!$

Define el término: $t_i = x^i / i!$

Recurrencia: $t_0 = 1$ $t_{i+1} = t_i * (x / (i+1))$

Con esto se calcula la suma con un solo ciclo: $ex = t_0 + t_1 + \dots + t_{n-1}$

Entrada

- Un entero $n \geq 1$ (términos).
- Un real x (float).

Salida

Imprime:

`exp(x) = <aprox>`

Complejidad

Tiempo: $O(n)$. Memoria: $O(1)$.

Note

Esta es la versión más eficiente de (009–011) porque evita recalcular potencias/factoriales.

Definition in file [Ejemplo011.c](#).

7.158.2 Function Documentation

7.158.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Punto de entrada. Calcula $\exp(x)$ con Taylor usando recurrencia $O(n)$.

Parameters

<i>argc</i>	No usado.
<i>argv</i>	No usado.

Returns

0 si finaliza correctamente.

Definition at line 44 of file [Ejemplo011.c](#).

```
00045 {
00046     (void)argc;
00047     (void)argv;
00048
00049     int n, i;
00050     float x, fct, ex;
00051
00052     do {
00053         printf("Ingrese el numero de terminos: ");
00054         scanf("%d", &n);
00055     } while (n < 1);
00056
00057     printf("Ingrese el valor de x: ");
00058     scanf("%f", &x);
00059
00060     /*
00061     Inicialización:
00062         fct = t_0 = 1
00063         ex = 0
00064
00065     En cada iteración i:
00066         ex += fct      (acumula t_i)
00067         fct *= x/(i+1) (convierte t_i en t_{i+1})
00068     */
00069     for (i = 0, ex = 0, fct = 1; i < n; i++)
00070     {
00071         ex += fct;
00072         fct *= (x / (i + 1));
00073     }
00074
00075     printf("exp(%f) = %f\n", x, ex);
00076     return 0;
00077 }
```

Here is the call graph for this function:



7.159 Ejemplo011.c

[Go to the documentation of this file.](#)

```

00001
00035
00036 #include <stdio.h>
00037
00044 int main(int argc, char *argv[])
00045 {
00046     (void)argc;
00047     (void)argv;
00048
00049     int n, i;
00050     float x, fct, ex;
00051
00052     do {
00053         printf("Ingrese el numero de terminos: ");
00054         scanf("%d", &n);
00055     } while (n < 1);
00056
00057     printf("Ingrese el valor de x: ");
00058     scanf("%f", &x);
00059
00060     /*
00061     Inicialización:
00062     fct = t_0 = 1
00063     ex = 0
00064
00065     En cada iteración i:
00066     ex += fct      (acumula t_i)
00067     fct *= x/(i+1) (convierte t_i en t_{i+1})
00068     */
00069     for (i = 0, ex = 0, fct = 1; i < n; i++)
00070     {
00071         ex += fct;
00072         fct *= (x / (i + 1));
00073     }
00074
00075     printf("exp(%f) = %f\n", x, ex);
00076     return 0;
00077 }

```

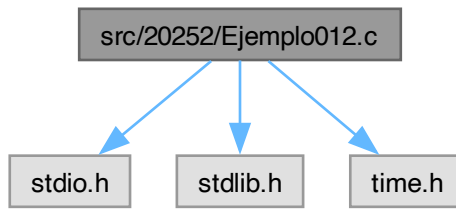
7.160 src/20252/Ejemplo012.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```


Include dependency graph for Ejemplo012.c:



Functions

- `int main (int argc, char *argv[])`

7.160.1 Function Documentation

7.160.1.1 main()

```
int main (  
    int argc,  
    char * argv[])
```

Definition at line 5 of file [Ejemplo012.c](#).

```
00006 {  
00007     int i, n;  
00008     float max, min, x;  
00009     srand(time(NULL));  
00010     do{  
00011         printf("Ingrese el numero de elementos: ");  
00012         scanf("%d", &n);  
00013     }while(n<1);  
00014     printf("Ingrese el valor maximo: ");  
00015     scanf("%f", &max);  
00016     printf("Ingrese el valor minimo: ");  
00017     scanf("%f", &min);  
00018     if(min>max)  
00019     {  
00020         if(max)  
00021         {  
00022             if(min)  
00023             {  
00024                 min*=max;  
00025                 max=min/max;  
00026                 min/=max;  
00027             }  
00028             else  
00029             {  
00030                 min = max;  
00031                 max = 0;  
00032             }  
00033         }  
00034         else  
00035         {  
00036             max = min;  
00037             min = 0;  
00038         }  
00039     }  
00040     for(i=0; i<n; i++)  
00041     {  
00042         x = ((max-min)*rand())/RAND_MAX+min;  
00043         printf("%d. %f\n", i+1, x);  
00044     }  
00045     return 0;  
00046 }
```

7.161 Ejemplo012.c

[Go to the documentation of this file.](#)

```

00001 #include<stdio.h>
00002 #include<stdlib.h>
00003 #include<time.h>
00004
00005 int main(int argc, char*argv[])
00006 {
00007     int i, n;
00008     float max, min, x;
00009     srand(time(NULL));
00010     do{
00011         printf("Ingrese el numero de elementos: ");
00012         scanf("%d", &n);
00013     }while(n<1);
00014     printf("Ingrese el valor maximo: ");
00015     scanf("%f", &max);
00016     printf("Ingrese el valor minimo: ");
00017     scanf("%f", &min);
00018     if(min>max)
00019     {
00020         if(max)
00021         {
00022             if(min)
00023             {
00024                 min*=max;
00025                 max=min/max;
00026                 min/=max;
00027             }
00028             else
00029             {
00030                 min = max;
00031                 max = 0;
00032             }
00033         }
00034         else
00035         {
00036             max = min;
00037             min = 0;
00038         }
00039     }
00040     for(i=0; i<n; i++)
00041     {
00042         x = ((max-min)*rand())/RAND_MAX+min;
00043         printf("%d. %f\n", i+1, x);
00044     }
00045     return 0;
00046 }

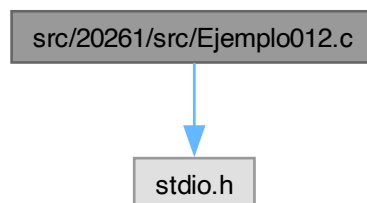
```

7.162 src/20261/src/Ejemplo012.c File Reference

Aproximación de $\ln(x)$ con serie tipo atanh calculando potencias desde cero.

#include <stdio.h>

Include dependency graph for Ejemplo012.c:



Functions

- `int main (int argc, char *argv[ ])`

7.162.1 Detailed Description

Aproximación de $\ln(x)$ con serie tipo atanh calculando potencias desde cero.

Para $x > 0$ se usa la identidad: $\ln(x) = 2 * [f + f^3/3 + f^5/5 + \dots]$ donde: $f = (x - 1) / (x + 1)$

Entonces: $\ln(x) = 2 * \sum_{i=0}^{n-1} f^{2i+1} / (2i+1)$

Este programa:

- Lee n (número de términos) y x (valor real positivo).
- Calcula $f = (x-1)/(x+1)$.
- Para cada término i calcula explícitamente f^{2i+1} multiplicando f varias veces.

Entrada

- Un entero $n \geq 1$ (número de términos).
- Un real $x > 0$.

Salida

Imprime la aproximación:

$\ln(x) = \text{valor}$

Precondiciones

- x debe ser positivo ($\ln(x)$ solo está definida para $x > 0$ en los reales).

Complejidad

Tiempo: $O(n^2)$ (por el doble ciclo al calcular potencias). Memoria: $O(1)$.

Note

La convergencia es mejor cuando x está cerca de 1 (porque $|f|$ es pequeño).

Definition in file [Ejemplo012.c](#).

7.162.2 Function Documentation

7.162.2.1 main()

```
int main (
    int argc,
    char * argv[ ])
```

Definition at line 41 of file [Ejemplo012.c](#).

```
00042 {
00043     (void)argc;
00044     (void)argv;
00045
00046     int n, i, j;
00047     int den;           /* denominador impar: 1,3,5,... */
00048     float x;           /* valor para evaluar ln(x) */
00049     float lnx;         /* acumulador de la serie */
00050     float fct;         /* f = (x-1)/(x+1) */
00051     float num;         /* potencia: f^{den} */
00052
00053     /* Leer n >= 1 */
00054     do {
00055         printf("Ingresa el numero de terminos: ");
00056         scanf("%d", &n);
00057     } while (n < 1);
00058
00059     /* Leer x > 0 */
```

```

00060     do {
00061         printf("Ingrese el valor de x: ");
00062         scanf("%f", &x);
00063     } while (x <= 0);
00064
00065     /*
00066     f = (x - 1) / (x + 1)
00067     y la serie es:
00068     ln(x) = 2 * sum_{i=0}^{n-1} f^{2i+1}/(2i+1)
00069     */
00070     for (i = 0, fct = (x - 1) / (x + 1), lnx = 0; i < n; i++)
00071     {
00072         den = 2*i + 1; /* 1, 3, 5, ... */
00073
00074         /*
00075         Calcular num = fct^den multiplicando fct den veces:
00076         num = fct * fct * ... * fct (den factores)
00077         */
00078         for (j = 0, num = 1; j < den; j++)
00079             num *= fct;
00080
00081         /* Sumar el término: f^{den}/den */
00082         lnx += num / den;
00083     }
00084
00085     /* Multiplicar por 2 según la identidad */
00086     lnx *= 2;
00087
00088     printf("ln(%f) = %f\n", x, lnx);
00089     return 0;
00090 }

```

Here is the call graph for this function:



7.163 Ejemplo012.c

[Go to the documentation of this file.](#)

```

00001
00038
00039 #include <stdio.h>
00040
00041 int main(int argc, char *argv[])
00042 {
00043     (void)argc;
00044     (void)argv;
00045
00046     int n, i, j;
00047     int den; /* denominador impar: 1,3,5,... */
00048     float x; /* valor para evaluar ln(x) */
00049     float lnx; /* acumulador de la serie */
00050     float fct; /* f = (x-1)/(x+1) */
00051     float num; /* potencia: f^{den} */
00052
00053     /* Leer n >= 1 */
00054     do {
00055         printf("Ingresa el numero de terminos: ");
00056         scanf("%d", &n);
00057     } while (n < 1);
00058
00059     /* Leer x > 0 */
00060     do {
00061         printf("Ingrese el valor de x: ");
00062         scanf("%f", &x);
00063     } while (x <= 0);
00064
00065     /*

```

```

00066     f = (x - 1) / (x + 1)
00067     y la serie es:
00068     ln(x) = 2 * sum_{i=0}^{n-1} f^{2i+1}/(2i+1)
00069 */
00070 for (i = 0, fct = (x - 1) / (x + 1), lnx = 0; i < n; i++)
00071 {
00072     den = 2*i + 1; /* 1, 3, 5, ... */
00073
00074     /*
00075      * Calcular num = fct^den multiplicando fct den veces:
00076      * num = fct * fct * ... * fct (den factores)
00077      */
00078     for (j = 0, num = 1; j < den; j++)
00079         num *= fct;
00080
00081     /* Sumar el término: f^{den}/den */
00082     lnx += num / den;
00083 }
00084
00085 /* Multiplicar por 2 según la identidad */
00086 lnx *= 2;
00087
00088 printf("ln(%f) = %f\n", x, lnx);
00089 return 0;
00090 }

```

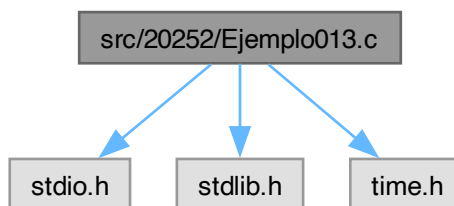
7.164 src/20252/Ejemplo013.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo013.c:



Macros

- #define [N](#) 10000

Functions

- int [main](#) (int argc, char *argv[])

7.164.1 Macro Definition Documentation

7.164.1.1 N

```
#define N 10000
```

Definition at line 5 of file [Ejemplo013.c](#).

7.164.2 Function Documentation

7.164.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 7 of file [Ejemplo013.c](#).

```
00008 {
00009     int i, j, n;
00010     float max, min, x[N], aux;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%d", &n);
00015     }while(n<1||n>N);
00016     printf("Ingrese el valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(min>max)
00021     {
00022         if(max)
00023         {
00024             if(min)
00025             {
00026                 min*=max;
00027                 max=min/max;
00028                 min/=max;
00029             }
00030             else
00031             {
00032                 min = max;
00033                 max = 0;
00034             }
00035         }
00036         else
00037         {
00038             max = min;
00039             min = 0;
00040         }
00041     }
00042     printf("Desordenado.\n");
00043     for(i=0; i<n; i++)
00044     {
00045         x[i] = ((max-min)*rand())/RAND_MAX+min;
00046         printf("X[%d] = %f\n", i+1, x[i]);
00047     }
00048     // Algoritmo Burbuja
00049     for(i=0; i<n-1; i++)
00050     {
00051         for(j=i+1; j<n; j++)
00052         {
00053             if(x[i]>x[j])
00054             {
00055                 aux = x[i];
00056                 x[i] = x[j];
00057                 x[j] = aux;
00058             }
00059         }
00060     }
00061     printf("Ordenado.\n");
00062     for(i=0; i<n; i++)
00063     {
00064         printf("X[%d] = %f\n", i+1, x[i]);
00065     }
00066     return 0;
00067 }
```

7.165 Ejemplo013.c

[Go to the documentation of this file.](#)

```
00001 #include<stdio.h>
00002 #include<stdlib.h>
00003 #include<time.h>
00004
00005 #define N 10000
00006
00007 int main(int argc, char*argv[])
00008 {
00009     int i, j, n;
00010     float max, min, x[N], aux;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%d", &n);
00015     }while(n<1||n>N);
```

```

00016     printf("Ingrese el valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(min>max)
00021     {
00022         if(max)
00023         {
00024             if(min)
00025             {
00026                 min*=max;
00027                 max=min/max;
00028                 min/=max;
00029             }
00030             else
00031             {
00032                 min = max;
00033                 max = 0;
00034             }
00035         }
00036         else
00037         {
00038             max = min;
00039             min = 0;
00040         }
00041     }
00042     printf("Desordenado.\n");
00043     for(i=0; i<n; i++)
00044     {
00045         x[i] = ((max-min)*rand())/RAND_MAX+min;
00046         printf("X[%d] = %f\n", i+1, x[i]);
00047     }
00048     // Algoritmo Burbuja
00049     for(i=0; i<n-1; i++)
00050         for(j=i+1; j<n; j++)
00051             if(x[i]>x[j])
00052             {
00053                 aux = x[i];
00054                 x[i] = x[j];
00055                 x[j] = aux;
00056             }
00057     printf("Ordenado.\n");
00058     for(i=0; i<n; i++)
00059         printf("X[%d] = %f\n", i+1, x[i]);
00060     return 0;
00061 }

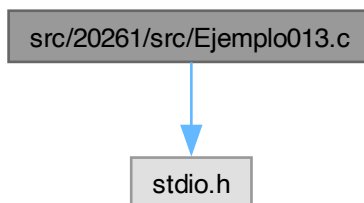
```

7.166 src/20261/src/Ejemplo013.c File Reference

Aproximación de $\ln(x)$ con serie tipo atanh usando recurrencia entre términos.

#include <stdio.h>

Include dependency graph for Ejemplo013.c:



Functions

- int [main](#) (int argc, char *argv[])

7.166.1 Detailed Description

Aproximación de $\ln(x)$ con serie tipo atanh usando recurrencia entre términos.

Se usa la misma identidad que en Ejemplo012: $\ln(x) = 2 * \sum_{i=0}^{\infty} f^{2i+1}/(2i+1)$, $f = (x - 1) / (x + 1)$, $x > 0$.

A diferencia del Ejemplo012, aquí NO se recalcula la potencia desde cero. Se usa la recurrencia de términos: $t_i = f^{2i+1}/(2i+1)$ $t_{i+1} = t_i * f^2 * (2i+1)/(2i+3)$

Entrada

- Un entero $n \geq 1$.
- Un real $x > 0$.

Salida

Imprime:

$\ln(x) = \text{valor}$

Precondiciones

- $x > 0$.

Complejidad

Tiempo: $O(n)$. Memoria: $O(1)$.

Note

Converge más rápido cuando x está cerca de 1 ($|f|$ pequeño).

Definition in file [Ejemplo013.c](#).

7.166.2 Function Documentation

7.166.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 37 of file [Ejemplo013.c](#).

```
00038 {
00039     (void)argc;
00040     (void)argv;
00041
00042     int n, i;
00043     float x;          /* valor para ln(x) */
00044     float lnx;         /* acumulador de la serie */
00045     float fct;         /* f = (x-1)/(x+1) */
00046     float fct2;        /* f^2 */
00047     float suma;        /* término actual t_i = f^{2i+1}/(2i+1) */
00048
00049     /* Leer n >= 1 */
00050     do {
00051         printf("Ingresa el numero de terminos: ");
00052         scanf("%d", &n);
00053     } while (n < 1);
00054
00055     /* Leer x > 0 */
00056     do {
00057         printf("Ingresa el valor de x: ");
00058         scanf("%f", &x);
00059     } while (x <= 0);
00060
00061     /*
00062      Inicialización:
00063      f = (x-1)/(x+1)
00064      f2 = f^2
00065      t0 = f^1/1 = f
00066      lnx = 0
00067     */
00068     for (i = 0, fct = (x - 1) / (x + 1), lnx = 0, fct2 = fct * fct, suma = fct;
00069          i < n;
00070          i++)
```



```

00071     {
00072         /* Acumular término t_i */
00073         lnx += suma;
00074
00075         /*
00076          Actualizar al siguiente término:
00077          t_{i+1} = t_i * f^2 * (2i+1)/(2i+3)
00078          (evita recalcular potencias desde cero)
00079          */
00080         suma *= (fct2 * (2*i + 1)) / (2*i + 3);
00081     }
00082
00083     /* ln(x) = 2 * serie */
00084     lnx *= 2;
00085
00086     printf("ln(%f) = %f\n", x, lnx);
00087     return 0;
00088 }

```

Here is the call graph for this function:



7.167 Ejemplo013.c

[Go to the documentation of this file.](#)

```

00001
00034
00035 #include <stdio.h>
00036
00037 int main(int argc, char *argv[])
00038 {
00039     (void)argc;
00040     (void)argv;
00041
00042     int n, i;
00043     float x;          /* valor para ln(x) */
00044     float lnx;         /* acumulador de la serie */
00045     float fct;         /* f = (x-1)/(x+1) */
00046     float fct2;        /* f^2 */
00047     float suma;       /* término actual t_i = f^{2i+1}/(2i+1) */
00048
00049     /* Leer n >= 1 */
00050     do {
00051         printf("Ingresa el numero de terminos: ");
00052         scanf("%d", &n);
00053     } while (n < 1);
00054
00055     /* Leer x > 0 */
00056     do {
00057         printf("Ingresa el valor de x: ");
00058         scanf("%f", &x);
00059     } while (x <= 0);
00060
00061     /*
00062      Inicialización:
00063      f = (x-1)/(x+1)
00064      f2 = f^2
00065      t0 = f^(1)/1 = f
00066      lnx = 0
00067      */
00068     for (i = 0, fct = (x - 1) / (x + 1), lnx = 0, fct2 = fct * fct, suma = fct;
00069          i < n;
00070          i++)
00071     {
00072         /* Acumular término t_i */
00073         lnx += suma;
00074

```

```

00075      /*
00076      Actualizar al siguiente término:
00077      t_{i+1} = t_i * f^2 * (2i+1)/(2i+3)
00078      (evita recalcular potencias desde cero)
00079      */
00080      suma *= (fct2 * (2*i + 1)) / (2*i + 3);
00081  }
00082
00083  /* ln(x) = 2 * serie */
00084  lnx *= 2;
00085
00086  printf("ln(%f) = %f\n", x, lnx);
00087  return 0;
00088 }

```

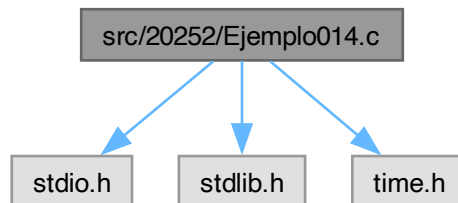
7.168 src/20252/Ejemplo014.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo014.c:



Macros

- `#define N 10000`

Functions

- `int main (int argc, char *argv[ ])`

7.168.1 Macro Definition Documentation

7.168.1.1 N

```
#define N 10000
```

Definition at line 5 of file [Ejemplo014.c](#).

7.168.2 Function Documentation

7.168.2.1 main()

```

int main (
    int argc,
    char * argv[ ])

```

Definition at line 7 of file [Ejemplo014.c](#).

```

00008 {
00009     int i, j, n;
00010     float max, min, x[N], aux;

```

```

00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%d", &n);
00015     }while(n<1||n>N);
00016     printf("Ingrese el valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(min>max)
00021     {
00022         if(max)
00023         {
00024             if(min)
00025             {
00026                 min*=max;
00027                 max=min/max;
00028                 min/=max;
00029             }
00030             else
00031             {
00032                 min = max;
00033                 max = 0;
00034             }
00035         }
00036         else
00037         {
00038             max = min;
00039             min = 0;
00040         }
00041     }
00042     printf("Desordenado.\n");
00043     for(i=0; i<n; i++)
00044     {
00045         x[i] = ((max-min)*rand())/RAND_MAX+min;
00046         printf("X[%d] = %f\n", i+1, x[i]);
00047     }
00048     // Algoritmo Pares y Nones
00049     for(i=0; i<n; i++)
00050     {
00051         for(j=i%2; j<(n+i%2)/2; j++)
00052             if(x[2*j-i%2]>x[2*j+1-i%2])
00053             {
00054                 aux = x[2*j-i%2];
00055                 x[2*j-i%2] = x[2*j+1-i%2];
00056                 x[2*j+1-i%2] = aux;
00057             }
00058     /*
00059     if(i%2)
00060         for(j=1; j<n/2; j++)
00061             x[2*i-1] x[2*i]
00062             x[1] x[2]
00063             x[3] x[4]
00064             x[5] x[6]
00065     else
00066         for(j=0; j<n/2; j++)
00067             x[2*i] x[2*i+1]
00068             x[0] x[1]
00069             x[2] x[3]
00070             x[4] x[5]
00071     */
00072     }
00073     printf("Ordenado.\n");
00074     for(i=0; i<n; i++)
00075         printf("X[%d] = %f\n", i+1, x[i]);
00076     return 0;
00077 }

```

7.169 Ejemplo014.c

[Go to the documentation of this file.](#)

```

00001 #include<stdio.h>
00002 #include<stdlib.h>
00003 #include<time.h>
00004
00005 #define N 10000
00006
00007 int main(int argc, char*argv[])
00008 {
00009     int i, j, n;
00010     float max, min, x[N], aux;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");

```

```

00014     scanf("%d", &n);
00015 }while (n<1||n>N);
00016 printf("Ingresa el valor maximo: ");
00017 scanf("%f", &max);
00018 printf("Ingresa el valor minimo: ");
00019 scanf("%f", &min);
00020 if(min>max)
00021 {
00022     if(max)
00023     {
00024         if(min)
00025         {
00026             min*=max;
00027             max=min/max;
00028             min/=max;
00029         }
00030         else
00031         {
00032             min = max;
00033             max = 0;
00034         }
00035     }
00036     else
00037     {
00038         max = min;
00039         min = 0;
00040     }
00041 }
00042 printf("Desordenado.\n");
00043 for(i=0; i<n; i++)
00044 {
00045     x[i] = ((max-min)*rand())/RAND_MAX+min;
00046     printf("X[%d] = %f\n", i+1, x[i]);
00047 }
00048 // Algoritmo Pares y Nones
00049 for(i=0; i<n; i++)
00050 {
00051     for(j=i%2; j<(n+i%2)/2; j++)
00052         if(x[2*j-i%2]>x[2*j+1-i%2])
00053         {
00054             aux = x[2*j-i%2];
00055             x[2*j-i%2] = x[2*j+1-i%2];
00056             x[2*j+1-i%2] = aux;
00057         }
00058     /*
00059     if(i%2)
00060         for(j=1; j<n/2; j++)
00061             x[2*i-1] x[2*i]
00062             x[1] x[2]
00063             x[3] x[4]
00064             x[5] x[6]
00065     else
00066         for(j=0; j<n/2; j++)
00067             x[2*i] x[2*i+1]
00068             x[0] x[1]
00069             x[2] x[3]
00070             x[4] x[5]
00071     */
00072 }
00073 printf("Ordenado.\n");
00074 for(i=0; i<n; i++)
00075     printf("X[%d] = %f\n", i+1, x[i]);
00076 return 0;
00077 }

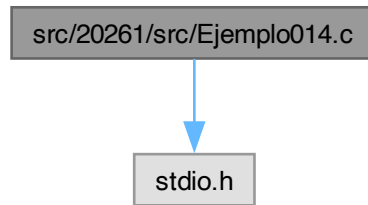
```

7.170 src/20261/src/Ejemplo014.c File Reference

Aproximación de $\sin(x)$ con serie de Taylor calculando potencia y factorial desde cero.

```
#include <stdio.h>
```

Include dependency graph for Ejemplo014.c:



Functions

- int [main](#) (int argc, char *argv[])

7.170.1 Detailed Description

Aproximación de $\sin(x)$ con serie de Taylor calculando potencia y factorial desde cero.

Serie de Taylor: $\sin(x) = \sum_{i=0}^{\infty} \frac{(-1)^i * x^{2i+1}}{(2i+1)!}$

Este programa aproxima con n términos: $\sin(x) \approx \sum_{i=0}^{n-1} \frac{(-1)^i * x^{2i+1}}{(2i+1)!}$

Implementación:

- Para cada i calcula $\text{num} = x^{2i+1}$ y $\text{den} = (2i+1)!$ con un ciclo interno.

Entrada

- Un entero $n \geq 1$.
- Un real x.

Salida

Imprime:

```
sen(x) = valor
```

Complejidad

Tiempo: $O(n^2)$. Memoria: $O(1)$.

Warning

Para $|x|$ grande, puede haber pérdida de precisión (no se hace reducción de rango). Además, el factorial crece rápido (aquí se guarda en int).

Definition in file [Ejemplo014.c](#).

7.170.2 Function Documentation

7.170.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 35 of file [Ejemplo014.c](#).

```

00036 {
00037     (void)argc;
00038     (void)argv;
00039
00040     int n, i, j;
00041     int den;      /* (2i+1)! */
00042     int signo;    /* +1 o -1 */
00043     float x;
00044     float sx;     /* acumulador de sen(x) */
00045     float num;    /* x^(2i+1) */
00046
00047     do {
00048         printf("Ingrese el numero de terminos: ");
00049         scanf("%d", &n);
00050     } while (n < 1);
00051
00052     printf("Ingrese el valor de x: ");
00053     scanf("%f", &x);
00054
00055     for (i = 0, sx = 0; i < n; i++)
00056     {
00057         /* signo = +1 si i par, -1 si i impar */
00058         signo = 1 - 2*(i % 2);
00059
00060         /*
00061          * Calcular:
00062          *   num = x^(2i+1)
00063          *   den = (2i+1)!
00064          * multiplicando desde cero en cada término.
00065          */
00066         for (j = 0, num = 1, den = 1; j < (2*i + 1); j++)
00067         {
00068             num *= x;      /* acumula potencia */
00069             den *= (j + 1); /* acumula factorial */
00070         }
00071
00072         /* Sumar el término: (-1)^i * num/den */
00073         sx += (signo * num / den);
00074     }
00075
00076     printf("sen(%f) = %f\n", x, sx);
00077     return 0;
00078 }

```

7.171 Ejemplo014.c

[Go to the documentation of this file.](#)

```

00001
00032
00033 #include <stdio.h>
00034
00035 int main(int argc, char *argv[])
00036 {
00037     (void)argc;
00038     (void)argv;
00039
00040     int n, i, j;
00041     int den;      /* (2i+1)! */
00042     int signo;    /* +1 o -1 */
00043     float x;
00044     float sx;     /* acumulador de sen(x) */
00045     float num;    /* x^(2i+1) */
00046
00047     do {
00048         printf("Ingrese el numero de terminos: ");
00049         scanf("%d", &n);
00050     } while (n < 1);
00051
00052     printf("Ingrese el valor de x: ");
00053     scanf("%f", &x);
00054
00055     for (i = 0, sx = 0; i < n; i++)
00056     {
00057         /* signo = +1 si i par, -1 si i impar */
00058         signo = 1 - 2*(i % 2);
00059
00060         /*
00061          * Calcular:
00062          *   num = x^(2i+1)
00063          *   den = (2i+1)!
00064          * multiplicando desde cero en cada término.
00065          */
00066         for (j = 0, num = 1, den = 1; j < (2*i + 1); j++)
00067         {

```

```

00068         num *= x;          /* acumula potencia */
00069         den *= (j + 1);     /* acumula factorial */
00070     }
00071
00072     /* Sumar el término: (-1)^i * num/den */
00073     sx += (signo * num / den);
00074 }
00075
00076 printf("sen(%f) = %f\n", x, sx);
00077 return 0;
00078 }

```

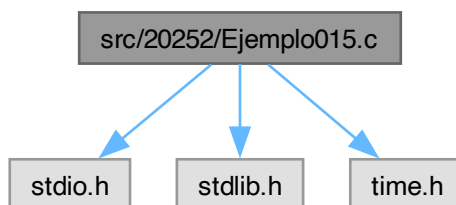
7.172 src/20252/Ejemplo015.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo015.c:



Macros

- `#define N 10000`

Functions

- `int main (int argc, char *argv[ ])`

7.172.1 Macro Definition Documentation

7.172.1.1 N

```
#define N 10000
```

Definition at line 5 of file [Ejemplo015.c](#).

7.172.2 Function Documentation

7.172.2.1 main()

```

int main (
    int argc,
    char * argv[ ])

```

Definition at line 7 of file [Ejemplo015.c](#).

```

00008 {
00009     int i, j, k, n;
00010     float max, min, x[N], aux;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");

```

```

00014     scanf("%d", &n);
00015 }while (n<1||n>N);
00016 printf("Ingrese el valor maximo: ");
00017 scanf("%f", &max);
00018 printf("Ingrese el valor minimo: ");
00019 scanf("%f", &min);
00020 if(min>max)
00021 {
00022     if(max)
00023     {
00024         if(min)
00025         {
00026             min*=max;
00027             max=min/max;
00028             min/=max;
00029         }
00030     }
00031     else
00032     {
00033         min = max;
00034         max = 0;
00035     }
00036 }
00037 else
00038 {
00039     max = min;
00040     min = 0;
00041 }
00042 printf("Desordenado.\n");
00043 for(i=0; i<n; i++)
00044 {
00045     x[i] = ((max-min)*rand())/RAND_MAX+min;
00046     printf("X[%d] = %f\n", i+1, x[i]);
00047 }
00048 // Algoritmo de Seleccion
00049 for(i=0; i<n-1; i++)
00050 {
00051     for(j=i+1, k=i; j<n; j++)
00052         if(x[k]>x[j])
00053             k = j;
00054     if(k!=i)
00055     {
00056         aux = x[i];
00057         x[i] = x[k];
00058         x[k] = aux;
00059     }
00060 }
00061 printf("Ordenado.\n");
00062 for(i=0; i<n; i++)
00063     printf("X[%d] = %f\n", i+1, x[i]);
00064 return 0;
00065 }

```

7.173 Ejemplo015.c

[Go to the documentation of this file.](#)

```

00001 #include<stdio.h>
00002 #include<stdlib.h>
00003 #include<time.h>
00004
00005 #define N 10000
00006
00007 int main(int argc, char*argv[])
00008 {
00009     int i, j, k, n;
00010     float max, min, x[N], aux;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%d", &n);
00015     }while (n<1||n>N);
00016     printf("Ingrese el valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(min>max)
00021     {
00022         if(max)
00023         {
00024             if(min)
00025             {
00026                 min*=max;
00027                 max=min/max;
00028                 min/=max;

```



```

00029         }
00030         else
00031         {
00032             min = max;
00033             max = 0;
00034         }
00035     }
00036     else
00037     {
00038         max = min;
00039         min = 0;
00040     }
00041 }
00042 printf("Desordenado.\n");
00043 for(i=0; i<n; i++)
00044 {
00045     x[i] = ((max-min)*rand())/RAND_MAX+min;
00046     printf("X[%d] = %f\n", i+1, x[i]);
00047 }
00048 // Algoritmo de Selecccion
00049 for(i=0; i<n-1; i++)
00050 {
00051     for(j=i+1, k=i; j<n; j++)
00052         if(x[k]>x[j])
00053             k = j;
00054     if(k!=i)
00055     {
00056         aux = x[i];
00057         x[i] = x[k];
00058         x[k] = aux;
00059     }
00060 }
00061 printf("Ordenado.\n");
00062 for(i=0; i<n; i++)
00063     printf("X[%d] = %f\n", i+1, x[i]);
00064 return 0;
00065 }

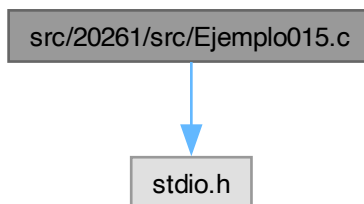
```

7.174 src/20261/src/Ejemplo015.c File Reference

Aproximación de $\sin(x)$ con Taylor usando producto por término (sin factorial entero explícito).

#include <stdio.h>

Include dependency graph for Ejemplo015.c:



Functions

- int [main](#) (int argc, char *argv[])

7.174.1 Detailed Description

Aproximación de $\sin(x)$ con Taylor usando producto por término (sin factorial entero explícito).

Serie: $\sin(x) = \sum_{i=0}^{\infty} \frac{(-1)^i * x^{2i+1}}{(2i+1)!}$

Para cada i , el término: $\frac{x^{2i+1}}{(2i+1)!} = \frac{1}{k!} \frac{x^{2i+1}}{(2i+1)!}$ (x/k)

Este programa calcula cada término como un producto: $fct = (x/1)*(x/2)*...*(x/(2i+1))$ y alterna el signo con $(-1)^i$.

Entrada

- Un entero $n \geq 1$.
- Un real x .

Salida

Imprime:

`sen(x) = valor`

Complejidad

Tiempo: $O(n^2)$. Memoria: $O(1)$.

Note

Evita factorial como entero, pero sigue recalculando cada término desde cero.

Definition in file [Ejemplo015.c](#).

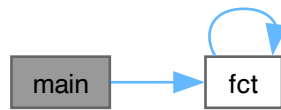
7.174.2 Function Documentation**7.174.2.1 main()**

```
int main (
    int argc,
    char * argv[])
```

Definition at line 35 of file [Ejemplo015.c](#).

```
00036 {
00037     (void)argc;
00038     (void)argv;
00039
00040     int n, i, j;
00041     int signo;
00042     float x;
00043     float sx; /* acumulador */
00044     float fct; /* término positivo: x^(2i+1)/(2i+1)! */
00045
00046     do {
00047         printf("Ingrese el numero de terminos: ");
00048         scanf("%d", &n);
00049     } while (n < 1);
00050
00051     printf("Ingrese el valor de x: ");
00052     scanf("%f", &x);
00053
00054     for (i = 0, sx = 0; i < n; i++)
00055     {
00056         signo = 1 - 2*(i % 2); /* +1, -1, +1, -1... */
00057
00058         /*
00059          fct = _{k=1}^{2i+1} (x/k)
00060          Se implementa con j=0...2i:
00061          multiplica x/(j+1)
00062          */
00063         for (j = 0, fct = 1; j < (2*i + 1); j++)
00064             fct *= (x / (j + 1));
00065
00066         sx += (signo * fct);
00067     }
00068
00069     printf("sen(%f) = %f\n", x, sx);
00070     return 0;
00071 }
```

Here is the call graph for this function:



7.175 Ejemplo015.c

[Go to the documentation of this file.](#)

```

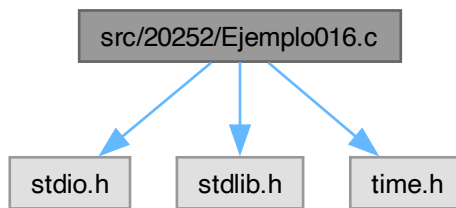
00001
00032
00033 #include <stdio.h>
00034
00035 int main(int argc, char *argv[])
00036 {
00037     (void)argc;
00038     (void)argv;
00039
00040     int n, i, j;
00041     int signo;
00042     float x;
00043     float sx; /* acumulador */
00044     float fct; /* término positivo: x^(2i+1)/(2i+1)! */
00045
00046     do {
00047         printf("Ingrese el numero de terminos: ");
00048         scanf("%d", &n);
00049     } while (n < 1);
00050
00051     printf("Ingrese el valor de x: ");
00052     scanf("%f", &x);
00053
00054     for (i = 0, sx = 0; i < n; i++)
00055     {
00056         signo = 1 - 2*(i % 2); /* +1, -1, +1, -1... */
00057
00058         /*
00059          fct = {k=1}^{2i+1} (x/k)
00060          Se implementa con j=0..2i:
00061          multiplica x/(j+1)
00062          */
00063         for (j = 0, fct = 1; j < (2*i + 1); j++)
00064             fct *= (x / (j + 1));
00065
00066         sx += (signo * fct);
00067     }
00068
00069     printf("sen(%f) = %f\n", x, sx);
00070     return 0;
00071 }
  
```

7.176 src/20252/Ejemplo016.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
  
```

Include dependency graph for Ejemplo016.c:



Macros

- `#define N 10000`

Functions

- `int main (int argc, char *argv[ ])`

7.176.1 Macro Definition Documentation

7.176.1.1 N

`#define N 10000`

Definition at line 5 of file [Ejemplo016.c](#).

7.176.2 Function Documentation

7.176.2.1 main()

```
int main (  
    int argc,  
    char * argv[ ])
```

Definition at line 7 of file [Ejemplo016.c](#).

```
00008 {  
00009     int i, j, k, n;  
00010     float max, min, x[N], aux;  
00011     srand(time(NULL));  
00012     do{  
00013         printf("Ingrese el numero de elementos: ");  
00014         scanf("%d", &n);  
00015     }while(n<1||n>N);  
00016     printf("Ingrese el valor maximo: ");  
00017     scanf("%f", &max);  
00018     printf("Ingrese el valor minimo: ");  
00019     scanf("%f", &min);  
00020     if(min>max)  
00021     {  
00022         if(max)  
00023         {  
00024             if(min)  
00025             {  
00026                 min*=max;  
00027                 max=min/max;  
00028                 min/=max;  
00029             }  
00030             else  
00031             {  
00032                 min = max;  
00033                 max = 0;  
00034             }  
00035         }  
00036     }
```

```

00036         else
00037         {
00038             max = min;
00039             min = 0;
00040         }
00041     }
00042     printf("Desordenado.\n");
00043     for(i=0; i<n; i++)
00044     {
00045         x[i] = ((max-min)*rand())/RAND_MAX+min;
00046         printf("X[%d] = %f\n", i+1, x[i]);
00047     }
00048     // Algoritmo de Insercion
00049     for(i=1; i<n; i++)
00050     {
00051         aux = x[i];
00052         j=i-1;
00053         while(aux<x[j])
00054         {
00055             x[j+1] = x[j];
00056             j--;
00057             if(j<0)
00058                 break;
00059         }
00060         x[j+1] = aux;
00061     }
00062     printf("Ordenado.\n");
00063     for(i=0; i<n; i++)
00064         printf("X[%d] = %f\n", i+1, x[i]);
00065     return 0;
00066 }

```

7.177 Ejemplo016.c

[Go to the documentation of this file.](#)

```

00001 #include<stdio.h>
00002 #include<stdlib.h>
00003 #include<time.h>
00004
00005 #define N 10000
00006
00007 int main(int argc, char*argv[])
00008 {
00009     int i, j, k, n;
00010     float max, min, x[N], aux;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%d", &n);
00015     }while(n<1||n>N);
00016     printf("Ingrese el valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(min>max)
00021     {
00022         if(max)
00023         {
00024             if(min)
00025             {
00026                 min*=max;
00027                 max=min/max;
00028                 min/=max;
00029             }
00030             else
00031             {
00032                 min = max;
00033                 max = 0;
00034             }
00035         }
00036         else
00037         {
00038             max = min;
00039             min = 0;
00040         }
00041     }
00042     printf("Desordenado.\n");
00043     for(i=0; i<n; i++)
00044     {
00045         x[i] = ((max-min)*rand())/RAND_MAX+min;
00046         printf("X[%d] = %f\n", i+1, x[i]);
00047     }
00048     // Algoritmo de Insercion
00049     for(i=1; i<n; i++)

```

```

00050      {
00051          aux = x[i];
00052          j=i-1;
00053          while (aux<x[j])
00054          {
00055              x[j+1] = x[j];
00056              j--;
00057              if (j<0)
00058                  break;
00059          }
00060          x[j+1] = aux;
00061      }
00062      printf("Ordenado.\n");
00063      for (i=0; i<n; i++)
00064          printf("X[%d] = %f\n", i+1, x[i]);
00065      return 0;
00066  }

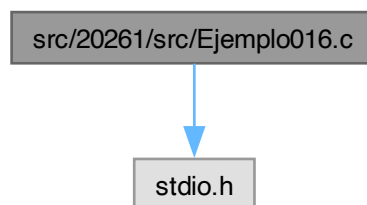
```

7.178 src/20261/src/Ejemplo016.c File Reference

Aproximación de $\sin(x)$ con Taylor usando recurrencia entre términos ($O(n)$).

#include <stdio.h>

Include dependency graph for Ejemplo016.c:



Functions

- int [main](#) (int argc, char *argv[])

7.178.1 Detailed Description

Aproximación de $\sin(x)$ con Taylor usando recurrencia entre términos ($O(n)$).

Serie: $\sin(x) = \sum_{i=0}^{\infty} (-1)^i * x^{2i+1} / (2i+1)!$

Definimos el término positivo: $t_i = x^{2i+1} / (2i+1)!$ con recurrencia: $t_0 = x$ $t_{i+1} = t_i * (x^2) / [(2i+2)(2i+3)]$

Luego: $\sin(x) = \sum_{i=0}^{n-1} (-1)^i * t_i$

Entrada

- Un entero $n \geq 1$.
- Un real x .

Salida

Imprime:

$\sin(x) = \text{valor}$

Complejidad

Tiempo: $O(n)$. Memoria: $O(1)$.

Note

Es la versión más eficiente (respecto a Ejemplo014 y Ejemplo015) porque evita recalcular potencias/↔ factoriales en cada término.

Definition in file [Ejemplo016.c](#).

7.178.2 Function Documentation**7.178.2.1 main()**

```
int main (
    int argc,
    char * argv[])
```

Definition at line 38 of file [Ejemplo016.c](#).

```
00039 {
00040     (void)argc;
00041     (void)argv;
00042
00043     int n, i;
00044     int signo; /* +1 o -1 alternando */
00045     float x;
00046     float sx; /* acumulador */
00047     float fct; /* término positivo t_i = x^(2i+1)/(2i+1)! */
00048
00049     do {
00050         printf("Ingrese el numero de terminos: ");
00051         scanf("%d", &n);
00052     } while (n < 1);
00053
00054     printf("Ingrese el valor de x: ");
00055     scanf("%f", &x);
00056
00057     /*
00058     Inicialización:
00059     t0 = x
00060     sx = 0
00061     En cada iteración:
00062     sx += (-1)^i * t_i
00063     t_{i+1} = t_i * (x^2) / [(2i+2) (2i+3)]
00064     */
00065     for (i = 0, sx = 0, fct = x; i < n; i++)
00066     {
00067         signo = 1 - 2*(i % 2); /* +1 si i par; -1 si i impar */
00068
00069         sx += (signo * fct);
00070
00071         /* Actualizar al siguiente término positivo */
00072         fct *= (x / (2*i + 2)) * (x / (2*i + 3));
00073     }
00074
00075     printf("sen(%f) = %f\n", x, sx);
00076     return 0;
00077 }
```

Here is the call graph for this function:

**7.179 Ejemplo016.c**

[Go to the documentation of this file.](#)

00001

```

00035
00036 #include <stdio.h>
00037
00038 int main(int argc, char *argv[])
00039 {
00040     (void)argc;
00041     (void)argv;
00042
00043     int n, i;
00044     int signo; /* +1 o -1 alternando */
00045     float x;
00046     float sx; /* acumulador */
00047     float fct; /* término positivo t_i = x^(2i+1)/(2i+1)! */
00048
00049     do {
00050         printf("Ingrese el numero de terminos: ");
00051         scanf("%d", &n);
00052     } while (n < 1);
00053
00054     printf("Ingrese el valor de x: ");
00055     scanf("%f", &x);
00056
00057     /*
00058     Inicialización:
00059         t0 = x
00060         sx = 0
00061     En cada iteración:
00062         sx += (-1)^i * t_i
00063         t_{i+1} = t_i * (x^2)/[(2i+2)(2i+3)]
00064     */
00065     for (i = 0, sx = 0, fct = x; i < n; i++)
00066     {
00067         signo = 1 - 2*(i % 2); /* +1 si i par; -1 si i impar */
00068
00069         sx += (signo * fct);
00070
00071         /* Actualizar al siguiente término positivo */
00072         fct *= (x / (2*i + 2)) * (x / (2*i + 3));
00073     }
00074
00075     printf("sen(%f) = %f\n", x, sx);
00076     return 0;
00077 }

```

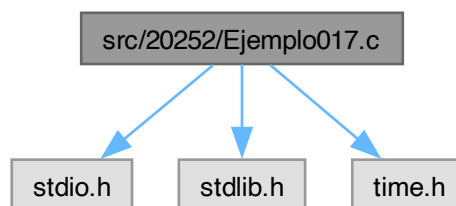
7.180 src/20252/Ejemplo017.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo017.c:



Macros

- #define N 10000

Functions

- `int main (int argc, char *argv[ ])`

7.180.1 Macro Definition Documentation

7.180.1.1 N

#define N 10000

Definition at line 5 of file [Ejemplo017.c](#).

7.180.2 Function Documentation

7.180.2.1 main()

```
int main (
    int argc,
    char * argv[ ])
```

Definition at line 7 of file [Ejemplo017.c](#).

```
00008 {
00009     int i, j, k, l, n, nc, cs[N], id[N], ncs;
00010     float max, min, x[N], xs[N], aux, rng_c;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%d", &n);
00015     }while(n<1||n>N);
00016     printf("Ingrese el valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(min>max)
00021     {
00022         if(max)
00023         {
00024             if(min)
00025             {
00026                 min*=max;
00027                 max=min/max;
00028                 min/=max;
00029             }
00030             else
00031             {
00032                 min = max;
00033                 max = 0;
00034             }
00035         }
00036         else
00037         {
00038             max = min;
00039             min = 0;
00040         }
00041     }
00042     printf("Desordenado.\n");
00043     for(i=0; i<n; i++)
00044     {
00045         x[i] = ((max-min)*rand())/RAND_MAX+min;
00046         cs[i] = 0;
00047         printf("X[%d] = %f\n", i+1, x[i]);
00048     }
00049     // Algoritmo de Bucket
00050     nc = 0;
00051     aux = n;
00052     while(aux>(2*nc+1))
00053     {
00054         aux -= (2*nc+1);
00055         nc++;
00056     }
00057     rng_c = (max-min)/nc;
00058     for(i=0; i<n; i++)
00059         cs[(int)((x[i]-min)/rng_c)]++;
00060     for(i=1, id[0]=0; i<n; i++)
00061         id[i] = id[i-1]+cs[i-1];
00062     for(i=0; i<n; i++)
00063     {
00064         ncs = (int)((x[i]-min)/rng_c);
00065         xs[id[ncs]] = x[i];
00066         id[ncs]++;
```

```

00067     }
00068     printf("%d\t%f\n", nc, rng_c);
00069     for(i=0; i<nc; i++)
00070         printf("CS[%d] = %d\n", i, cs[i]);
00071     for(i=0; i<nc; i++)
00072         printf("%f\t", min+rng_c*(i+1));
00073     printf("Pseudo-ordenado.\n");
00074     for(i=0; i<n; i++)
00075         printf("X[%d] = %f\t%d\t%f\n", i+1, x[i], (int)((x[i]-min)/rng_c), xs[i]);
00076     for(i=1, id[0]=0; i<n; i++)
00077         id[i] = id[i-1]+cs[i-1];
00078     for(l=0; l<nc; l++)
00079     {
00080         // Algoritmo de Seleccion
00081         for(i=id[l]; i<(id[l]+cs[l])-1; i++)
00082         {
00083             for(j=i+1, k=i; j<(id[l]+cs[l]); j++)
00084                 if(xs[k]>xs[j])
00085                     k = j;
00086             if(k!=i)
00087             {
00088                 aux = xs[i];
00089                 xs[i] = xs[k];
00090                 xs[k] = aux;
00091             }
00092         }
00093     }
00094     printf("Ordenado.\n");
00095     for(i=0; i<n; i++)
00096         printf("X[%d] = %f\n", i+1, xs[i]);
00097     return 0;
00098 }

```

7.181 Ejemplo017.c

[Go to the documentation of this file.](#)

```

00001 #include<stdio.h>
00002 #include<stdlib.h>
00003 #include<time.h>
00004
00005 #define N 10000
00006
00007 int main(int argc, char*argv[])
00008 {
00009     int i, j, k, l, n, nc, cs[N], id[N], ncs;
00010     float max, min, x[N], xs[N], aux, rng_c;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%d", &n);
00015     }while(n<1||n>N);
00016     printf("Ingrese el valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(min>max)
00021     {
00022         if(max)
00023         {
00024             if(min)
00025             {
00026                 min*=max;
00027                 max=min/max;
00028                 min/=max;
00029             }
00030             else
00031             {
00032                 min = max;
00033                 max = 0;
00034             }
00035         }
00036         else
00037         {
00038             max = min;
00039             min = 0;
00040         }
00041     }
00042     printf("Desordenado.\n");
00043     for(i=0; i<n; i++)
00044     {
00045         x[i] = ((max-min)*rand())/RAND_MAX+min;
00046         cs[i] = 0;
00047         printf("X[%d] = %f\n", i+1, x[i]);
00048     }

```

```

00049 // Algoritmo de Bucket
00050 nc = 0;
00051 aux = n;
00052 while(aux>(2*nc+1))
00053 {
00054     aux -= (2*nc+1);
00055     nc++;
00056 }
00057 rng_c = (max-min)/nc;
00058 for(i=0; i<n; i++)
00059     cs[(int)((x[i]-min)/rng_c)]++;
00060 for(i=1, id[0]=0; i<n; i++)
00061     id[i] = id[i-1]+cs[i-1];
00062 for(i=0; i<n; i++)
00063 {
00064     ncs = (int)((x[i]-min)/rng_c);
00065     xs[id[ncs]] = x[i];
00066     id[ncs]++;
00067 }
00068 printf("%d\t%f\n", nc, rng_c);
00069 for(i=0; i<nc; i++)
00070     printf("CS[%d] = %d\n", i, cs[i]);
00071 for(i=0; i<nc; i++)
00072     printf("%f\t", min+rng_c*(i+1));
00073 printf("Pseudo-ordenado.\n");
00074 for(i=0; i<n; i++)
00075     printf("X[%d] = %f\t%d\t%f\n", i+1, x[i], (int)((x[i]-min)/rng_c), xs[i]);
00076 for(i=1, id[0]=0; i<n; i++)
00077     id[i] = id[i-1]+cs[i-1];
00078 for(l=0; l<nc; l++)
00079 {
00080     // Algoritmo de Seleccion
00081     for(i=id[l]; i<(id[l]+cs[l])-1; i++)
00082     {
00083         for(j=i+1, k=i; j<(id[l]+cs[l]); j++)
00084             if(xs[k]>xs[j])
00085                 k = j;
00086         if(k!=i)
00087         {
00088             aux = xs[i];
00089             xs[i] = xs[k];
00090             xs[k] = aux;
00091         }
00092     }
00093 }
00094 printf("Ordenado.\n");
00095 for(i=0; i<n; i++)
00096     printf("X[%d] = %f\n", i+1, xs[i]);
00097 return 0;
00098 }

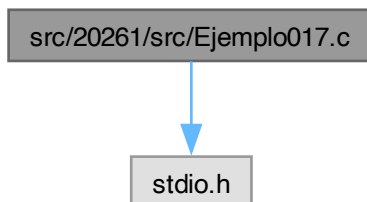
```

7.182 src/20261/src/Ejemplo017.c File Reference

Aproximación de $\arcsin(x)$ mediante serie de Taylor (Maclaurin) para $|x| < 1$.

#include <stdio.h>

Include dependency graph for Ejemplo017.c:



Functions

- `int main (int argc, char *argv[ ])`

Punto de entrada. Aproxima $\arcsin(x)$ con n términos de la serie.

7.182.1 Detailed Description

Aproximación de $\arcsin(x)$ mediante serie de Taylor (Maclaurin) para $|x| < 1$.

Este programa aproxima el seno inverso (arcoseno):

$$\arcsin(x)$$

usando la serie de Maclaurin (Taylor alrededor de 0), válida para $|x| < 1$:

$$\arcsin(x) = \sum_{i=0}^{\infty} \frac{(2i)!}{4^i (i!)^2 (2i+1)} x^{2i+1}, \quad |x| < 1.$$

En lugar de calcular factoriales grandes, la implementación construye cada término de forma incremental:

- `num` acumula la potencia x^{2i+1} a partir de la anterior ($\times x^2$).
- `fact` acumula el coeficiente $C_i = \frac{(2i)!}{4^i (i!)^2}$ mediante la recurrencia:

$$C_i = C_{i-1} \cdot \frac{2i-1}{2i}.$$

Cada iteración suma:

$$\frac{C_i x^{2i+1}}{2i+1}.$$

Entrada estándar

- 1) Un entero n (número de términos), con $n \geq 1$
- 2) Un real x con $-1 < x < 1$

Salida estándar

Imprime la aproximación:

`asen(x) = valor`

Precondiciones

- Se requiere $-1 < x < 1$. La serie converge más lentamente cuando $|x| \rightarrow 1$.

Complejidad

Tiempo: $O(n)$. Memoria: $O(1)$.

Note

- El programa usa `float`; si se desea mayor precisión, usar `double`.
- El nombre impreso es `asen(x)` (arcoseno).

Definition in file [Ejemplo017.c](#).

7.182.2 Function Documentation

7.182.2.1 main()

```
int main (
    int argc,
    char * argv[ ])
```

Punto de entrada. Aproxima $\arcsin(x)$ con n términos de la serie.

Parameters

<code>argc</code>	No usado.
-------------------	-----------

<code>argv</code>	No usado.
-------------------	-----------

Returns

0 si finaliza correctamente.

Definition at line 59 of file [Ejemplo017.c](#).

```

00060 {
00061     (void)argc;
00062     (void)argv;
00063
00064     /*
00065      n   : número de términos de la serie (n >= 1)
00066      i   : índice del término (inicia en 1 porque el término i=0 se inicializa con asx=x)
00067      den : denominador impar (2i+1) del término
00068      x   : valor para evaluar arcsin(x), con -1 < x < 1
00069      asx : acumulador de la suma (aproximación final)
00070      num : potencia acumulada x^(2i+1)
00071      fct : coeficiente acumulado C_i = (2i)! / (4^i (i!)^2)
00072     */
00073     int n, i, den;
00074     float x, asx, num, fct;
00075
00076     /* Leer n >= 1 */
00077     do {
00078         printf("Ingrese el numero de terminos: ");
00079         scanf("%d", &n);
00080     } while (n < 1);
00081
00082     /* Leer x con -1 < x < 1 */
00083     do {
00084         printf("Ingrese el valor de x: ");
00085         scanf("%f", &x);
00086     } while (-1 >= x || x >= 1);
00087
00088     /*
00089      Término inicial (i = 0):
00090      C_0 = 1
00091      x^(2*0+1) = x
00092      (2*0+1) = 1
00093      => arcsin(x)  x
00094     */
00095     for (i = 1, asx = x, num = x, fct = 1; i < n; i++)
00096     {
00097         /*
00098          Actualizar coeficiente:
00099          C_i = C_{i-1} * (2i-1)/(2i)
00100          Se usa 2*i - 1.0f para forzar división en punto flotante.
00101         */
00102         fct *= ((2*i - 1.0f) / (2*i));
00103
00104         /*
00105          Actualizar potencia:
00106          x^(2i+1) = x^(2i-1) * x^2
00107         */
00108         num *= (x * x);
00109
00110         /* Denominador del término: (2i + 1) */
00111         den = (2*i + 1);
00112
00113         /* Sumar término: (C_i * x^(2i+1)) / (2i+1) */
00114         asx += (fct * num / den);
00115     }
00116
00117     printf("asen(%f) = %f\n", x, asx);
00118     return 0;
00119 }
```

Here is the call graph for this function:



7.183 Ejemplo017.c

[Go to the documentation of this file.](#)

```

00001
00050
00051 #include <stdio.h>
00052
00059 int main(int argc, char *argv[])
00060 {
00061     (void)argc;
00062     (void)argv;
00063
00064     /*
00065      n   : número de términos de la serie (n >= 1)
00066      i   : índice del término (inicia en 1 porque el término i=0 se inicializa con asx=x)
00067      den : denominador impar (2i+1) del término
00068      x   : valor para evaluar arcsin(x), con -1 < x < 1
00069      asx : acumulador de la suma (aproximación final)
00070      num : potencia acumulada x^(2i+1)
00071      fct : coeficiente acumulado C_i = (2i)! / (4^i (i!)^2)
00072     */
00073     int n, i, den;
00074     float x, asx, num, fct;
00075
00076     /* Leer n >= 1 */
00077     do {
00078         printf("Ingrese el numero de terminos: ");
00079         scanf("%d", &n);
00080     } while (n < 1);
00081
00082     /* Leer x con -1 < x < 1 */
00083     do {
00084         printf("Ingrese el valor de x: ");
00085         scanf("%f", &x);
00086     } while (-1 >= x || x >= 1);
00087
00088     /*
00089      Término inicial (i = 0):
00090      C_0 = 1
00091      x^(2*0+1) = x
00092      (2*0+1) = 1
00093      => arcsin(x)  x
00094     */
00095     for (i = 1, asx = x, num = x, fct = 1; i < n; i++)
00096     {
00097         /*
00098          Actualizar coeficiente:
00099          C_i = C_{i-1} * (2i-1)/(2i)
00100          Se usa 2*i - 1.0f para forzar división en punto flotante.
00101         */
00102         fct *= ((2*i - 1.0f) / (2*i));
00103
00104         /*
00105          Actualizar potencia:
00106          x^(2i+1) = x^(2i-1) * x^2
00107         */
00108         num *= (x * x);
00109
00110         /* Denominador del término: (2i + 1) */
00111         den = (2*i + 1);
00112
00113         /* Sumar término: (C_i * x^(2i+1)) / (2i+1) */
00114         asx += (fct * num / den);
00115     }

```

```

00116
00117     printf("asen(%f) = %f\n", x, asx);
00118     return 0;
00119 }

```

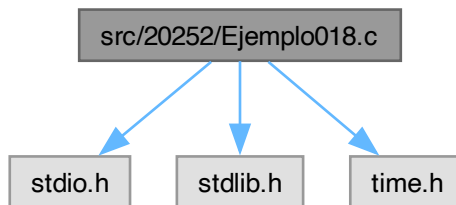
7.184 src/20252/Ejemplo018.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo018.c:



Macros

- #define [N](#) 1000000

Functions

- int [main](#) (int argc, char *argv[])

7.184.1 Macro Definition Documentation

7.184.1.1 N

```
#define N 1000000
```

Definition at line 5 of file [Ejemplo018.c](#).

7.184.2 Function Documentation

7.184.2.1 main()

```

int main (
    int argc,
    char * argv[])

```

Definition at line 7 of file [Ejemplo018.c](#).

```

00008 {
00009     int i, j, k, n;
00010     float max, min, x[N], aux;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%d", &n);
00015     }while(n<1||n>N);
00016     printf("Ingrese el valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(min>max)

```

```

00021     {
00022         if(max)
00023         {
00024             if(min)
00025             {
00026                 min*=max;
00027                 max=min/max;
00028                 min/=max;
00029             }
00030             else
00031             {
00032                 min = max;
00033                 max = 0;
00034             }
00035         }
00036         else
00037         {
00038             max = min;
00039             min = 0;
00040         }
00041     }
00042     printf("Desordenado.\n");
00043     for(i=0; i<n; i++)
00044     {
00045         x[i] = ((max-min)*rand())/RAND_MAX+min;
00046         printf("X[%d] = %f\n", i+1, x[i]);
00047     }
00048     // Algoritmo de Bidireccional
00049     for(i=0; i<n-1; i++)
00050     {
00051         for(j=(i%2?n-1-i/2:i/2+1);
00052             (i%2?j>=(i/2+1):(j<=n-1-i/2));
00053             i%2?j--:j++)
00054         {
00055             if(i%2?(x[i%2?n-i/2-1:i/2]<x[j])
00056                 :(x[i%2?n-i/2-1:i/2]>x[j]))
00057             {
00058                 aux = x[j];
00059                 x[j] = x[i%2?n-i/2-1:i/2];
00060                 x[i%2?n-i/2-1:i/2] = aux;
00061             }
00062             // printf("i=%d\tx[k], k=%d\tj=%d\n", i,
00063             //         i%2?n-i/2-1:i/2, j);
00064         }
00065     }
00066     printf("Ordenado.\n");
00067     for(i=0; i<n; i++)
00068         printf("X[%d] = %f\n", i+1, x[i]);
00069     return 0;
00070 }

```

7.185 Ejemplo018.c

[Go to the documentation of this file.](#)

```

00001 #include<stdio.h>
00002 #include<stdlib.h>
00003 #include<time.h>
00004
00005 #define N 1000000
00006
00007 int main(int argc, char*argv[])
00008 {
00009     int i, j, k, n;
00010     float max, min, x[N], aux;
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%d", &n);
00015     }while(n<1||n>N);
00016     printf("Ingrese el valor maximo: ");
00017     scanf("%f", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%f", &min);
00020     if(min>max)
00021     {
00022         if(max)
00023         {
00024             if(min)
00025             {
00026                 min*=max;
00027                 max=min/max;
00028                 min/=max;
00029             }
00030             else

```



```

00031         {
00032             min = max;
00033             max = 0;
00034         }
00035     }
00036     else
00037     {
00038         max = min;
00039         min = 0;
00040     }
00041 }
00042 printf("Desordenado.\n");
00043 for(i=0; i<n; i++)
00044 {
00045     x[i] = ((max-min)*rand())/RAND_MAX+min;
00046     printf("X[%d] = %f\n", i+1, x[i]);
00047 }
00048 // Algoritmo de Bidireccional
00049 for(i=0; i<n-1; i++)
00050 {
00051     for(j=(i%2?n-1-i/2:i/2+1);
00052         (i%2?j>=(i/2+1):(j<=n-1-i/2));
00053         i%2?j--:j++)
00054     {
00055         if(i%2?(x[i%2?n-i/2-1:i/2]<x[j])
00056             :(x[i%2?n-i/2-1:i/2]>x[j]))
00057         {
00058             aux = x[j];
00059             x[j] = x[i%2?n-i/2-1:i/2];
00060             x[i%2?n-i/2-1:i/2] = aux;
00061         }
00062         // printf("i=%d\tx[k], k=%d\tj=%d\n", i,
00063         //         i%2?n-i/2-1:i/2, j);
00064     }
00065 }
00066 printf("Ordenado.\n");
00067 for(i=0; i<n; i++)
00068     printf("X[%d] = %f\n", i+1, x[i]);
00069 return 0;
00070 }

```

7.186 src/20261/src/Ejemplo018.c File Reference

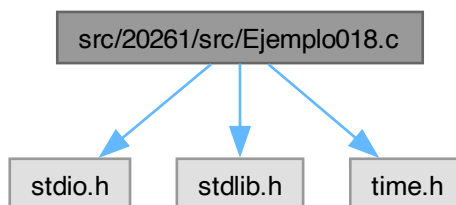
Generación de un arreglo de enteros aleatorios en el rango [min, max].

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo018.c:



Macros

- #define `N` 100000

Functions

- int `main` (int argc, char *argv[])

7.186.1 Detailed Description

Generación de un arreglo de enteros aleatorios en el rango $[\text{min}, \text{max}]$.

El programa: 1) Solicita al usuario el número de elementos n ($1 \leq n \leq N$). 2) Solicita un valor máximo y un valor mínimo. 3) Si el usuario ingresa $\text{min} > \text{max}$, los intercambia. 4) Inicializa el generador pseudoaleatorio con `srand(time(NULL))`. 5) Genera n enteros aleatorios uniformemente distribuidos (aprox.) en el intervalo $[\text{min}, \text{max}]$ usando: `rand() % (max - min + 1) + min` 6) Imprime todos los elementos generados.

Entrada estándar

- `n` (long int): número de elementos (1..N)
- `max` (int): valor máximo permitido
- `min` (int): valor mínimo permitido

Salida estándar

Imprime el arreglo generado:

```
x[1] = ...
x[2] = ...
...
x[n] = ...
```

Restricciones

- `N` es el tamaño máximo del arreglo (100000).
- Se asume que `max - min + 1` cabe en `int`.

Complejidad

Tiempo: $O(n)$. Memoria: $O(N)$ por el arreglo estático.

Note

- La operación `rand() % rango` puede introducir “sesgo de módulo” si el rango no divide a `RAND_MAX`. Para fines didácticos suele ser suficiente.

```
gcc Ejemplo018.c -o Ejemplo018
./Ejemplo018
```

Definition in file [Ejemplo018.c](#).

7.186.2 Macro Definition Documentation

7.186.2.1 N

```
#define N 100000
```

Tamaño máximo permitido del arreglo.

Definition at line 52 of file [Ejemplo018.c](#).

7.186.3 Function Documentation

7.186.3.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 54 of file [Ejemplo018.c](#).

```
00055 {
00056     (void)argc;
00057     (void)argv;
00058
00059     long int n, i;          /* n: número de elementos, i: índice de recorrido */
00060     int min, max;          /* límites del rango */
00061     int x[N];              /* arreglo con capacidad máxima N */
00062 }
```

```

00063      /* Leer n y validar que esté en el rango [1, N]. */
00064      do {
00065          printf("Ingrese el numero de elementos: ");
00066          scanf("%ld", &n);
00067      } while (n < 1 || n > N);
00068
00069      /* Leer límites del rango. */
00070      printf("Ingrese el valor maximo: ");
00071      scanf("%d", &max);
00072
00073      printf("Ingrese el valor minimo: ");
00074      scanf("%d", &min);
00075
00076      /*
00077      Si el usuario capturó min > max, se intercambian.
00078      Se usa intercambio sin variable temporal (suma/resta).
00079      Nota: para valores extremos podría haber overflow; en uso típico es OK.
00080      */
00081      if (min > max)
00082      {
00083          max += min;
00084          min = max - min;
00085          max -= min;
00086      }
00087
00088      /*
00089      Inicializar semilla del generador pseudoaleatorio.
00090      time(NULL) cambia cada segundo, así que los valores cambian en cada ejecución.
00091      */
00092      srand(time(NULL));
00093
00094      /*
00095      Generación:
00096      rand() % (max - min + 1) produce valores en [0, max-min]
00097      luego + min desplaza al intervalo [min, max].
00098      */
00099      for (i = 0; i < n; i++)
00100      {
00101          x[i] = rand() % (max - min + 1) + min;
00102          printf("x[%ld] = %d\n", i + 1, x[i]); /* se imprime desde 1 por estética */
00103      }
00104
00105      return 0;
00106 }

```

7.187 Ejemplo018.c

[Go to the documentation of this file.](#)

```

00001
00046
00047 #include <stdio.h>
00048 #include <stdlib.h>
00049 #include <time.h>
00050
00052 #define N 100000
00053
00054 int main(int argc, char *argv[])
00055 {
00056     (void)argc;
00057     (void)argv;
00058
00059     long int n, i;          /* n: número de elementos, i: índice de recorrido */
00060     int min, max;          /* límites del rango */
00061     int x[N];              /* arreglo con capacidad máxima N */
00062
00063     /* Leer n y validar que esté en el rango [1, N]. */
00064     do {
00065         printf("Ingrese el numero de elementos: ");
00066         scanf("%ld", &n);
00067     } while (n < 1 || n > N);
00068
00069     /* Leer límites del rango. */
00070     printf("Ingrese el valor maximo: ");
00071     scanf("%d", &max);
00072
00073     printf("Ingrese el valor minimo: ");
00074     scanf("%d", &min);
00075
00076     /*
00077     Si el usuario capturó min > max, se intercambian.
00078     Se usa intercambio sin variable temporal (suma/resta).
00079     Nota: para valores extremos podría haber overflow; en uso típico es OK.
00080     */
00081     if (min > max)

```

```

00082     {
00083         max += min;
00084         min = max - min;
00085         max -= min;
00086     }
00087
00088     /*
00089     Inicializar semilla del generador pseudoaleatorio.
00090     time(NULL) cambia cada segundo, así que los valores cambian en cada ejecución.
00091     */
00092     srand(time(NULL));
00093
00094     /*
00095     Generación:
00096     rand() % (max - min + 1) produce valores en [0, max-min]
00097     luego + min desplaza al intervalo [min, max].
00098     */
00099     for (i = 0; i < n; i++)
00100     {
00101         x[i] = rand() % (max - min + 1) + min;
00102         printf("x[%ld] = %d\n", i + 1, x[i]); /* se imprime desde 1 por estética */
00103     }
00104
00105     return 0;
00106 }

```

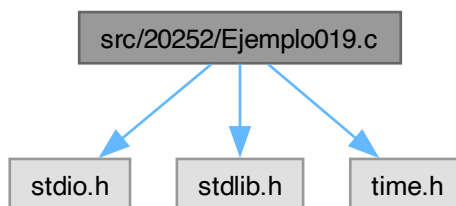
7.188 src/20252/Ejemplo019.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo019.c:



Macros

- `#define N 1000000`

Functions

- `int main (int argc, char *argv[])`

7.188.1 Macro Definition Documentation

7.188.1.1 N

```
#define N 1000000
```

Definition at line 5 of file [Ejemplo019.c](#).

7.188.2 Function Documentation

7.188.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 7 of file [Ejemplo019.c](#).

```
00008 {
00009     int i, j, k, n;
00010     int max, min, x[N], h[N];
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%d", &n);
00015     }while(n<1||n>N);
00016     printf("Ingrese el valor maximo: ");
00017     scanf("%d", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%d", &min);
00020     if(min>max)
00021     {
00022         if(max)
00023         {
00024             if(min)
00025             {
00026                 min+=max;
00027                 max=min-max;
00028                 min-=max;
00029             }
00030             else
00031             {
00032                 min = max;
00033                 max = 0;
00034             }
00035         }
00036         else
00037         {
00038             max = min;
00039             min = 0;
00040         }
00041     }
00042     printf("Desordenado.\n");
00043     for(i=0; i<n; i++)
00044     {
00045         x[i] = rand()%(max-min+1)+min;
00046         printf("X[%d] = %d\n", i+1, x[i]);
00047     }
00048     // Algoritmo de Conteo
00049     for(i=0; i<(max-min+1); i++)
00050         h[i]=0;
00051     for(i=0; i<n; i++)
00052         h[x[i]-min]++;
00053     for(j=0, i=0; j<(max-min+1); j++)
00054     {
00055         while(h[j])
00056         {
00057             x[i] = j+min;
00058             h[j]--;
00059             i++;
00060         }
00061         if(i==n)
00062             break;
00063     }
00064     printf("Ordenado.\n");
00065     for(i=0; i<n; i++)
00066         printf("X[%d] = %d\n", i+1, x[i]);
00067     return 0;
00068 }
```

7.189 Ejemplo019.c

[Go to the documentation of this file.](#)

```
00001 #include<stdio.h>
00002 #include<stdlib.h>
00003 #include<time.h>
00004
00005 #define N 1000000
00006
00007 int main(int argc, char*argv[])
00008 {
```

```

00009     int i, j, k, n;
00010     int max, min, x[N], h[N];
00011     srand(time(NULL));
00012     do{
00013         printf("Ingrese el numero de elementos: ");
00014         scanf("%d", &n);
00015     }while(n<1||n>N);
00016     printf("Ingrese el valor maximo: ");
00017     scanf("%d", &max);
00018     printf("Ingrese el valor minimo: ");
00019     scanf("%d", &min);
00020     if(min>max)
00021     {
00022         if(max)
00023         {
00024             if(min)
00025             {
00026                 min+=max;
00027                 max=min-max;
00028                 min-=max;
00029             }
00030             else
00031             {
00032                 min = max;
00033                 max = 0;
00034             }
00035         }
00036         else
00037         {
00038             max = min;
00039             min = 0;
00040         }
00041     }
00042     printf("Desordenado.\n");
00043     for(i=0; i<n; i++)
00044     {
00045         x[i] = rand()%(max-min+1)+min;
00046         printf("X[%d] = %d\n", i+1, x[i]);
00047     }
00048     // Algoritmo de Conteo
00049     for(i=0; i<(max-min+1); i++)
00050         h[i]=0;
00051     for(i=0; i<n; i++)
00052         h[x[i]-min]++;
00053     for(j=0, i=0; j<(max-min+1); j++)
00054     {
00055         while (h[j])
00056         {
00057             x[i] = j+min;
00058             h[j]--;
00059             i++;
00060         }
00061         if(i==n)
00062             break;
00063     }
00064     printf("Ordenado.\n");
00065     for(i=0; i<n; i++)
00066         printf("X[%d] = %d\n", i+1, x[i]);
00067     return 0;
00068 }

```

7.190 src/20261/src/Ejemplo019.c File Reference

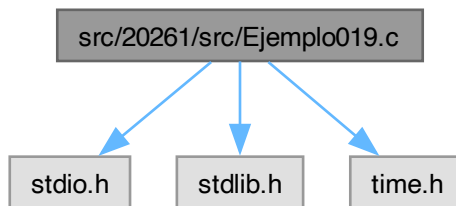
Generación y ordenamiento de un arreglo de enteros aleatorios (orden ascendente).

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo019.c:



Macros

- `#define N 100000`

Functions

- `int main (int argc, char *argv[ ])`

7.190.1 Detailed Description

Generación y ordenamiento de un arreglo de enteros aleatorios (orden ascendente).

El programa: 1) Solicita n ($1 \leq n \leq N$), y los límites `min` y `max`. 2) Si `min` > `max`, intercambia los valores. 3) Genera un arreglo `x` de tamaño n con enteros aleatorios en $[\text{min}, \text{max}]$. 4) Muestra el arreglo “Desordenado”. 5) Ordena el arreglo en orden ascendente mediante un algoritmo $O(n^2)$ basado en comparación e intercambio (similar a selección/intercambio):

- Para cada i , compara `x[i]` con `x[j]` ($j > i$) y si `x[i] > x[j]`, intercambia. 6) Muestra el arreglo “Ordenado”.

Entrada estándar

- `n` (long int): número de elementos ($1..N$)
- `max` (int): valor máximo permitido
- `min` (int): valor mínimo permitido

Salida estándar

- Arreglo desordenado (generado).
- Arreglo ordenado en forma ascendente.

Restricciones

- `N = 100000` (capacidad máxima del arreglo).

Complejidad

- Generación + despliegue: $O(n)$
- Ordenamiento: $O(n^2)$ por los dos ciclos anidados
- Memoria: $O(N)$

Note

Para n grande (cerca de 100000), $O(n^2)$ será muy lento. Este ejemplo se usa con propósitos didácticos para mostrar un ordenamiento básico.

```
gcc Ejemplo019.c -o Ejemplo019
./Ejemplo019
```

Definition in file [Ejemplo019.c](#).

7.190.2 Macro Definition Documentation

7.190.2.1 N

#define N 100000

Tamaño máximo permitido del arreglo.

Definition at line 48 of file [Ejemplo019.c](#).

7.190.3 Function Documentation

7.190.3.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 50 of file [Ejemplo019.c](#).

```
00051 {
00052     (void)argc;
00053     (void)argv;
00054
00055     long int n, i, j; /* n: tamaño; i,j: índices para recorrido/ordenamiento */
00056     int min, max; /* límites del rango */
00057     int x[N]; /* arreglo */
00058     int aux; /* auxiliar para intercambio */
00059
00060     /* Leer n y validar. */
00061     do {
00062         printf("Ingrese el numero de elementos: ");
00063         scanf("%ld", &n);
00064     } while (n < 1 || n > N);
00065
00066     /* Leer límites (max y min). */
00067     printf("Ingrese el valor maximo: ");
00068     scanf("%d", &max);
00069
00070     printf("Ingrese el valor minimo: ");
00071     scanf("%d", &min);
00072
00073     /* Asegurar min <= max intercambiando si es necesario. */
00074     if (min > max)
00075     {
00076         max += min;
00077         min = max - min;
00078         max -= min;
00079     }
00080
00081     /* Semilla para valores aleatorios distintos por ejecución. */
00082     srand(time(NULL));
00083
00084     /* Generar e imprimir arreglo inicial (sin ordenar). */
00085     printf("Desordenado.\n");
00086     for (i = 0; i < n; i++)
00087     {
00088         x[i] = rand() % (max - min + 1) + min;
00089         printf("x[%ld] = %d\n", i + 1, x[i]);
00090     }
00091
00092     /*
00093     Ordenamiento ascendente por comparación e intercambio:
00094     - Para cada i, se busca si existe algún x[j] (j>i) menor que x[i].
00095     - Si x[i] > x[j], se intercambian.
00096     Este patrón es similar a una selección por intercambios y tiene costo O(n^2).
00097     */
00098     for (i = 0; i < n - 1; i++)
00099         for (j = i + 1; j < n; j++)
00100             if (x[i] > x[j])
00101             {
00102                 aux = x[i];
00103                 x[i] = x[j];
00104                 x[j] = aux;
00105             }
00106
00107     /* Imprimir arreglo ordenado. */
00108     printf("Ordenado.\n");
00109     for (i = 0; i < n; i++)
00110         printf("x[%ld] = %d\n", i + 1, x[i]);
00111
00112     return 0;
00113 }
```


7.191 Ejemplo019.c

[Go to the documentation of this file.](#)

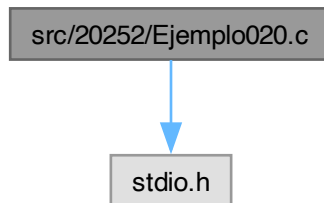
```

00001
00042
00043 #include <stdio.h>
00044 #include <stdlib.h>
00045 #include <time.h>
00046
00048 #define N 100000
00049
00050 int main(int argc, char *argv[])
00051 {
00052     (void)argc;
00053     (void)argv;
00054
00055     long int n, i, j; /* n: tamaño; i,j: índices para recorrido/ordenamiento */
00056     int min, max; /* límites del rango */
00057     int x[N]; /* arreglo */
00058     int aux; /* auxiliar para intercambio */
00059
00060     /* Leer n y validar. */
00061     do {
00062         printf("Ingrese el numero de elementos: ");
00063         scanf("%ld", &n);
00064     } while (n < 1 || n > N);
00065
00066     /* Leer límites (max y min). */
00067     printf("Ingrese el valor maximo: ");
00068     scanf("%d", &max);
00069
00070     printf("Ingrese el valor minimo: ");
00071     scanf("%d", &min);
00072
00073     /* Asegurar min <= max intercambiando si es necesario. */
00074     if (min > max)
00075     {
00076         max += min;
00077         min = max - min;
00078         max -= min;
00079     }
00080
00081     /* Semilla para valores aleatorios distintos por ejecución. */
00082     srand(time(NULL));
00083
00084     /* Generar e imprimir arreglo inicial (sin ordenar). */
00085     printf("Desordenado.\n");
00086     for (i = 0; i < n; i++)
00087     {
00088         x[i] = rand() % (max - min + 1) + min;
00089         printf("x[%ld] = %d\n", i + 1, x[i]);
00090     }
00091
00092     /*
00093     Ordenamiento ascendente por comparación e intercambio:
00094     - Para cada i, se busca si existe algún x[j] (j>i) menor que x[i].
00095     - Si x[i] > x[j], se intercambian.
00096     Este patrón es similar a una selección por intercambios y tiene costo O(n^2).
00097     */
00098     for (i = 0; i < n - 1; i++)
00099         for (j = i + 1; j < n; j++)
00100             if (x[i] > x[j])
00101             {
00102                 aux = x[i];
00103                 x[i] = x[j];
00104                 x[j] = aux;
00105             }
00106
00107     /* Imprimir arreglo ordenado. */
00108     printf("Ordenado.\n");
00109     for (i = 0; i < n; i++)
00110         printf("x[%ld] = %d\n", i + 1, x[i]);
00111
00112     return 0;
00113 }
```

7.192 src/20252/Ejemplo020.c File Reference

#include <stdio.h>

Include dependency graph for Ejemplo020.c:



Macros

- #define [N](#) 100
- #define [M](#) 100

Functions

- int [main](#) (int argc, char *argv[])

7.192.1 Macro Definition Documentation

7.192.1.1 M

#define M 100

Definition at line 4 of file [Ejemplo020.c](#).

7.192.1.2 N

#define N 100

Definition at line 3 of file [Ejemplo020.c](#).

7.192.2 Function Documentation

7.192.2.1 main()

```
int main (  
    int argc,  
    char * argv[ ])
```

Definition at line 6 of file [Ejemplo020.c](#).

```
00007 {  
00008     float A[N][M], B[N][M], C[N][M];  
00009     int n, m, i, j;  
00010     do{  
00011         printf("Ingrese el numero de filas: ");  
00012         scanf("%d", &n);  
00013     }while(n<1||n>N);  
00014     do{  
00015         printf("Ingrese el numero de columnas: ");  
00016         scanf("%d", &m);  
00017     }while(m<1||m>M);  
00018     for(i=0; i<n; i++)  
00019         for(j=0; j<m; j++)  
00020     {
```

```

00021         printf("A[%d][%d] = ", i+1, j+1);
00022         scanf("%f", &(A[i][j]));
00023     }
00024     for(i=0; i<n; i++)
00025         for(j=0; j<m; j++)
00026         {
00027             printf("B[%d][%d] = ", i+1, j+1);
00028             scanf("%f", &(B[i][j]));
00029         }
00030     for(i=0; i<n; i++)
00031         for(j=0; j<m; j++)
00032         {
00033             C[i][j] = A[i][j] + B[i][j];
00034             printf("C[%d][%d] = %f\n", i+1, j+1, C[i][j]);
00035         }
00036     return 0;
00037 }

```

7.193 Ejemplo020.c

[Go to the documentation of this file.](#)

```

00001 #include<stdio.h>
00002
00003 #define N 100
00004 #define M 100
00005
00006 int main(int argc, char *argv[])
00007 {
00008     float A[N][M], B[N][M], C[N][M];
00009     int n, m, i, j;
00010     do{
00011         printf("Ingrese el numero de filas: ");
00012         scanf("%d", &n);
00013     }while(n<1||n>N);
00014     do{
00015         printf("Ingrese el numero de columnas: ");
00016         scanf("%d", &m);
00017     }while(m<1||m>M);
00018     for(i=0; i<n; i++)
00019         for(j=0; j<m; j++)
00020         {
00021             printf("A[%d][%d] = ", i+1, j+1);
00022             scanf("%f", &(A[i][j]));
00023         }
00024     for(i=0; i<n; i++)
00025         for(j=0; j<m; j++)
00026         {
00027             printf("B[%d][%d] = ", i+1, j+1);
00028             scanf("%f", &(B[i][j]));
00029         }
00030     for(i=0; i<n; i++)
00031         for(j=0; j<m; j++)
00032         {
00033             C[i][j] = A[i][j] + B[i][j];
00034             printf("C[%d][%d] = %f\n", i+1, j+1, C[i][j]);
00035         }
00036     return 0;
00037 }

```

7.194 src/20261/src/Ejemplo020.c File Reference

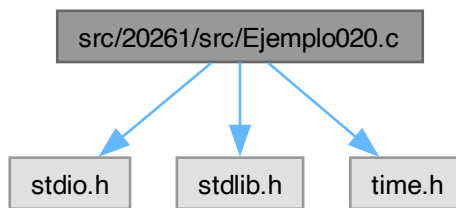
Genera un arreglo de enteros aleatorios y lo ordena con Odd–Even Sort (Brick Sort).

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo020.c:



Macros

- `#define N 100000`

Functions

- `int main (int argc, char *argv[ ])`

7.194.1 Detailed Description

Genera un arreglo de enteros aleatorios y lo ordena con Odd–Even Sort (Brick Sort).

El programa: 1) Lee el tamaño n ($1 \leq n \leq N$) y un rango entero $[\text{min}, \text{max}]$. 2) Genera n enteros pseudoaleatorios uniformes (aprox.) en $[\text{min}, \text{max}]$. 3) Imprime el arreglo "Desordenado". 4) Ordena el arreglo con **Odd–Even Transposition Sort** (también llamado Brick Sort):

- En pasadas alternadas, compara e intercambia pares adyacentes:
 - pasada par: (0,1), (2,3), (4,5), ...
 - pasada impar: (1,2), (3,4), (5,6), ...
- Se realizan aproximadamente $n-1$ pasadas. 5) Imprime el arreglo "Ordenado".

Entrada estándar

- n (long int): número de elementos (1..N)
- `max` (int): límite superior del rango
- `min` (int): límite inferior del rango

Salida estándar

- Lista de valores "Desordenado"
- Lista de valores "Ordenado"

Complejidad

- Generación/impresión: $O(n)$
- Ordenamiento (Odd–Even Sort): $O(n^2)$
- Memoria: $O(N)$ (arreglo estático en pila)

Warning

- El intercambio usa suma/resta (sin variable temporal). Puede desbordar si los enteros son muy grandes. Para un uso robusto conviene usar una variable auxiliar.
- `rand()` % `rango` puede introducir sesgo de módulo; suficiente para fines didácticos.

```
gcc Ejemplo020.c -o Ejemplo020
./Ejemplo020
```

Definition in file [Ejemplo020.c](#).

7.194.2 Macro Definition Documentation

7.194.2.1 N

```
#define N 100000
```

Definition at line 46 of file [Ejemplo020.c](#).

7.194.3 Function Documentation

7.194.3.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 48 of file [Ejemplo020.c](#).

```
00049 {
00050     (void)argc;
00051     (void)argv;
00052
00053     long int n, i, j;
00054     long int i0, i1;    /* índices del par a comparar */
00055     int min, max;
00056     int x[N];           /* arreglo de enteros */
00057
00058     /* Leer n y validar rango [1, N] */
00059     do {
00060         printf("Ingrese el numero de elementos: ");
00061         scanf("%ld", &n);
00062     } while (n < 1 || n > N);
00063
00064     /* Leer rango [min, max] (si viene invertido, se corrige) */
00065     printf("Ingrese el valor maximo: ");
00066     scanf("%d", &max);
00067     printf("Ingrese el valor minimo: ");
00068     scanf("%d", &min);
00069
00070     if (min > max)
00071     {
00072         /* Intercambio max <-> min sin variable auxiliar (puede desbordar en casos extremos) */
00073         max += min;
00074         min = max - min;
00075         max -= min;
00076     }
00077
00078     /* Semilla del generador pseudoaleatorio */
00079     srand(time(NULL));
00080
00081     /* Generar e imprimir arreglo desordenado */
00082     printf("Desordenado.\n");
00083     for (i = 0; i < n; i++)
00084     {
00085         x[i] = rand() % (max - min + 1) + min;
00086         printf("x[%ld] = %d\n", i + 1, x[i]);
00087     }
00088
00089     /*
00090     ODD-EVEN SORT (Brick Sort):
00091     - i representa la pasada (0..n-2).
00092     - si i es par: compara (0,1), (2,3), ...
00093     - si i es impar: compara (1,2), (3,4), ...
00094
00095     Aquí se implementa mapeando un índice j de pares:
00096     i0 = 2*j - (i%2)
00097     i1 = i0 + 1
00098     */
00099     for (i = 0; i < n - 1; i++)
00100     {
```

```

00101         for (j = i % 2; j < (n + i % 2) / 2; j++)
00102         {
00103             i0 = 2 * j - i % 2;
00104             i1 = 2 * j + 1 - i % 2;
00105
00106             /* Compare-exchange: si están invertidos, intercambia */
00107             if (x[i0] > x[i1])
00108             {
00109                 /* Swap por suma/resta (riesgo de overflow si valores grandes) */
00110                 x[i0] += x[i1];
00111                 x[i1] = x[i0] - x[i1];
00112                 x[i0] -= x[i1];
00113             }
00114         }
00115     }
00116
00117     /* Imprimir arreglo ordenado */
00118     printf("Ordenado.\n");
00119     for (i = 0; i < n; i++)
00120         printf("x[%ld] = %d\n", i + 1, x[i]);
00121
00122     return 0;
00123 }

```

7.195 Ejemplo020.c

[Go to the documentation of this file.](#)

```

00001
00041
00042 #include <stdio.h>
00043 #include <stdlib.h>
00044 #include <time.h>
00045
00046 #define N 100000
00047
00048 int main(int argc, char *argv[])
00049 {
00050     (void)argc;
00051     (void)argv;
00052
00053     long int n, i, j;
00054     long int i0, i1; /* índices del par a comparar */
00055     int min, max;
00056     int x[N]; /* arreglo de enteros */
00057
00058     /* Leer n y validar rango [1, N] */
00059     do {
00060         printf("Ingrese el numero de elementos: ");
00061         scanf("%ld", &n);
00062     } while (n < 1 || n > N);
00063
00064     /* Leer rango [min, max] (si viene invertido, se corrige) */
00065     printf("Ingrese el valor maximo: ");
00066     scanf("%d", &max);
00067     printf("Ingrese el valor minimo: ");
00068     scanf("%d", &min);
00069
00070     if (min > max)
00071     {
00072         /* Intercambio max <-> min sin variable auxiliar (puede desbordar en casos extremos) */
00073         max += min;
00074         min = max - min;
00075         max -= min;
00076     }
00077
00078     /* Semilla del generador pseudoaleatorio */
00079     srand(time(NULL));
00080
00081     /* Generar e imprimir arreglo desordenado */
00082     printf("Desordenado.\n");
00083     for (i = 0; i < n; i++)
00084     {
00085         x[i] = rand() % (max - min + 1) + min;
00086         printf("x[%ld] = %d\n", i + 1, x[i]);
00087     }
00088
00089     /*
00090     ODD-EVEN SORT (Brick Sort):
00091     - i representa la pasada (0..n-2).
00092     - si i es par: compara (0,1), (2,3), ...
00093     - si i es impar: compara (1,2), (3,4), ...
00094
00095     Aquí se implementa mapeando un índice j de pares:
00096     i0 = 2*j - (i%2)

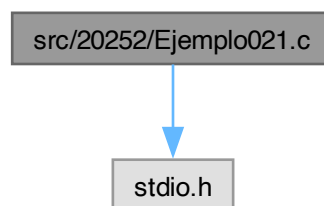
```

```
00097     i1 = i0 + 1
00098     */
00099     for (i = 0; i < n - 1; i++)
00100     {
00101         for (j = i % 2; j < (n + i % 2) / 2; j++)
00102         {
00103             i0 = 2 * j - i % 2;
00104             i1 = 2 * j + 1 - i % 2;
00105
00106             /* Compare-exchange: si están invertidos, intercambia */
00107             if (x[i0] > x[i1])
00108             {
00109                 /* Swap por suma/resta (riesgo de overflow si valores grandes) */
00110                 x[i0] += x[i1];
00111                 x[i1] = x[i0] - x[i1];
00112                 x[i0] -= x[i1];
00113             }
00114         }
00115     }
00116
00117     /* Imprimir arreglo ordenado */
00118     printf("Ordenado.\n");
00119     for (i = 0; i < n; i++)
00120         printf("x[%ld] = %d\n", i + 1, x[i]);
00121
00122     return 0;
00123 }
```

7.196 src/20252/Ejemplo021.c File Reference

#include <stdio.h>

Include dependency graph for Ejemplo021.c:



Macros

- #define [N](#) 100
- #define [M](#) 100

Functions

- int [main](#) (int argc, char *argv[])

7.196.1 Macro Definition Documentation

7.196.1.1 M

#define M 100

Definition at line 4 of file [Ejemplo021.c](#).

7.196.1.2 N

```
#define N 100
```

Definition at line 3 of file [Ejemplo021.c](#).

7.196.2 Function Documentation

7.196.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 6 of file [Ejemplo021.c](#).

```
00007 {
00008     float A[N][M], B[N][M], C[N][M];
00009     int nA, mA, nB, mB, i, j, k;
00010     do{
00011         printf("Ingrese el numero de filas de A: ");
00012         scanf("%d", &nA);
00013     }while(nA<1||nA>N);
00014     do{
00015         printf("Ingrese el numero de columnas de A: ");
00016         scanf("%d", &mA);
00017     }while(mA<1||mA>M);
00018     do{
00019         printf("Ingrese el numero de filas de B: ");
00020         scanf("%d", &nB);
00021     }while(nB!=mA);
00022     do{
00023         printf("Ingrese el numero de columnas de B: ");
00024         scanf("%d", &mB);
00025     }while(mB<1||mB>M);
00026     for(i=0; i<nA; i++)
00027         for(j=0; j<mA; j++)
00028         {
00029             printf("A[%d][%d] = ", i+1, j+1);
00030             scanf("%f", &(A[i][j]));
00031         }
00032     for(i=0; i<nB; i++)
00033         for(j=0; j<mB; j++)
00034         {
00035             printf("B[%d][%d] = ", i+1, j+1);
00036             scanf("%f", &(B[i][j]));
00037         }
00038     for(i=0; i<nA; i++)
00039         for(j=0; j<mB; j++)
00040         {
00041             for(k=0; C[i][j] = 0; k<mA; k++)
00042                 C[i][j] += (A[i][k]*B[k][j]);
00043             printf("C[%d][%d] = %f\n", i+1, j+1, C[i][j]);
00044         }
00045     return 0;
00046 }
```

7.197 Ejemplo021.c

[Go to the documentation of this file.](#)

```
00001 #include<stdio.h>
00002
00003 #define N 100
00004 #define M 100
00005
00006 int main(int argc, char *argv[])
00007 {
00008     float A[N][M], B[N][M], C[N][M];
00009     int nA, mA, nB, mB, i, j, k;
00010     do{
00011         printf("Ingrese el numero de filas de A: ");
00012         scanf("%d", &nA);
00013     }while(nA<1||nA>N);
00014     do{
00015         printf("Ingrese el numero de columnas de A: ");
00016         scanf("%d", &mA);
00017     }while(mA<1||mA>M);
00018     do{
00019         printf("Ingrese el numero de filas de B: ");
00020         scanf("%d", &nB);
00021     }while(nB!=mA);
```



```

00022     do{
00023         printf("Ingrese el numero de columnas de B: ");
00024         scanf("%d", &mB);
00025     }while(mB<1||mB>M);
00026     for(i=0; i<nA; i++)
00027         for(j=0; j<mA; j++)
00028     {
00029         printf("A[%d][%d] = ", i+1, j+1);
00030         scanf("%f", &(A[i][j]));
00031     }
00032     for(i=0; i<nB; i++)
00033         for(j=0; j<mB; j++)
00034     {
00035         printf("B[%d][%d] = ", i+1, j+1);
00036         scanf("%f", &(B[i][j]));
00037     }
00038     for(i=0; i<nA; i++)
00039         for(j=0; j<mB; j++)
00040     {
00041         for(k=0, C[i][j] = 0; k<mA; k++)
00042             C[i][j] += (A[i][k]*B[k][j]);
00043         printf("C[%d][%d] = %f\n", i+1, j+1, C[i][j]);
00044     }
00045     return 0;
00046 }

```

7.198 src/20261/src/Ejemplo021.c File Reference

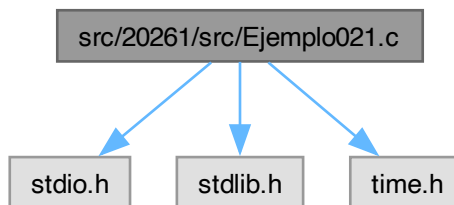
Genera un arreglo de flotantes aleatorios y lo ordena con Selection Sort.

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo021.c:



Macros

- #define N 100000

Functions

- int main(int argc, char *argv[])

7.198.1 Detailed Description

Genera un arreglo de flotantes aleatorios y lo ordena con Selection Sort.

El programa: 1) Lee n (1..N) y un rango real [min, max]. 2) Genera n flotantes uniformes (aprox.) en [min, max] usando: $x[i] = ((\text{max}-\text{min}) * \text{rand}()) / \text{RAND_MAX} + \text{min}$ 3) Imprime el arreglo "Desordenado". 4) Ordena con **Selection Sort**:

- Primero ubica el mínimo global en $x[0]$.
- Después, para cada posición i , selecciona el mínimo del subarreglo $[i..n-1]$ y lo intercambia con $x[i]$.

Entrada estándar

- `n` (long int): número de elementos (1..N)
- `max` (float): límite superior
- `min` (float): límite inferior

Salida estándar

- Arreglo desordenado
- Arreglo ordenado (ascendente)

Complejidad

- Generación/impresión: $O(n)$
- Selection Sort: $O(n^2)$
- Memoria: $O(N)$

Warning

- El intercambio de flotantes se hace con multiplicación/división. Si alguno de los valores involucrados es 0, puede provocar división entre 0 o NaN/Inf. Para una versión robusta, usa una variable auxiliar (`aux`).
- También el intercambio inicial de min/max usa multiplicación/división y falla si `min=0`.

```
gcc Ejemplo021.c -o Ejemplo021
./Ejemplo021
```

Definition in file [Ejemplo021.c](#).

7.198.2 Macro Definition Documentation**7.198.2.1 N**

```
#define N 100000
```

Definition at line 46 of file [Ejemplo021.c](#).

7.198.3 Function Documentation**7.198.3.1 main()**

```
int main (
    int argc,
    char * argv[])
```

Definition at line 48 of file [Ejemplo021.c](#).

```
00049 {
00050     (void)argc;
00051     (void)argv;
00052
00053     long int n, i, j, i_min;
00054     float min, max;
00055     float x[N];
00056
00057     /* Leer n y validar */
00058     do {
00059         printf("Ingrese el numero de elementos: ");
00060         scanf("%ld", &n);
00061     } while (n < 1 || n > N);
00062
00063     /* Leer rango [min,max] */
00064     printf("Ingrese el valor maximo: ");
00065     scanf("%f", &max);
00066     printf("Ingrese el valor minimo: ");
00067     scanf("%f", &min);
00068
00069     if (min > max)
00070     {
00071         /*
00072          * Intercambio sin variable auxiliar usando multiplicación/división.
00073          * WARNING: falla si min==0 o max==0 (división entre 0).
00074          */
00075     }
```

```

00074         */
00075         max *= min;
00076         min = max / min;
00077         max /= min;
00078     }
00079
00080     srand(time(NULL));
00081
00082     /* Generar e imprimir arreglo desordenado */
00083     printf("Desordenado.\n");
00084     for (i = 0; i < n; i++)
00085     {
00086         /* Escalamiento lineal a [min, max] */
00087         x[i] = ((max - min) * rand()) / RAND_MAX + min;
00088         printf("x[%ld] = %f\n", i + 1, x[i]);
00089     }
00090
00091     /*
00092     Paso 1: localizar el mínimo global y ponerlo en x[0].
00093     Esto no es estrictamente necesario para Selection Sort, pero el código lo hace.
00094     */
00095     for (i = 1, i_min = 0; i < n; i++)
00096         if (x[i] < x[i_min])
00097             i_min = i;
00098
00099     if (i_min)
00100     {
00101         /* Swap por multiplicación/división (WARNING: falla si x[0]==0 o x[i_min]==0) */
00102         x[i_min] *= x[0];
00103         x[0] = x[i_min] / x[0];
00104         x[i_min] /= x[0];
00105     }
00106
00107     /*
00108     Paso 2: Selection Sort desde i=1:
00109     - en cada i, encontrar el mínimo en [i..n-1]
00110     - intercambiarlo con x[i]
00111     */
00112     for (i = 1; i < n - 1; i++)
00113     {
00114         for (j = i + 1, i_min = i; j < n; j++)
00115             if (x[j] < x[i_min])
00116                 i_min = j;
00117
00118         if (i_min != i)
00119         {
00120             /* Swap por multiplicación/división (mismas advertencias) */
00121             x[i_min] *= x[i];
00122             x[i] = x[i_min] / x[i];
00123             x[i_min] /= x[i];
00124         }
00125     }
00126
00127     /* Imprimir arreglo ordenado */
00128     printf("Ordenado.\n");
00129     for (i = 0; i < n; i++)
00130         printf("x[%ld] = %f\n", i + 1, x[i]);
00131
00132     return 0;
00133 }

```

7.199 Ejemplo021.c

[Go to the documentation of this file.](#)

```

00001
00041
00042 #include <stdio.h>
00043 #include <stdlib.h>
00044 #include <time.h>
00045
00046 #define N 100000
00047
00048 int main(int argc, char *argv[])
00049 {
00050     (void)argc;
00051     (void)argv;
00052
00053     long int n, i, j, i_min;
00054     float min, max;
00055     float x[N];
00056
00057     /* Leer n y validar */
00058     do {
00059         printf("Ingrese el numero de elementos: ");

```

```

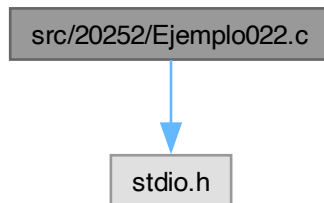
00060     scanf("%ld", &n);
00061 } while (n < 1 || n > N);
00062
00063 /* Leer rango [min,max] */
00064 printf("Ingrese el valor maximo: ");
00065 scanf("%f", &max);
00066 printf("Ingrese el valor minimo: ");
00067 scanf("%f", &min);
00068
00069 if (min > max)
00070 {
00071     /*
00072      * Intercambio sin variable auxiliar usando multiplicación/división.
00073      * WARNING: falla si min==0 o max==0 (división entre 0).
00074      */
00075     max *= min;
00076     min = max / min;
00077     max /= min;
00078 }
00079
00080 srand(time(NULL));
00081
00082 /* Generar e imprimir arreglo desordenado */
00083 printf("Desordenado.\n");
00084 for (i = 0; i < n; i++)
00085 {
00086     /* Escalamiento lineal a [min, max] */
00087     x[i] = ((max - min) * rand()) / RAND_MAX + min;
00088     printf("x[%ld] = %f\n", i + 1, x[i]);
00089 }
00090
00091 /*
00092  * Paso 1: localizar el mínimo global y ponerlo en x[0].
00093  * Esto no es estrictamente necesario para Selection Sort, pero el código lo hace.
00094  */
00095 for (i = 1, i_min = 0; i < n; i++)
00096     if (x[i] < x[i_min])
00097         i_min = i;
00098
00099 if (i_min)
00100 {
00101     /* Swap por multiplicación/división (WARNING: falla si x[0]==0 o x[i_min]==0) */
00102     x[i_min] *= x[0];
00103     x[0] = x[i_min] / x[0];
00104     x[i_min] /= x[0];
00105 }
00106
00107 /*
00108  * Paso 2: Selection Sort desde i=1:
00109  * - en cada i, encontrar el mínimo en [i..n-1]
00110  * - intercambiarlo con x[i]
00111  */
00112 for (i = 1; i < n - 1; i++)
00113 {
00114     for (j = i + 1, i_min = i; j < n; j++)
00115         if (x[j] < x[i_min])
00116             i_min = j;
00117
00118     if (i_min != i)
00119     {
00120         /* Swap por multiplicación/división (mismas advertencias) */
00121         x[i_min] *= x[i];
00122         x[i] = x[i_min] / x[i];
00123         x[i_min] /= x[i];
00124     }
00125 }
00126
00127 /* Imprimir arreglo ordenado */
00128 printf("Ordenado.\n");
00129 for (i = 0; i < n; i++)
00130     printf("x[%ld] = %f\n", i + 1, x[i]);
00131
00132 return 0;
00133 }

```

7.200 src/20252/Ejemplo022.c File Reference

```
#include <stdio.h>
```

Include dependency graph for Ejemplo022.c:



Macros

- `#define N 100`

Functions

- `int main (int argc, char *argv[ ])`

7.200.1 Macro Definition Documentation

7.200.1.1 N

```
#define N 100
```

Definition at line 3 of file [Ejemplo022.c](#).

7.200.2 Function Documentation

7.200.2.1 main()

```
int main (
    int argc,
    char * argv[ ])
```

Definition at line 5 of file [Ejemplo022.c](#).

```

00006 {
00007     int n, i, j, k;
00008     float A[N][N], b[N], fct, x[N];
00009     do{
00010         printf("Ingrese el numero de incognitas: ");
00011         scanf("%d", &n);
00012     }while(n<3||n>N);
00013     for(i=0; i<n; i++)
00014     {
00015         for(j=0; j<n; j++)
00016         {
00017             printf("A[%d][%d] = ", i+1, j+1);
00018             scanf("%f", &(A[i][j]));
00019         }
00020         printf("b[%d] = ", i+1);
00021         scanf("%f", &b[i]);
00022         x[i] = 0;
00023     }
00024     for(i=0; i<n; i++)
00025     {
00026         printf("%.2fx1", A[i][0]);
00027         for(j=1; j<n; j++)
00028     {
```

```

00029         printf("%.2fx%d", A[i][j], j+1);
00030     }
00031     printf("=%.2f\n", b[i]);
00032 }
00033 for(i=1; i<n; i++)
00034     for(j=i; j<n; j++)
00035     {
00036         for(k=0, fct=A[j][i-1]/A[i-1][i-1]; k<n; k++)
00037             A[j][k] -= (fct*A[i-1][k]);
00038         b[j] -= (fct*b[i-1]);
00039     }
00040 /*
00041 for(i=0; i<n; i++)
00042 {
00043     for(j=0; j<n; j++)
00044         printf("%.2f\t", A[i][j]);
00045     printf("%.2f\n", b[i]);
00046 }
00047 */
00048 for(i=n-1; i>-1; i--)
00049 {
00050     x[i] = b[i];
00051     for(j=i+1; j<n; j++)
00052         x[i] -= A[i][j]*x[j];
00053     x[i] /= A[i][i];
00054 }
00055 for(i=0; i<n; i++)
00056     printf("X[%d] = %f\n", i+1, x[i]);
00057 return 0;
00058 }

```

Here is the call graph for this function:



7.201 Ejemplo022.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002
00003 #define N 100
00004
00005 int main(int argc, char *argv[])
00006 {
00007     int n, i, j, k;
00008     float A[N][N], b[N], fct, x[N];
00009     do{
00010         printf("Ingrese el numero de incognitas: ");
00011         scanf("%d", &n);
00012     }while(n<3||n>N);
00013     for(i=0; i<n; i++)
00014     {
00015         for(j=0; j<n; j++)
00016         {
00017             printf("A[%d][%d] = ", i+1, j+1);
00018             scanf("%f", &(A[i][j]));
00019         }
00020         printf("b[%d] = ", i+1);
00021         scanf("%f", &b[i]);
00022         x[i] = 0;
00023     }
00024     for(i=0; i<n; i++)
00025     {
00026         printf("%.2fx1", A[i][0]);
00027         for(j=1; j<n; j++)
00028         {
00029             printf("%.2fx%d", A[i][j], j+1);
00030         }

```

```

00031     printf("%.2f\n", b[i]);
00032 }
00033 for(i=1; i<n; i++)
00034     for(j=i; j<n; j++)
00035     {
00036         for(k=0, fct=A[j][i-1]/A[i-1][i-1]; k<n; k++)
00037             A[j][k] -= (fct*A[i-1][k]);
00038         b[j] -= (fct*b[i-1]);
00039     }
00040 /*
00041 for(i=0; i<n; i++)
00042 {
00043     for(j=0; j<n; j++)
00044         printf("%.2f\t", A[i][j]);
00045     printf("%.2f\n", b[i]);
00046 }
00047 */
00048 for(i=n-1; i>-1; i--)
00049 {
00050     x[i] = b[i];
00051     for(j=i+1; j<n; j++)
00052         x[i] -= A[i][j]*x[j];
00053     x[i] /= A[i][i];
00054 }
00055 for(i=0; i<n; i++)
00056     printf("X[%d] = %f\n", i+1, x[i]);
00057 return 0;
00058 }

```

7.202 src/20261/src/Ejemplo022.c File Reference

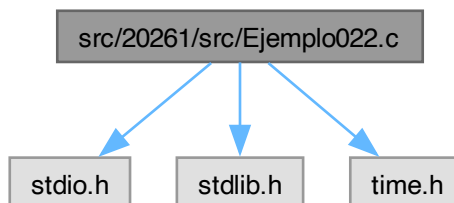
Genera un arreglo de flotantes aleatorios y lo ordena con Insertion Sort.

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

Include dependency graph for Ejemplo022.c:



Macros

- #define `N` 100000

Functions

- int `main` (int argc, char *argv[])

7.202.1 Detailed Description

Genera un arreglo de flotantes aleatorios y lo ordena con Insertion Sort.

El programa: 1) Lee `n` (1..N) y un rango real [min, max]. 2) Genera `n` flotantes en [min,max]. 3) Imprime el arreglo "Desordenado". 4) Ordena el arreglo con **Insertion Sort** (orden ascendente):

- Recorre `i = 1..n-1`, toma `x[i]` como `aux`,

- desplaza hacia la derecha todos los elementos mayores que `aux`,
- inserta `aux` en la posición correcta. 5) Imprime el arreglo "Ordenado".

Entrada estándar

- `n` (long int): número de elementos (1..N)
- `max` (float): límite superior
- `min` (float): límite inferior

Salida estándar

- Arreglo desordenado
- Arreglo ordenado (ascendente)

Complejidad

- Generación/impresión: $O(n)$
- Insertion Sort: $O(n^2)$ en el peor caso; $O(n)$ si ya está casi ordenado.
- Memoria: $O(N)$

Warning

- El intercambio de min/max se hace con multiplicación/división; falla si `min==0`.
- El arreglo `x[N]` vive en la pila; en algunos entornos con pila pequeña puede fallar.

```
gcc Ejemplo022.c -o Ejemplo022
./Ejemplo022
```

Definition in file [Ejemplo022.c](#).

7.202.2 Macro Definition Documentation

7.202.2.1 N

```
#define N 100000
```

Definition at line 44 of file [Ejemplo022.c](#).

7.202.3 Function Documentation

7.202.3.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 46 of file [Ejemplo022.c](#).

```
00047 {
00048     (void)argc;
00049     (void)argv;
00050
00051     long int n, i, j;
00052     float min, max;
00053     float x[N], aux;
00054
00055     /* Leer n y validar */
00056     do {
00057         printf("Ingrese el numero de elementos: ");
00058         scanf("%ld", &n);
00059     } while (n < 1 || n > N);
00060
00061     /* Leer rango [min,max] */
00062     printf("Ingrese el valor maximo: ");
00063     scanf("%f", &max);
00064     printf("Ingrese el valor minimo: ");
00065     scanf("%f", &min);
00066 }
```



```

00067     if (min > max)
00068     {
00069         /* Swap por multiplicación/división (WARNING: falla si min==0) */
00070         max *= min;
00071         min = max / min;
00072         max /= min;
00073     }
00074
00075     srand(time(NULL));
00076
00077     /* Generar e imprimir arreglo desordenado */
00078     printf("Desordenado.\n");
00079     for (i = 0; i < n; i++)
00080     {
00081         x[i] = ((max - min) * rand()) / RAND_MAX + min;
00082         printf("x[%ld] = %f\n", i + 1, x[i]);
00083     }
00084
00085     /*
00086     INSERTION SORT:
00087     - i recorre desde 1 hasta n-1.
00088     - aux almacena el elemento actual a insertar.
00089     - j recorre hacia atrás desplazando elementos mayores que aux.
00090     */
00091     for (i = 1; i < n; i++)
00092     {
00093         j = i - 1;
00094         aux = x[i];
00095
00096         /* Desplazar hacia la derecha mientras x[j] > aux */
00097         while (x[j] > aux)
00098         {
00099             x[j + 1] = x[j];
00100             j--;
00101
00102             /* Si j se vuelve negativo, se detiene para evitar acceder fuera de rango */
00103             if (j < 0)
00104                 break;
00105         }
00106
00107         /* Insertar aux en la posición correcta */
00108         x[j + 1] = aux;
00109     }
00110
00111     /* Imprimir arreglo ordenado */
00112     printf("Ordenado.\n");
00113     for (i = 0; i < n; i++)
00114         printf("x[%ld] = %f\n", i + 1, x[i]);
00115
00116     return 0;
00117 }

```

7.203 Ejemplo022.c

[Go to the documentation of this file.](#)

```

00001
00039
00040 #include <stdio.h>
00041 #include <stdlib.h>
00042 #include <time.h>
00043
00044 #define N 100000
00045
00046 int main(int argc, char *argv[])
00047 {
00048     (void)argc;
00049     (void)argv;
00050
00051     long int n, i, j;
00052     float min, max;
00053     float x[N], aux;
00054
00055     /* Leer n y validar */
00056     do {
00057         printf("Ingrese el numero de elementos: ");
00058         scanf("%ld", &n);
00059     } while (n < 1 || n > N);
00060
00061     /* Leer rango [min,max] */
00062     printf("Ingrese el valor maximo: ");
00063     scanf("%f", &max);
00064     printf("Ingrese el valor minimo: ");
00065     scanf("%f", &min);
00066

```

```

00067     if (min > max)
00068     {
00069         /* Swap por multiplicación/división (WARNING: falla si min==0) */
00070         max *= min;
00071         min = max / min;
00072         max /= min;
00073     }
00074
00075     srand(time(NULL));
00076
00077     /* Generar e imprimir arreglo desordenado */
00078     printf("Desordenado.\n");
00079     for (i = 0; i < n; i++)
00080     {
00081         x[i] = ((max - min) * rand()) / RAND_MAX + min;
00082         printf("x[%ld] = %f\n", i + 1, x[i]);
00083     }
00084
00085     /*
00086     INSERTION SORT:
00087     - i recorre desde 1 hasta n-1.
00088     - aux almacena el elemento actual a insertar.
00089     - j recorre hacia atrás desplazando elementos mayores que aux.
00090     */
00091     for (i = 1; i < n; i++)
00092     {
00093         j = i - 1;
00094         aux = x[i];
00095
00096         /* Desplazar hacia la derecha mientras x[j] > aux */
00097         while (x[j] > aux)
00098         {
00099             x[j + 1] = x[j];
00100             j--;
00101
00102             /* Si j se vuelve negativo, se detiene para evitar acceder fuera de rango */
00103             if (j < 0)
00104                 break;
00105         }
00106
00107         /* Insertar aux en la posición correcta */
00108         x[j + 1] = aux;
00109     }
00110
00111     /* Imprimir arreglo ordenado */
00112     printf("Ordenado.\n");
00113     for (i = 0; i < n; i++)
00114         printf("x[%ld] = %f\n", i + 1, x[i]);
00115
00116     return 0;
00117 }

```

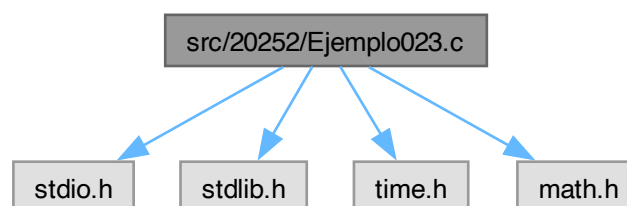
7.204 src/20252/Ejemplo023.c File Reference

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>

```

Include dependency graph for Ejemplo023.c:



Macros

- `#define N 1000`

Functions

- `int main (int argc, char *argv[ ])`

7.204.1 Macro Definition Documentation

7.204.1.1 N

`#define N 1000`

Definition at line 6 of file [Ejemplo023.c](#).

7.204.2 Function Documentation

7.204.2.1 main()

```
int main (
    int argc,
    char * argv[ ])
```

Definition at line 8 of file [Ejemplo023.c](#).

```
00009 {
00010     int n, i, flag, op;
00011     float max, min, aux, x[N], vmx, vmn, md, st, z[N], cn, nr;
00012     srand(time(NULL));
00013     do{
00014         printf("Ingrese el numero de datos: ");
00015         scanf("%d", &n);
00016     }while(n<1||n>N);
00017     printf("Ingrese el numero maximo: ");
00018     scanf("%f", &max);
00019     printf("Ingrese el numero minimo: ");
00020     scanf("%f", &min);
00021     if(max<min)
00022     {
00023         aux = max;
00024         max = min;
00025         min = aux;
00026     }
00027     for(i=0, vmn=max, vmx=min, md=0, st=0; i<n; i++)
00028     {
00029         x[i] = ((max-min)*rand())/RAND_MAX+min;
00030         printf("X[%d] = %f\n", i+1, x[i]);
00031         if(vmn>x[i])
00032             vmn = x[i];
00033         if(vmx<x[i])
00034             vmx = x[i];
00035         md+=x[i];
00036         st+=(x[i]*x[i]);
00037     }
00038     md/=n;
00039     st/=n;
00040     st=(md*md);
00041     st=sqrt(st);
00042     printf("Max: %f\n Min = %f\n Media = %f\n D. E. = %f\n",
00043         vmx, vmn, md, st);
00044     flag = 0;
00045     printf("Desea centrar: ");
00046     scanf("%d", &op);
00047     if(op)
00048     {
00049         printf("Por rango: ");
00050         scanf("%d", &op);
00051         flag|=(op?1:4);
00052     }
00053     printf("Desea normalizar: ");
00054     scanf("%d", &op);
00055     if(op)
00056     {
00057         printf("Por rango: ");
00058         scanf("%d", &op);
00059         flag|=(op?2:8);
00060     }
00061     cn = (flag&1)?vmn:(flag&4?md:0);
00062     nr = (flag&2)?vmx-vmn:(flag&8?st:1);
```

```

00063     printf("%d\t%f\t%f\n", flag, cn, nr);
00064     for(i=0; i<n; i++)
00065     {
00066         z[i] = (x[i]-cn)/nr;
00067         printf("Z[%d] = %f\n", i+1, z[i]);
00068     }
00069     return 0;
00070 }

```

7.205 Ejemplo023.c

[Go to the documentation of this file.](#)

```

00001 #include <stdio.h>
00002 #include <stdlib.h>
00003 #include <time.h>
00004 #include <math.h>
00005
00006 #define N 1000
00007
00008 int main(int argc, char *argv[])
00009 {
00010     int n, i, flag, op;
00011     float max, min, aux, x[N], vmx, vmn, md, st, z[N], cn, nr;
00012     srand(time(NULL));
00013     do{
00014         printf("Ingrese el numero de datos: ");
00015         scanf("%d", &n);
00016     }while(n<1||n>N);
00017     printf("Ingrese el numero maximo: ");
00018     scanf("%f", &max);
00019     printf("Ingrese el numero minimo: ");
00020     scanf("%f", &min);
00021     if(max<min)
00022     {
00023         aux = max;
00024         max = min;
00025         min = aux;
00026     }
00027     for(i=0, vmn=max, vmx=min, md=0, st=0; i<n; i++)
00028     {
00029         x[i] = ((max-min)*rand())/RAND_MAX+min;
00030         printf("X[%d] = %f\n", i+1, x[i]);
00031         if(vmn>x[i])
00032             vmn = x[i];
00033         if(vmx<x[i])
00034             vmx = x[i];
00035         md+=x[i];
00036         st+=(x[i]*x[i]);
00037     }
00038     md/=n;
00039     st/=n;
00040     st-=(md*md);
00041     st=sqrt(st);
00042     printf("Max: %f\n Min = %f\n Media = %f\n D. E. = %f\n",
00043         vmx, vmn, md, st);
00044     flag = 0;
00045     printf("Desea centrar: ");
00046     scanf("%d", &op);
00047     if(op)
00048     {
00049         printf("Por rango: ");
00050         scanf("%d", &op);
00051         flag|=(op?1:4);
00052     }
00053     printf("Desea normalizar: ");
00054     scanf("%d", &op);
00055     if(op)
00056     {
00057         printf("Por rango: ");
00058         scanf("%d", &op);
00059         flag|=(op?2:8);
00060     }
00061     cn = (flag&1)?vmn:(flag&4?md:0);
00062     nr = (flag&2)?vmx-vmn:(flag&8?st:1);
00063     printf("%d\t%f\t%f\n", flag, cn, nr);
00064     for(i=0; i<n; i++)
00065     {
00066         z[i] = (x[i]-cn)/nr;
00067         printf("Z[%d] = %f\n", i+1, z[i]);
00068     }
00069     return 0;
00070 }

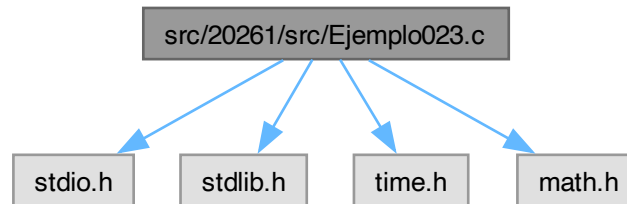
```

7.206 src/20261/src/Ejemplo023.c File Reference

Clasificación por “casilleros” (bucket/distribution) de números reales en un rango [min, max].

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
```

Include dependency graph for Ejemplo023.c:



Macros

- `#define N 100000`

Functions

- `int main (int argc, char *argv[ ])`

Punto de entrada. Genera valores y los agrupa por casilleros.

7.206.1 Detailed Description

Clasificación por “casilleros” (bucket/distribution) de números reales en un rango [min, max].

Este programa genera n números aleatorios (float) dentro del rango [min, max] y los reordena usando un enfoque tipo **bucket / distribution sort**:

- 1) Se elige el número de divisiones (casilleros) como: $nd = \text{floor}(\text{sqrt}(n))$
- 2) Se calcula el tamaño de casillero: $\text{rango} = \text{max} - \text{min}$ $Dx = \text{rango} / nd$
- 3) Para cada valor $x[i]$ se calcula el índice del casillero: $j = \text{floor}((x[i] - \text{min}) / Dx)$
- 4) Se cuenta cuántos elementos caen en cada casillero (histograma).
- 5) Se calcula un arreglo de posiciones iniciales (prefijos) y se construye un nuevo arreglo $xc[]$ colocando cada elemento en su casillero correspondiente.

El arreglo xc queda **agrupado por intervalos** (primero todos los del casillero 0, luego los del 1, etc.). **No ordena dentro de cada casillero**, por lo que NO es un ordenamiento total, sino una etapa típica de un bucket sort (faltaría ordenar cada bucket).

Entrada estándar

- n (int): número de elementos, $1 \leq n \leq N$
- max (float): valor máximo del rango
- min (float): valor mínimo del rango

Salida estándar

- Imprime los valores generados (“Desordenado”).
- Imprime el número de casilleros `nd`, rango y `Dx`.
- Imprime los intervalos de cada casillero.
- Imprime cada elemento con su índice de casillero.
- Imprime conteos y prefijos por casillero.
- Imprime el arreglo `xc` (reordenado por casilleros) con su casillero.

Precondiciones

- Se espera que `min` $\leq$ `max`. Si `min` $>$ `max`, se intercambian.
- `nd` $>$ 0 y `Dx` $>$ 0 (se cumple si `n` $\geq$ 1 y `max` $>$ `min`).

Complejidad

- Conteo por casilleros: $O(n)$
- Cálculo de prefijos: $O(nd)$
- Distribución a `xc`: $O(n)$
- Memoria: $O(N)$

Warning

1) El intercambio `min/max` se hace con multiplicación/división: si alguno es 0, puede fallar (división entre 0). Para robustez usa una variable auxiliar. 2) Cuando `x[i] == max`, el índice `j` puede resultar igual a `nd` (fuera del rango `0..nd-1`). En código de producción conviene “clamp”: `if (j  $\geq$  nd) j = nd - 1;`

```
gcc Ejemplo023.c -o Ejemplo023 -lm
./Ejemplo023
```

Definition in file [Ejemplo023.c](#).

7.206.2 Macro Definition Documentation**7.206.2.1 N**

```
#define N 100000
```

Definition at line 69 of file [Ejemplo023.c](#).

7.206.3 Function Documentation**7.206.3.1 main()**

```
int main (
    int argc,
    char * argv[])
```

Punto de entrada. Genera valores y los agrupa por casilleros.

Parameters

<code>argc</code>	No usado.
<code>argv</code>	No usado.

Returns

0 si finaliza correctamente.

Definition at line 77 of file [Ejemplo023.c](#).

```

00078 {
00079     /*
00080      n   : número de elementos a generar (1..N)
00081      i,j : índices auxiliares
00082      nd  : número de divisiones/casilleros ( sqrt(n))
00083
00084      c[] : arreglo de enteros usado en dos zonas:
00085           - c[0..nd-1]      = conteos por casillero (histograma)
00086           - c[nd..2*nd-1]   = posiciones iniciales/prefijos (índices de escritura en xc)
00087     */
00088     int n, i, j, nd, c[N];
00089
00090     /*
00091      min, max : rango de generación
00092      x[]      : arreglo original "desordenado"
00093      xc[]     : arreglo reordenado por casilleros
00094      rango    : max - min
00095      Dx       : ancho de cada casillero (rango/nd)
00096     */
00097     float min, max;
00098     float x[N], rango, Dx, xc[N];
00099
00100     /* Leer n */
00101     do {
00102         printf("Ingrese el numero de elementos: ");
00103         scanf("%d", &n);
00104     } while (n < 1 || n > N);
00105
00106     /* Leer rango */
00107     printf("Ingrese el valor maximo: ");
00108     scanf("%f", &max);
00109     printf("Ingrese el valor minimo: ");
00110     scanf("%f", &min);
00111
00112     /*
00113      Si el usuario capturó min > max, se intercambian.
00114      WARNING: este intercambio por multiplicación/división puede fallar si min==0 o max==0.
00115     */
00116     if (min > max)
00117     {
00118         max *= min;
00119         min = max / min;
00120         max /= min;
00121     }
00122
00123     srand(time(NULL));
00124
00125     /* Generar arreglo original y (por simplicidad) inicializar c[i]=0 para i<n */
00126     printf("Desordenado.\n");
00127     for (i = 0; i < n; i++)
00128     {
00129         /* Genera valores en [min, max] (puede incluir max si rand()==RAND_MAX) */
00130         x[i] = ((max - min) * rand()) / RAND_MAX + min;
00131
00132         /* Inicialización (basta con 0..2*nd-1, pero aquí se inicializa 0..n-1) */
00133         c[i] = 0;
00134
00135         printf("x[%02d] = %f\n", i + 1, x[i]);
00136     }
00137
00138     /* Definir número de casilleros */
00139     nd = (int)sqrt((double)n);
00140
00141     /* Calcular rango y ancho de casillero */
00142     rango = max - min;
00143     Dx = rango / nd;
00144
00145     printf("ND = %d\tRango = %f\tDx = %f\n", nd, rango, Dx);
00146
00147     /* Mostrar intervalos de los casilleros */
00148     for (i = 0; i < nd; i++)
00149         printf("%d. [%f, %f]\n", i, min + i*Dx, min + (i+1)*Dx);
00150
00151     /*
00152      1) Conteo por casillero:
00153          j = floor((x[i]-min)/Dx)
00154          c[j]++
00155     */
00156     printf("Desordenado con numero de casillero.\n");
00157     for (i = 0; i < n; i++)
00158     {
00159         j = (int)((x[i] - min) / Dx);

```

```

00160
00161      /*
00162      WARNING: si x[i] == max, j puede ser nd. Para robustez:
00163      if (j >= nd) j = nd - 1;
00164      */
00165
00166      printf("x[%02d] = %f\t%d\n", i + 1, x[i], j);
00167      c[j]++;
00168  }
00169
00170  /*
00171  2) Prefijos / posiciones iniciales:
00172  Se construye en la zona c[nd..2*nd-1]:
00173      c[nd]      = 0
00174      c[nd+1]    = c[0]
00175      c[nd+2]    = c[0] + c[1]
00176      ...
00177  donde c[nd+k] es el índice de inicio para escribir el casillero k en xc[].
00178  */
00179  for (i = 1; i < nd; i++)
00180      c[i + nd] = c[i + nd - 1] + c[i - 1];
00181
00182  /* Mostrar conteos y prefijos */
00183  for (i = 0; i < nd; i++)
00184      printf("%d. [%+f, %+f]\t%d\t%d\n",
00185             i, min + i*Dx, min + (i+1)*Dx, c[i], c[i + nd]);
00186
00187  /*
00188  3) Distribución a xc:
00189      - Se vuelve a calcular el casillero j para cada x[i]
00190      - Se coloca x[i] en la siguiente posición libre del casillero j:
00191          xc[ c[nd + j] ] = x[i]
00192      y se incrementa esa posición para el siguiente elemento del mismo casillero.
00193  */
00194  for (i = 0; i < n; i++)
00195  {
00196      j = (int)((x[i] - min) / Dx);
00197
00198      /* WARNING: clamp recomendado para evitar j==nd */
00199      /* if (j >= nd) j = nd - 1; */
00200
00201      xc[c[j + nd]] = x[i];
00202      c[j + nd]++;
00203  }
00204
00205  /* Mostrar arreglo reordenado por casilleros */
00206  for (i = 0; i < n; i++)
00207  {
00208      j = (int)((xc[i] - min) / Dx);
00209      /* if (j >= nd) j = nd - 1; */
00210      printf("x[%02d] = %f\t%d\n", i + 1, xc[i], j);
00211  }
00212
00213  return 0;
00214  }

```

7.207 Ejemplo023.c

[Go to the documentation of this file.](#)

```

00001
00063
00064 #include <stdio.h>
00065 #include <stdlib.h>
00066 #include <time.h>
00067 #include <math.h>
00068
00069 #define N 100000
00070
00077 int main(int argc, char *argv[])
00078 {
00079     /*
00080     n : número de elementos a generar (1..N)
00081     i, j : índices auxiliares
00082     nd : número de divisiones/casilleros ( sqrt(n))
00083
00084     c[] : arreglo de enteros usado en dos zonas:
00085         - c[0..nd-1] = conteos por casillero (histograma)
00086         - c[nd..2*nd-1] = posiciones iniciales/prefijos (índices de escritura en xc)
00087     */
00088     int n, i, j, nd, c[N];
00089
00090     /*
00091     min, max : rango de generación
00092     x[] : arreglo original "desordenado"

```



```

00093     xc[]      : arreglo reordenado por casilleros
00094     rango     : max - min
00095     Dx        : ancho de cada casillero (rango/nd)
00096     */
00097     float min, max;
00098     float x[N], rango, Dx, xc[N];
00099
00100     /* Leer n */
00101     do {
00102         printf("Ingrese el numero de elementos: ");
00103         scanf("%d", &n);
00104     } while (n < 1 || n > N);
00105
00106     /* Leer rango */
00107     printf("Ingrese el valor maximo: ");
00108     scanf("%f", &max);
00109     printf("Ingrese el valor minimo: ");
00110     scanf("%f", &min);
00111
00112     /*
00113     Si el usuario capturó min > max, se intercambian.
00114     WARNING: este intercambio por multiplicación/división puede fallar si min==0 o max==0.
00115     */
00116     if (min > max)
00117     {
00118         max *= min;
00119         min = max / min;
00120         max /= min;
00121     }
00122
00123     srand(time(NULL));
00124
00125     /* Generar arreglo original y (por simplicidad) inicializar c[i]=0 para i<n */
00126     printf("Desordenado.\n");
00127     for (i = 0; i < n; i++)
00128     {
00129         /* Genera valores en [min, max] (puede incluir max si rand()==RAND_MAX) */
00130         x[i] = ((max - min) * rand()) / RAND_MAX + min;
00131
00132         /* Inicialización (basta con 0..2*nd-1, pero aquí se inicializa 0..n-1) */
00133         c[i] = 0;
00134
00135         printf("x[%02d] = %f\n", i + 1, x[i]);
00136     }
00137
00138     /* Definir número de casilleros */
00139     nd = (int)sqrt((double)n);
00140
00141     /* Calcular rango y ancho de casillero */
00142     rango = max - min;
00143     Dx = rango / nd;
00144
00145     printf("ND = %d\tRango = %f\tDx = %f\n", nd, rango, Dx);
00146
00147     /* Mostrar intervalos de los casilleros */
00148     for (i = 0; i < nd; i++)
00149         printf("%d. [%f, %f]\n", i, min + i*Dx, min + (i+1)*Dx);
00150
00151     /*
00152     1) Conteo por casillero:
00153         j = floor((x[i]-min)/Dx)
00154         c[j]++
00155     */
00156     printf("Desordenado con numero de casillero.\n");
00157     for (i = 0; i < n; i++)
00158     {
00159         j = (int)((x[i] - min) / Dx);
00160
00161         /*
00162         WARNING: si x[i] == max, j puede ser nd. Para robustez:
00163         if (j >= nd) j = nd - 1;
00164         */
00165
00166         printf("x[%02d] = %f\t%d\n", i + 1, x[i], j);
00167         c[j]++;
00168     }
00169
00170     /*
00171     2) Prefijos / posiciones iniciales:
00172     Se construye en la zona c[nd..2*nd-1]:
00173         c[nd] = 0
00174         c[nd+1] = c[0]
00175         c[nd+2] = c[0] + c[1]
00176         ...
00177     donde c[nd+k] es el índice de inicio para escribir el casillero k en xc[].
00178     */
00179     for (i = 1; i < nd; i++)

```

```

00180         c[i + nd] = c[i + nd - 1] + c[i - 1];
00181
00182     /* Mostrar conteos y prefijos */
00183     for (i = 0; i < nd; i++)
00184         printf("%d. [%+f, %+f]\t%d\t%d\n",
00185             i, min + i*Dx, min + (i+1)*Dx, c[i], c[i + nd]);
00186
00187     /*
00188     3) Distribución a xc:
00189         - Se vuelve a calcular el casillero j para cada x[i]
00190         - Se coloca x[i] en la siguiente posición libre del casillero j:
00191             xc[ c[nd + j] ] = x[i]
00192             y se incrementa esa posición para el siguiente elemento del mismo casillero.
00193     */
00194     for (i = 0; i < n; i++)
00195     {
00196         j = (int)((x[i] - min) / Dx);
00197
00198         /* WARNING: clamp recomendado para evitar j==nd */
00199         /* if (j >= nd) j = nd - 1; */
00200
00201         xc[c[j + nd]] = x[i];
00202         c[j + nd]++;
00203     }
00204
00205     /* Mostrar arreglo reordenado por casilleros */
00206     for (i = 0; i < n; i++)
00207     {
00208         j = (int)((xc[i] - min) / Dx);
00209         /* if (j >= nd) j = nd - 1; */
00210         printf("x[%02d] = %f\t%d\n", i + 1, xc[i], j);
00211     }
00212
00213     return 0;
00214 }

```

7.208 src/template/Template.c File Reference

Functions

- int [main](#) (int argc, char *argv[])

7.208.1 Function Documentation

7.208.1.1 main()

```

int main (
    int argc,
    char * argv[])

```

Definition at line 24 of file [Template.c](#).

```

00025 {
00026     // (Opcional) Evitar advertencias de compilación si no usamos argc/argv
00027     (void)argc;
00028     (void)argv;
00029
00030     // Termina el programa indicando "éxito".
00031     return 0;
00032 }

```

7.209 Template.c

[Go to the documentation of this file.](#)

```

00001
00002
00003
00004 int main(int argc, char *argv[])
00005 {
00006     // (Opcional) Evitar advertencias de compilación si no usamos argc/argv
00007     (void)argc;
00008     (void)argv;
00009
00010     // Termina el programa indicando "éxito".
00011     return 0;
00012 }

```